

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Kiến trúc Phần mềm (CO3017)

BÁO CÁO BÀI TẬP LỚN

Intelligent Restaurant Management System (IRMS)

Giảng viên: Trần Trương Tuấn Phát

Lớp học phần: L01

Nhóm: 8

Khoa: Khoa học và Kỹ thuật Máy tính

MSSV	Họ và tên	Email
2311896	Đỗ Quang Long	long.doquang1896@hcmut.edu.vn
2311982	Lê Hoàng Luân	luan.le2311982@hcmut.edu.vn
2310990	Lê Thúy Hiền	hien.lethuy@hcmut.edu.vn
2213698	Nguyễn Hải Trung	trung.nguyen2004@hcmut.edu.vn
2211384	Tô Thế Hưng	hung.totothehung@hcmut.edu.vn
2310737	Trần Nhật Đăng	dang.tranmh388@hcmut.edu.vn

Thành phố Hồ Chí Minh, tháng 4 năm 2026

Mục lục

1	Tổng quan hệ thống	3
1.0.1	Bối cảnh nghiệp vụ	3
1.0.2	Mục tiêu của hệ thống	3
1.0.3	Phạm vi hệ thống	3
1.1	Phạm vi hệ thống	4
1.2	Các bên liên quan	5
1.2.1	Cơ sở đo tải và ràng buộc dự án	5
1.3	Yêu cầu chức năng	6
1.4	Yêu cầu phi chức năng ở góc nhìn toàn dự án	6
2	Mô hình hóa hệ thống	8
2.1	Danh mục ca sử dụng	8
2.2	Góc nhìn sử dụng	12
2.2.1	Kịch bản 1: Từ đặt bàn đến tiếp nhận và xếp bàn	12
2.2.2	Kịch bản 2: Từ gọi món đến điều phối bếp và phục vụ	12
2.2.3	Kịch bản 3: Từ hóa đơn đến thanh toán và biên lai	13
2.2.4	Kịch bản 4: Từ cảnh báo mức tồn thấp đến quyết định của quản lý	13
2.3	Đặc tả ca sử dụng	21
3	Kiến trúc phần mềm	40
3.1	Tổng quan kiến trúc	40
3.2	Góc nhìn logic	42
3.2.1	Sơ đồ tổng quan	42
3.3	Tổng quan cho các góc nhìn chính	44
3.3.1	Tổng quan cho góc nhìn logic	44
3.3.2	Góc nhìn logic cho chuỗi phục vụ lỗi	45
3.3.3	Tổng quan cho góc nhìn mô-đun	46
3.3.4	Góc nhìn mô-đun cho tuyến order-to-cash	48
3.3.5	Tổng quan cho góc nhìn phân bổ	50
3.4	Đặc trưng kiến trúc	51
3.5	Góc nhìn phân rã	52
3.5.1	Các mô-đun phía người dùng	52
3.5.2	Các dịch vụ nghiệp vụ phía sau	52
3.5.3	Sơ đồ tổng quan	53
3.6	Góc nhìn thành phần và kết nối	59
3.6.1	Các thành phần chính	59
3.6.2	Các kiểu kết nối chính	60
3.6.3	Lý do chọn mô hình kết nối hỗn hợp	60
3.6.4	Sơ đồ thành phần cho tuyến lệnh vận hành	65
3.6.5	Sơ đồ thành phần cho xử lý nền, quản trị và mô hình đọc	68
3.7	Góc nhìn triển khai	69
3.7.1	Lý do thiết kế	69
3.7.2	Sơ đồ triển khai cho tính sẵn sàng cao và cô lập lỗi	76
3.8	Nguyên tắc thiết kế	78
3.9	Áp dụng nguyên lý SOLID vào thiết kế	79
3.9.1	Nguyên lý trách nhiệm đơn nhất	79
3.9.2	Nguyên lý đóng để ổn định, mở để mở rộng	79
3.9.3	Nguyên lý thay thế tương thích	80
3.9.4	Nguyên lý phân tách giao diện	80
3.9.5	Nguyên lý đảo ngược phụ thuộc	80

3.10	Khả năng mở rộng trong tương lai	81
4	Thiết kế chi tiết	82
4.1	Góc nhìn lớp	82
4.1.1	Phạm vi bao phủ của sơ đồ lớp	82
4.2	Giải thích theo từng cụm lớp	88
4.3	Thiết kế hành vi dựa trên ca sử dụng	100
4.3.1	Nhóm đặt bàn và bàn ăn	101
4.3.2	Nhóm gọi món và điều phối bếp	107
4.3.3	Nhóm hóa đơn, thanh toán và biên lai	114
4.3.4	Nhóm tồn kho, cảnh báo và phân tích	119
4.3.5	Nhóm quản trị cấu hình và vận hành	123
4.4	Phản tư về toàn bộ báo cáo và định hướng hiện thực	125
A	Phụ lục	126
A.1	Lý do chọn kiến trúc và hồ sơ quyết định kiến trúc	126
A.1.1	So sánh các kiểu kiến trúc ứng viên	127
A.2	Phân công công việc	129

1 Tổng quan hệ thống

Bối cảnh nghiệp vụ

IRMS được thiết kế cho môi trường vận hành nhà hàng có phục vụ tại bàn, khu bếp, quầy thanh toán và hoạt động điều phối theo ca. Trong bối cảnh thực tế, các sai sót thường xuất hiện đúng vào giờ cao điểm: đơn gọi món ghi nhầm món, ghi chú dị ứng bị bỏ sót, trạng thái bàn thay đổi nhưng không được cập nhật kịp, hóa đơn bị lệch khi có tách hóa đơn hoặc áp dụng khuyến mãi, và tồn kho cuối ngày không phản ánh đúng mức tiêu hao thực tế.

Bài toán vì vậy không chỉ là “ghi đơn gọi món trên màn hình”. IRMS phải đóng vai trò như một nền tảng số trung tâm liên kết khu vực phục vụ phía trước, khu vực bếp và khu vực quản trị: đặt bàn, xếp bàn, gọi món, điều phối bếp, lập hóa đơn, thanh toán, tồn kho, báo cáo, kiểm soát truy cập, nhật ký kiểm toán và các tích hợp bên ngoài. Đây là một hệ thống nghiệp vụ hoàn chỉnh, có đường xử lý lỗi rõ ràng, có luồng hỗ trợ bất đồng bộ và có yêu cầu mở rộng về sau.

Mục tiêu của hệ thống

Các mục tiêu thiết kế chính của IRMS không chỉ dừng ở mức “có đủ chức năng”, mà còn phải chuyển thành kết quả vận hành có thể kiểm chứng trong phạm vi một dự án phần mềm cho nhà hàng quy mô trung bình. Vì vậy, báo cáo đặt ra các mục tiêu sau:

1. **Giảm sai sót nghiệp vụ trên tuyến gọi món:** mục tiêu là giữ tỷ lệ đơn gọi món bị sửa do nhập sai dưới 2% tổng số đơn đã xác nhận trong một ca, thông qua thực đơn số hóa, tùy chọn món có ràng buộc, cảnh báo ghi chú dị ứng và bước xác nhận trước khi gửi bếp.
2. **Rút ngắn độ trễ từ phục vụ xuống bếp:** phiếu món phải xuất hiện trên màn hình bếp trong vòng tối đa 2 giây ở điều kiện thường và không quá 3 giây ở giờ cao điểm, để tránh việc phục vụ phải gọi miệng hoặc ghi tay.
3. **Đồng bộ trạng thái vận hành theo thời gian gần thực:** trạng thái bàn, đơn gọi món, phiếu bếp, hóa đơn và thanh toán phải phản ánh trên các màn hình liên quan với độ trễ quan sát được không vượt quá 5 giây đối với các thay đổi quan trọng.
4. **Rút ngắn thời gian đóng phiên phục vụ:** với một bàn tiêu chuẩn không có tách hóa đơn phức tạp, thời gian từ lúc yêu cầu thanh toán đến lúc phát hành biên lai nên dưới 2 phút; với tách hóa đơn tiêu chuẩn cho 2-4 người, mục tiêu là dưới 4 phút.
5. **Tăng khả năng kiểm soát và truy vết:** 100% thao tác nhạy cảm như hoàn tiền, hủy đơn đã gửi bếp, ghi đè giá, mở lại hóa đơn hoặc đổi trạng thái thanh toán phải được ghi nhận đầy đủ người thực hiện, thời điểm và lý do.
6. **Tạo nền tảng mở rộng về sau:** khi cần thêm phương thức thanh toán, kênh gửi thông báo, thiết bị tự phục vụ hoặc mở rộng sang nhiều chi nhánh, nhóm phát triển phải có thể mở rộng qua giao diện và dịch vụ mới mà không buộc sửa chéo logic lõi của toàn bộ hệ thống.

Phạm vi hệ thống

Trong phạm vi

- **Đặt bàn và bàn ăn:** đặt bàn, danh sách chờ, tiếp nhận khách, gán bàn, giải phóng bàn, theo dõi phiên phục vụ tại bàn;
- **Gọi món số hóa:** tạo đơn gọi món, chỉnh sửa đơn gọi món, tùy chọn món, món kết hợp, ghi chú dị ứng, hướng dẫn đặc biệt;
- **Quản lý thực đơn:** quản lý món, nhóm món, nhóm tùy chọn, trạng thái còn bán, khuyến mãi cơ bản;
- **Điều phối bếp:** phát phiếu theo trạm, cập nhật trạng thái món, theo dõi hàng chờ của bếp;

- **Lập hóa đơn và thanh toán:** tạo hóa đơn, tách hóa đơn, thuế, phí phục vụ, tiền boa, tiền mặt, thẻ hoặc ví điện tử;
- **Theo dõi tồn kho:** trừ nguyên liệu, lưu giao dịch kho, cảnh báo mức tồn thấp;
- **Phân tích và báo cáo:** doanh thu, món bán chạy, giờ cao điểm, điểm nghẽn, thống kê hoàn tiền;
- **Quản trị:** người dùng, vai trò, quyền, nhật ký kiểm toán, cấu hình thực đơn, chính sách giá.

Ngoài phạm vi

- kế toán doanh nghiệp và đối soát tài chính ở mức tập đoàn;
- hệ quản trị chuỗi nhiều chi nhánh ở mức triển khai đầy đủ;
- học máy nâng cao cho dự báo nhu cầu hoặc định giá động;
- tích hợp nền tảng giao hàng bên thứ ba ở mức hoàn chỉnh;
- hệ quản trị quan hệ khách hàng và nền tảng khách hàng thân thiết mở rộng.

1.1 Phạm vi hệ thống

IRMS bao phủ các nhóm chức năng sau:

- **Đặt bàn và xếp bàn:** đặt bàn, tiếp nhận khách, danh sách chờ, sơ đồ bàn, trạng thái bàn và vòng quay bàn.
- **Gọi món và thực đơn:** tra cứu thực đơn, tạo đơn gọi món, ghi chú dị ứng, tùy chọn món, món kết hợp và quản lý trạng thái món.
- **Điều phối bếp:** chuyển phiếu đến từng trạm bếp, theo dõi tiến độ chế biến, đồng bộ thông tin món sẵn sàng và món đã phục vụ.
- **Lập hóa đơn và thanh toán:** tạo hóa đơn, tách hóa đơn, tiền boa, thanh toán nhiều phương thức, biên lai và hoàn tiền.
- **Theo dõi tồn kho:** trừ tồn theo công thức món, theo dõi mức tồn thấp và hỗ trợ quyết định ẩn món.
- **Phân tích và quản trị:** báo cáo doanh thu, báo cáo vận hành, quản trị thực đơn, giá, khuyến mãi, vai trò và lịch làm việc.

1.2 Các bên liên quan

Bảng 1: Các bên liên quan chính của IRMS

Bên liên quan	Vai trò	Mối quan tâm chính
Khách hàng	Người dùng/đơn vị vận hành	Tạo/cập nhật đặt bàn, nhận xác nhận, thanh toán và nhận biên lai điện tử.
Lễ tân	Người dùng/đơn vị vận hành	Quản lý đặt bàn, danh sách chờ, tiếp nhận khách và tối ưu sơ đồ bàn.
Phục vụ	Người dùng/đơn vị vận hành	Tạo đơn gọi món, ghi chú tùy biến, theo dõi món, phục vụ tại bàn và hỗ trợ tính tiền.
Nhân viên bếp	Người dùng/đơn vị vận hành	Nhận phiếu bếp, cập nhật tiến độ chế biến và phát hiện nghẽn vận hành.
Thu ngân	Người dùng/đơn vị vận hành	Tạo hoá đơn, tách hóa đơn, xử lý thanh toán, in hoặc gửi biên lai và hoàn tiền theo quyền.
Quản lý	Người dùng/đơn vị vận hành	Quản lý menu, giá, khuyến mãi, ca làm, báo cáo và xử lý ngoại lệ nghiệp vụ.
Quản trị viên	Người dùng/đơn vị vận hành	Quản lý tài khoản, vai trò, phân quyền và nhật ký kiểm toán toàn hệ thống.

Cơ sở đo tải và ràng buộc dự án

Để các yêu cầu phi chức năng không bị mô tả chung chung, báo cáo dùng **hai điều kiện vận hành chuẩn** làm mốc đánh giá. Đây là giả định thiết kế cho một nhà hàng tầm trung, được dùng thống nhất ở toàn bộ báo cáo khi nói về hiệu năng, độ tin cậy và khả năng mở rộng.

Bảng 2: Hai điều kiện vận hành dùng làm cơ sở thiết kế của IRMS

Điều kiện	Tải giả định	Ý nghĩa với kiến trúc
Điều kiện 1 — Ca thông thường	20–25 bàn hoạt động, 8–12 nhân viên đăng nhập đồng thời, tối đa 35 đơn gọi món mỗi giờ, khoảng 120 dòng món mỗi giờ.	Hệ thống phải phản hồi mượt cho các tác vụ tạo đơn gọi món, cập nhật màn hình bếp, đổi trạng thái bàn và chốt hóa đơn mà chưa cần mở rộng hay hy sinh trải nghiệm thao tác.
Điều kiện 2 — Giờ cao điểm	35–40 bàn hoạt động, 15–18 nhân viên đăng nhập đồng thời, tối đa 60 đơn gọi món mỗi giờ, khoảng 220 dòng món mỗi giờ và 20 hóa đơn mỗi giờ.	Kiến trúc phải giữ ổn định tuyến giao dịch nóng; các luồng nền như báo cáo, cảnh báo mức tồn thấp và đồng bộ thông báo không được làm nghẽn gọi món, điều phối bếp hay thanh toán.

Trong hai điều kiện trên, dự án chịu các ràng buộc thực tế sau:

- **Ràng buộc phạm vi:** phiên bản hiện tại phục vụ một nhà hàng đơn lẻ, chưa tối ưu cho vận hành nhiều chi nhánh ở quy mô đầy đủ, nhưng kiến trúc phải chừa điểm mở cho mở rộng về sau.

- **Ràng buộc thiết bị:** nhân viên thao tác qua máy tính bảng gọi món, màn hình bếp, thiết bị thu ngân hoặc giao diện web nội bộ; trải nghiệm phải ưu tiên thao tác nhanh, ít bước và dễ học theo ca.
- **Ràng buộc mạng:** mạng nội bộ có thể chấp chừa ngắn hạn trong phạm vi dưới 60 giây; hệ thống được phép thử lại nhưng không được làm mất đơn gọi món đã xác nhận hoặc tạo phiếu trùng.
- **Ràng buộc thanh toán:** thanh toán điện tử luôn đi qua cổng đối tác ngoài; IRMS không lưu dữ liệu thẻ thô mà chỉ lưu kết quả giao dịch và mã tham chiếu.
- **Ràng buộc kiểm soát:** dữ liệu nghiệp vụ phải được lưu tập trung và có nhật ký kiểm toán cho hoàn tiền, hủy bỏ, ghi đề hoặc thay đổi nhạy cảm.
- **Ràng buộc học phần:** nhóm có thể chỉ hiện thực một phần module lõi trong học phần, nhưng kiến trúc phải phản ánh đầy đủ bức tranh hệ thống để chứng minh tư duy thiết kế tổng thể.

1.3 Yêu cầu chức năng

Các yêu cầu chức năng ở mức hạt thô được tổ chức thành năm cụm:

1. **Đặt bàn và xếp bàn:** tạo hoặc cập nhật đặt bàn, quản lý danh sách chờ, tiếp nhận khách, mở hoặc đóng phiên bàn.
2. **Gọi món và bếp:** tạo đơn gọi món, chỉnh sửa dòng món, chuyển phiếu sang màn hình bếp, cập nhật tiến độ và ra món.
3. **Thanh toán:** tạo hóa đơn, áp dụng khuyến mãi, tách hóa đơn, tiền boa, thanh toán, biên lai, hoàn tiền.
4. **Tồn kho và thực đơn:** quản lý thực đơn, giá, tùy chọn món, công thức, trừ tồn và cảnh báo mức tồn thấp.
5. **Quản trị và phân tích:** phân quyền theo vai trò, nhật ký kiểm toán, lịch ca, báo cáo doanh thu, hiệu suất nhân viên và điểm nghề của bếp.

1.4 Yêu cầu phi chức năng ở góc nhìn toàn dự án

Khác với ca sử dụng vốn mô tả hành vi theo từng kịch bản, yêu cầu phi chức năng của IRMS phải được nhìn ở mức **toàn dự án**. Chúng quyết định kiến trúc nào được chọn, mô-đun nào phải tách riêng, dữ liệu nào cần đồng bộ mạnh và luồng nào có thể xử lý bất đồng bộ. Vì vậy, mỗi yêu cầu dưới đây đều được gắn với **số liệu mục tiêu, điều kiện áp dụng** và **cách đáp ứng trong kiến trúc**.

Bảng 3: Yêu cầu phi chức năng của IRMS ở góc nhìn toàn dự án

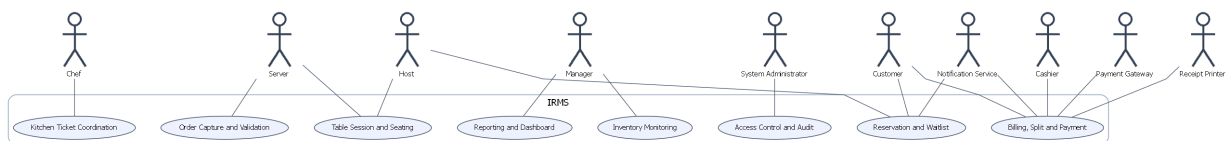
Thuộc tính	Mục tiêu đo được	Điều kiện áp dụng	Cách đáp ứng trong kiến trúc
Hiệu năng & độ phản hồi	Tra cứu thực đơn và trạng thái bàn có thời gian phản hồi bách phân vị thứ 95 dưới 1 giây; thao tác xác nhận đơn gọi món và gửi bếp có thời gian phản hồi bách phân vị thứ 95 dưới 2 giây ở Điều kiện 1 và dưới 3 giây ở Điều kiện 2; phiếu phải hiển thị trên màn hình bếp không quá 2 giây sau khi đơn được xác nhận.	Áp dụng cho toàn bộ giờ mở cửa, đặc biệt tại màn hình gọi món, màn hình bếp và quầy thu ngân là ba điểm chạm không được gây nhện thao tác người dùng.	Tách riêng các miền Ordering , Kitchen Coordination và Billing để giảm cạnh tranh tài nguyên; dùng mô hình đọc hoặc bộ nhớ đệm cho thực đơn và trạng thái thường đọc; giữ các tác vụ trên tuyến giao dịch nóng ở dạng gọi đồng bộ ngắn, còn báo cáo và cảnh báo chuyển sang xử lý nền.
Độ tin cậy & toàn vẹn giao dịch	Không làm mất đơn gọi món đã xác nhận; tỷ lệ tạo phiếu trùng hoặc ghi nhận thanh toán trùng phải thấp hơn 0.1%; dữ liệu gọi món và thanh toán có mục tiêu điểm khôi phục bằng 0 và khôi phục sau lỗi một nút ứng dụng trong vòng tối đa 10 phút.	Áp dụng ở cả Điều kiện 1 và Điều kiện 2; vẫn phải giữ đúng khi mạng nội bộ gián đoạn ngắn hạn dưới 60 giây hoặc khi đối tác thanh toán phản hồi chậm.	Dùng giao dịch cục bộ theo dịch vụ, khóa chống xử lý lặp cho gọi món, thanh toán và hoàn tiền, khóa lạc quan khi cập nhật trạng thái, cùng hộp thư ra và bộ truyền sự kiện cho sự kiện miền để tránh vừa mất dữ liệu vừa xử lý lặp.
Khả năng mở rộng	Hệ thống phải chịu được tối thiểu 1.5 lần tải của Điều kiện 2 trong các đợt cao điểm ngắn 10–15 phút mà không cần thiết kế lại; riêng các miền Ordering , Kitchen Coordination và Billing phải có thể tăng số phiên bản chạy độc lập khi tải tăng.	Áp dụng cho bữa trưa, bữa tối, ngày cuối tuần hoặc các đơn đặt nhóm lớn; báo cáo và phân tích không được kéo chậm tuyến giao dịch nóng.	Chọn kiến trúc tách miền nghiệp vụ rõ ràng để phân bổ tải theo nhu cầu thực tế; mở rộng theo chiều ngang các miền có nhu cầu cao; cô lập báo cáo bằng mô hình đọc và ảnh chụp dữ liệu để truy vấn phân tích không tranh chấp với cơ sở dữ liệu giao dịch.
Bảo mật & khả năng kiểm toán	100% thao tác hoàn tiền, hủy món đã gửi bếp, ghi đề giá, mở lại hóa đơn và thay đổi quyền phải có nhật ký người thực hiện, thời điểm, giá trị trước, giá trị sau và lý do; phiên thu ngân hoặc quản lý tự khóa sau 15 phút không hoạt động; hệ thống không lưu dữ liệu thẻ gốc.	Áp dụng liên tục trong toàn bộ vòng đời vận hành và đặc biệt quan trọng ở các ca đối nhân sự, bàn giao ca hoặc xử lý khiếu nại sau thanh toán.	Dùng IAM tập trung, phân quyền theo vai trò, chính sách kiểm tra quyền trong từng dịch vụ, nhật ký kiểm toán bất biến cho hành động nhạy cảm, và tích hợp thanh toán qua đối tác ngoài để giới hạn phạm vi dữ liệu nhạy cảm nằm trong IRMS.
Khả năng quan sát & vận hành	100% yêu cầu ghi dữ liệu phải mang mã liên kết xuyên suốt; cảnh báo mức tồn thấp phải xuất hiện trong vòng 60 giây sau khi vượt ngưỡng; bảng điều khiển vận hành có độ trễ dữ liệu không quá 5 phút; lỗi ở cổng vào hoặc cổng thanh toán phải sinh cảnh báo vận hành trong vòng 1 phút.	Áp dụng cho cả thời gian phục vụ và giai đoạn chốt ca, khi quản lý cần đối chiếu chênh lệch đơn gọi món, hóa đơn, tồn kho và thao tác hoàn tiền.	Ghi nhật ký tập trung theo dịch vụ, truy vết xuyên suốt bằng mã liên kết, thu thập số đo cho độ trễ phiếu và thanh toán, tổng hợp bảng điều khiển từ mô hình đọc, cùng với dấu thời gian sự kiện để có thể truy vết một đơn từ lúc tạo đến lúc đóng hóa đơn.

Thuộc tính	Mục tiêu đo được	Điều kiện áp dụng	Cách đáp ứng trong kiến trúc
Khả năng bảo trì & mở rộng	Khi thêm một phương thức thanh toán hoặc một kênh thông báo mới, thay đổi chủ yếu chỉ nằm ở bộ kết nối, cấu hình và kiểm thử tích hợp; thay đổi thực đơn, giá, khuyến mãi hoặc ngưỡng mức tồn thấp không được buộc sửa chéo bộ điều khiển, thanh toán và tồn kho cùng lúc.	Áp dụng trong suốt vòng đời dự án, đặc biệt khi nhà hàng đổi chính sách giá, khuyến mãi theo mùa hoặc cần thí điểm thêm kênh thanh toán mới.	Phân rã theo miền nghiệp vụ, áp dụng nguyên lý SOLID và đảo ngược phụ thuộc cho cổng ngoài, tách lớp điều phối ứng dụng khỏi quy tắc miền, chuẩn hóa hợp đồng giữa các dịch vụ để mỗi thay đổi được khu trú trong biên chức năng tương ứng.

2 Mô hình hóa hệ thống

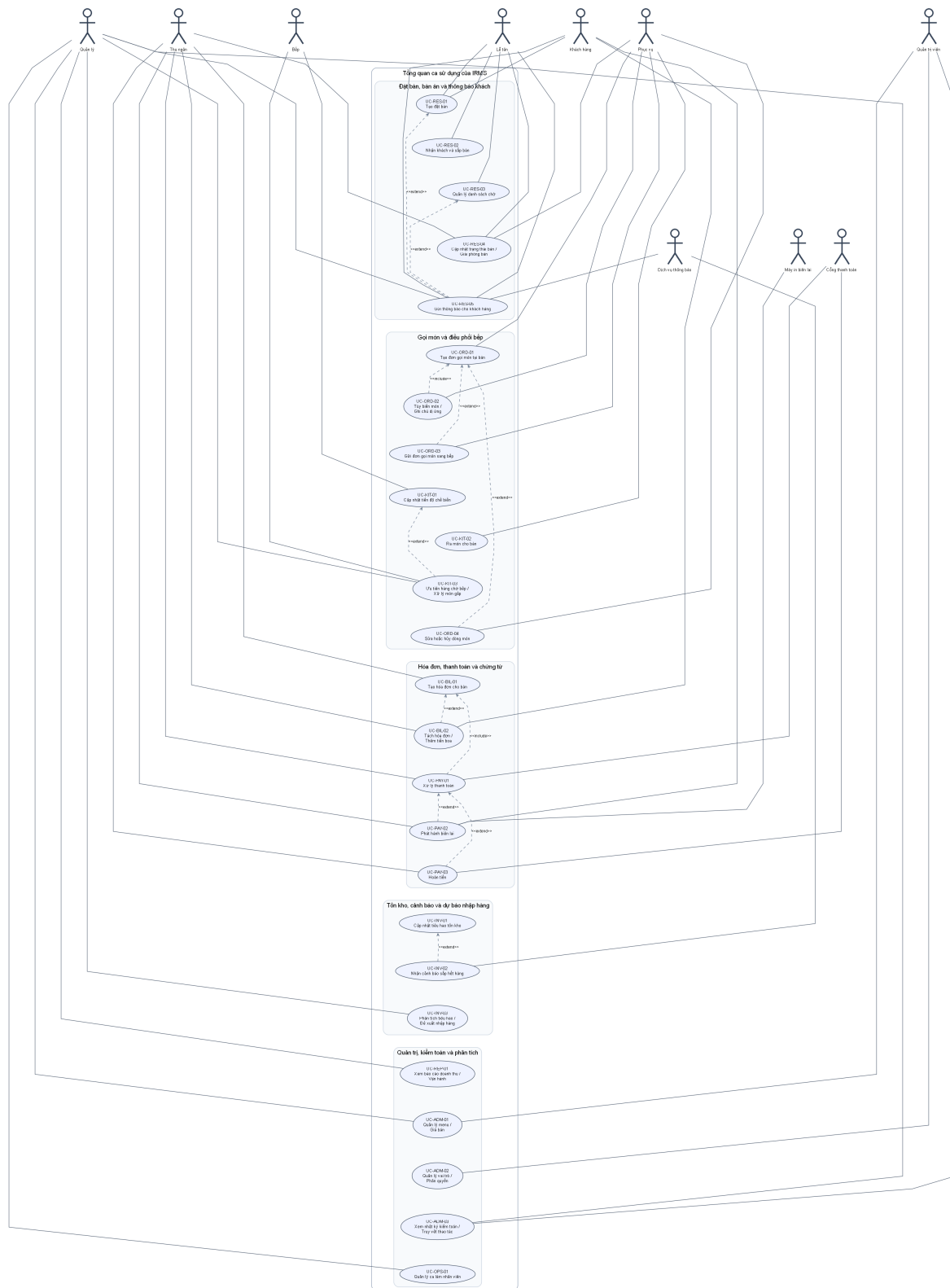
2.1 Danh mục ca sử dụng

Ba hình dưới đây không chỉ có vai trò minh họa mà còn là phần chuyển tiếp quan trọng trước khi đi vào đặc tả ca sử dụng. Chúng lần lượt trả lời ba câu hỏi nền tảng: **IRMS giao tiếp với ai, IRMS phải hỗ trợ những nhóm nghiệp vụ nào và một vòng vận hành đầu-cuối trong nhà hàng diễn ra ra sao.** Việc diễn giải ba hình này giúp phần ca sử dụng phía sau bám đúng bối cảnh toàn hệ thống thay vì bị nhìn như những kịch bản rời rạc.



Hình 1: Sơ đồ ngữ cảnh của IRMS

Hình 1 chốt rõ ranh giới hệ thống và các điểm giao tiếp bên ngoài của IRMS. Ở lớp ngoài cùng là các tác nhân nghiệp vụ như khách hàng, lễ tân, phục vụ, bếp, thu ngân và quản lý; ở lớp tích hợp là các đối tác như cổng thanh toán, kênh gửi thông báo hoặc hạ tầng báo cáo. Ý nghĩa của hình này là xác nhận rằng IRMS không phải một ứng dụng gọi món đơn lẻ, mà là trục điều phối trung tâm của cả khu vực phục vụ phía trước, khu vực bếp và khu vực quản trị.

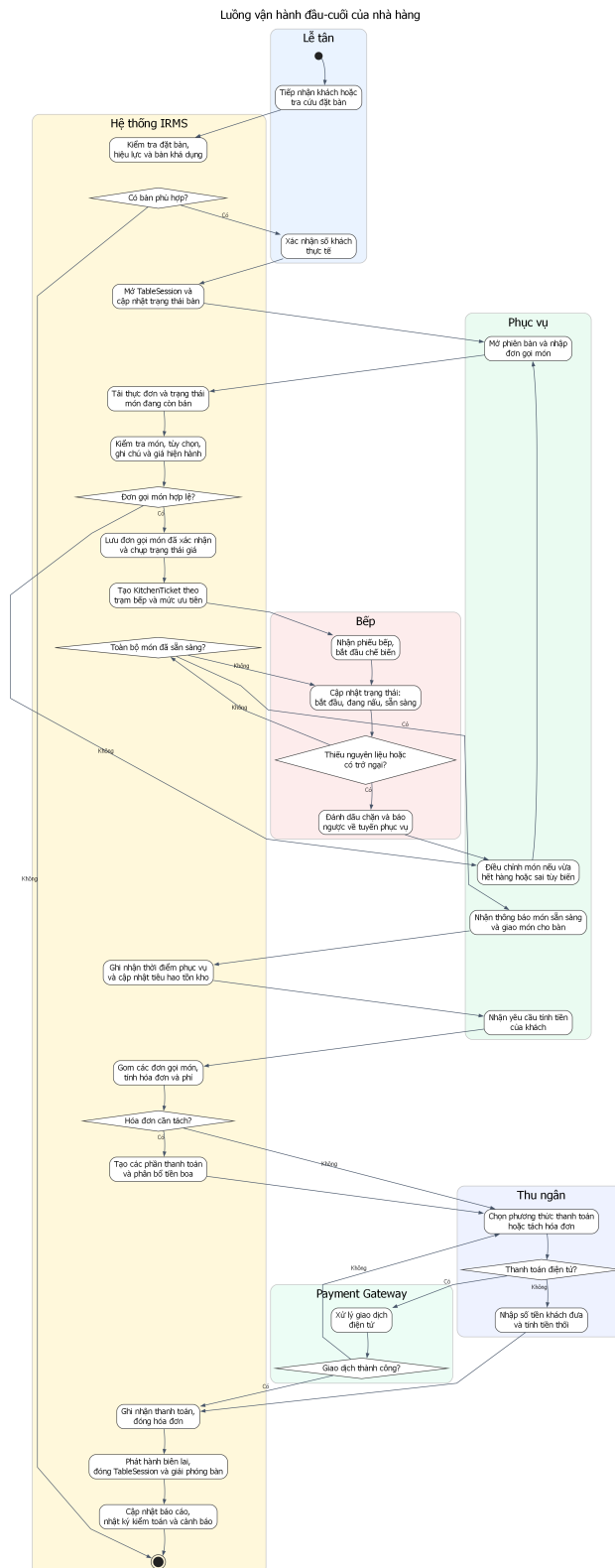


Hình 2: Sơ đồ tổng quan ca sử dụng của IRMS

Hình 2 gom các tương tác nghiệp vụ thành năm cụm lớn: đặt bàn và vòng đời bàn, gọi món và điều phối bếp, hóa đơn và thanh toán, tồn kho và dự báo nhập hàng, cùng quản trị, kiểm toán và phân tích. Cách tổ chức này cho thấy phạm vi của IRMS không dừng ở các thao tác trực tiếp trước khách hàng, mà bao trùm cả phần kết thúc vòng đời bàn, giao tiếp chủ động với khách hàng, điều phối ưu tiên trong bếp, hỗ trợ quyết định nhập hàng từ dữ liệu tiêu hao lịch sử và truy vết thao tác nhạy cảm ở tầng quản trị. Vì vậy, sơ đồ tổng quan ca sử dụng không chỉ là danh sách chức năng,

mà là bản đồ nghiệp vụ đủ kín để dẫn sang phần kiến trúc và thiết kế chi tiết.

Nhìn theo góc độ nghiệp vụ, năm cụm này cũng tương ứng với năm lớp trách nhiệm khác nhau trong hệ thống. Cụm đặt bàn và bàn ăn quản lý năng lực tiếp nhận khách theo thời gian thực; cụm gọi món và bếp điều khiển luồng thực thi nóng nhất của nhà hàng; cụm hóa đơn và thanh toán chốt quyết định tài chính; cụm tồn kho và dự báo hỗ trợ điều hành vật tư; còn cụm quản trị, kiểm toán và phân tích bảo đảm hệ thống có thể được kiểm soát, truy vết và cải tiến sau vận hành. Vì vậy, sơ đồ tổng quan ca sử dụng không chỉ là danh mục chức năng, mà còn là bản đồ chỉ ra nơi nào cần phản hồi tức thời, nơi nào cần độ đúng đắn cao, và nơi nào có thể chấp nhận xử lý sau trên tuyến nền.



Hình 3: Sơ đồ hoạt động của luồng vận hành đầu-cuối nhà hàng

Hình 3 mô tả luồng vận hành đầu-cuối tiêu biểu của nhà hàng, bắt đầu từ khâu tiếp nhận khách hoặc kiểm tra đặt bàn, chuyển sang mở phiên bàn, tạo đơn gọi món, gửi phiếu xuống bếp, cập nhật tiến độ chế biến, giao món, lập hóa đơn, xử lý thanh toán và kết thúc bằng giải phóng bàn. Điểm quan trọng của sơ đồ hoạt động này là nó cho thấy những trạng thái nào phải được đồng bộ mạnh giữa các bộ phận: một bàn chỉ được mở một phiên phục vụ, một đơn đã xác nhận phải sinh phiếu bếp, và một hóa đơn chỉ được đóng khi thanh toán hoàn tất hoặc được ghi nhận là thất bại có kiểm

soát.

Nhìn dưới góc độ kiến trúc, luồng đầu-cuối này cũng chỉ ra ranh giới giữa **đường giao dịch nóng** và **đường xử lý nền**. Các bước như xác nhận đơn gọi món, cập nhật trạng thái phiếu Ready, ghi nhận thanh toán thành công và đóng `TableSession` thuộc về tuyến đồng bộ bắt buộc vì chúng tác động trực tiếp đến trải nghiệm khách hàng. Ngược lại, các bước như cập nhật báo cáo, phát cảnh báo mức tồn thấp, gửi thông báo xác nhận hoặc đồng bộ số liệu quản trị có thể được đẩy sang luồng bất đồng bộ để giảm tải cho các điểm chạm thời gian thực.

Bảng 4: Danh mục 25 ca sử dụng chính được đặc tả chi tiết

Nhóm	Mã ca sử dụng
Nhóm đặt bàn và bàn ăn	UC-RES-01, UC-RES-02, UC-RES-03, UC-RES-04, UC-RES-05
Nhóm gọi món và điều phối bếp	UC-ORD-01, UC-ORD-02, UC-ORD-03, UC-KIT-01, UC-KIT-02, UC-KIT-03, UC-ORD-04
Nhóm thanh toán	UC-BIL-01, UC-BIL-02, UC-PAY-01, UC-PAY-02, UC-PAY-03
Nhóm tồn kho, cảnh báo và dự báo	UC-INV-01, UC-INV-02, UC-INV-03
Nhóm quản trị, kiểm toán và phân tích	UC-REP-01, UC-ADM-01, UC-ADM-02, UC-ADM-03, UC-OPS-01

Năm ca sử dụng giữ vai trò khóa kín phạm vi nghiệp vụ ở phần này là UC-RES-04, UC-RES-05, UC-KIT-03, UC-INV-03 và UC-ADM-03. Chúng làm rõ các mắt xích dễ bị xem nhẹ khi mô tả hệ thống nhà hàng ở mức khái quát: kết thúc vòng đời bàn sau thanh toán, giao tiếp chủ động với khách hàng, điều phối ưu tiên trong bếp, hỗ trợ quyết định nhập hàng từ dữ liệu lịch sử và truy vết thao tác nhạy cảm ở tầng quản trị.

2.2 Góc nhìn sử dụng

Góc nhìn sử dụng cho thấy cách các tác nhân tương tác với các dịch vụ qua các kịch bản nghiệp vụ tiêu biểu. Về mặt lý thuyết, góc nhìn sử dụng là công cụ rất tốt để **kiểm chứng kiến trúc bằng kịch bản**. Một phương án phân rã đẹp trên góc nhìn mô-đun vẫn có thể thất bại khi đi qua tình huống thực, nếu chuỗi tương tác quá dài, lỗi khó giải thích hoặc ranh giới giao dịch không rõ. Do đó, bốn kịch bản dưới đây được chọn vì chúng chạm vào bốn luồng cốt lõi nhất của IRMS.

Kịch bản 1: Từ đặt bàn đến tiếp nhận và xếp bàn

1. Khách hàng hoặc lễ tân tạo đặt bàn.
2. `Reservation Service` kiểm tra khả dụng và gán bàn hoặc giữ chỗ.
3. Khi khách đến, `Host UI` xác nhận tiếp nhận và mở `TableSession`.
4. `TableSession` được chuyển tới `Server POS UI` để phục vụ gọi món.

Kịch bản 2: Từ gọi món đến điều phối bếp và phục vụ

1. Phục vụ tạo đơn gọi món và cấu hình tùy chọn món hoặc gợi ý dĩa ứng trên `Server POS UI`.
2. `Ordering Service` xác nhận đơn gọi món và phát sự kiện tạo phiếu bếp.
3. `Kitchen Coordination Service` tách phiếu theo trạm bếp và hiển thị lên `Kitchen Display UI`.
4. Bếp cập nhật trạng thái từng dòng phiếu; màn hình gọi món nhận lại trạng thái sẵn sàng.
5. Phục vụ giao món và đánh dấu đã phục vụ; `Inventory Service` nhận sự kiện để trừ tồn.

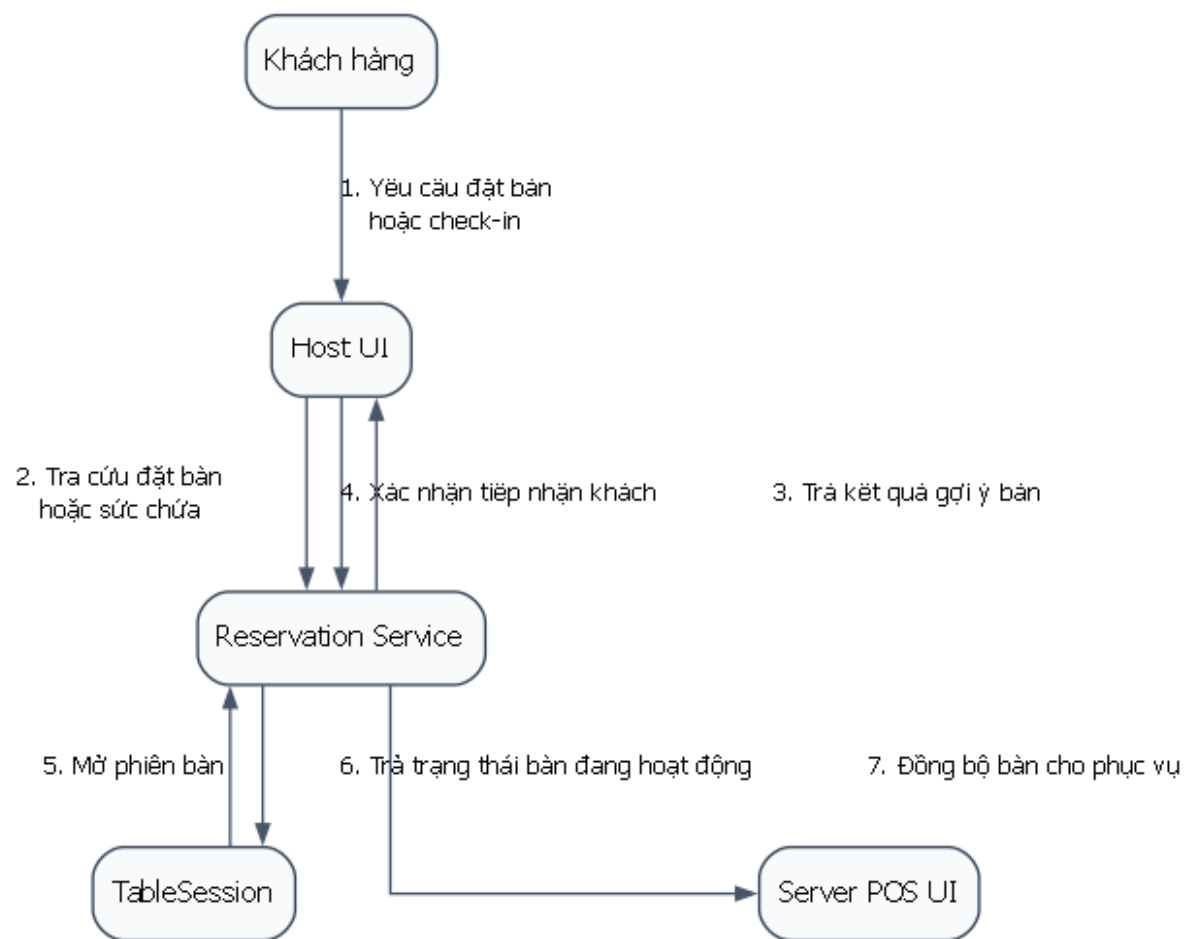
Kịch bản 3: Từ hóa đơn đến thanh toán và biên lai

1. Thu ngân hoặc phục vụ gom các đơn gọi món của bàn thành hóa đơn.
2. **Billing Service** áp dụng khuyến mãi, tiền boa và tách hóa đơn nếu cần.
3. Thanh toán được thực hiện qua tiền mặt hoặc cổng thanh toán điện tử.
4. Khi thanh toán đủ, biên lai được phát hành và **TableSession** chuyển sang chuẩn bị đóng.

Kịch bản 4: Từ cảnh báo mức tồn thấp đến quyết định của quản lý

1. **Inventory Service** cập nhật lượng tồn thực tế sau mỗi món đã phục vụ hoặc sau mỗi lần điều chỉnh thủ công.
2. Nếu xuống dưới ngưỡng, hệ thống tạo sự kiện cảnh báo mức tồn thấp.
3. **Manager Portal** nhận cảnh báo, hiển thị nguyên liệu bị ảnh hưởng và món có nguy cơ hết hàng.
4. Quản lý quyết định nhập thêm hàng hoặc tạm ẩn món trong menu.

Giải thích chi tiết cho góc nhìn sử dụng. Góc nhìn sử dụng đặc biệt quan trọng với IRMS vì hệ thống không phải một khối xử lý tĩnh; nó là một chuỗi phối hợp giữa người và máy theo thời gian thực. Mỗi kịch bản ở trên đều cho thấy ranh giới nào cần đồng bộ mạnh, chẳng hạn như thanh toán, và ranh giới nào chấp nhận xử lý bất đồng bộ, chẳng hạn như báo cáo hoặc cảnh báo mức tồn thấp.



Hình 4: Sơ đồ trình tự cho kịch bản từ đặt bàn đến tiếp nhận và xếp bàn

Hình 4 cho thấy **Reservation** và **TableSession** không phải là cùng một thực thể nghiệp vụ dù chúng liên quan trực tiếp. Điểm mạnh của cách tách này là hệ thống xử lý được nhiều trường hợp thực tế: khách có đặt bàn nhưng không đến, khách vào trực tiếp không có đặt bàn nhưng vẫn mở phiên phục vụ, hoặc lễ tân phải đổi bàn khi năng lực phục vụ thay đổi. Nếu trộn hai khái niệm này thành một, kiến trúc sẽ khó biểu diễn các trạng thái chuyển tiếp quan trọng của vận hành nhà hàng.

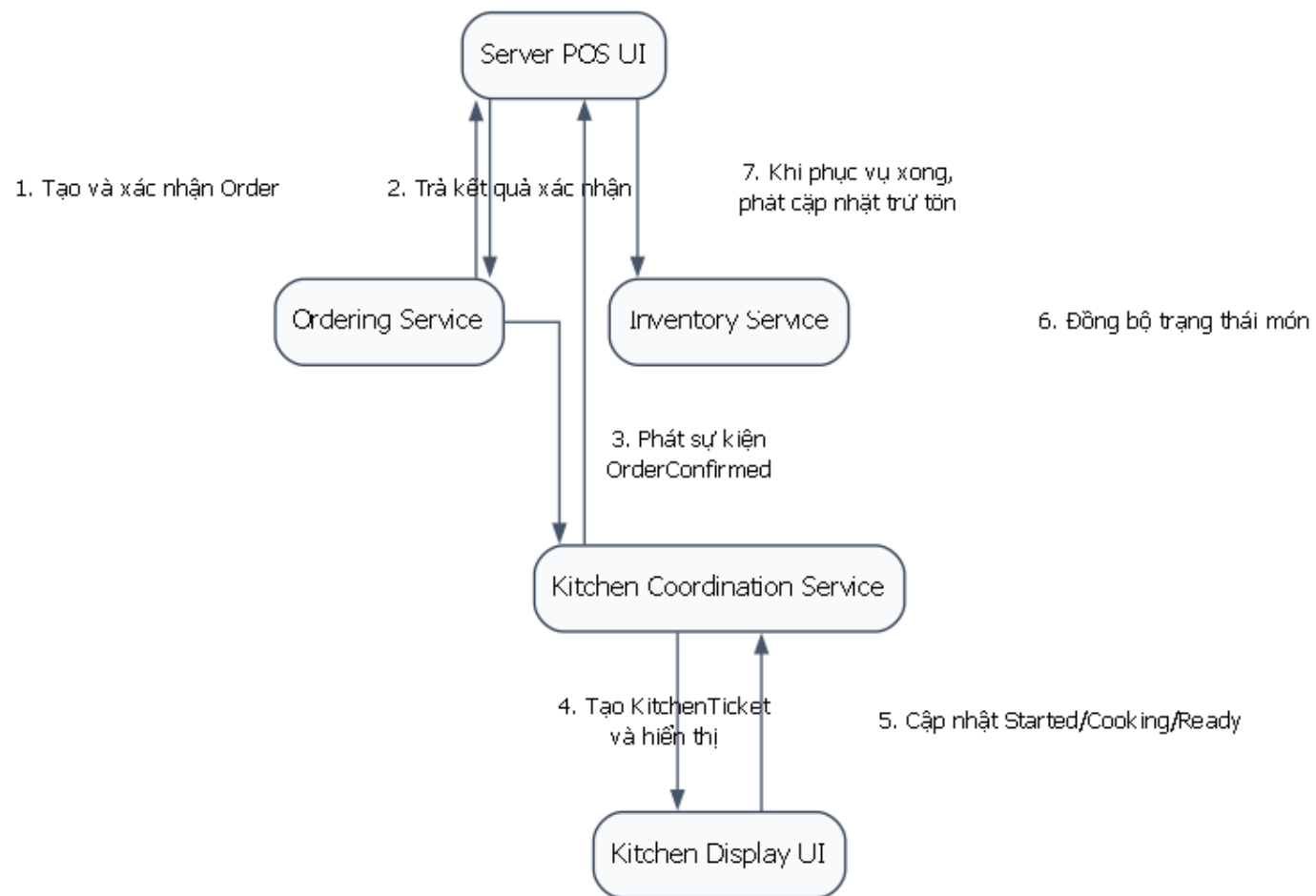
Đánh đổi là luồng xếp bàn phải cẩn thận hơn về đồng bộ trạng thái: đặt bàn có thể còn hiệu lực nhưng bàn ăn chưa sẵn sàng; bàn có thể vừa được giải phóng nhưng vẫn đang dọn dẹp; thao tác tiếp nhận có thể đến muộn so với thời điểm giữ chỗ ban đầu. Do đó, góc nhìn sử dụng này nhắc rằng đặt bàn và xếp bàn tuy nhìn đơn giản, nhưng thực ra là nơi chứa nhiều quy tắc nghiệp vụ về thời gian và năng lực bàn.

Các thành phần chính thể hiện trong hình.

- **Khách hàng và Host UI:** khởi tạo yêu cầu đặt bàn hoặc check-in.
- **Reservation Service:** chịu trách nhiệm xác thực trạng thái đặt bàn và gợi ý bàn.
- **TableSession và Server POS UI:** nhận bàn đã mở để chuyển sang luồng phục vụ thực tế.

Lý do thiết kế của sơ đồ này.

- Sơ đồ làm rõ ranh giới giữa dữ liệu đặt bàn và phiên bàn thực tế.
- Đây là kịch bản giúp chứng minh tại sao bài toán xếp bàn cần được mô hình hóa riêng thay vì gộp vào một trạng thái bàn đơn giản.



Hình 5: Sơ đồ trình tự cho kịch bản từ gọi món đến điều phối bếp và phục vụ

Hình 5 là kịch bản quan trọng nhất để kiểm chứng **khả năng đáp ứng**. Nguyên tắc thiết kế nên là: *lưu đơn gọi món thành công trước, rồi mới lan truyền sang bếp*. Điều này bảo vệ tuyến đầu theo hai cách. Thứ nhất, màn hình gọi món nhận phản hồi nhanh hơn vì không phải đợi toàn bộ chuỗi bếp, tồn kho và báo cáo. Thứ hai, khi việc chuyển sang bếp gặp trục trặc, hệ thống vẫn có thể thử lại từ trạng thái đơn đã xác nhận thay vì mất dữ liệu đơn gọi món.

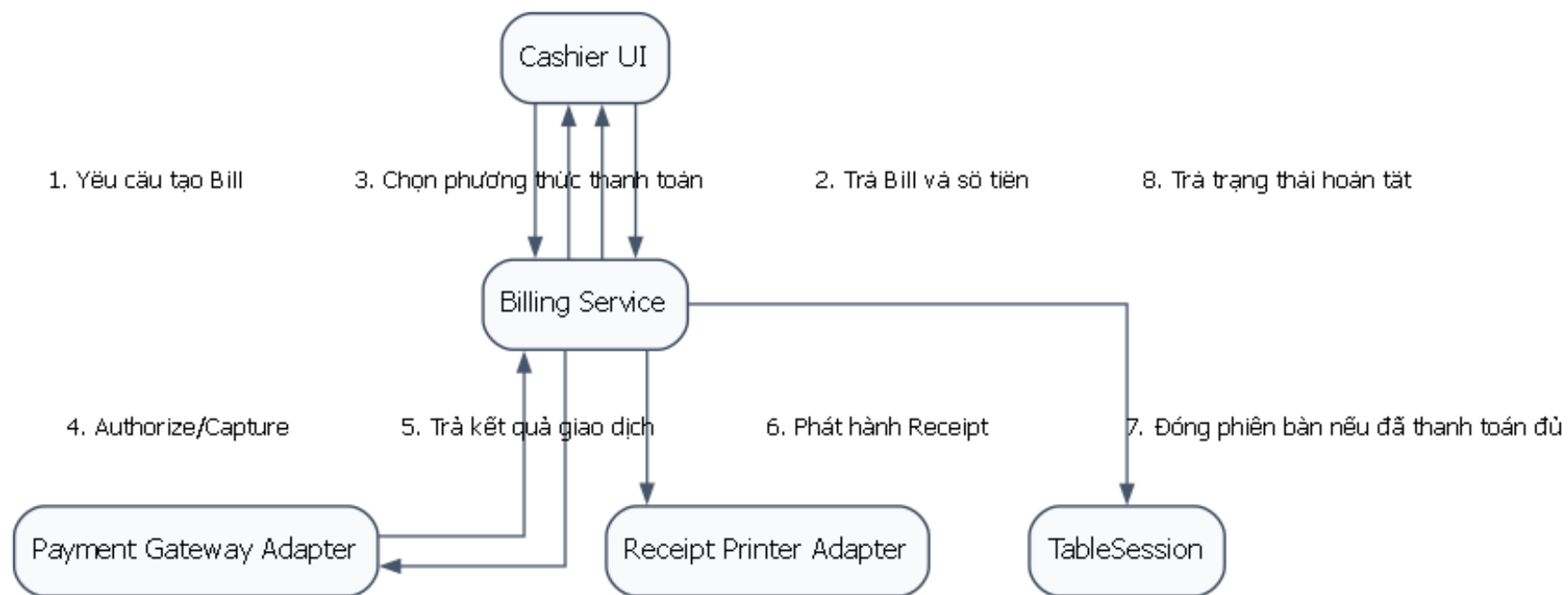
Ưu điểm của mô hình này là tách được **bước chốt giao dịch** khỏi **bước lan truyền vận hành**. Tuy nhiên, đánh đổi là màn hình bếp và màn hình gọi món phải xử lý được trạng thái trung gian: đơn đã xác nhận nhưng phiếu chưa hiện lên màn hình bếp ngay lập tức; một dòng món có thể đã sẵn sàng ở bếp nhưng màn hình gọi món chưa nhận được cập nhật cuối cùng; việc trừ tồn cũng có thể đi sau sự kiện phục vụ xong. Vì thế, góc nhìn sử dụng này là bằng chứng rằng cơ chế nhất quán trễ ở IRMS phải được dùng có kiểm soát, không dùng tùy tiện.

Các thành phần chính thể hiện trong hình.

- **Server POS UI**: là điểm bắt đầu của thao tác gọi món.
- **Ordering Service**: là nơi chốt giao dịch gọi món.
- **Kitchen Coordination Service và Kitchen Display UI**: nhận luồng lan truyền sang bếp sau khi đơn đã được xác nhận.
- **Inventory Service**: chỉ nhận cập nhật sau khi món đã đi qua luồng phục vụ phù hợp.

Lý do thiết kế của sơ đồ này.

- Sơ đồ được viết để nhấn mạnh thứ tự **?xác nhận trước, lan truyền sau?** của tuyến gọi món.
- Đây là kịch bản quan trọng nhất để giải thích vì sao kiến trúc phải cân bằng giữa phản hồi nhanh và nhất quán có kiểm soát.



Hình 6: Sơ đồ trình tự cho kịch bản từ hóa đơn đến thanh toán và biên lai

Hình 6 là nơi kiến trúc phải nghiêng mạnh về **độ tin cậy**. Ở kịch bản này, khối lập hóa đơn không chỉ làm phép cộng cuối cùng, mà còn là nơi áp dụng khuyến mãi, tiền boa, tách hóa đơn, chọn phương thức thanh toán, giao tiếp với cổng thanh toán, phát hành biên lai và đổi trạng thái phiên phục vụ. Đây là luồng có mật độ chuyển trạng thái cao nhất và hệ quả nghiệp vụ lớn nhất nếu xử lý sai.

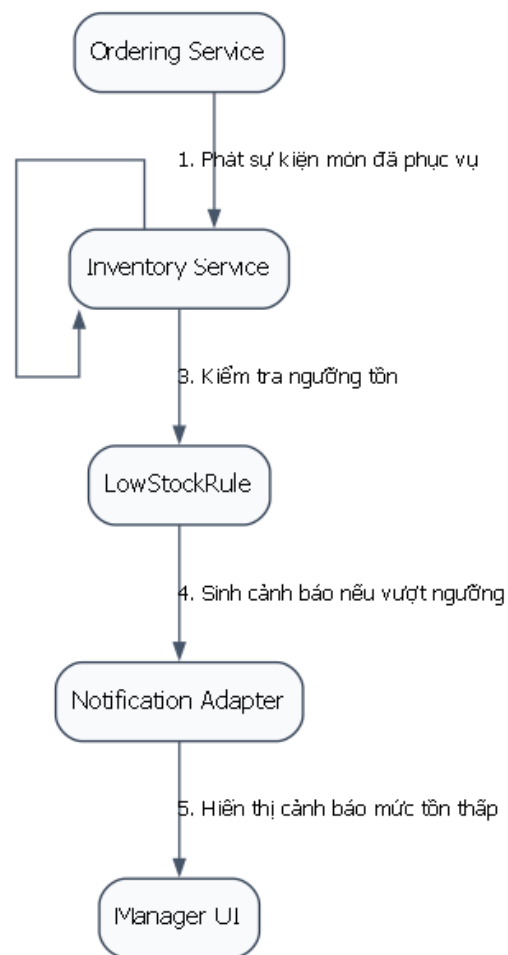
Ưu điểm của việc giữ thanh toán trong luồng đồng bộ là hệ thống có thể trả lời rất rõ: giao dịch thành công hay thất bại, hóa đơn còn mở hay đã đóng, phiên phục vụ còn tiếp diễn hay đã kết thúc. Ngược lại, nếu thanh toán bị đẩy sang bất đồng bộ quá mức, thu ngân và khách hàng sẽ rất khó hiểu trạng thái thật sự của giao dịch. Nhược điểm của cách làm đồng bộ là đường xử lý thanh toán trở thành điểm nhảy cảm với hệ ngoài, đặc biệt khi cổng thanh toán chậm hoặc lỗi. Vì thế, đây là nơi cần thời gian chờ rõ ràng, khóa chống xử lý lặp, nhật ký kiểm toán và cơ chế xử lý lỗi minh bạch nhất.

Các thành phần chính thể hiện trong hình.

- **Cashier UI**: là điểm điều khiển nghiệp vụ của thu ngân.
- **Billing Service**: giữ trung tâm của tuyến quyết toán.
- **Payment Gateway Adapter**: đại diện cho tích hợp thanh toán ngoài hệ thống.
- **Receipt Printer Adapter và TableSession**: kết thúc luồng bằng phát hành biên lai và đóng phiên bàn.

Lý do thiết kế của sơ đồ này.

- Đây là luồng nhạy cảm nhất về độ đúng đắn nghiệp vụ, nên cần được tách riêng để làm rõ ranh giới trách nhiệm và ranh giới giao dịch.
- Việc tách rõ cổng thanh toán, phát hành biên lai và đóng phiên bàn giúp người đọc thấy ranh giới trách nhiệm của từng bước sau khi thanh toán hoàn tất.



Hình 7: Sơ đồ trình tự cho kịch bản từ cảnh báo mức tồn thấp đến quyết định của quản lý

Hình 7 cho thấy rõ vì sao quản lý tồn kho và gửi thông báo không nên chen trực tiếp vào tuyến giao dịch nóng. Việc theo dõi tồn kho cần chính xác ở mức vận hành, nhưng không nhất thiết phải khóa trải nghiệm gọi món của phục vụ chỉ vì một cảnh báo quản lý chưa được gửi xong. Cách đặt việc trừ tồn và cảnh báo mức tồn thấp trên luồng sự kiện giúp hệ thống giữ được **khả năng sửa đổi**: khi chính sách trừ kho, ánh xạ công thức hay ngưỡng cảnh báo thay đổi, phần ảnh hưởng chủ yếu nằm trong khối tồn kho và khối gửi thông báo.

Nhược điểm là lượng tồn hiển thị cho quản lý có thể hơi chậm so với giao dịch mới nhất nếu bộ xử lý nền bị trễ hoặc sự kiện đang được thử lại. Điều này chấp nhận được trong phạm vi IRMS, miễn là không nhầm lẫn cảnh báo tồn kho với xác nhận thanh toán. Nói cách khác, góc nhìn sử dụng này giúp chứng minh nơi nào được phép chấp nhận độ trễ đồng bộ, và nơi nào thì tuyệt đối không nên chấp nhận điều đó.

Các thành phần chính thể hiện trong hình.

- **Ordering Service**: là nguồn phát sinh biến động vận hành cần suy diễn sang kho.
- **Inventory Service và LowStockRule**: chịu trách nhiệm cập nhật tồn và đánh giá ngưỡng cảnh báo.
- **Notification Adapter và Manager UI**: là đầu ra quản trị của luồng cảnh báo.

Lý do thiết kế của sơ đồ này.

- Sơ đồ cho thấy rõ tồn kho và cảnh báo nên đi trên một tuyến hỗ trợ thay vì chen vào đường giao dịch nóng.
- Đây là ví dụ rõ nhất để giải thích sự khác nhau giữa **đúng về vận hành** và **phải phản hồi tức thời** trong kiến trúc.

2.3 Đặc tả ca sử dụng

UC-RES-01 — Tạo đặt bàn

Tác nhân chính: Khách hàng, Lễ tân

Mục tiêu: Tạo một đặt bàn hợp lệ cho ngày, giờ và số lượng khách cụ thể.

Kích hoạt: Khách hàng gửi yêu cầu đặt bàn qua quầy lễ tân, điện thoại hoặc web/app.

Tiền điều kiện

- Nhà hàng có mở nhận đặt bàn trong khung giờ yêu cầu.
- Thông tin liên hệ của khách hàng được cung cấp tối thiểu gồm tên và số điện thoại.
- Sơ đồ bàn và năng lực phục vụ của chi nhánh đã được cấu hình.

Hậu điều kiện

- Một bản ghi đặt bàn được tạo ở trạng thái **Pending** hoặc **Confirmed**.
- Hệ thống giữ chỗ sơ bộ hoặc gợi ý danh sách bàn phù hợp cho lễ tân.
- Khách hàng nhận được mã đặt bàn và thông báo xác nhận.

Luồng chính

1. Lễ tân mở màn hình đặt bàn và nhập thời gian, số khách, tên khách, số điện thoại.
2. Hệ thống kiểm tra năng lực tiếp nhận theo chi nhánh, ca phục vụ và bàn còn trống.
3. Hệ thống gợi ý bàn hoặc nhóm bàn đáp ứng sức chứa.
4. Lễ tân xác nhận lựa chọn bàn và các ghi chú đặc biệt của khách.
5. Hệ thống tạo đặt bàn, sinh mã đặt bàn và lưu lịch sử thao tác.
6. Hệ thống gửi thông báo xác nhận qua SMS/email hoặc in phiếu xác nhận.

Luồng thay thế

- Nếu chưa thể gán ngay một bàn cụ thể, hệ thống tạo đặt bàn ở chế độ giữ chỗ theo khoảng sức chứa.
- Nếu khách yêu cầu khu vực riêng, tầng riêng hoặc bàn gần cửa sổ, lễ tân lọc danh sách gợi ý theo thuộc tính bàn.

Ngoại lệ

- Không còn sức chứa phù hợp: hệ thống đề xuất khung giờ khác hoặc mời vào danh sách chờ.
- Thông tin khách không hợp lệ: hệ thống yêu cầu cập nhật số điện thoại hoặc tên liên hệ.
- Lỗi đồng bộ sơ đồ bàn: hệ thống khóa thao tác xác nhận và yêu cầu tải lại dữ liệu mới nhất.

Quy tắc nghiệp vụ

- Không cho phép nhận đặt bàn vượt quá sức chứa tối đa của nhóm bàn.
- Đặt bàn cho nhóm lớn hơn ngưỡng cấu hình phải được lễ tân hoặc quản lý xác nhận.

UC-RES-02 — Nhận khách và sắp bàn

Tác nhân chính: Lễ tân, Phục vụ

Mục tiêu: Tiếp nhận khách đến nhà hàng và gán khách vào bàn hoặc nhóm bàn phù hợp.

Kích hoạt: Khách đến quầy lễ tân hoặc được gọi từ danh sách chờ.

Tiền điều kiện

- Khách có đặt bàn hoặc có thể được xếp chỗ tại chỗ.
- Sơ đồ bàn và trạng thái bàn đang được đồng bộ theo thời gian thực.

Hậu điều kiện

- Một phiên phục vụ bàn (`TableSession`) được mở.
- Bàn được chuyển sang trạng thái `Occupied` hoặc `Reserved-Arrived`.
- Phục vụ phụ trách khu vực nhận được thông báo tiếp nhận bàn mới.

Luồng chính

1. Lễ tân tìm đặt bàn theo mã, số điện thoại hoặc tên khách.
2. Hệ thống hiển thị trạng thái đặt bàn, thời điểm đến và bàn gợi ý.
3. Lễ tân xác nhận khách đến, số khách thực tế và thời gian check-in.
4. Nếu cần, lễ tân điều chỉnh bàn hoặc ghép bàn theo số khách thực tế.
5. Hệ thống mở phiên phục vụ bàn và phân công bàn cho phục vụ khu vực.
6. Phục vụ nhận thông báo và chuẩn bị tiếp nhận đơn gọi món đầu tiên.

Luồng thay thế

- Khách vắng lai không đặt trước: lễ tân chọn chức năng xếp bàn trực tiếp và hệ thống gợi ý bàn còn trống.
- Khách đến thiếu người hoặc tăng số lượng khách: lễ tân cập nhật lại sức chứa trước khi mở phiên phục vụ.

Ngoại lệ

- Bàn dự kiến chưa sẵn sàng: hệ thống chuyển khách sang danh sách chờ hoặc gợi ý bàn thay thế.
- Đặt bàn đã hết hiệu lực vì quá giờ giữ chỗ: hệ thống yêu cầu quản lý xác nhận phục hồi.

- Mất kết nối màn hình sơ đồ bàn: thao tác check-in tạm ngừng để tránh gán trùng bàn.

Quy tắc nghiệp vụ

- Chỉ một phiên phục vụ được phép hoạt động trên một bàn tại cùng thời điểm.
- Thời gian giữ chỗ tối đa sau giờ đặt được cấu hình theo ca phục vụ.

UC-RES-03 — Quản lý danh sách chờ

Tác nhân chính: Lễ tân

Mục tiêu: Ghi nhận khách chờ bàn và chủ động điều phối khi có bàn phù hợp.

Kích hoạt: Nhà hàng hết bàn phù hợp cho khách đến trực tiếp hoặc khách đặt bàn bị trì hoãn.

Tiền điều kiện

- Lễ tân có quyền truy cập màn hình danh sách chờ.
- Hệ thống đã ghi nhận thời gian chờ ước tính theo tải hiện tại.

Hậu điều kiện

- Khách hàng được thêm vào danh sách chờ với thời gian chờ dự kiến.
- Khi có bàn phù hợp, khách được gọi theo mức ưu tiên và thời gian vào hàng.

Luồng chính

1. Lễ tân nhập thông tin khách, số lượng khách và kênh liên hệ.
2. Hệ thống ước tính thời gian chờ dựa trên vòng quay bàn và loại bàn yêu cầu.
3. Lễ tân xác nhận thêm khách vào danh sách chờ.
4. Khi một bàn được giải phóng, hệ thống dò các bản ghi phù hợp theo sức chứa và ưu tiên.
5. Lễ tân chọn khách để gọi, hệ thống gửi thông báo và đặt trạng thái **Called**.
6. Sau khi khách xác nhận, lễ tân chuyển khách sang ca sử dụng Nhận khách và sắp bàn.

Luồng thay thế

- Lễ tân có thể ưu tiên khách thành viên hạng cao hoặc khách đã đặt trước nhưng đến muộn.
- Khách có thể được cập nhật số điện thoại, thay đổi số người hoặc hủy yêu cầu trong lúc chờ.

Ngoại lệ

- Khách không phản hồi trong thời gian giữ lượt gọi: hệ thống chuyển trạng thái sang **Skipped**.
- Thời gian chờ tăng đột biến do cao điểm: hệ thống yêu cầu lễ tân xác nhận lại ước tính trước khi gửi thông báo.

Quy tắc nghiệp vụ

- Ưu tiên danh sách chờ mặc định là FIFO trong cùng một nhóm sức chứa.
- Chỉ bàn đáp ứng sức chứa tối thiểu và phạm vi khu vực yêu cầu mới được dùng để gọi khách.

UC-RES-04 — Cập nhật trạng thái bàn và giải phóng bàn

Tác nhân chính: Lễ tân, Phục vụ, Thu ngân

Mục tiêu: Cập nhật đúng trạng thái vận hành của bàn, đóng phiên phục vụ khi đủ điều kiện và giải phóng bàn cho lượt khách tiếp theo.

Kích hoạt: Bàn hoàn tất phục vụ, khách đã rời bàn, bàn cần chuyển sang dọn dẹp, mở lại hoặc tạm khóa.

Tiền điều kiện

- Bàn tồn tại trong sơ đồ bàn của chi nhánh.
- Người thao tác có quyền cập nhật trạng thái bàn theo vai trò.
- Nếu đang có **TableSession**, hệ thống phải truy xuất được trạng thái hóa đơn liên quan.

Hậu điều kiện

- Trạng thái bàn được cập nhật nhất quán trên sơ đồ vận hành.
- Nếu đủ điều kiện, **TableSession** được đóng và bàn chuyển sang **Cleaning** hoặc **Available**.
- Lịch sử thay đổi trạng thái bàn được ghi nhận để phục vụ audit và phân tích vòng quay bàn.

Luồng chính

1. Người dùng chọn bàn cần cập nhật trên sơ đồ bàn.
2. Hệ thống hiển thị trạng thái hiện tại của bàn, phiên phục vụ đang hoạt động và công nợ liên quan nếu có.
3. Người dùng chọn trạng thái đích như **Cleaning**, **Available** hoặc **OutOfService**.
4. Nếu mục tiêu là giải phóng bàn, hệ thống kiểm tra các hóa đơn hoặc split còn mở.
5. Nếu mọi nghĩa vụ thanh toán đã hoàn tất, hệ thống đóng **TableSession** và chuyển bàn sang **Cleaning**.
6. Sau khi nhân viên xác nhận dọn bàn xong, hệ thống chuyển bàn sang **Available**.
7. Nếu có khách chờ phù hợp, hệ thống làm mới ước tính chờ hoặc gợi ý gọi khách tiếp theo.

Luồng thay thế

- Bàn có thể được chuyển sang **OutOfService** khi bảo trì, ghép bàn hoặc khóa khu vực.
- Nếu khách được chuyển sang bàn khác trong lúc phục vụ, hệ thống cập nhật ánh xạ bàn mới mà vẫn giữ nguyên phiên phục vụ.

Ngoại lệ

- Hóa đơn hoặc split còn chưa tất toán: hệ thống từ chối giải phóng bàn và yêu cầu xử lý phần công nợ trước.
- Có cập nhật đồng thời từ thiết bị khác: hệ thống phát hiện xung đột trạng thái và yêu cầu tải lại sơ đồ bàn.
- Vai trò hiện tại không đủ quyền đổi sang trạng thái đặc biệt như **OutOfService**: hệ thống chặn thao tác.

Quy tắc nghiệp vụ

- Một bàn chỉ được chuyển sang **Available** khi không còn **TableSession** đang hoạt động và không còn hóa đơn mở.
- Mọi thao tác giải phóng bàn, khóa bàn hoặc mở lại bàn phải được ghi nhận thời điểm và người thực hiện.

UC-RES-05 — Gửi thông báo cho khách hàng

Tác nhân chính: Hệ thống, Lễ tân

Mục tiêu: Gửi thông báo nhất quán cho khách hàng khi đặt bàn được xác nhận, thay đổi, đến lượt chờ hoặc khi có sự kiện cần phản hồi.

Kích hoạt: Phát sinh sự kiện như tạo đặt bàn mới, đổi giờ, xác nhận danh sách chờ, nhắc giờ đến hoặc thông báo bàn đã sẵn sàng.

Tiền điều kiện

- Bản ghi đặt bàn hoặc danh sách chờ có thông tin liên hệ hợp lệ.
- Mẫu thông báo và kênh gửi đã được cấu hình.
- Hệ thống có khả năng kết nối tới dịch vụ gửi thông báo hoặc hàng đợi retry.

Hậu điều kiện

- Một bản ghi `NotificationMessage` được tạo với trạng thái đã gửi, đã xếp hàng hoặc cần xử lý thủ công.
- Khách hàng nhận được thông tin cần thiết qua SMS, email hoặc kênh đã chọn.
- Lịch sử gửi thông báo được lưu để tránh gửi trùng và để hỗ trợ truy vết.

Luồng chính

1. Hệ thống nhận một sự kiện nghiệp vụ cần thông báo cho khách.
2. Hệ thống xác định loại thông báo, mẫu nội dung và thời điểm gửi phù hợp.
3. Hệ thống kiểm tra thông tin liên hệ của khách hàng và kênh gửi ưu tiên.
4. Hệ thống cá nhân hóa nội dung bằng mã đặt bàn, thời gian, khu vực hoặc thông tin chờ.
5. Hệ thống tạo `NotificationMessage` và gửi thông báo qua kênh đã chọn.
6. Hệ thống ghi nhận kết quả gửi và cập nhật trạng thái của thông báo.

Luồng thay thế

- Lễ tân có thể kích hoạt gửi lại thủ công nếu khách yêu cầu hoặc nếu cần xác nhận lại.
- Nếu SMS lỗi nhưng email khả dụng, hệ thống có thể chuyển sang kênh dự phòng theo cấu hình.

Ngoại lệ

- Thiếu số điện thoại hoặc email hợp lệ: hệ thống đánh dấu cần liên hệ thủ công.
- Dịch vụ thông báo ngoài không phản hồi: hệ thống đưa thông báo vào hàng chờ retry và cảnh báo cho lễ tân khi cần.
- Thông báo bị chặn vì vượt ngưỡng chống spam: hệ thống bỏ qua bản gửi trùng và lưu lý do.

Quy tắc nghiệp vụ

- Mỗi thông báo phải tham chiếu tới một thực thể nghiệp vụ nguồn như `Reservation` hoặc `WaitlistEntry`.
- Các thông báo có tính thời điểm như gọi khách từ danh sách chờ phải kèm khung thời gian phản hồi rõ ràng.

UC-ORD-01 — Tạo đơn gọi món tại bàn

Tác nhân chính: Phục vụ

Mục tiêu: Tạo đơn gọi món cho một bàn đang phục vụ, bao gồm món, số lượng và ghi chú.

Kích hoạt: Phục vụ nhấn ?Tạo đơn gọi món? trên màn hình bàn đang mở phiên phục vụ.

Tiền điều kiện

- Phiên phục vụ bàn đã được mở.
- Phục vụ có quyền thao tác trên khu vực phụ trách.

Hậu điều kiện

- Đơn gọi món được lưu ở trạng thái `Draft` hoặc `Confirmed`.
- Các dòng món được gán với bàn và phiên phục vụ tương ứng.

Luồng chính

1. Phục vụ mở phiên bàn và chọn chức năng tạo đơn gọi món.
2. Hệ thống hiển thị thực đơn, các món gợi ý và danh sách combo.
3. Phục vụ chọn món, số lượng, ghi chú đặc biệt và xác nhận từng dòng món.
4. Hệ thống tính tạm tính theo giá hiện hành.
5. Phục vụ lưu đơn gọi món nhập hoặc xác nhận gửi bếp ngay.

Luồng thay thế

- Phục vụ có thể tạo nhiều đơn gọi món nối tiếp cho cùng một bàn trong suốt phiên phục vụ.
- Phục vụ có thể dùng mẫu đơn gọi món thường gặp cho bàn đoàn hoặc thực đơn theo set.

Ngoại lệ

- Món vừa chuyển sang hết hàng: hệ thống chặn thêm món và yêu cầu chọn thay thế.
- Bàn đã đóng phiên phục vụ: hệ thống từ chối tạo đơn gọi món mới và yêu cầu mở lại bàn nếu được phép.

Quy tắc nghiệp vụ

- Mỗi đơn gọi món phải gắn đúng một phiên phục vụ bàn.
- Giá của dòng món được chụp tại thời điểm xác nhận đơn gọi món.

UC-ORD-02 — Tùy biến món và ghi chú dị ứng

Tác nhân chính: Phục vụ, Bếp

Mục tiêu: Ghi nhận các lựa chọn tùy chọn món, mức độ chế biến và cảnh báo dị ứng cho từng món.

Kích hoạt: Phục vụ chọn một món đã thêm vào đơn gọi món để cấu hình chi tiết.

Tiền điều kiện

- Món ăn hỗ trợ nhóm tùy chọn hoặc cho phép nhập ghi chú.
- Danh sách tùy chọn và quy tắc chọn đã được cấu hình.

Hậu điều kiện

- Các tùy chọn được lưu thành `OrderItemModifier` và hiển thị trên phiếu bếp.
- Cảnh báo dị ứng hoặc yêu cầu đặc biệt đi kèm từng món.

Luồng chính

1. Phục vụ mở chi tiết dòng món đã chọn.
2. Hệ thống hiển thị các nhóm modifier bắt buộc/tùy chọn và giới hạn chọn.
3. Phục vụ chọn các tùy biến như không cay, thêm phô mai, không hành hoặc mức chín.
4. Phục vụ nhập ghi chú dị ứng hoặc hướng dẫn riêng cho bếp.
5. Hệ thống kiểm tra số lượng lựa chọn tối thiểu/tối đa và lưu cấu hình.
6. Khi đơn gọi món được gửi bếp, toàn bộ ghi chú xuất hiện trên phiếu bếp.

Luồng thay thế

- Nếu nhà hàng dùng cấu hình chuẩn, phục vụ có thể chọn nhanh một cấu hình có sẵn.
- Một số ghi chú nhạy cảm như dị ứng đậu phộng được hệ thống tự gắn cờ màu nổi bật.

Ngoại lệ

- Vi phạm quy tắc chọn tùy chọn: hệ thống yêu cầu bổ sung hoặc bỏ bớt lựa chọn.

- Ghi chú vượt quá độ dài cho phép: hệ thống yêu cầu rút gọn hoặc chuyển thành ghi chú nội bộ.

Quy tắc nghiệp vụ

- Ghi chú dị ứng phải được gửi đến bếp dưới định dạng không thể bị ẩn khỏi phiếu bếp.
- Một tùy chọn có thể làm thay đổi đơn giá của dòng món.

UC-ORD-03 — Gửi đơn gọi món sang bếp

Tác nhân chính: Phục vụ, Bếp

Mục tiêu: Chuyển đơn gọi món đã xác nhận từ thiết bị phục vụ sang các trạm bếp phù hợp để chế biến.

Kích hoạt: Phục vụ nhấn ?Gửi bếp? hoặc đơn gọi món đạt điều kiện gửi tự động.

Tiền điều kiện

- Đơn gọi món có ít nhất một dòng món hợp lệ.
- Kênh kết nối tới hệ thống điều phối bếp khả dụng.

Hậu điều kiện

- Một hoặc nhiều `KitchenTicket` được tạo theo trạm bếp.
- Trạng thái đơn gọi món chuyển sang `Confirmed/Queued`.

Luồng chính

1. Phục vụ xác nhận các dòng món cần gửi.
2. Hệ thống kiểm tra trạng thái món, tồn kho logic và quy tắc định tuyến trạm bếp.
3. Hệ thống tách phiếu bếp theo loại trạm như bếp nướng, bếp chiên, tráng miệng hoặc quầy đồ uống.
4. Phiếu bếp được xếp hàng theo độ ưu tiên, thời gian đến và hạn phục vụ.
5. Màn hình bếp hiển thị phiếu bếp mới và phát cảnh báo âm thanh hoặc màu sắc.

Luồng thay thế

- Nếu đơn gọi món có món phục vụ sau, phục vụ có thể đánh dấu chế độ gửi sau.
- Nếu cùng một bàn có nhiều đơn gọi món sát nhau, hệ thống có thể gộp phiếu bếp trong cùng khoảng thời gian cấu hình.

Ngoại lệ

- Không xác định được trạm bếp cho món: hệ thống chặn gửi và tạo cảnh báo cho quản lý menu.
- Mất kết nối với màn hình bếp: hệ thống lưu hàng đợi gửi lại và cảnh báo cho phục vụ.

Quy tắc nghiệp vụ

- Mỗi dòng món phải được định tuyến đến ít nhất một trạm bếp chịu trách nhiệm.
- Chỉ các dòng món ở trạng thái hợp lệ mới được gửi bếp.

UC-KIT-01 — Cập nhật tiến độ chế biến

Tác nhân chính: Bếp

Mục tiêu: Theo dõi và cập nhật trạng thái món từ lúc nhận phiếu bếp đến khi sẵn sàng phục vụ.

Kích hoạt: Đầu bếp chọn một phiếu bếp trên màn hình bếp.

Tiền điều kiện

- Ticket đã được tạo và gán cho đúng trạm bếp.
- Nhân viên bếp đã đăng nhập vào trạm làm việc.

Hậu điều kiện

- Trạng thái phiếu bếp và từng món phản ánh đúng thực tế chế biến.
- Phục vụ nhìn thấy tiến độ mới nhất trên POS.

Luồng chính

1. Nhân viên bếp mở phiếu bếp đang chờ.
2. Bếp đánh dấu từng món **Started**, **Cooking**, **Ready**.
3. Hệ thống tính thời gian chuẩn bị thực tế và so sánh với SLA nội bộ.
4. Khi toàn bộ món của phiếu bếp sẵn sàng, hệ thống đánh dấu phiếu bếp ở trạng thái **Ready**.
5. POS của phục vụ nhận thông báo để ra món.

Luồng thay thế

- Bếp có thể ưu tiên phiếu bếp khẩn do khách VIP hoặc yêu cầu ra trước món khai vị.
- Nếu món cần chờ đồng bộ với món khác, bếp có thể đánh dấu **Hold for service**.

Ngoại lệ

- Món bị thiếu nguyên liệu trong lúc chế biến: bếp đánh dấu **Blocked** và gửi yêu cầu thay món.
- Màn hình KDS mất kết nối: hệ thống chuyển sang chế độ hàng đợi cục bộ và tự đồng bộ lại khi có mạng.

Quy tắc nghiệp vụ

- Chỉ bếp thuộc đúng trạm mới được đổi trạng thái phiếu bếp của trạm đó.
- Thời gian chuyển trạng thái phải được ghi vào nhật ký kiểm toán vận hành.

UC-KIT-02 — Ra món cho bàn

Tác nhân chính: Phục vụ

Mục tiêu: Nhận món đã hoàn tất từ bếp và giao đúng bàn, đúng thứ tự phục vụ.

Kích hoạt: Hệ thống thông báo phiếu bếp hoặc món đã ở trạng thái **Ready**.

Tiền điều kiện

- Món đã được bếp hoàn tất.
- Phiên bàn vẫn đang mở.

Hậu điều kiện

- Dòng món được đánh dấu **Served**.
- Thời điểm ra món được ghi nhận cho phân tích SLA phục vụ.

Luồng chính

1. Phục vụ mở danh sách món sẵn sàng và xác nhận nhận món.
2. Hệ thống hiển thị bàn, seat number và ghi chú phục vụ liên quan.
3. Phục vụ giao món cho khách tại bàn.
4. Phục vụ nhấn ?Đã phục vụ?; hệ thống cập nhật trạng thái từng dòng món.
5. Nếu toàn bộ món đã ra đủ, hệ thống cập nhật nhịp phục vụ của bàn.

Luồng thay thế

- Nhà hàng có thể chọn chế độ xác nhận theo từng món hoặc theo toàn bộ phiếu bếp.

- Nếu cần đồng bộ ra món theo course, phục vụ có thể giữ món sẵn sàng và phát lệnh ra sau.

Ngoại lệ

- Phục vụ phát hiện món sai bàn hoặc sai tùy biến: hệ thống cho phép trả lại bếp với lý do.
- Ticket đã sẵn sàng nhưng bàn tạm vắng: phục vụ đánh dấu trì hoãn phục vụ.

Quy tắc nghiệp vụ

- Mọi thao tác trả món hoặc phục vụ lại phải có lý do để phục vụ báo cáo vận hành.
- Không được đánh dấu **Served** khi món chưa đạt trạng thái **Ready**.

UC-KIT-03 — Ưu tiên hàng chờ bếp và xử lý món gấp

Tác nhân chính: Bếp, Quản lý

Mục tiêu: Điều chỉnh mức ưu tiên của ticket hoặc dòng món để giảm nguy cơ trễ hạn, xử lý món gấp và điều phối tốt hơn trong giờ cao điểm.

Kích hoạt: Ticket gần vượt SLA, có yêu cầu phục vụ gấp, khách VIP, hoặc quản lý cần can thiệp vào thứ tự hàng chờ.

Tiền điều kiện

- KitchenTicket hoặc KitchenTicketItem đã tồn tại trên hàng chờ của trạm bếp.
- Hệ thống có dữ liệu về độ trễ, thứ tự đến và chính sách ưu tiên.
- Người thao tác có quyền thay đổi mức ưu tiên hoặc có thể yêu cầu quản lý xác nhận.

Hậu điều kiện

- Mức ưu tiên của ticket hoặc item được cập nhật và phản ánh trên KDS.
- Lý do ưu tiên được ghi nhận để phục vụ audit vận hành.
- Hàng chờ của trạm bếp được sắp xếp lại theo chính sách mới.

Luồng chính

1. Nhân viên bếp hoặc quản lý mở hàng chờ của trạm chế biến.
2. Hệ thống hiển thị độ trễ hiện tại, thời hạn phục vụ và các ticket có nguy cơ trễ hạn.
3. Người thao tác chọn ticket hoặc dòng món cần ưu tiên.
4. Hệ thống áp dụng chính sách ưu tiên dựa trên SLA, loại món, phụ thuộc giữa các course và cấu hình VIP nếu có.
5. Người thao tác nhập lý do ưu tiên hoặc xác nhận đề xuất tự động của hệ thống.
6. Hệ thống cập nhật mức ưu tiên, sắp xếp lại hàng chờ và làm nổi bật ticket gấp trên KDS.

Luồng thay thế

- Hệ thống có thể tự động đẩy mức ưu tiên nếu ticket sắp vượt ngưỡng thời gian phục vụ.
- Chỉ một phần của ticket, ví dụ món khai vị hoặc món cần ra cùng course, có thể được ưu tiên riêng.

Ngoại lệ

- Người thao tác không đủ quyền đổi ưu tiên: hệ thống yêu cầu xác nhận của quản lý hoặc giữ nguyên thứ tự cũ.
- Trạm bếp đang bị quá tải nghiêm trọng: hệ thống cảnh báo rủi ro và gợi ý điều phối sang trạm hỗ trợ nếu mô hình bếp cho phép.
- Dữ liệu thời gian phục vụ không khả dụng: hệ thống chỉ cho phép ưu tiên thủ công có lý do.

Quy tắc nghiệp vụ

- Mọi hành động expedite phải có lý do chuẩn hóa hoặc ghi chú giải thích.
- Chính sách ưu tiên không được làm một ticket cũ bị trì hoãn vô hạn nếu không có phê duyệt vận hành phù hợp.

UC-ORD-04 — Sửa hoặc hủy dòng món

Tác nhân chính: Phục vụ, Quản lý

Mục tiêu: Điều chỉnh đơn gọi món khi khách đổi ý, món hết hàng hoặc phát sinh sai sót.

Kích hoạt: Phục vụ mở chi tiết đơn gọi món và chọn dòng món cần sửa/hủy.

Tiền điều kiện

- Order đang còn hiệu lực và chưa thanh toán xong.
- Người thực hiện có quyền sửa/hủy theo trạng thái món.

Hậu điều kiện

- Dòng món được cập nhật số lượng, ghi chú hoặc trạng thái hủy.
- Nếu món đã gửi bếp, hệ thống tạo thông báo bếp và ghi nhật ký kiểm toán.

Luồng chính

1. Phục vụ chọn dòng món cần sửa.
2. Hệ thống hiển thị trạng thái hiện tại của món và mức độ được phép chỉnh sửa.
3. Phục vụ chỉnh số lượng, tùy biến hoặc chọn hủy dòng món.
4. Nếu món đã gửi bếp, hệ thống yêu cầu nhập lý do và có thể cần quản lý phê duyệt.
5. Hệ thống cập nhật đơn gọi món, phát sự kiện cho KDS và tính lại tạm tính.

Luồng thay thế

- Trước khi gửi bếp, phục vụ có thể chỉnh sửa tự do không cần phê duyệt.
- Sau khi món đã chế biến xong, hệ thống chỉ cho phép tạo món bù hoặc hủy có phê duyệt.

Ngoại lệ

- Bếp đã bắt đầu chế biến và vai trò hiện tại không đủ quyền: hệ thống từ chối thao tác.
- Cập nhật đồng thời từ hai thiết bị POS: hệ thống phát hiện xung đột và yêu cầu tải lại đơn gọi món.

Quy tắc nghiệp vụ

- Mọi hủy món sau khi đã gửi bếp phải có lý do và người phê duyệt nếu vượt ngưỡng cấu hình.
- Điều chỉnh đơn gọi món không được làm mất lịch sử giá và trạng thái gốc.

UC-BIL-01 — Tạo hóa đơn cho bàn

Tác nhân chính: Phục vụ, Thu ngân

Mục tiêu: Tổng hợp các đơn gọi món của bàn thành hóa đơn có thể thanh toán.

Kích hoạt: Khách yêu cầu tính tiền hoặc phục vụ chọn ?Tạo hóa đơn?.

Tiền điều kiện

- Phiên bàn đang mở và có ít nhất một đơn gọi món hợp lệ.
- Các món cần tính tiền đã ở trạng thái cho phép xuất hóa đơn.

Hậu điều kiện

- Một hóa đơn được tạo với tổng tiền, phí, giảm giá và trạng thái thanh toán ban đầu.

- Bàn có thể được chuyển sang trạng thái chờ thanh toán.

Luồng chính

1. Phục vụ chọn bàn cần tính tiền.
2. Hệ thống gom toàn bộ đơn gọi món chưa được quyết toán của bàn.
3. Hệ thống tính subtotal, phụ phí, thuế, khuyến mãi và tổng thanh toán.
4. Phục vụ/thu ngân xem trước hóa đơn và xác nhận phát hành.
5. Hệ thống tạo bản ghi Bill và danh sách phân bổ thanh toán mặc định.

Luồng thay thế

- Nhà hàng có thể tạo hóa đơn tạm thời để khách kiểm tra trước khi thanh toán.
- Có thể tạo nhiều hóa đơn con nếu bàn yêu cầu tách theo ghế hoặc theo nhóm món.

Ngoại lệ

- Một đơn gọi món vừa bị hủy hoặc điều chỉnh trong lúc tạo hóa đơn: hệ thống yêu cầu tính lại.
- Chưa có cấu hình thuế hoặc phí cho chi nhánh: hệ thống chặn phát hành hóa đơn.

Quy tắc nghiệp vụ

- Một đơn gọi món không được gắn vào hơn một hóa đơn đang mở cùng lúc.
- Công thức tính phí và thuế phải được chụp lại tại thời điểm tạo hóa đơn.

UC-BIL-02 — Tách hóa đơn và thêm tiền boa

Tác nhân chính: Thu ngân, Phục vụ

Mục tiêu: Chia hóa đơn thành nhiều phần thanh toán và ghi nhận tiền boa.

Kích hoạt: Khách yêu cầu thanh toán riêng theo đầu người, theo món hoặc theo số tiền.

Tiền điều kiện

- Hóa đơn đã được tạo.
- Hóa đơn chưa đóng hoàn toàn.

Hậu điều kiện

- Các phần thanh toán (BillSplit) được tạo và sẵn sàng nhận giao dịch thanh toán.
- Tiền boa được phân bổ theo chính sách nhà hàng.

Luồng chính

1. Thu ngân mở hóa đơn cần tách.
2. Hệ thống hiển thị các món, số ghế, subtotal và các quy tắc chia hóa đơn.
3. Thu ngân chọn kiểu chia: đều, theo món, theo ghế, theo số tiền hoặc kết hợp.
4. Thu ngân nhập tiền boa hoặc chọn mức tiền boa gợi ý.
5. Hệ thống tính lại số tiền của từng phần thanh toán và hiển thị chênh lệch nếu có.
6. Thu ngân xác nhận cấu hình tách hóa đơn.

Luồng thay thế

- Khách có thể chỉ tách một phần món ra thanh toán trước, phần còn lại giữ nguyên hóa đơn gốc.
- Tiền boa có thể áp dụng cho toàn bộ hóa đơn hoặc gắn riêng từng split.

Ngoại lệ

- Tổng các split không khớp tổng hóa đơn: hệ thống từ chối lưu và yêu cầu cân bằng lại.

- Một split đã thanh toán không được phép thay đổi phạm vi món trừ khi hoàn tiền/phê duyệt.

Quy tắc nghiệp vụ

- Tổng các split cộng tiền bo phải khớp với công thức tiền của hóa đơn.
- Tiền bo có thể bị giới hạn tối đa theo chính sách địa phương hoặc hệ thống POS.

UC-PAY-01 — Xử lý thanh toán

Tác nhân chính: Thu ngân

Mục tiêu: Thu tiền cho một hóa đơn hoặc một split bằng tiền mặt, thẻ hoặc ví điện tử.

Kích hoạt: Thu ngân chọn ?Thanh toán? trên hóa đơn hoặc split cụ thể.

Tiền điều kiện

- Hóa đơn/split ở trạng thái chờ thanh toán.
- Cổng thanh toán và cấu hình máy POS khả dụng đối với phương thức không tiền mặt.

Hậu điều kiện

- Một bản ghi **Payment** được tạo với trạng thái thành công, thất bại hoặc chờ xác nhận.
- Nếu đã thanh toán đủ, hóa đơn được đóng.

Luồng chính

1. Thu ngân chọn split cần thanh toán và phương thức thanh toán.
2. Hệ thống hiển thị số tiền phải thu và các bước xác nhận.
3. Nếu thanh toán điện tử, hệ thống gọi **PaymentGateway** để ủy quyền và ghi nhận giao dịch.
4. Nếu thanh toán tiền mặt, thu ngân nhập số tiền khách đưa và hệ thống tính tiền thối.
5. Hệ thống lưu kết quả thanh toán, cập nhật số dư còn lại và trạng thái hóa đơn.
6. Nếu hóa đơn đã được thanh toán đủ, hệ thống đóng hóa đơn và mở bước phát hành biên lai.

Luồng thay thế

- Một hóa đơn có thể được thanh toán bằng nhiều phương thức trong nhiều lần khác nhau.
- Nhà hàng có thể cho phép đặt cọc trước và trừ vào tổng hóa đơn khi check-out.

Ngoại lệ

- Cổng thanh toán trả về thất bại hoặc quá thời gian chờ: hệ thống ghi nhận trạng thái **Failed/Pending Review**.
- Thiết bị thanh toán ngoại vi không phản hồi: hệ thống cho phép chuyển phương thức khác.
- Số tiền thanh toán vượt mức còn lại: hệ thống yêu cầu xác nhận hoặc tự giới hạn theo cấu hình.

Quy tắc nghiệp vụ

- Mọi thanh toán điện tử phải lưu mã tham chiếu giao dịch.
- Hóa đơn chỉ đóng khi tổng thanh toán hợp lệ lớn hơn hoặc bằng tổng phải thu.

UC-PAY-02 — Phát hành biên lai

Tác nhân chính: Thu ngân, Khách hàng

Mục tiêu: Tạo và gửi biên lai sau khi thanh toán thành công.

Kích hoạt: Một hóa đơn hoặc split được thanh toán thành công.

Tiền điều kiện

- Thanh toán đã được ghi nhận hợp lệ.
- Kênh phát hành biên lai (in giấy, email, SMS) đã được cấu hình.

Hậu điều kiện

- Một Receipt được tạo và liên kết với hóa đơn/thanh toán.
- Khách hàng nhận biên lai theo kênh đã chọn.

Luồng chính

1. Hệ thống tạo nội dung biên lai gồm món, giảm giá, phí, phương thức thanh toán và thời gian.
2. Thu ngân chọn in giấy hoặc gửi điện tử.
3. Hệ thống phát hành biên lai và cập nhật trạng thái thành công.
4. Nếu cần, hệ thống gửi bản sao tới email khách hàng hoặc số điện thoại đã lưu.

Luồng thay thế

- Khách có thể yêu cầu xuất biên lai rút gọn hoặc chi tiết theo chính sách nhà hàng.
- Trong trường hợp split bill, mỗi split có thể có biên lai riêng.

Ngoại lệ

- Máy in lỗi hoặc hết giấy: hệ thống gợi ý chuyển sang gửi điện tử hoặc in lại sau.
- Email/SMS không gửi được: hệ thống vẫn lưu biên lai và cho phép gửi lại thủ công.

Quy tắc nghiệp vụ

- Biên lai phải tham chiếu tới ít nhất một thanh toán thành công.
- Việc phát hành lại biên lai không được tạo thêm thanh toán mới.

UC-PAY-03 — Hoàn tiền

Tác nhân chính: Thu ngân, Quản lý

Mục tiêu: Thực hiện hoàn tiền một phần hoặc toàn phần cho giao dịch đã thanh toán.

Kích hoạt: Khách khiếu nại, hủy món sau thanh toán hoặc cần điều chỉnh hóa đơn.

Tiền điều kiện

- Đã tồn tại thanh toán thành công cần hoàn.
- Người thao tác có quyền hoàn tiền hoặc được quản lý phê duyệt.

Hậu điều kiện

- Một bản ghi Refund được tạo và liên kết với giao dịch thanh toán gốc.
- Sổ audit ghi nhận người hoàn, lý do và số tiền hoàn.

Luồng chính

1. Thu ngân chọn giao dịch thanh toán cần hoàn.
2. Hệ thống hiển thị số tiền đã thanh toán, số tiền đã hoàn trước đó và mức hoàn còn khả dụng.
3. Thu ngân nhập số tiền hoàn và lý do.
4. Nếu vượt ngưỡng cho phép, hệ thống yêu cầu quản lý xác nhận.
5. Hệ thống gửi yêu cầu hoàn tiền tới gateway hoặc ghi nhận hoàn tiền mặt.
6. Hệ thống cập nhật trạng thái hoàn tiền, số dư hóa đơn và nhật ký kiểm toán.

Luồng thay thế

- Có thể hoàn tiền một phần cho riêng món bị lỗi mà không đóng lại toàn bộ hóa đơn.

- Quản lý có thể chuyển hoàn tiền thành tín dụng khách hàng nếu nhà hàng áp dụng chính sách ví nội bộ.

Ngoại lệ

- Gateway từ chối hoàn tiền: hệ thống giữ trạng thái **Pending Review** và cảnh báo kế toán.
- Số tiền hoàn vượt mức thanh toán ròng còn lại: hệ thống chặn thao tác.
- Giao dịch thanh toán gốc không cho phép hoàn tiền sau thời hạn quy định: hệ thống yêu cầu xử lý ngoại lệ.

Quy tắc nghiệp vụ

- Mỗi khoản hoàn tiền phải tham chiếu tới giao dịch thanh toán gốc và lý do hoàn tiền chuẩn hóa.
- Refund vượt ngưỡng cấu hình phải có phê duyệt của quản lý.

UC-INV-01 — Cập nhật tiêu hao tồn kho sau phục vụ

Tác nhân chính: Hệ thống, Quản lý kho

Mục tiêu: Khấu trừ tồn kho theo món đã bán hoặc điều chỉnh thủ công khi cần.

Kích hoạt: Dòng món được xác nhận đã phục vụ hoặc quản lý kho tạo giao dịch điều chỉnh.

Tiền điều kiện

- Món ăn đã được ánh xạ công thức nguyên liệu qua **RecipeLine**.
- Tồn kho của chi nhánh đang hoạt động.

Hậu điều kiện

- Các giao dịch kho (**StockTransaction**) được ghi nhận.
- Số lượng tồn (**onHand**) của nguyên liệu được cập nhật.

Luồng chính

1. Khi món được phục vụ, hệ thống tra công thức nguyên liệu tương ứng.
2. Hệ thống tính tổng định mức theo số lượng món đã ra.
3. Hệ thống tạo các giao dịch kho trừ tồn cho từng nguyên liệu.
4. Nếu có điều chỉnh thủ công, quản lý kho nhập số lượng và lý do điều chỉnh.
5. Hệ thống cập nhật tồn kho hiện tại và lịch sử giao dịch.

Luồng thay thế

- Nhà hàng có thể cấu hình khấu trừ khi gửi bếp thay vì khi phục vụ để phản ánh thực tế chế biến.
- Các nguyên liệu hao hụt do prep đầu ca được ghi nhận bằng giao dịch riêng.

Ngoại lệ

- Thiếu ánh xạ công thức: hệ thống không tự trừ tồn và tạo cảnh báo cho quản lý menu/kho.
- Cập nhật đồng thời dẫn đến âm kho bất thường: hệ thống chặn hoặc đưa vào hàng chờ rà soát.

Quy tắc nghiệp vụ

- Mọi điều chỉnh thủ công phải có lý do chuẩn hóa như waste, spoilage, recount, prep.
- Tồn kho và lịch sử giao dịch phải được tách theo chi nhánh.

UC-INV-02 — Nhận cảnh báo sắp hết hàng

Tác nhân chính: Quản lý, Bếp, Hệ thống

Mục tiêu: Phát hiện nguyên liệu xuống dưới ngưỡng và gửi cảnh báo vận hành kịp thời.

Kích hoạt: Tồn kho sau cập nhật nhỏ hơn ngưỡng trong LowStockRule.

Tiền điều kiện

- Ngưỡng cảnh báo tồn kho đã được cấu hình cho từng nguyên liệu và chi nhánh.
- Kênh gửi cảnh báo nội bộ hoạt động.

Hậu điều kiện

- Cảnh báo sắp hết hàng được gửi đến đúng vai trò chịu trách nhiệm.
- Người dùng có thể xem danh sách nguyên liệu cần hoặc khóa bán món liên quan.

Luồng chính

1. Sau mỗi lần cập nhật tồn, hệ thống kiểm tra điều kiện sắp hết hàng.
2. Khi vượt ngưỡng cảnh báo, hệ thống tạo sự kiện và thông báo nội bộ.
3. Quản lý mở danh sách cảnh báo, xem nguyên liệu ảnh hưởng và các món liên quan.
4. Quản lý quyết định đặt hàng, điều chỉnh ngưỡng hoặc tạm ẩn món phụ thuộc.

Luồng thay thế

- Nếu nhà hàng dùng tích hợp nhà cung cấp, hệ thống có thể gợi ý lượng đặt hàng theo lịch sử tiêu thụ.
- Cảnh báo có thể được gom theo ca hoặc gửi ngay lập tức với nguyên liệu trọng yếu.

Ngoại lệ

- Ngưỡng cấu hình thiếu hoặc sai đơn vị: hệ thống ghi cảnh báo cấu hình thay vì gửi cảnh báo vận hành.
- Kênh thông báo lỗi: hệ thống lưu thông báo chờ gửi lại để thực hiện lại sau.

Quy tắc nghiệp vụ

- Một cảnh báo đang mở không được gửi lặp lại quá thường xuyên trong cùng khoảng thời gian chống spam.
- Nguyên liệu trọng yếu có thể kích hoạt tự động ẩn món phụ thuộc theo chính sách chi nhánh.

UC-INV-03 — Phân tích tiêu hao và đề xuất nhập hàng

Tác nhân chính: Quản lý, Quản lý kho

Mục tiêu: Phân tích lịch sử tiêu hao, hao hụt và cảnh báo tồn để đề xuất lượng nhập hàng phù hợp cho kỳ tiếp theo.

Kích hoạt: Quản lý mở màn hình phân tích nhập hàng theo ngày, tuần hoặc kỳ vận hành.

Tiền điều kiện

- Hệ thống đã lưu lịch sử StockTransaction, cảnh báo sắp hết hàng và số liệu tiêu hao theo chi nhánh.
- Quy tắc đặt lại hàng như lead time, mức an toàn hoặc min-max đã được cấu hình ở mức tối thiểu.
- Người dùng có quyền xem dữ liệu tồn kho và báo cáo vận hành.

Hậu điều kiện

- Một tập đề xuất nhập hàng được sinh ra với lượng khuyến nghị và mức độ tin cậy.

- Quản lý có thể lưu snapshot, xuất báo cáo hoặc dùng kết quả để lập kế hoạch đặt hàng.
- Các nguyên liệu rủi ro cao, tiêu hao bất thường hoặc hao hụt lớn được làm nổi bật.

Luồng chính

1. Quản lý chọn kỳ phân tích, chi nhánh và nhóm nguyên liệu cần xem.
2. Hệ thống tải lịch sử tiêu hao, cảnh báo sắp hết hàng, dữ liệu hao hụt và thời gian cung ứng.
3. Hệ thống tính mức tiêu hao bình quân, độ biến động theo ca và xác suất thiếu hàng trong kỳ kế tiếp.
4. Hệ thống sinh lượng tồn mục tiêu, điểm đặt lại hàng và lượng nhập khuyến nghị.
5. Quản lý rà soát các đề xuất, điều chỉnh nếu cần và lưu hoặc xuất báo cáo.

Luồng thay thế

- Nếu dữ liệu lịch sử chưa đủ dày, hệ thống chuyển sang quy tắc min-max và mức an toàn mặc định.
- Quản lý có thể so sánh nhiều kỳ như cuối tuần, ngày lễ hoặc giờ cao điểm để chọn phương án nhập hàng.

Ngoại lệ

- Dữ liệu đơn vị tính không nhất quán hoặc thiếu ánh xạ nguyên liệu: hệ thống đánh dấu cần rà soát thủ công.
- Lịch sử giao dịch quá ít: hệ thống vẫn cho phép tạo đề xuất nhưng phải gắn cảnh báo mức tin cậy thấp.

Quy tắc nghiệp vụ

- Đề xuất nhập hàng là khuyến nghị ra quyết định, không tự động sinh giao dịch mua hàng.
- Việc tính lượng nhập phải xét theo từng chi nhánh vì nhu cầu, lead time và mức hao hụt có thể khác nhau.

UC-REP-01 — Xem báo cáo doanh thu và vận hành

Tác nhân chính: Quản lý

Mục tiêu: Theo dõi doanh thu, giờ cao điểm, món bán chạy, độ trễ bếp và hiệu suất phục vụ.

Kích hoạt: Quản lý mở bảng điều khiển phân tích hoặc yêu cầu xuất báo cáo.

Tiền điều kiện

- Dữ liệu giao dịch, gọi món, bếp và tồn kho đã được tổng hợp vào mô hình đọc báo cáo.
- Người dùng có quyền xem báo cáo.

Hậu điều kiện

- Quản lý có được ảnh chụp báo cáo hiện tại hoặc tệp xuất PDF/CSV.
- Các chỉ số dùng cho hợp vận hành được chuẩn hóa.

Luồng chính

1. Quản lý chọn khoảng thời gian, chi nhánh và bộ lọc báo cáo.
2. Hệ thống truy xuất **ReportSnapshot** hoặc tính toán số liệu mới nhất.
3. Hệ thống hiển thị doanh thu, số bàn quay vòng, món bán chạy, phiếu bếp chậm, tỷ lệ hủy món và hiệu suất nhân viên.
4. Quản lý phân tích biểu đồ, đi sâu theo ca, trạm bếp hoặc khu vực.
5. Nếu cần, quản lý chọn xuất báo cáo sang PDF/CSV.

Luồng thay thế

- Bảng điều khiển có thể hiển thị cảnh báo bất thường như thời gian ra món tăng mạnh hoặc hoàn tiền tăng đột biến.
- Quản lý có thể lưu bộ lọc ưa thích để dùng lại.

Ngoại lệ

- Read model báo cáo chưa được làm mới: hệ thống hiển thị thời điểm đồng bộ cuối cùng.
- Khối lượng dữ liệu quá lớn: hệ thống chuyển truy vấn sang chế độ bất đồng bộ và thông báo khi hoàn tất.

Quy tắc nghiệp vụ

- Báo cáo phân tích dùng dữ liệu tổng hợp, không truy cập trực tiếp vào luồng ghi giao dịch thời gian thực.
- Mỗi lần xuất báo cáo phải lưu dấu vết ai xuất và thời điểm xuất.

UC-ADM-01 — Quản lý menu và giá bán

Tác nhân chính: Quản lý

Mục tiêu: Tạo, cập nhật, tạm ẩn món, thay đổi giá và chương trình khuyến mãi.

Kích hoạt: Quản lý mở màn hình cấu hình menu.

Tiền điều kiện

- Người dùng có quyền quản trị menu.
- Chi nhánh mục tiêu đang được cấu hình và có danh mục món cơ sở.

Hậu điều kiện

- Thực đơn mới hoặc thay đổi giá được lưu và có thể công bố theo lịch hiệu lực.
- Những món liên quan đến đơn gọi món đang mở vẫn giữ giá đã được chụp lại trước đó.

Luồng chính

1. Quản lý chọn danh mục món hoặc món cụ thể.
2. Hệ thống hiển thị thông tin giá, tình trạng bán, modifier và món liên quan.
3. Quản lý chỉnh sửa tên, giá, tình trạng bán, hình thức khuyến mãi hoặc thuộc tính món.
4. Quản lý lưu thay đổi và chọn công bố ngay hoặc theo lịch.
5. Hệ thống kiểm tra tác động tới POS/KDS và phát hành bản menu mới.

Luồng thay thế

- Có thể công bố theo ca sáng, ca tối hoặc theo ngày lễ.
- Khuyến mãi có thể áp dụng theo món, danh mục hoặc mức hóa đơn.

Ngoại lệ

- Dữ liệu cấu hình thiếu bắt buộc, ví dụ chưa gán trạm bếp hoặc thiếu thuế suất: hệ thống không cho công bố.
- Xung đột lịch hiệu lực giữa nhiều chương trình giá: hệ thống yêu cầu người dùng giải quyết.

Quy tắc nghiệp vụ

- Không được thay đổi trực tiếp giá của đơn gọi món đã xác nhận.
- Mọi lần công bố thực đơn phải được ghi vào nhật ký kiểm toán và có phiên bản.

UC-ADM-02 — Quản lý vai trò và phân quyền

Tác nhân chính: Quản trị viên, Quản lý

Mục tiêu: Gán vai trò và quyền thao tác cho lễ tân, phục vụ, bếp, thu ngân và quản lý.

Kích hoạt: Quản trị viên mở màn hình quản lý người dùng.

Tiền điều kiện

- Tài khoản người dùng đã tồn tại hoặc đang được tạo mới.
- Chính sách RBAC và danh mục quyền đã được định nghĩa.

Hậu điều kiện

- Tài khoản người dùng được gán đúng vai trò/quyền theo chi nhánh.
- Các thao tác nhạy cảm bị kiểm soát bởi `AuthorizationPolicy`.

Luồng chính

1. Quản trị viên tìm người dùng cần cập nhật.
2. Hệ thống hiển thị vai trò hiện tại, chi nhánh và phạm vi quyền.
3. Quản trị viên thêm/bỏ vai trò hoặc gán quyền theo chính sách.
4. Hệ thống lưu thay đổi và yêu cầu người dùng đăng nhập lại nếu cần.
5. Audit log ghi nhận toàn bộ thay đổi quyền.

Luồng thay thế

- Có thể gán vai trò tạm thời cho ca làm việc hoặc theo khoảng thời gian hiệu lực.
- Nhà hàng nhiều chi nhánh có thể áp dụng vai trò khác nhau theo từng địa điểm.

Ngoại lệ

- Người dùng đang tự gỡ quyền quản trị cuối cùng của hệ thống: thao tác bị chặn.
- Vai trò xung đột với chính sách hiện hành: hệ thống yêu cầu xác nhận hoặc điều chỉnh.

Quy tắc nghiệp vụ

- Nguyên tắc tối thiểu quyền được áp dụng mặc định.
- Hoàn tiền, ghi nợ giá và điều chỉnh tồn kho là các quyền nhạy cảm phải được kiểm soát riêng.

UC-ADM-03 — Xem nhật ký kiểm toán và truy vết thao tác nhạy cảm

Tác nhân chính: Quản lý, Quản trị viên

Mục tiêu: Tra cứu, đối chiếu và truy vết các hành động nhạy cảm như hoàn tiền, hủy món, ghi nợ giá, đổi quyền hoặc mở lại hóa đơn.

Kích hoạt: Người có thẩm quyền mở màn hình kiểm toán để điều tra sự cố, rà soát cuối ca hoặc phục vụ kiểm tra nội bộ.

Tiền điều kiện

- Hệ thống đang ghi nhận `AuditLog` cho các hành động nhạy cảm.
- Người dùng có quyền xem nhật ký kiểm toán ở phạm vi chi nhánh hoặc toàn hệ thống theo vai trò.
- Dữ liệu correlation id, thời gian và đối tượng bị tác động có thể truy xuất được.

Hậu điều kiện

- Người dùng xem được chuỗi hành động liên quan đến một giao dịch hoặc một người thao tác.
- Hệ thống lưu lại dấu vết việc xem hoặc xuất audit log nếu chính sách yêu cầu.

- Có thể tạo báo cáo hoặc gắn cờ theo dõi cho bản ghi cần điều tra thêm.

Luồng chính

1. Người dùng mở màn hình nhật ký kiểm toán.
2. Hệ thống kiểm tra quyền truy cập vào dữ liệu kiểm toán nhạy cảm.
3. Người dùng nhập bộ lọc như khoảng thời gian, người thao tác, thực thể, loại hành động hoặc mã liên kết.
4. Hệ thống truy vấn các bản ghi audit bất biến và hiển thị danh sách kết quả phù hợp.
5. Người dùng mở chi tiết một bản ghi để xem giá trị trước, giá trị sau, lý do, correlation id và ngữ cảnh liên quan.
6. Nếu cần, người dùng xuất báo cáo hoặc đánh dấu follow-up để tiếp tục điều tra.

Luồng thay thế

- Người dùng có thể đi từ một hóa đơn, payment hoặc order cụ thể sang chuỗi audit liên quan.
- Các trường nhạy cảm như thông tin liên hệ hoặc dữ liệu thanh toán tham chiếu có thể được làm mờ theo quyền.

Ngoại lệ

- Người dùng không đủ quyền xem chi tiết audit log: hệ thống từ chối truy cập hoặc chỉ cho xem bản rút gọn.
- Không có bản ghi phù hợp với bộ lọc: hệ thống thông báo không có kết quả và cho phép điều chỉnh bộ lọc.
- Kho audit tạm thời chậm hoặc không sẵn sàng: hệ thống chuyển sang chế độ chỉ đọc hạn chế và cảnh báo quản trị.

Quy tắc nghiệp vụ

- Bản ghi kiểm toán đã phát sinh không được phép sửa nội dung theo cách làm mất dấu vết gốc.
- Việc xem hoặc xuất audit log mức nhạy cảm cao cũng phải được ghi nhận vào audit log.

UC-OPS-01 — Quản lý ca làm nhân viên

Tác nhân chính: Quản lý

Mục tiêu: Phân công nhân sự cho từng chi nhánh, khu vực và ca làm việc.

Kích hoạt: Quản lý mở lịch làm việc và tạo/cập nhật ca.

Tiền điều kiện

- Tài khoản nhân viên đã tồn tại.
- Mẫu ca làm và cấu hình khu vực phục vụ đã được tạo.

Hậu điều kiện

- Một hoặc nhiều **ShiftAssignment** được tạo hoặc cập nhật.
- Nhân viên nhìn thấy ca của mình trong lịch làm việc.

Luồng chính

1. Quản lý chọn ngày, chi nhánh và loại ca.
2. Hệ thống hiển thị danh sách nhân viên khả dụng cùng kỹ năng/vai trò.
3. Quản lý gán nhân viên cho ca và khu vực phụ trách.
4. Hệ thống kiểm tra trùng ca, thiếu vị trí bắt buộc và giới hạn giờ làm.

5. Quản lý lưu lịch; hệ thống gửi thông báo cho nhân viên.

Luồng thay thế

- Quản lý có thể sao chép lịch từ tuần trước rồi tinh chỉnh.
- Ca thiếu người có thể được đánh dấu chờ xác nhận hoặc mở đăng ký tự nguyện.

Ngoại lệ

- Nhân viên bị phân ca trùng thời gian hoặc vượt giới hạn giờ làm: hệ thống chặn lưu.
- Người dùng không thuộc chi nhánh được chọn: hệ thống yêu cầu thay đổi phạm vi gán.

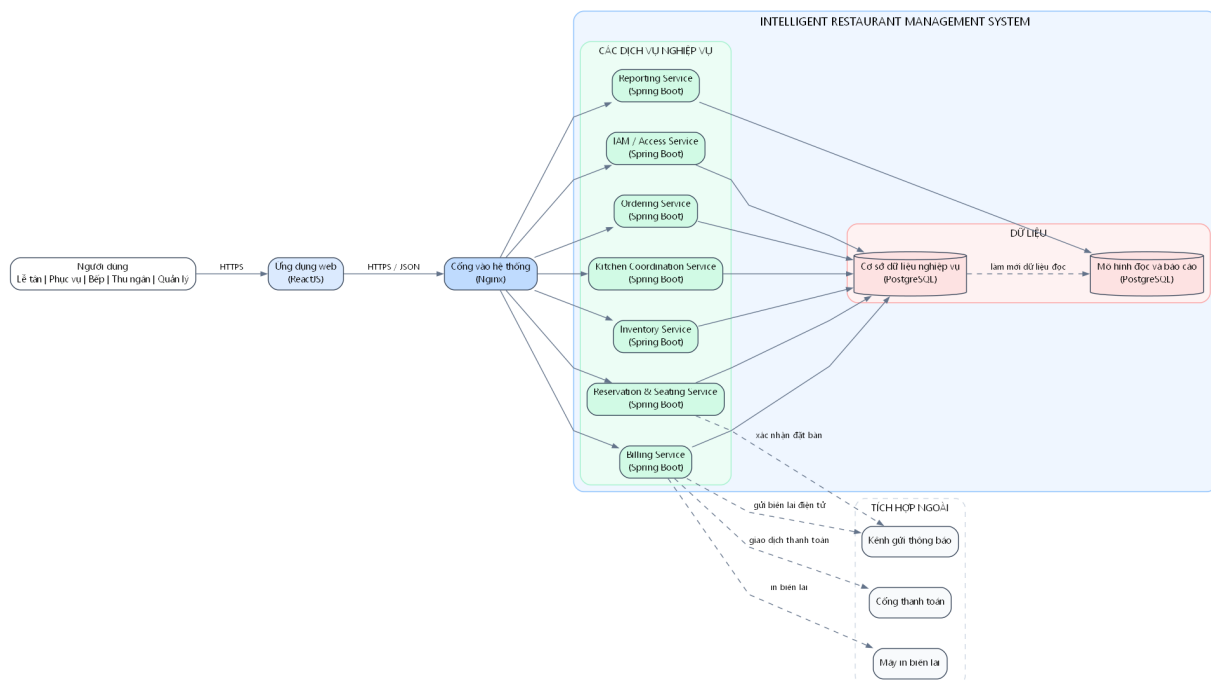
Quy tắc nghiệp vụ

- Một nhân viên không được có hai ca chồng lấn ở cùng thời điểm.
- Ca bếp, ca thu ngân và ca quản lý phải đáp ứng số lượng tối thiểu theo chính sách vận hành.

3 Kiến trúc phần mềm

3.1 Tổng quan kiến trúc

Hệ thống IRMS được tổ chức theo kiến trúc dịch vụ bám theo miền nghiệp vụ của nhà hàng. Ở phía ngoài, người dùng thao tác thông qua một ứng dụng web thống nhất được xây dựng bằng ReactJS. Mọi yêu cầu từ giao diện đều đi qua Nginx trước khi được định tuyến vào các dịch vụ nghiệp vụ ở phía sau. Các dịch vụ này được phát triển bằng Spring Boot và tách theo các vùng chức năng chính như đặt bàn và xếp bàn, gọi món, điều phối bếp, thanh toán, tồn kho, báo cáo và kiểm soát truy cập. Dữ liệu vận hành và dữ liệu đọc báo cáo được lưu trên PostgreSQL; các tích hợp như cổng thanh toán, kênh gửi thông báo và máy in biên lai được đặt ở rìa hệ thống.



Hình 8: Sơ đồ tổng quan kiến trúc của IRMS

Hình 8 là sơ đồ nền tảng để đọc toàn bộ chương kiến trúc. Hình này không chỉ trả lời câu hỏi ?IRMS có những khối nào?, mà còn trả lời ba câu hỏi quan trọng hơn: yêu cầu đi vào hệ thống từ

đâu, quyết định nghiệp vụ được giữ ở tầng nào, và đâu là ranh giới giữa phần lõi có thể kiểm soát với phần tích hợp ngoài cần cách ly rủi ro.

Ở lớp ngoài cùng, **ReactJS** đóng vai trò mặt trước thống nhất cho lễ tân, phục vụ, bếp, thu ngân và quản lý. Việc dùng một nền giao diện thống nhất không có nghĩa mọi vai trò dùng cùng một màn hình, mà có nghĩa mọi vai trò đi qua cùng một cơ chế xác thực, cùng quy tắc điều hướng và cùng cách đồng bộ trạng thái quan trọng như bàn, đơn gọi món, phiếu bếp và hóa đơn. Đây là quyết định quan trọng vì **IRMS** là hệ thống có nhiều điểm chạm nghiệp vụ nhưng cần một cảm giác vận hành liền mạch. Nếu mỗi vai trò bị tách thành những ứng dụng rời rạc ngay từ đầu, chi phí đồng bộ trạng thái, quản lý phiên làm việc và kiểm soát truy cập sẽ tăng rất nhanh.

Ở lớp cổng vào, **Nginx** được đặt làm điểm tiếp nhận thống nhất. Vai trò của nó không chỉ là chuyển tiếp yêu cầu. Về kiến trúc, đây là nơi phù hợp để đặt các kiểm tra chung như xác thực sơ bộ, giới hạn lưu lượng, ghi nhật ký ở biên, chuẩn hóa tiêu đề yêu cầu và tách ứng dụng khách khỏi cấu trúc bên trong của hệ thống. Nhờ có lớp này, giao diện người dùng không cần biết trực tiếp từng mô-đun phía sau đang chạy ra sao, ở đâu, hay được tách thành bao nhiêu phần. Điều đó giúp cấu trúc bên trong được thay đổi dần theo thời gian mà không làm gãy các điểm chạm ở tuyến đầu.

Phần quan trọng nhất của hình là lớp dịch vụ xử lý phía sau. Ở đây, **IRMS** được tổ chức theo ranh giới nghiệp vụ thay vì theo tầng kỹ thuật thuần túy. **Reservation & Seating Service** giữ bối cảnh đặt bàn, danh sách chờ và phiên bàn; **Ordering Service** giữ dữ liệu gọi món và ảnh chụp giá trị tại thời điểm xác nhận; **Kitchen Coordination Service** quản lý việc chuyển từ đơn gọi món sang thực thi tại bếp; **Billing Service** chịu trách nhiệm cho toàn bộ chuỗi lập hóa đơn, tách hóa đơn, thanh toán và hoàn tiền; **Inventory Service** xử lý mức tồn, quy tắc cảnh báo và lịch sử biến động; **Reporting Service** xây dựng phía đọc phục vụ quản trị; còn **IAM / Access Service** bảo vệ biên truy cập và các quyết định nhảy cảm. Cách chia này làm rõ một nguyên tắc quan trọng: phần mềm phải phản ánh cách nhà hàng vận hành ngoài đời, không phản ánh cách nhóm lập trình thuận tay chia thư mục mã nguồn.

Lớp dữ liệu trong hình cũng cần được đọc kỹ. **PostgreSQL** không chỉ là nơi lưu bảng dữ liệu, mà là nơi neo giữ tính nhất quán của những quyết định có hậu quả nghiệp vụ cao như xác nhận đơn gọi món, đổi trạng thái bàn, đóng hóa đơn hay ghi nhận hoàn tiền. Đồng thời, sơ đồ cũng chủ ý cho thấy có phần dữ liệu đọc phục vụ báo cáo, thay vì bắt tuyến quản trị truy vấn trực tiếp lên dữ liệu giao dịch nóng. Đây là một quyết định cân bằng rất thực tế: giao diện phục vụ tại bàn, màn hình bếp và giao diện thu ngân phải phản hồi trước; báo cáo chi tiết có thể đi sau vài giây hoặc vài phút miễn là đúng và truy vết được.

Cuối cùng, các tích hợp ngoài như cổng thanh toán, kênh gửi thông báo và máy in biên lai được đặt ở rìa hệ thống. Đây là chi tiết kiến trúc rất quan trọng. Những thành phần này có thể chậm, lỗi, thay nhà cung cấp hoặc thay giao thức tích hợp. Nếu chúng chen trực tiếp vào miền nghiệp vụ, mọi thay đổi ngoài hệ thống sẽ lan ngược vào lõi. Khi bị đẩy ra rìa và đi qua các bộ kết nối trừu tượng, **IRMS** có thể giữ ổn định phần trung tâm ngay cả khi phải thay đổi cách gửi thông báo, đổi thiết bị in hoặc đổi đối tác thanh toán.

1. **Lớp giao diện người dùng (ReactJS).** Đây là nơi tối ưu cho thao tác nhanh, ít bước và nhất quán giữa nhiều vai trò vận hành. Kiến trúc lựa chọn một nền giao diện chung để giảm chi phí học hệ thống, giữ một mô hình điều hướng thống nhất và đơn giản hóa quản lý phiên làm việc theo ca.
2. **Cổng vào giao diện lập trình ứng dụng (Nginx).** Thành phần này tạo một biên truy cập rõ ràng, là nơi phù hợp để gom các kiểm tra chung trước khi yêu cầu đi sâu vào lõi nghiệp vụ. Nó cũng giúp che giấu cấu trúc bên trong khỏi ứng dụng khách.
3. **Các dịch vụ xử lý phía sau (Spring Boot).** Mỗi dịch vụ giữ một ranh giới trách nhiệm rõ, nhờ đó thay đổi ở vùng gọi món không tự động làm vỡ vùng thanh toán, còn thay đổi ở vùng tồn kho không ép vùng đặt bàn phải sửa theo.
4. **Kho dữ liệu (PostgreSQL).** Dữ liệu nghiệp vụ cốt lõi và dữ liệu đọc phục vụ báo cáo được lưu trên **PostgreSQL**. Việc tách dữ liệu giao dịch khỏi dữ liệu đọc giúp truy vấn quản trị không làm nặng tuyến giao dịch nóng như gọi món, điều phối bếp và thanh toán.

5. **Các tích hợp ngoài.** Những điểm phụ thuộc đối tác được đặt ở rìa kiến trúc để dễ thay thế, dễ cô lập lỗi và không làm ô nhiễm mô hình miền nghiệp vụ.

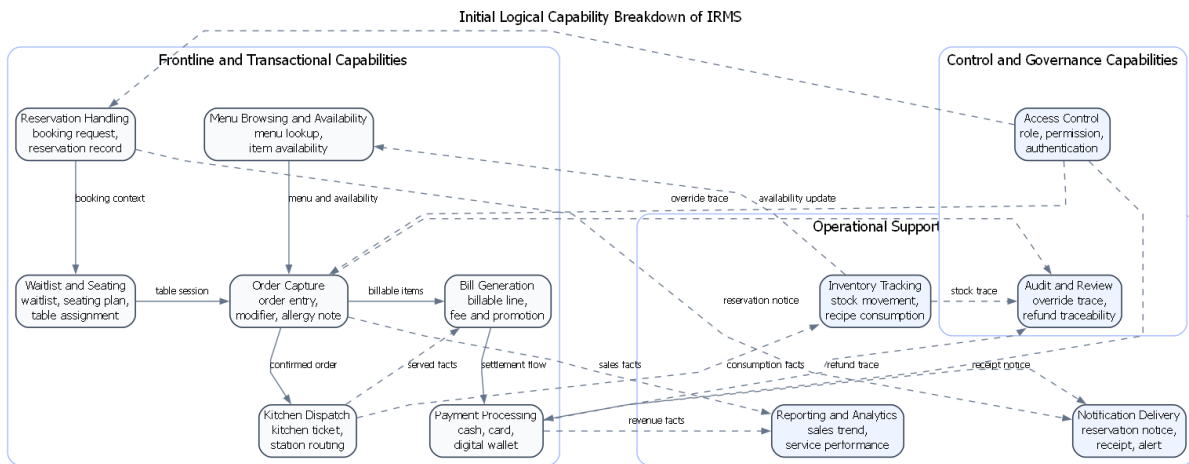
Nhìn tổng thể, sơ đồ này cho thấy IRMS đi theo một tư duy kiến trúc rất thực dụng: giữ tuyến đầu gọn, giữ lõi nghiệp vụ có ranh giới rõ, và đẩy phần dễ biến động ra ngoài biên. Chính vì vậy, các góc nhìn logic, mô-đun, thành phần và triển khai ở các mục sau không phải là những bức tranh rời rạc, mà là những lát cắt khác nhau của cùng một quyết định kiến trúc.

3.2 Góc nhìn logic

Ở góc nhìn logic, trọng tâm không còn là đường đi từ người dùng vào hệ thống, mà là cách IRMS tách các năng lực nghiệp vụ thành những miền trách nhiệm rõ ràng. Trong báo cáo này, góc nhìn logic được dùng theo hướng **capability and responsibility mapping**: hệ thống được đọc như một tập các năng lực nghiệp vụ, mỗi năng lực giữ một vùng quyết định riêng, có mối liên hệ rõ với các năng lực còn lại nhưng không bị trộn vào cùng một khối xử lý mơ hồ. Cách nhìn đó phù hợp với mục tiêu của báo cáo kiến trúc vì nó giúp kiểm tra xem kiến trúc có thật sự phản ánh vận hành nhà hàng hay chỉ mới phản ánh danh sách chức năng.

Hai sơ đồ dưới đây lần lượt thể hiện bước phân rã logic ban đầu và bước gom các năng lực thành ranh giới kiến trúc ổn định hơn để dùng nhất quán ở các phần sau.

Sơ đồ tổng quan



Hình 9: Sơ đồ thành phần logic ban đầu của IRMS

Hình 9 phản ánh bước tách năng lực nghiệp vụ ban đầu. Ở đây, hệ thống được nhìn như một tập hợp các năng lực chức năng còn khá gần với thao tác người dùng: đặt bàn và xếp bàn, duyệt thực đơn, tiếp nhận đơn gọi món, điều phối bếp, lập hóa đơn, xử lý thanh toán, theo dõi tồn kho, báo cáo, kiểm soát truy cập, rà soát kiểm toán và gửi thông báo. Cách nhìn này có ích ở giai đoạn đầu vì nó bám sát đầu bài và tác nhân, giúp nhóm tránh bỏ sót các chức năng bắt buộc.

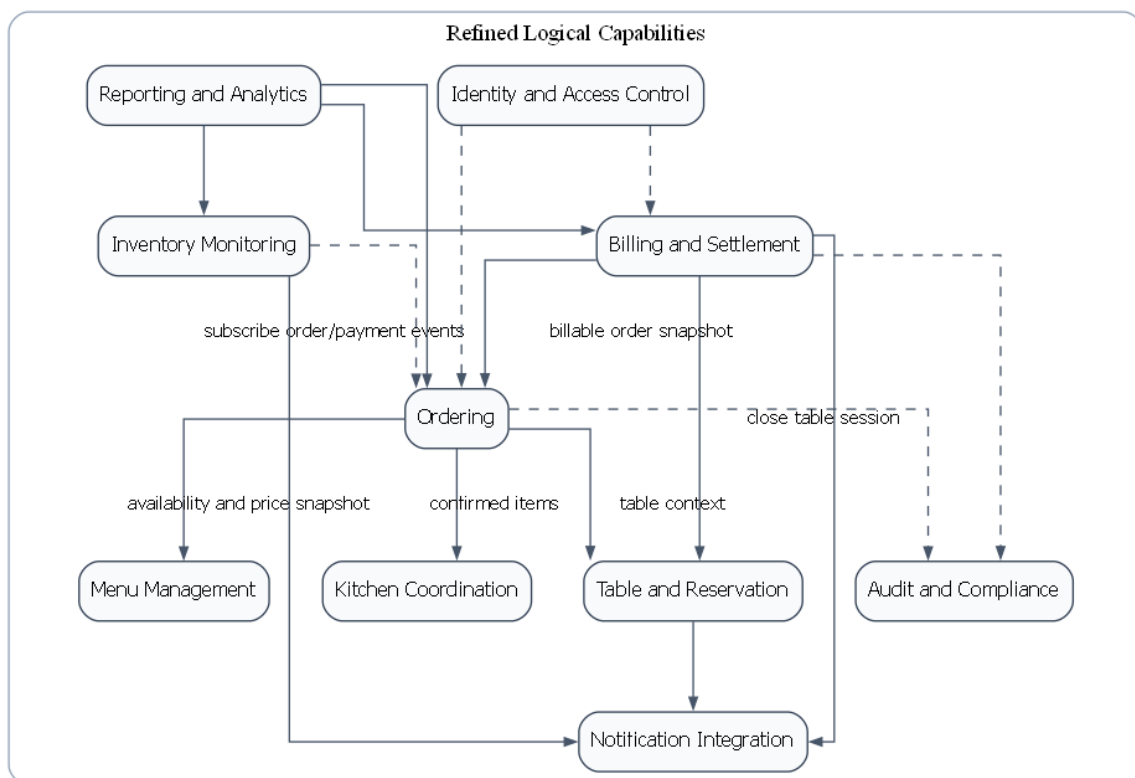
Tuy nhiên, nếu dừng ở mức này thì kiến trúc sẽ còn khá *thiên về chức năng bề mặt*. Ví dụ, Bill Generation và Payment Processing thực ra nằm trong cùng một vùng quyết toán nghiệp vụ; Access Control và Audit Review cũng chưa được phân biệt rõ giữa chính sách kiểm soát và dữ liệu kiểm toán. Vì thế, sơ đồ này nên được hiểu là bước khám phá năng lực nghiệp vụ, không phải phương án phân rã cuối cùng.

Các thành phần chính thể hiện trong hình.

- **Nhóm tuyến đầu:** gồm các năng lực gần trực tiếp với thao tác người dùng như đặt bàn, gọi món, bếp và thanh toán.
- **Nhóm hỗ trợ vận hành:** gồm tồn kho, báo cáo và gửi thông báo.
- **Nhóm kiểm soát hệ thống:** gồm kiểm soát truy cập và rà soát kiểm toán.

Lý do thiết kế của sơ đồ này.

- Sơ đồ ban đầu giúp quan sát xuất phát điểm theo chức năng bề mặt trước khi hệ thống được gom lại theo các ranh giới kiến trúc chặt chẽ hơn.
- Việc đặt các năng lực gần nhau trong cùng một hình giúp người đọc thấy xuất phát điểm thiết kế trước khi báo cáo khóa lại thành các miền nghiệp vụ rõ ràng hơn.



Hình 10: Sơ đồ phân rã năng lực logic của IRMS

Hình 10 khóa lại phương án phân rã logic được sử dụng xuyên suốt báo cáo. Ở mức này, các năng lực nghiệp vụ được gom thành những ranh giới phù hợp hơn cho kiến trúc: **Table and Reservation**, **Menu Management**, **Ordering**, **Kitchen Coordination**, **Billing and Settlement**, **Inventory Monitoring**, **Reporting and Analytics**, **Identity and Access Control**, **Audit and Compliance**, và **Notification Integration**. Đây là điểm rất quan trọng vì nó cho thấy kiến trúc không còn chỉ phản ánh thao tác giao diện, mà đã chuyển sang phản ánh **các ranh giới năng lực nghiệp vụ**.

Về mặt kiến trúc, cách phân rã này giúp ba việc. Thứ nhất, nó tăng **tính kết tụ**: mỗi năng lực nghiệp vụ gom những quy tắc gần nhau vào cùng một ranh giới. Thứ hai, nó giảm **mức độ phụ thuộc**: tồn kho, báo cáo và gửi thông báo không bị kéo vào cùng tuyến giao dịch nóng của gọi món. Thứ ba, nó tạo nền cho việc chọn kiểu kết nối phù hợp: **Ordering** và **Billing and**

Settlement cần đồng bộ mạnh hơn, trong khi Inventory Monitoring, Reporting and Analytics và Notification Integration có thể đi qua sự kiện hoặc bộ xử lý nền.

Các thành phần chính thể hiện trong hình.

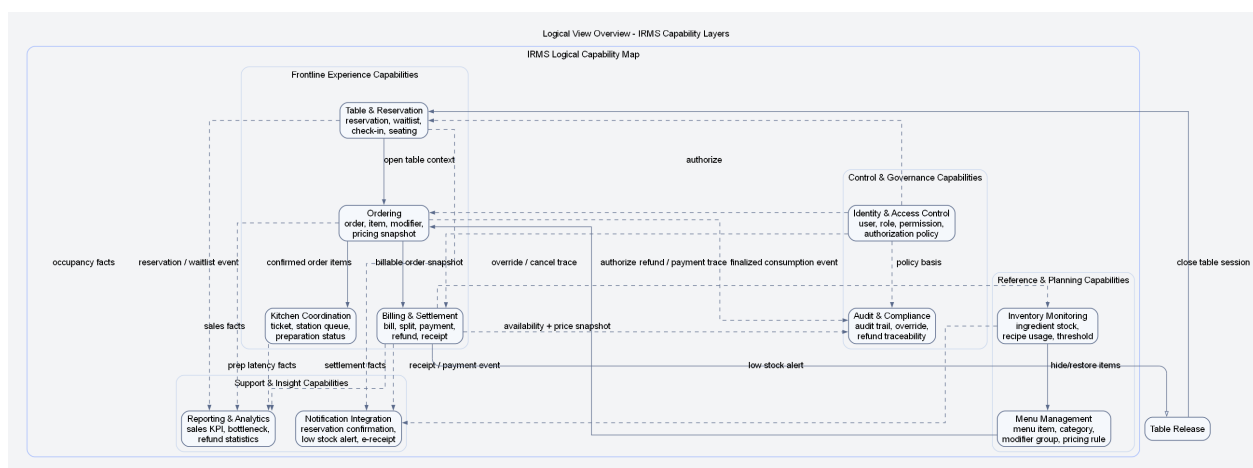
- Các năng lực nghiệp vụ lõi: Table and Reservation, Menu Management, Ordering, Kitchen Coordination và Billing and Settlement.
- Các năng lực hỗ trợ: Inventory Monitoring, Reporting and Analytics và Notification Integration.
- Các năng lực kiểm soát: Identity and Access Control cùng Audit and Compliance.

Lý do thiết kế của sơ đồ này.

- Đây là sơ đồ tổng quan quan trọng nhất của góc nhìn logic vì nó khóa ranh giới miền nghiệp vụ trước khi đi sang mô-đun và thành phần thực thi.
- Việc làm rõ các nhóm lõi, hỗ trợ và kiểm soát giúp những phần sau của báo cáo giải thích được vì sao có luồng đồng bộ và luồng bất đồng bộ trong kiến trúc.

3.3 Tổng quan cho các góc nhìn chính

Tổng quan cho góc nhìn logic



Hình 11: Sơ đồ tổng quan cho góc nhìn logic của IRMS

Hình 11 làm rõ tầng khái niệm của góc nhìn logic, tức là trả lời câu hỏi ?trong bức tranh nghiệp vụ tổng thể, các năng lực của IRMS được nhóm thành những lớp nào và quan hệ chính giữa các lớp đó là gì?. Khác với hình logic ở mức liệt kê năng lực riêng lẻ, hình này nhấn mạnh **bốn tầng năng lực**: tuyến phục vụ trực tiếp với khách, nhóm tham chiếu và hoạch định, nhóm kiểm soát và nhóm hỗ trợ phân tích. Cách nhìn đó giúp phần logic trở thành một bản đồ năng lực có cấu trúc, thay vì một danh sách chức năng rời rạc.

Các nhóm năng lực chính thể hiện trong hình.

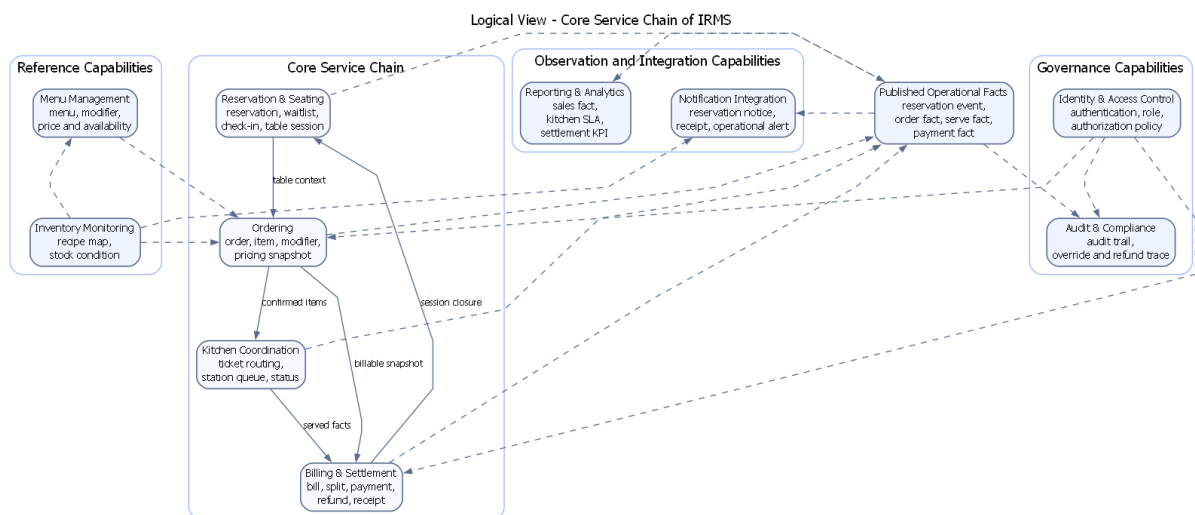
- Nhóm năng lực tuyến đầu: gồm Table & Reservation, Ordering, Kitchen Coordination và Billing & Settlement; đây là tuyến giao dịch nóng, nơi mọi thao tác phải phản hồi nhanh và nhất quán.
- Nhóm năng lực tham chiếu và hoạch định: gồm Menu Management và Inventory Monitoring; hai năng lực này cung cấp dữ liệu điều kiện và quy tắc cho tuyến vận hành, nhưng không nên làm nghẽn tuyến phục vụ.

- **Nhóm năng lực kiểm soát và quản trị:** gồm Identity & Access Control và Audit & Compliance; đây là lớp bảo vệ và truy vết cho những hành động nhạy cảm.
- **Nhóm năng lực hỗ trợ và phân tích:** gồm Notification Integration và Reporting & Analytics; đây là lớp mở rộng giá trị vận hành, thiên về xử lý bất đồng bộ hoặc tổng hợp dữ liệu.

Ý nghĩa thiết kế của sơ đồ tổng quan này.

- Nó khóa lại ranh giới logic ở mức **nhóm năng lực**, giúp những phần sau của báo cáo giải thích được vì sao có luồng đồng bộ mạnh ở tuyến đầu nhưng lại có luồng dựa trên sự kiện cho cảnh báo, báo cáo và gửi thông báo.
- Nó cho thấy rõ **chiều phụ thuộc đúng**: thực đơn và tồn kho phục vụ gọi món; gọi món phát sinh dữ liệu cho bếp và quyết toán; thanh toán hoàn tất mới kéo theo cập nhật tồn kho cuối cùng, thông báo và báo cáo.
- Nó tạo nền cho góc nhìn phân bổ vì mỗi năng lực sau đó sẽ được ánh xạ sang vùng thực thi, nơi lưu trữ và bộ kết nối phù hợp thay vì ánh xạ ngẫu nhiên theo giao diện người dùng.

Góc nhìn logic cho chuỗi phục vụ lõi



Hình 12: Góc nhìn logic cho chuỗi phục vụ lõi của IRMS

Hình 12 đi từ bản đồ năng lực sang chuỗi năng lực lõi, tức phần quyết định trực tiếp trải nghiệm phục vụ của khách hàng từ lúc khách được nhận bàn cho tới khi bàn được giải phóng sau thanh toán. Trong góc nhìn logic, chuỗi này gồm bốn miền chính: **Reservation & Seating** mở bối cảnh phục vụ, **Ordering** giữ dữ liệu giao dịch của món, **Kitchen Coordination** biến dữ liệu bán hàng thành công việc chế biến, và **Billing & Settlement** chốt nghĩa vụ tài chính cũng như đóng vòng đời phiên bàn. Sơ đồ giúp người đọc thấy rõ hệ thống không vận hành như một chuỗi màn hình, mà như một chuỗi năng lực nghiệp vụ có đầu vào, đầu ra và trách nhiệm riêng.

Điểm quan trọng nhất của hình là **mỗi miền logic chỉ công bố loại thông tin cần thiết cho miền kế tiếp**. **Reservation & Seating** không truyền toàn bộ chi tiết đặt bàn sang gọi món; nó chỉ truyền bối cảnh phiên bàn hợp lệ. **Ordering** không giao thẳng các cấu trúc giao diện sang bếp; nó truyền ảnh chụp đơn gọi món đã xác nhận và có khả năng định tuyến. **Kitchen Coordination** không quyết toán thay cho thu ngân; nó chỉ phát hành trạng thái thực thi như **Ready**, **Blocked** hoặc **Served**. **Billing & Settlement** sau đó sử dụng ảnh chụp giao dịch đủ tin cậy để tính tiền,

thu tiền và khép lại phiên bàn. Nhờ giới hạn như vậy, chuỗi nghiệp vụ giữ được độ kết tụ cao mà không làm các miền trở nên phụ thuộc quá mức.

Một chi tiết quan trọng khác là nút **Published Operational Facts** ở phần dưới của hình. Nút này không phải một thành phần triển khai, mà là cách biểu diễn ở mức logic cho nhóm dữ kiện được phát ra sau khi các quyết định lõi đã được chốt, ví dụ sự kiện đặt bàn, ảnh chụp đơn gọi món, trạng thái phục vụ món hay kết quả quyết toán. Cách biểu diễn này giúp tách rõ hai loại quan hệ: quan hệ trên **trục phục vụ chính** và quan hệ trên **trục quan sát, tích hợp, truy vết**. Nhờ đó, sơ đồ vừa giữ được chuỗi lõi dễ đọc, vừa vẫn thể hiện được vì sao báo cáo, thông báo và kiểm toán có thể bám theo giao dịch mà không chen vào quyết định lõi.

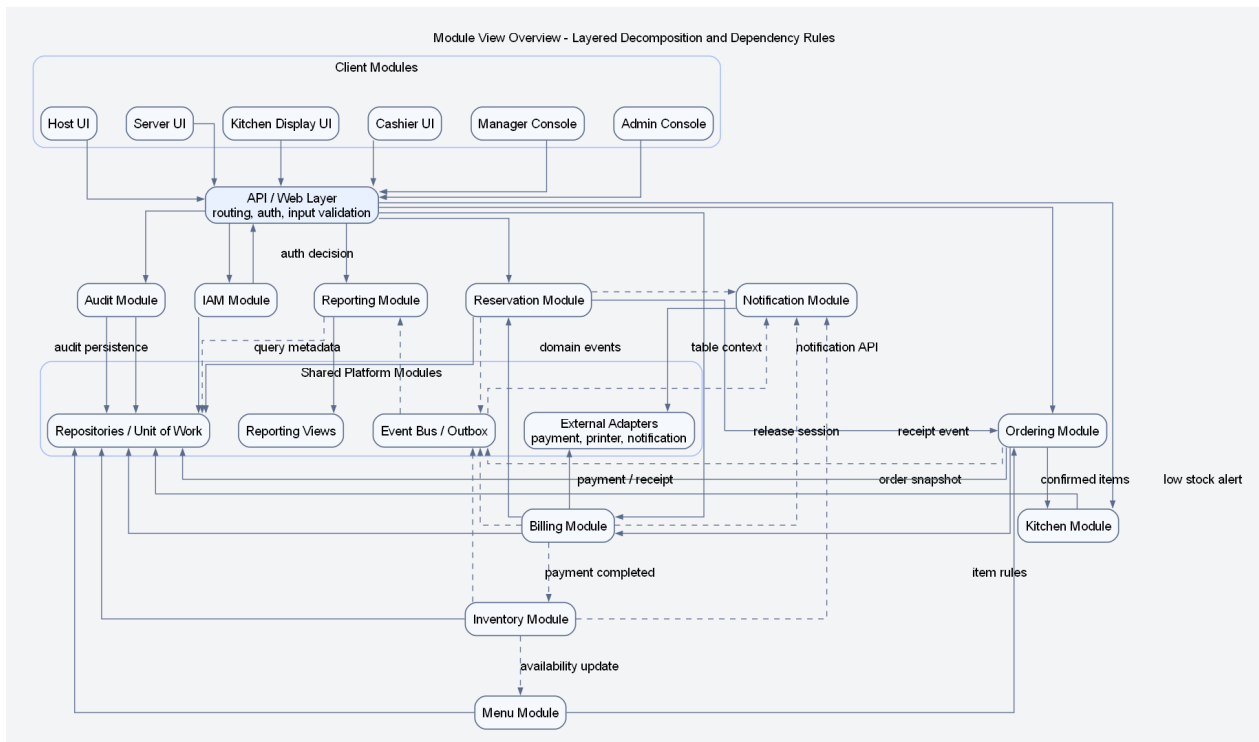
Sơ đồ cũng cho thấy vị trí của các miền tham chiếu và kiểm soát xung quanh chuỗi lõi. **Menu Management** và **Inventory Monitoring** không đứng trên trục phục vụ chính, nhưng chúng cung cấp điều kiện vận hành như khả dụng món, giá, ảnh xạ công thức và tín hiệu ẩn món. **Identity & Access Control** cùng **Audit & Compliance** lại bao quanh chuỗi lõi như một lớp kiểm soát: không tác động trực tiếp vào giá trị món ăn hay số tiền phải thu, nhưng quyết định ai được phép ghi đề, hoàn tiền, hủy món hay xem dữ liệu nhạy cảm. Đây là cách nhìn logic phù hợp với chuẩn mô tả kiến trúc: phân biệt rõ **năng lực nghiệp vụ chính**, **năng lực tham chiếu** và **năng lực điều tiết**.

Những điểm chính cần đọc từ hình.

- **Chuỗi lõi** đi theo thứ tự **Reservation & Seating** → **Ordering** → **Kitchen Coordination** → **Billing & Settlement**, phản ánh đúng vòng đời phục vụ trong nhà hàng.
- **Menu Management** và **Inventory Monitoring** đóng vai trò năng lực điều kiện: chúng không thay mặt tuyến đầu xử lý giao dịch nhưng ảnh hưởng trực tiếp tới tính hợp lệ của món và quyết định ẩn hoặc mở bán.
- **Published Operational Facts** biểu diễn lớp dữ kiện được công bố ra ngoài chuỗi lõi sau khi các quyết định chính đã được chốt, làm cầu nối tới báo cáo, thông báo và truy vết.
- **Identity & Access Control** cùng **Audit & Compliance** là lớp bảo vệ xuyên suốt, giúp những hành động nhạy cảm có cơ sở quyền hạn và dấu vết truy vết rõ ràng.
- **Notification Integration** và **Reporting & Analytics** nằm ở vòng ngoài vì chúng tiêu thụ dữ kiện vận hành hơn là điều khiển giao dịch lõi theo thời gian thực.

Tổng quan cho góc nhìn mô-đun

Trong báo cáo này, góc nhìn mô-đun được trình bày theo tinh thần **decomposition view** kết hợp với **uses/layered constraints**. Nói cách khác, phần mô-đun không chỉ liệt kê hệ thống có những khối nào, mà còn chỉ ra khối nào được phép phụ thuộc vào khối nào, thông tin nào là phụ thuộc tĩnh, và đâu là những liên hệ chỉ nên xuất hiện dưới dạng hệ quả sự kiện sau giao dịch.



Hình 13: Sơ đồ tổng quan cho góc nhìn mô-đun của IRMS

Hình 13 làm rõ quy tắc phụ thuộc theo lớp trong góc nhìn mô-đun. Góc nhìn mô-đun không chỉ trả lời hệ thống có những mô-đun nào, mà còn phải trả lời mô-đun nào được phép gọi trực tiếp mô-đun nào, mô-đun nào chỉ nên giao tiếp qua sự kiện, và mô-đun nào là nền tảng dùng chung chứ không phải mô-đun nghiệp vụ. Vì vậy hình này tách rõ Client Modules, Business Modules và Shared Platform Modules.

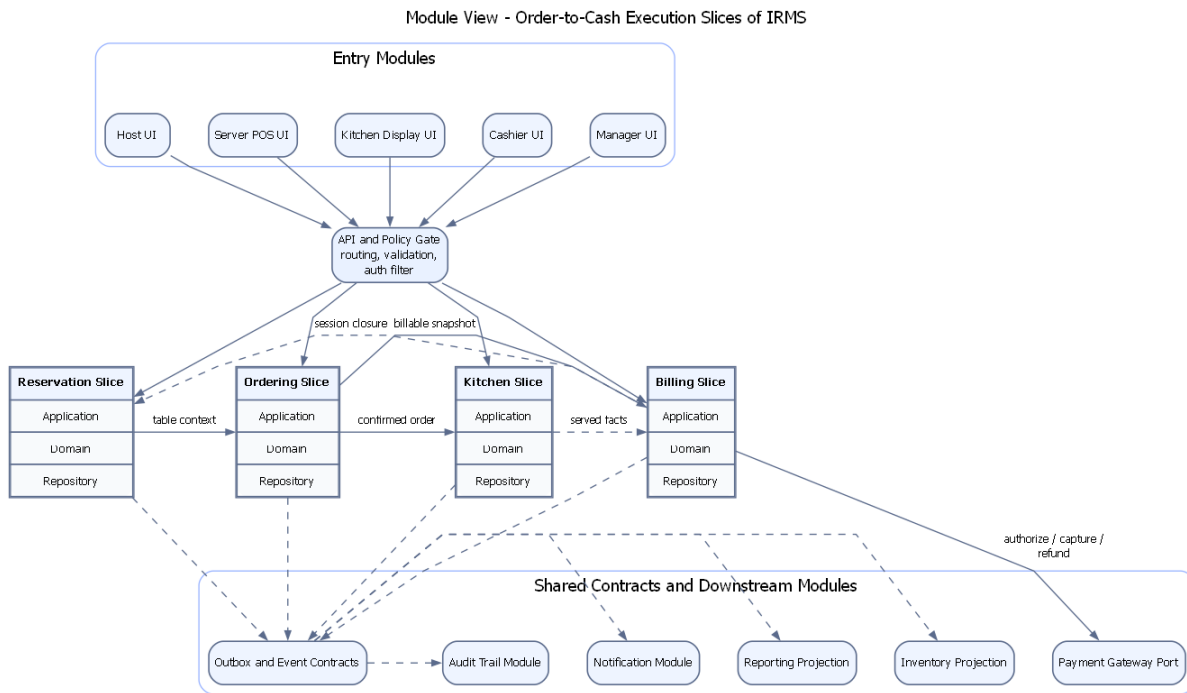
Các ý chính cần đọc từ hình.

- Các giao diện người dùng chỉ đi vào API / Web Layer; chúng không được gọi chéo vào lớp kho lưu trữ hay bộ kết nối.
- Các mô-đun nghiệp vụ giữ trách nhiệm riêng, ví dụ Ordering không ?ôm? logic thanh toán, còn Billing không trực tiếp điều phối trạng thái bếp.
- Những năng lực như Repositories / Unit of Work, Event Bus / Outbox, Reporting Views và External Adapters là hạ tầng dùng chung, được phụ thuộc theo hướng có kiểm soát.

Vì sao sơ đồ tổng quan này quan trọng.

- Nó giúp phần góc nhìn mô-đun không bị dừng ở mức ?danh sách mô-đun?, mà nâng lên thành quy tắc tổ chức mã nguồn và phụ thuộc.
- Nó tạo cầu nối trực tiếp sang góc nhìn thành phần: từ phụ thuộc ở mức mô-đun, người đọc có thể suy ra vì sao ở sơ đồ thành phần tồn tại bộ điều khiển, dịch vụ ứng dụng, kho lưu trữ, hộp thư ra và bộ kết nối như những loại thành phần khác nhau.
- Nó cũng hỗ trợ phần SOLID vì từ sơ đồ có thể thấy rõ đường phụ thuộc được bẻ vào lớp trừu tượng hoặc hạ tầng dùng chung thay vì gọi chéo trực tiếp giữa các mô-đun.

Góc nhìn mô-đun cho tuyến order-to-cash



Hình 14: Góc nhìn mô-đun cho tuyến order-to-cash của IRMS

Hình 14 làm rõ ở mức chi tiết hơn tuyến từ nhận bàn, gọi món, điều phối bếp, quyết toán cho tới phát sinh dữ kiện báo cáo và thông báo. Ở mức này, hệ thống được nhìn như một chuỗi lát cắt mô-đun nối tiếp nhau thay vì một tập tên mô-đun đứng cạnh nhau. Đây là sơ đồ cần thiết vì trong nhiều báo cáo kiến trúc, phần mô-đun thường chỉ dừng ở danh sách mô-đun mà không chỉ ra đường phụ thuộc tính của một tuyến nghiệp vụ quan trọng. Với IRMS, tuyến order-to-cash là nơi mọi quyết định về ranh giới mô-đun lộ ra rõ nhất.

Điểm cốt lõi của sơ đồ là **mỗi lát cắt mô-đun sở hữu dữ liệu, luật nghiệp vụ và nhịp thực thi của riêng mình**. Trong hình, mỗi lát cắt được biểu diễn như một khối dọc gồm ba tầng **Application**, **Domain** và **Repository**. Đây là cách biểu diễn súc tích nhưng vẫn đủ mạnh về mặt kiến trúc: **Reservation** sở hữu bối cảnh phiên bàn và trạng thái bàn; **Ordering** sở hữu **Order**, **OrderItem** và ảnh chụp thực đơn; **Kitchen** sở hữu **KitchenTicket** và hàng chờ chế biến; **Billing** sở hữu **Bill**, **BillSplit**, **Payment** và **Refund**. Các phụ thuộc giữa chúng đi qua các giao diện công khai hoặc ảnh chụp nghiệp vụ, chứ không qua việc truy cập thẳng vào cấu trúc bên trong nhau. Nhờ đó, thay đổi ở một mô-đun không kéo theo nghĩa vụ sửa mã nội bộ của mô-đun khác.

Hình này còn nhấn mạnh sự khác nhau giữa **đường thực thi đồng bộ** và **đường hệ quả sau giao dịch**. Đường thực thi đồng bộ xuất hiện ở các mũi tên đi từ **Reservation Slice** sang **Ordering Slice**, từ **Ordering Slice** sang **Kitchen Slice**, từ **Ordering Slice** sang **Billing Slice** và từ **Billing Slice** quay lại **Reservation Slice** để khép phiên bàn. Đường hệ quả sau giao dịch lại đi qua **Outbox and Event Contracts** rồi mới lan sang **Inventory Projection**, **Reporting Projection**, **Notification Module** và **Audit Trail Module**. Việc tách hai đường này là một nguyên tắc quan trọng của báo cáo: tác vụ tuyến đầu phải ngắn và giải thích được trạng thái, còn tác vụ hỗ trợ phải được tách nhịp để không làm nghẽn giao dịch.

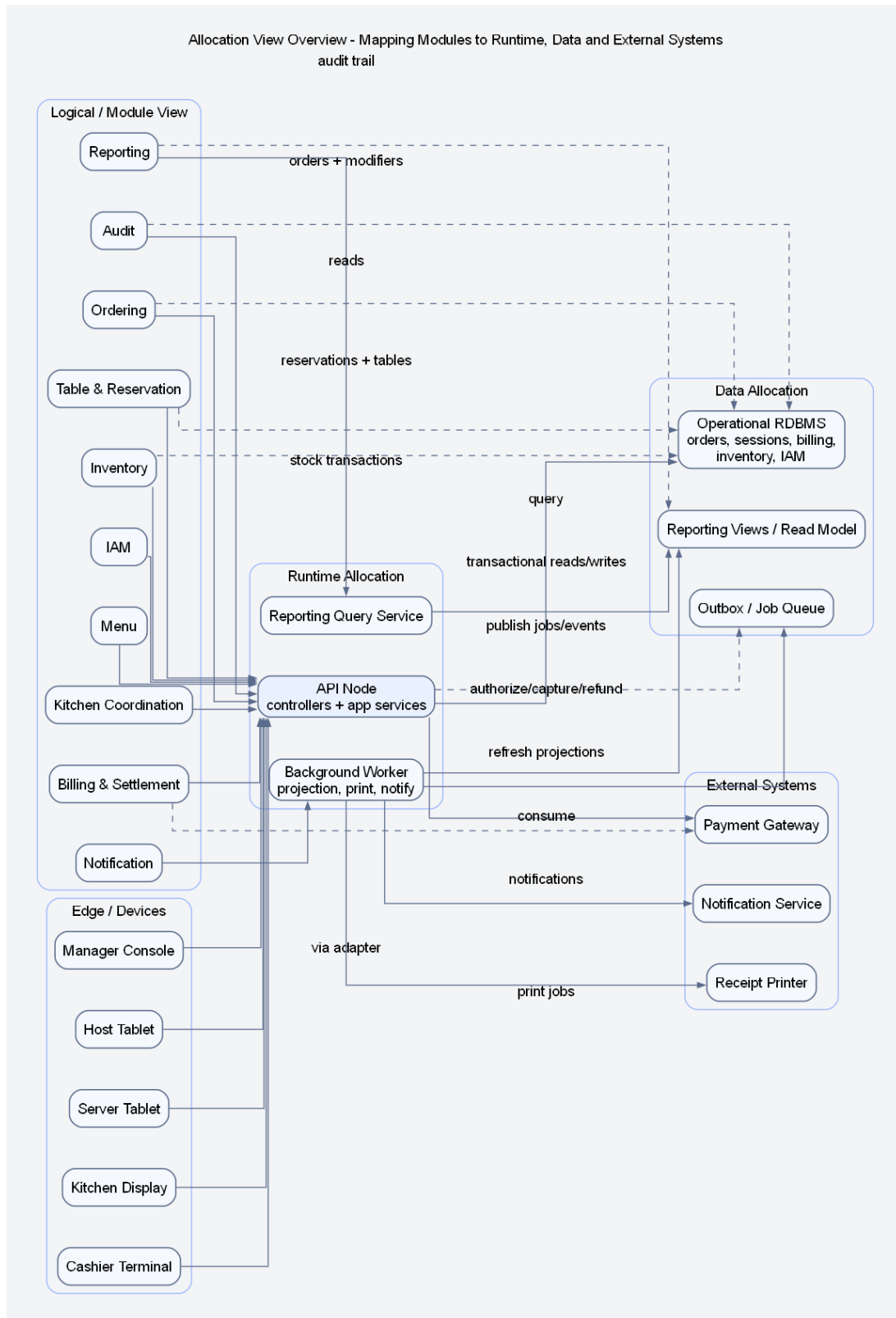
Ngoài ra, sơ đồ còn cho thấy vai trò của **API and Policy Gate** như một mô-đun biên nằm trước các lát cắt nghiệp vụ. Trong thực tế hiện thực, đây là nơi hợp lý để gom các kiểm tra đầu vào, xác thực, kiểm tra quyền ở mức yêu cầu và chuẩn hóa lời gọi vào các mô-đun lõi. Nhờ có tầng biên này,

các lát cắt phía sau không phải lặp lại quá nhiều logic kỹ thuật, trong khi phần điều phối ứng dụng vẫn giữ được biên giao dịch riêng của từng mô-đun.

Những điểm chính cần đọc từ hình.

- **Lát cắt mô-đun** được tổ chức theo chiều dọc: mỗi mô-đun có lớp ứng dụng, lớp miền và kho lưu trữ riêng, thay vì dùng chung một lớp kỹ thuật cho toàn bộ hệ thống.
- **Tuyên phụ thuộc tĩnh** chỉ đi qua giao diện công khai và ảnh chụp nghiệp vụ cần thiết, nhờ đó tránh phụ thuộc chéo vào chi tiết nội bộ.
- **Tuyên hệ quả sự kiện** được tách sang Outbox, Reporting, Notification và một phần Inventory, bảo vệ đường nóng khởi tác vụ phụ.
- **API and Policy Gate** là biên chung trước các lát cắt nghiệp vụ, giúp gom các kiểm tra kỹ thuật và kiểm tra quyền trước khi lời gọi đi vào từng mô-đun.
- **Quy tắc mô-đun** ở hình này là cơ sở để kiểm chứng phần hiện thực mã nguồn về sau: nếu mô-đun gọi xuyên qua nội tạng của nhau, kiến trúc sẽ bị phá vỡ ngay ở mức code.

Tổng quan cho góc nhìn phân bố



Hình 15: Sơ đồ tổng quan cho góc nhìn phân bố của IRMS

Hình 15 đưa ra bức tranh tổng quan cho góc nhìn phân bố. Nếu góc nhìn logic trả lời ?hệ thống gồm những năng lực nào? và góc nhìn mô-đun trả lời ?hệ thống phân rã thành những mô-đun nào?, thì góc nhìn phân bố phải trả lời ?những mô-đun và năng lực đó cuối cùng được đặt ở đâu

trong vùng thực thi, dữ liệu và hệ thống ngoài?. Hình này vì vậy ánh xạ ba chiều: **logic/mô-đun** → **phân bố thực thi** → **phân bố dữ liệu**.

Các lớp phân bố chính thể hiện trong hình.

- **Lớp biên và thiết bị:** là nơi phát sinh thao tác nghiệp vụ thực tế như máy của lễ tân, máy của phục vụ, màn hình bếp, máy thu ngân và màn hình quản lý.
- **Lớp phân bố thực thi:** gồm nút giao diện lập trình ứng dụng, bộ xử lý nền và dịch vụ truy vấn báo cáo; đây là nơi các năng lực được hiện thực thành tiến trình hoặc vùng thực thi cụ thể.
- **Lớp phân bố dữ liệu:** gồm Operational RDBMS, Reporting Views / Read Model và Outbox / Job Queue; chúng tách dữ liệu giao dịch, dữ liệu đọc tổng hợp và hàng đợi xử lý nền.
- **Lớp hệ thống ngoài:** gồm Payment Gateway, Notification Service và Receipt Printer; chúng là các điểm tích hợp nằm ngoài lõi IRMS.

Lý do thiết kế của sơ đồ này.

- Sơ đồ giúp người đọc nhìn ra ngay **mô-đun nào chạy trên nút giao diện lập trình ứng dụng, mô-đun nào đi qua bộ xử lý nền, và mô-đun nào đọc từ mô hình báo cáo thay vì đánh trực tiếp vào cơ sở dữ liệu giao dịch.**
- Nó cho thấy rõ vì sao Notification và một phần Reporting không nên bị buộc vào tuyến giao dịch nóng.
- Nó là phần nối đúng giữa góc nhìn thành phần và góc nhìn triển khai: sơ đồ thành phần cho thấy thành phần hợp tác ra sao; góc nhìn phân bố cho thấy các thành phần đó được đặt trên vùng thực thi và nơi lưu trữ nào.

3.4 Đặc trưng kiến trúc

Dựa trên các yêu cầu phi chức năng đã được lượng hóa ở phần tổng quan, những *đặc trưng kiến trúc* quan trọng nhất của IRMS là:

1. **Khả năng đáp ứng:** màn hình gọi món, màn hình bếp và màn hình thu ngân phải phản hồi nhanh để không làm chậm tuyến phục vụ trực tiếp với khách.
2. **Độ tin cậy:** đơn gọi món, thanh toán, hoàn tiền và cập nhật trạng thái bàn là các luồng không được mất dữ liệu hoặc xử lý lặp sai lệch.
3. **Khả năng mở rộng:** điểm nóng thường dồn vào giờ cao điểm và tập trung ở các miền Ordering, Kitchen Coordination và Billing thay vì toàn hệ thống.
4. **Bảo mật và khả năng kiểm toán:** mọi thao tác nhạy cảm phải được kiểm soát bằng quyền và truy vết lại được về sau.
5. **Khả năng quan sát:** hệ thống phải đủ nhật ký, vết thực thi và số đo để thấy độ trễ của phiếu bếp, thanh toán và các điểm nghẽn vận hành.
6. **Khả năng sửa đổi:** thực đơn, giá, khuyến mãi, ngưỡng cảnh báo tồn kho, kênh gửi thông báo và bộ kết nối thanh toán thay đổi thường xuyên nên cần biên thay đổi hẹp.

Sáu đặc trưng trên không độc lập tuyệt đối mà thường kéo nhau theo hướng căng thẳng. Ví dụ, để tăng **khả năng đáp ứng**, hệ thống có xu hướng rút ngắn chuỗi gọi đồng bộ; nhưng để giữ **độ tin cậy**, một số bước như thanh toán vẫn phải nằm trong giao dịch hoặc giao thức chặt hơn. Tương tự, **khả năng sửa đổi** khuyến khích tách ranh giới nghiệp vụ, nhưng nếu tách quá sớm hoặc quá mạnh thì có thể làm tăng độ phức tạp tích hợp, ảnh hưởng ngược lại **khả năng đáp ứng** và **khả năng bảo trì**. Vì vậy, các quyết định kiến trúc ở phần sau đều được đọc dưới góc nhìn cân bằng giữa các đặc trưng này, không phải tối ưu một đặc tính duy nhất.

3.5 Góc nhìn phân rã

Ở góc nhìn phân rã, hệ thống được chia thành hai lớp lớn: **các ứng dụng phía người dùng** và **các dịch vụ nghiệp vụ phía sau**. Việc phân rã không dựa trên tầng kỹ thuật thuần túy, mà bám theo miền nghiệp vụ của nhà hàng.

Các mô-đun phía người dùng

- **Host/Reservation UI:** phục vụ đặt bàn, danh sách chờ, sơ đồ bàn và check-in.
- **Server POS UI:** phục vụ gọi món tại bàn, chỉnh món, theo dõi phiếu bếp và hỗ trợ tính tiền.
- **Kitchen Display UI:** hiển thị phiếu bếp theo trạm, mức ưu tiên, hạn hoàn thành và tiến độ chế biến.
- **Cashier UI:** tạo hóa đơn, tách hóa đơn, thanh toán, biên lai và hoàn tiền.
- **Manager Portal:** quản trị thực đơn, chương trình khuyến mãi, cảnh báo tồn kho, lịch làm việc và bảng điều khiển báo cáo.

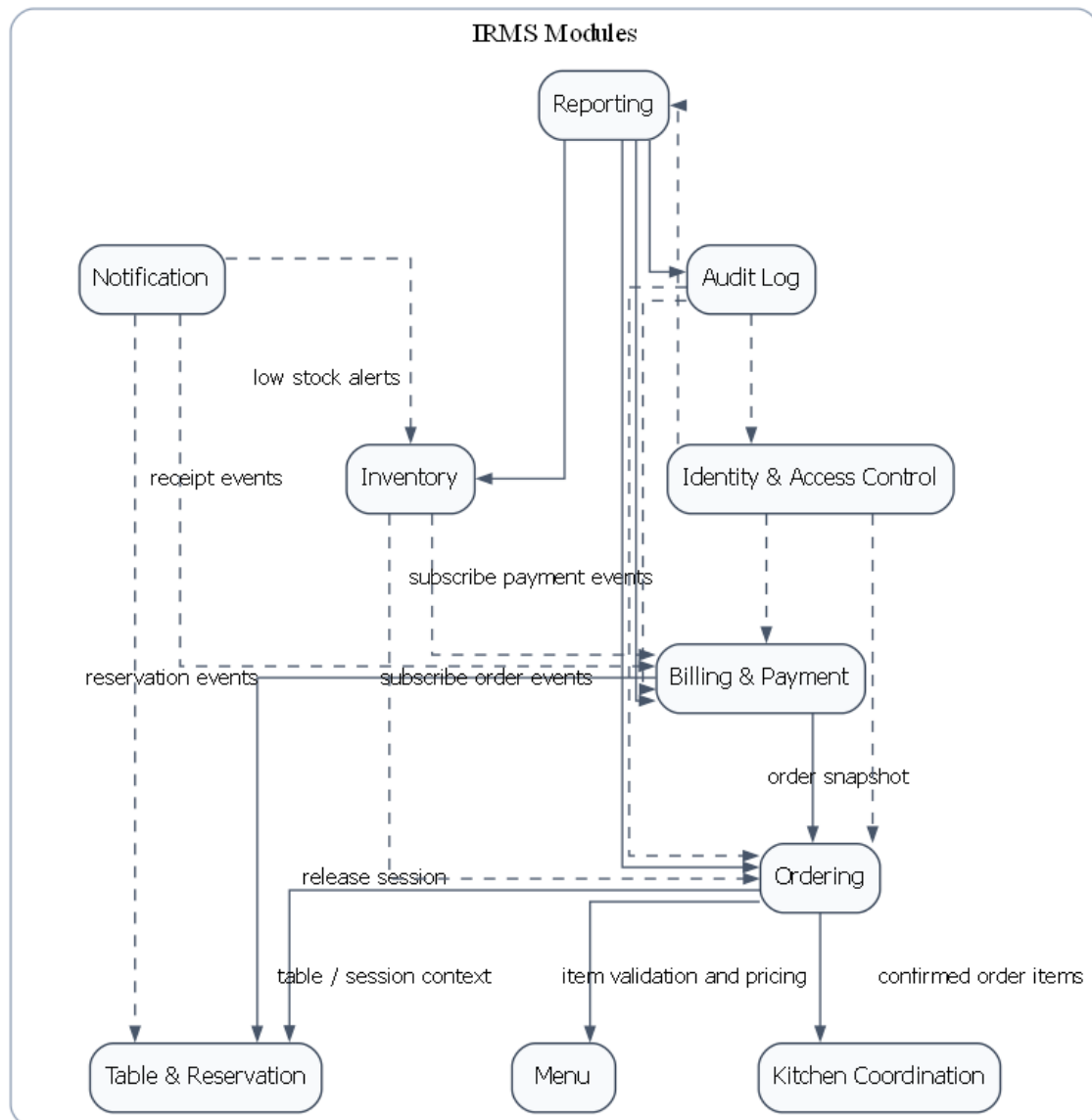
Các dịch vụ nghiệp vụ phía sau

- **Reservation & Seating Service:** quản lý đặt bàn, danh sách chờ, bàn ăn và phiên phục vụ tại bàn.
- **Ordering Service:** quản lý thực đơn, tùy chọn món, đơn gọi món, dòng món và trạng thái của đơn từ đầu đến cuối.
- **Kitchen Coordination Service:** điều phối phiếu bếp, trạm bếp và cam kết thời gian ra món.
- **Billing Service:** quản lý hóa đơn, tách hóa đơn, thanh toán, biên lai, hoàn tiền và áp dụng khuyến mãi.
- **Inventory Service:** quản lý công thức, mặt hàng tồn kho, giao dịch kho và quy tắc cảnh báo mức tồn thấp.
- **Reporting Service:** quản lý mô hình đọc, ảnh chụp dữ liệu báo cáo và xuất báo cáo.
- **IAM/Access Service:** định danh, vai trò, quyền và thực thi chính sách truy cập.

Lưu ý về thuật ngữ ?dịch vụ?. Trong góc nhìn phân rã này, từ ?dịch vụ? được dùng theo nghĩa **ranh giới năng lực nghiệp vụ ở mức logic**, không mặc nhiên có nghĩa rằng mỗi dịch vụ phải được triển khai thành một tiến trình mạng độc lập. Điểm này rất quan trọng để tránh nhầm lẫn giữa *ranh giới mô-đun* và *cách bố trí triển khai*. Một hệ thống có thể phân rã theo dịch vụ rất rõ về mặt logic, nhưng khi triển khai vẫn chạy tập trung trong một khối xử lý phía sau nếu điều đó phù hợp hơn với quy mô và ranh giới giao dịch.

Giải thích chi tiết cho góc nhìn phân rã. Phân rã như trên giúp ranh giới trách nhiệm rõ ràng: thay đổi ở bếp không kéo trực tiếp vào thanh toán; thay đổi ở thực đơn không làm nứt mô hình đặt bàn; báo cáo không đung vào luồng ghi thời gian thực. Đây là cách phân rã phù hợp với IRMS vì ranh giới miền nghiệp vụ của nhà hàng đã rất rõ theo thực tế vận hành.

Sơ đồ tổng quan



Hình 16: Góc nhìn mô-đun tổng quan của IRMS

Hình 16 thể hiện góc nhìn mô-đun ở mức đủ rộng để nhìn ra toàn bộ cấu trúc trách nhiệm của hệ thống mà chưa sa vào chi tiết cách hiện thực. Đây là một trong những sơ đồ quan trọng nhất của báo cáo vì nó trả lời câu hỏi mà bất kỳ người phản biện nào cũng sẽ quan tâm: hệ thống được chia thành các khối nào, mỗi khối thật sự chịu trách nhiệm cho điều gì, và ranh giới nào là ranh giới kiến trúc đáng bảo vệ.

Điều cần đọc đầu tiên ở hình này không phải là số lượng mô-đun, mà là **cách chia mô-đun theo nghĩa nghiệp vụ**. **Reservation** không chỉ là nơi lưu đặt bàn; nó giữ toàn bộ quy tắc về tiếp nhận khách, danh sách chờ, sức chứa và mở phiên bàn. **Ordering** không chỉ là nơi ghi dòng món; nó giữ ảnh chụp thực đơn tại thời điểm gọi, các tùy chọn món, ghi chú đặc biệt và trạng thái đơn gọi món. **Kitchen** không phải là phần mở rộng nhỏ của gọi món; nó là miền chịu trách nhiệm cho hàng chờ chế biến, ưu tiên, đồng bộ trạng thái và bàn giao món. **Billing** là miền quyết toán có

vòng đời riêng. Khi được tách như vậy, mô-đun trở thành nơi trú ngụ của quyết định nghiệp vụ, không chỉ là vỏ bọc kỹ thuật.

Một điểm quan trọng khác của sơ đồ là **các phụ thuộc giữa mô-đun**. **Ordering** cần biết ngữ cảnh bàn đang phục vụ và cần lấy thông tin món từ menu, nhưng không vì vậy mà được ôm luôn toàn bộ quy tắc của bàn hay toàn bộ vòng đời khuyến mãi. **Billing** cần ảnh chụp đáng tin cậy của những gì đã được gọi và phục vụ, nhưng không nên quay ngược trở lại để điều khiển trực tiếp **Kitchen**. **Inventory**, **Reporting**, **Notification**, **Audit** và **IAM** được bố trí thành các mô-đun hỗ trợ hoặc xuyên suốt nhằm nhấn mạnh rằng chúng rất quan trọng, nhưng không nên chen vào mọi quyết định ở tuyến giao dịch nóng. Cách đặt đó giúp người đọc thấy ngay nơi nào phải ưu tiên tính đúng đắn tức thời, nơi nào có thể chấp nhận độ trễ nhỏ và nơi nào phải ưu tiên khả năng truy vết.

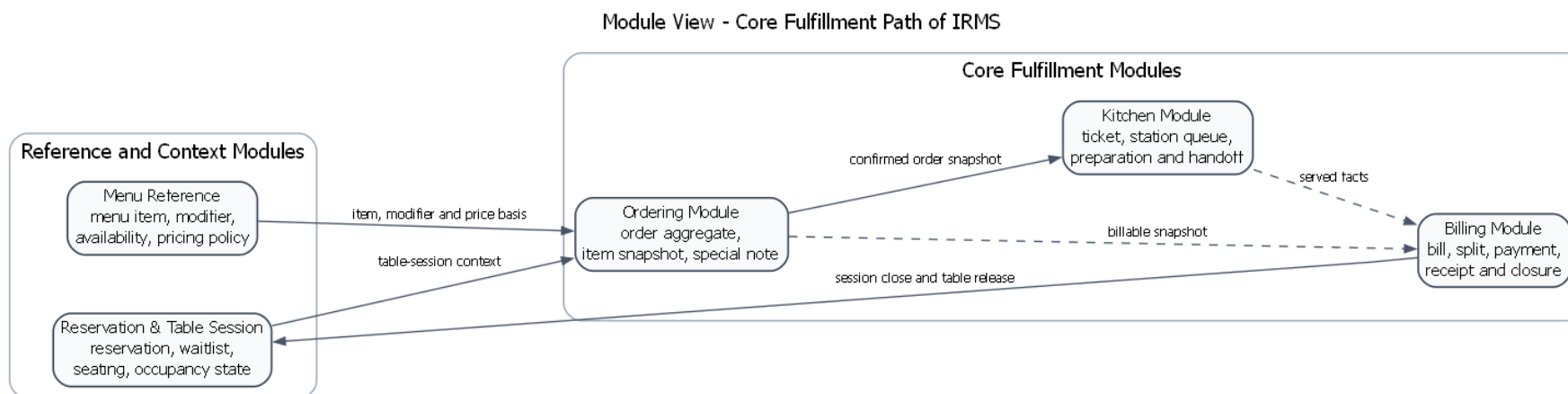
Sơ đồ còn quan trọng ở chỗ nó làm rõ **điểm dễ hỏng nếu thiết kế kém kỷ luật**. Trong một hệ thống nhà hàng, rất nhiều khái niệm có liên hệ chặt với nhau: **Order** liên quan đến **Bill**; **KitchenTicket** liên quan đến **OrderItem**; tồn kho lại suy diễn từ các món đã phục vụ hoặc các quy tắc nghiệp vụ khác. Nếu không có ranh giới mô-đun đủ chặt, hệ thống sẽ rất dễ rơi vào tình trạng bất kỳ thay đổi nhỏ nào ở khuyến mãi, thực đơn hay thanh toán cũng kéo theo một chuỗi sửa chéo trên nhiều vùng mã. Hình này vì vậy không chỉ mô tả cấu trúc hiện tại, mà còn là công cụ để tự kiểm tra chất lượng kiến trúc trong suốt vòng đời phát triển.

Các thành phần chính thể hiện trong hình.

- **Các mô-đun nghiệp vụ lõi:** **Reservation**, **Ordering**, **Kitchen**, **Billing**.
- **Các mô-đun hỗ trợ và xuyên suốt:** **Inventory**, **Reporting**, **Notification**, **Audit**, **IAM**.
- **Các phụ thuộc mô-đun:** cho thấy dữ liệu hoặc quyết định nào đi qua biên mô-đun nào.

Lý do thiết kế của sơ đồ này.

- Đây là sơ đồ tổng quan của góc nhìn mô-đun vì nó mô tả toàn bộ cấu trúc phân rã trước khi tách riêng tuyến lõi và tuyến hỗ trợ.
- Việc nhìn dependencies ở mức mô-đun giúp người đọc đánh giá nhanh mức kết tụ và mức phụ thuộc chéo của kiến trúc.



Hình 17: Góc nhìn mô-đun của luồng thực thi nghiệp vụ lõi

Hình 17 đi sâu vào tuyến thực thi nghiệp vụ lõi của IRMS, tức tuyến mà mỗi giây chậm đi hoặc mỗi quyết định sai lệch đều tác động trực tiếp tới khách hàng đang ngồi tại bàn. Đây là vùng kiến trúc phải được đọc cẩn thận nhất vì nó quyết định hệ thống có thực sự dùng được trong giờ cao điểm hay không.

Trung tâm của hình là **Ordering**. Cách đặt **Ordering** ở tâm không nhằm làm mô-đun này trở nên ?quyền lực?, mà để phản ánh đúng bản chất dữ liệu của bài toán. Tại thời điểm gọi món, hệ thống phải biết khách đang ở bàn nào, phiên bản có còn hiệu lực hay không, món còn bán hay không, cấu hình tùy chọn có hợp lệ hay không, giá nào cần được chụp lại để về sau hóa đơn không bị sai lệch, và sau khi xác nhận thì thông tin nào phải được lan truyền sang bếp. Tất cả các quyết định đó gặp nhau ở **Ordering**. Vì vậy, nếu đặt trung tâm vào nơi khác, mô hình sẽ méo theo kỹ thuật chứ không còn bám đúng nghiệp vụ.

Tuy nhiên, chính vì ở trung tâm nên **Ordering** cũng là mô-đun dễ bị phình to nhất. Nếu nhóm phát triển đưa cả khuyến mãi phức tạp, tính toán kho, gửi thông báo, phân tích và kiểm toán chi tiết vào cùng một chỗ, mô-đun này sẽ nhanh chóng trở thành ?điểm dồn mọi thứ?. Hình này vì vậy không chỉ cho thấy vai trò trung tâm của **Ordering**, mà còn nhắc rất rõ biên của nó: nó phải tập trung vào dữ liệu gọi món và chuyển tiếp nghiệp vụ trực tiếp; còn các hệ quả hỗ trợ cần được chuyển sang mô-đun khác hoặc luồng xử lý khác.

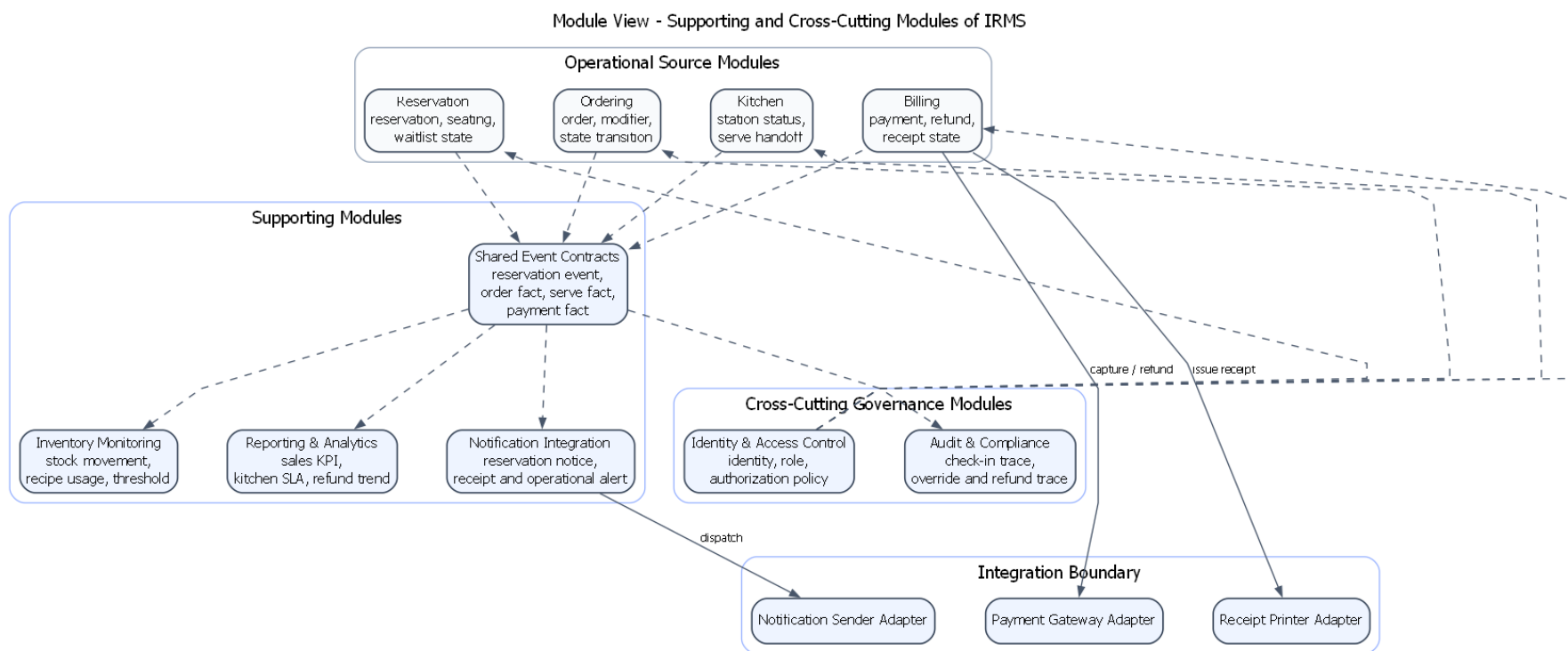
Mối quan hệ giữa **Ordering**, **Kitchen** và **Billing** trong hình cũng rất quan trọng. Sau khi đơn gọi món được xác nhận, **Kitchen** phải nhận đủ thông tin để tổ chức chế biến nhưng không nên quyết định ngược trở lại những bất biến gốc của đơn gọi món. Tương tự, **Billing** cần một ảnh chụp đáng tin cậy của món đã gọi và món đã phục vụ; nhưng miền quyết toán không nên trở thành nơi quay lại chỉnh sửa cấu trúc gọi món gốc. Cách sắp xếp này giúp chuỗi nghiệp vụ có hướng đi rõ ràng: từ ngữ cảnh bàn, sang xác nhận đơn gọi món, sang điều phối bếp, rồi tới quyết toán. Khi hướng đi rõ, ranh giới giao dịch, khóa đồng thời và xử lý ngoại lệ cũng dễ được xác định hơn.

Các thành phần chính thể hiện trong hình.

- **Reservation và TableSession:** cung cấp ngữ cảnh bàn đang phục vụ.
- **Ordering:** giữ vai trò trục trung tâm của tuyến nghiệp vụ nóng.
- **Kitchen và Billing:** là hai hướng lan truyền quan trọng sau khi đơn được xác nhận.

Lý do thiết kế của sơ đồ này.

- Sơ đồ được tách riêng để người đọc nhìn rõ đường nghiệp vụ nóng thay vì bị chìm trong toàn bộ hệ thống.
- Việc nhấn mạnh tâm của **Ordering** giúp giải thích vì sao mô-đun này cần được bảo vệ khỏi việc ôm thêm quá nhiều trách nhiệm.



Hình 18: Góc nhìn mô-đun của các mô-đun hỗ trợ và xuyên suốt

Hình 18 giải thích phần còn lại của kiến trúc, tức những mô-đun không trực tiếp đứng trên đường giao dịch nóng nhưng vẫn quyết định chất lượng vận hành của hệ thống trong dài hạn. Đây là phần thường bị trình bày quá mỏng trong nhiều tài liệu, khiến các năng lực hỗ trợ bị nhìn như phần phụ. Với IRMS, điều đó không đúng: **Inventory, Reporting, Notification, Identity & Access Control** và **Audit & Compliance** đều là những mô-đun hạng nhất, chỉ khác ở chỗ chúng phục vụ mục tiêu khác với mô-đun gọi món và thanh toán.

Inventory là mô-đun suy diễn từ kết quả vận hành sang biến động kho; **Reporting** chuyển dữ liệu nghiệp vụ thành góc nhìn quản trị; **Notification** chịu trách nhiệm lan truyền thông tin ra ngoài; còn **Identity & Access Control** và **Audit Log** bảo vệ tính kiểm soát của toàn hệ thống. Sơ đồ hiện tại làm rõ thêm một điểm rất quan trọng: các mô-đun hỗ trợ không tồn tại lơ lửng, mà được nuôi bởi **các mô-đun nguồn vận hành** như **Reservation, Ordering, Kitchen** và **Billing** thông qua một lớp **Shared Event Contracts**. Cách tổ chức này làm rõ sự khác nhau giữa mô-đun sinh dữ kiện vận hành, hợp đồng trung gian mang nghĩa nghiệp vụ, mô-đun tiêu thụ dữ kiện và bộ kết nối đặt ở rìa hệ thống.

Tuy vậy, tách ra riêng không có nghĩa là xem nhẹ. Chính các mô-đun hỗ trợ này mới giúp IRMS trở thành một hệ thống vận hành hoàn chỉnh thay vì chỉ là công cụ nhập món. Nhà quản lý cần số liệu; bộ phận vận hành cần cảnh báo tồn kho; tổ chức cần khả năng truy vết thao tác; hệ thống cần phân quyền đủ chặt để tránh lạm dụng quyền hạn. Có thể hiểu ngắn gọn rằng tuyến lỗi phản ánh trạng thái phục vụ đang diễn ra, còn nhóm mô-đun hỗ trợ phản ánh cách toàn bộ nhà hàng được điều hành, quan sát và kiểm soát.

Đánh đổi đi cùng cách tổ chức này là hệ thống phải chấp nhận một mức **độ trễ quan sát có kiểm soát**. Báo cáo có thể chậm hơn vài giây hoặc vài phút; cảnh báo mức tồn thấp có thể đến sau giao dịch gốc một khoảng nhỏ; dữ liệu kiểm toán có thể được làm giàu thêm chi tiết sau khi hành động chính đã hoàn tất. Nhưng đó là đánh đổi đúng trong bối cảnh nhà hàng, vì cái cần phản hồi ngay là đơn gọi món, bếp và thanh toán; còn cái cần đúng, đủ và truy vết được ở tầng quản trị có thể đi sau một nhịp nhỏ mà vẫn hoàn toàn chấp nhận được.

Một điểm cần nhấn mạnh thêm là **Identity & Access Control** không đi cùng **Shared Event Contracts**. Đây là chủ ý thiết kế. Phân quyền là ràng buộc được áp trước hoặc ngay trong lúc xử lý yêu cầu, không phải hệ quả đi sau giao dịch. Vì vậy, mô-đun này được vẽ như một lớp điều tiết trực tiếp lên các mô-đun nguồn. Ngược lại, **Audit & Compliance** vừa có phần kiểm soát chính sách, vừa có phần tiêu thụ dấu vết sau hành động; bởi vậy trong sơ đồ nó đứng giữa vai trò governance và vai trò hậu kiểm. Cách đặt đó giúp hình không chỉ liệt kê mô-đun, mà còn làm rõ nhịp can thiệp của từng mô-đun vào đời sống vận hành của hệ thống.

Các thành phần chính thể hiện trong hình.

- **Reservation, Ordering, Kitchen và Billing:** là các mô-đun nguồn phát sinh dữ kiện vận hành để nhóm hỗ trợ tiêu thụ.
- **Shared Event Contracts:** là lớp hợp đồng trung gian chuẩn hóa các dữ kiện được phát ra từ mô-đun nguồn trước khi sang nhóm hỗ trợ.
- **Inventory, Reporting và Notification:** đại diện cho nhóm mô-đun suy diễn, tổng hợp và tích hợp có thể chạy lệch nhịp với đường nóng.
- **Identity & Access Control và Audit & Compliance:** đại diện cho nhóm kiểm soát xuyên suốt.
- **Các bộ kết nối ngoài:** cho thấy ranh giới cuối cùng giữa mô-đun hỗ trợ của IRMS và hạ tầng hoặc đối tác bên ngoài.

Lý do thiết kế của sơ đồ này.

- Tách sơ đồ này ra khỏi sơ đồ tuyến lỗi giúp người đọc thấy rõ phần nào không nên chen trực tiếp vào tuyến giao dịch.

- Hình này cũng làm rõ tại sao một số phần có thể chấp nhận độ trễ nhỏ nhưng vẫn phải giữ được tính truy vết và khả năng kiểm soát.

3.6 Góc nhìn thành phần và kết nối

Ở góc nhìn thành phần và kết nối, hệ thống được mô tả bằng các thành phần thực thi chính và các kiểu liên kết giữa chúng. Góc nhìn này đặc biệt quan trọng vì nó cho biết một yêu cầu đi qua những khối nào, khối nào giữ quyết định nghiệp vụ và khối nào chỉ đóng vai trò tích hợp hoặc hỗ trợ kỹ thuật.

Các thành phần chính

- **Các ứng dụng giao diện:** gồm các giao diện cho lễ tân, phục vụ, bếp, thu ngân và quản lý. Nhóm này chịu trách nhiệm nhận thao tác của người dùng và hiển thị kết quả theo đúng ngữ cảnh công việc.
- **Cổng vào giao diện lập trình ứng dụng:** là điểm vào thống nhất của hệ thống, chịu trách nhiệm xác thực, kiểm tra yêu cầu, định tuyến và ghi nhật ký ở biên.
- **Các dịch vụ ứng dụng:** điều phối các ca sử dụng thuộc các miền đặt bàn, gọi món, điều phối bếp, thanh toán, tồn kho, báo cáo và quản trị.
- **Các mô-đun miền nghiệp vụ:** chứa thực thể gốc, dịch vụ miền, chính sách và các bất biến nghiệp vụ cốt lõi.
- **Lớp lưu trữ dữ liệu:** thực hiện lưu trữ và truy xuất dữ liệu giao dịch thông qua kho lưu trữ, bộ quản lý giao dịch và bộ kết nối cơ sở dữ liệu.
- **Bộ phát sự kiện và bộ xử lý nền:** phát sự kiện, xử lý tác vụ nền, thử lại khi lỗi và làm mới mô hình đọc.
- **Các bộ kết nối ngoài:** kết nối tới cổng thanh toán, kênh gửi thông báo và thiết bị in biên lai mà không để logic nghiệp vụ phụ thuộc trực tiếp vào đối tác ngoài.

Các kiểu kết nối chính

Bảng 5: Các kiểu kết nối chính trong góc nhìn thành phần và kết nối

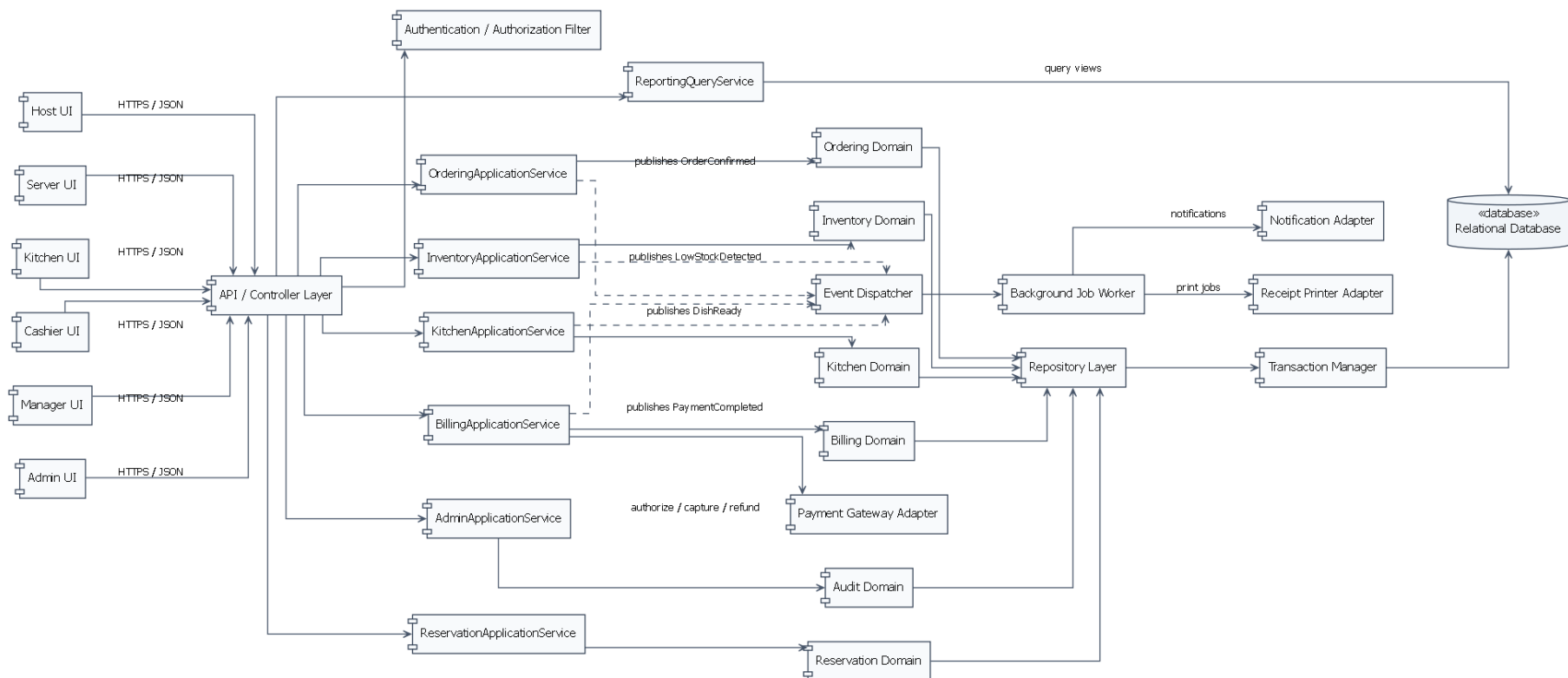
Kiểu kết nối	Vai trò	Ví dụ trong IRMS
Giao thức HTTP và lớp giao tiếp ứng dụng	giao tiếp giữa thiết bị người dùng và khối xử lý phía sau, ưu tiên phản hồi nhanh và thông điệp lỗi rõ ràng	các giao diện lễ tân, phục vụ và thu ngân gửi yêu cầu nghiệp vụ
Lời gọi nội bộ trong tiến trình	điều phối ca sử dụng bên trong khối xử lý phía sau	bộ điều khiển gọi Application Service
Kết nối kho lưu trữ và giao dịch	đọc ghi dữ liệu giao dịch và bảo vệ ranh giới giao dịch	ghi Order, Bill, Payment, TableSession
Công bố và tiếp nhận sự kiện	lan truyền trạng thái hoặc kích hoạt xử lý phụ	OrderConfirmed, PaymentCompleted, LowStockDetected
Bộ kết nối thích nghi	giao tiếp với hệ thống ngoài mà không để miền nghiệp vụ phụ thuộc trực tiếp	thanh toán, gửi thông báo, in biên lai
Kết nối truy vấn	đọc mô hình đọc hoặc khung nhìn báo cáo	bảng điều khiển quản lý, xuất báo cáo doanh thu

Lý do chọn mô hình kết nối hỗn hợp

IRMS không thể dùng một kiểu kết nối duy nhất cho mọi luồng nghiệp vụ. Nếu tất cả đều gọi đồng bộ nối chuỗi, hệ thống sẽ dễ nghẽn vào giờ cao điểm, đặc biệt trên các màn hình gọi món, màn hình bếp và màn hình thu ngân. Ngược lại, nếu tất cả đều chuyển sang xử lý bất đồng bộ, người dùng sẽ khó biết thao tác nào đã thành công, thao tác nào đang chờ và thao tác nào thất bại.

Vì vậy, kiến trúc dùng mô hình kết nối hỗn hợp với hai nguyên tắc. Thứ nhất, các bước nằm trên tuyến phục vụ trực tiếp cho khách như xác nhận đơn gọi món, chuyển món sang bếp, cập nhật trạng thái sẵn sàng và ghi nhận thanh toán phải ưu tiên phản hồi ngay. Thứ hai, các luồng hỗ trợ như cập nhật báo cáo, gửi thông báo, làm giàu nhật ký kiểm toán hoặc cảnh báo tồn kho nên đi qua sự kiện và bộ xử lý nền để không chặn tuyến giao dịch chính.

Giải thích chi tiết cho góc nhìn thành phần và kết nối. Điểm then chốt của IRMS là phân biệt *kết nối phục vụ trải nghiệm người dùng* với *kết nối phục vụ phối hợp nội bộ*. Chọn đúng kiểu kết nối cho từng nhóm thành phần sẽ quyết định trực tiếp đến độ trễ phản hồi, khả năng giải thích lỗi và khả năng mở rộng của toàn hệ thống.



Hình 19: Sơ đồ thành phần và kết nối tổng quan của IRMS

Hình 19 cho thấy cấu trúc thực thi tổng quan của hệ thống theo cách nhìn thành phần. Thiết bị người dùng chỉ giao tiếp với lớp cổng vào; từ đó yêu cầu đi qua lớp điều phối ca sử dụng, xuống miền nghiệp vụ, lớp lưu trữ và các bộ kết nối ra ngoài. Ý nghĩa học thuật của hình này là chứng minh hệ thống không được tổ chức như một khối gọi chéo tùy ý, mà có hướng phụ thuộc rõ ràng từ ngoài vào trong.

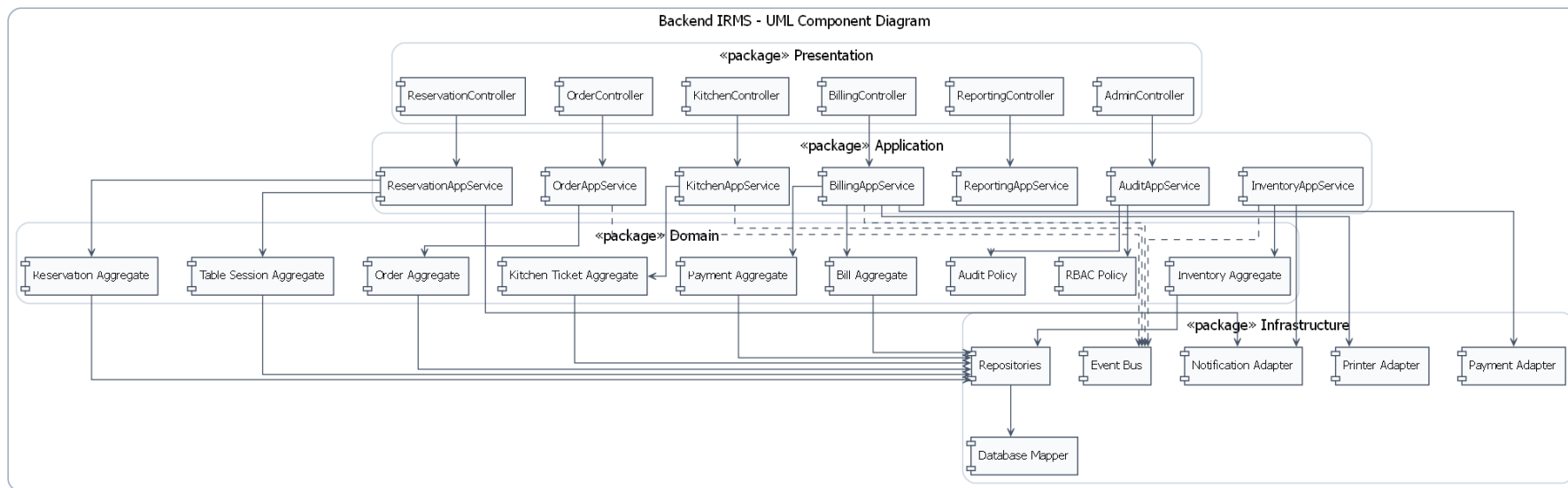
Các thành phần chính thể hiện trong hình.

- **Các ứng dụng giao diện:** là nơi phát sinh yêu cầu từ lễ tân, phục vụ, bếp, thu ngân và quản lý.
- **Cổng vào giao diện lập trình ứng dụng:** tiếp nhận yêu cầu, xác thực, kiểm tra đầu vào và chuyển tiếp tới đúng ca sử dụng.
- **Các dịch vụ ứng dụng:** điều phối từng ca sử dụng cụ thể như tạo đặt bàn, xác nhận đơn gọi món, cập nhật phiếu bếp hoặc chốt hóa đơn.
- **Các mô-đun miền nghiệp vụ:** giữ luật nghiệp vụ cốt lõi, bất biến dữ liệu và quyết định nghiệp vụ của nhà hàng.
- **Lớp lưu trữ dữ liệu:** chịu trách nhiệm lưu trữ trạng thái giao dịch và bảo vệ ranh giới giao dịch.
- **Bộ phát sự kiện và bộ xử lý nền:** xử lý sự kiện và tác vụ nền không nên nằm trên tuyến giao dịch nóng.
- **Các bộ kết nối ngoài:** bao bọc các tích hợp ra ngoài như thanh toán, gửi thông báo và in biên lai.

Lý do thiết kế của sơ đồ này.

- Việc đặt **Cổng vào giao diện lập trình ứng dụng** ở phía trước giúp thống nhất xác thực, kiểm tra yêu cầu và ghi nhật ký ở biên thay vì lặp lại ở mọi điểm vào.
- Việc tách **Các dịch vụ ứng dụng** khỏi **Các mô-đun miền nghiệp vụ** giúp phần điều phối ca sử dụng không lẫn với luật nghiệp vụ cốt lõi.
- Việc đẩy **Các bộ kết nối ngoài** ra rìa giúp miền nghiệp vụ không phụ thuộc trực tiếp vào cổng thanh toán, dịch vụ gửi thông báo hay thiết bị in.
- Việc tách **Bộ phát sự kiện và bộ xử lý nền** thành thành phần riêng giúp các tác vụ phụ không làm chậm các thao tác đang diễn ra trực tiếp trước khách hàng.

Ưu điểm của sơ đồ này là chiều phụ thuộc được kiểm soát rõ: giao diện không phụ thuộc trực tiếp vào miền nghiệp vụ, miền nghiệp vụ không gọi ngược bộ điều khiển, còn bộ kết nối ngoài chỉ xuất hiện ở rìa hệ thống. Nhược điểm là nhóm phát triển phải giữ kỷ luật hiện thực; nếu đưa luật nghiệp vụ vào bộ điều khiển hoặc để miền nghiệp vụ gọi thẳng bộ kết nối ngoài thì giá trị của kiến trúc sẽ bị phá vỡ.



Hình 20: Sơ đồ thành phần chi tiết của khối xử lý phía sau trong IRMS

Hình 20 là sơ đồ quan trọng nhất của góc nhìn thành phần vì nó mở lớp vỏ của khối xử lý phía sau và cho thấy hệ thống được tổ chức theo trật tự trách nhiệm như thế nào. Rất nhiều tài liệu chỉ dừng ở việc nói rằng hệ thống có giao diện, dịch vụ và cơ sở dữ liệu. Hình này đi xa hơn: nó chỉ ra thành phần nào được quyền giữ luật nghiệp vụ, thành phần nào chỉ nên điều phối tiến trình, và thành phần nào tuyệt đối không được chen ngược vào lõi.

Lớp tiếp nhận chỉ làm việc của lớp tiếp nhận: nhận yêu cầu, kiểm tra hình dạng dữ liệu, chuyển đổi dữ liệu vào ra và trả kết quả. Điều quan trọng ở đây là nó không giữ luật nghiệp vụ. Khi lớp này phình to, mọi quy tắc quan trọng sẽ bị rải khắp các bộ điều khiển và hệ thống rất khó kiểm thử. **Lớp điều phối ứng dụng** là nơi ghép các bước của từng ca sử dụng: mở giao dịch, gọi đúng đối tượng miền nghiệp vụ, điều phối kho lưu trữ và quyết định điểm nào cần phát sự kiện. Lớp này không nên chứa bản thân các bất biến nghiệp vụ, nhưng phải đủ gần nghiệp vụ để điều phối đúng trình tự. **Lớp miền nghiệp vụ** mới là nơi cư trú của các đối tượng gốc, chính sách, quy tắc và bất biến cốt lõi. Đây là trái tim của thiết kế. Cuối cùng, **lớp hạ tầng kỹ thuật** giữ các chi tiết thay đổi nhanh như cơ sở dữ liệu, cổng thanh toán, kênh gửi thông báo và cơ chế phát sự kiện.

Sự phân lớp này đặc biệt quan trọng với IRMS vì hệ thống có nhiều loại thay đổi với tốc độ khác nhau. Quy trình thu ngân có thể thay đổi nhẹ theo chính sách nhà hàng; cách tính phí và khuyến mãi có thể đổi theo mùa; đối tác thanh toán hoặc máy in biên lai có thể thay. Nếu không có ranh giới giữa miền nghiệp vụ và chi tiết công nghệ, mọi thay đổi dạng đó sẽ lan thẳng vào lõi. Hình này cho thấy kiến trúc đã chủ động chặn điều đó: bộ kết nối ngoài bị giữ ở rìa, kho lưu trữ bị ẩn sau giao diện trừu tượng, và phần quyết định nghiệp vụ được bảo vệ ở trung tâm.

Một giá trị khác của hình là nó chứng minh rằng các nguyên lý thiết kế được nói ở phần sau không chỉ mang tính khẩu hiệu. Khi quan sát sơ đồ này, người đọc có thể thấy trực tiếp nơi áp dụng trách nhiệm đơn nhất, nơi áp dụng phân tách giao diện và nơi áp dụng đảo ngược phụ thuộc. Nói cách khác, đây không chỉ là sơ đồ đẹp; đây là sơ đồ dùng để kiểm tra xem những lời biện minh về thiết kế có thực sự được phản ánh vào cấu trúc hay không.

Các thành phần chính thể hiện trong hình.

- **Lớp tiếp nhận:** tiếp nhận yêu cầu, chuyển đổi dữ liệu vào và dữ liệu ra, nhưng không giữ luật nghiệp vụ.
- **Lớp điều phối ứng dụng:** điều phối từng ca sử dụng, kiểm soát ranh giới giao dịch và gọi đúng đối tượng miền nghiệp vụ.
- **Lớp miền nghiệp vụ:** chứa thực thể gốc, chính sách nghiệp vụ, quy tắc kiểm tra và các bất biến cốt lõi.
- **Lớp hạ tầng kỹ thuật:** cung cấp lưu trữ, công bố sự kiện, kết nối cơ sở dữ liệu và tích hợp với hệ thống ngoài.
- **Kho lưu trữ, bộ kết nối, trục sự kiện nội bộ:** là các thành phần kỹ thuật cụ thể, được đặt ở lớp hạ tầng thay vì chen vào lõi.

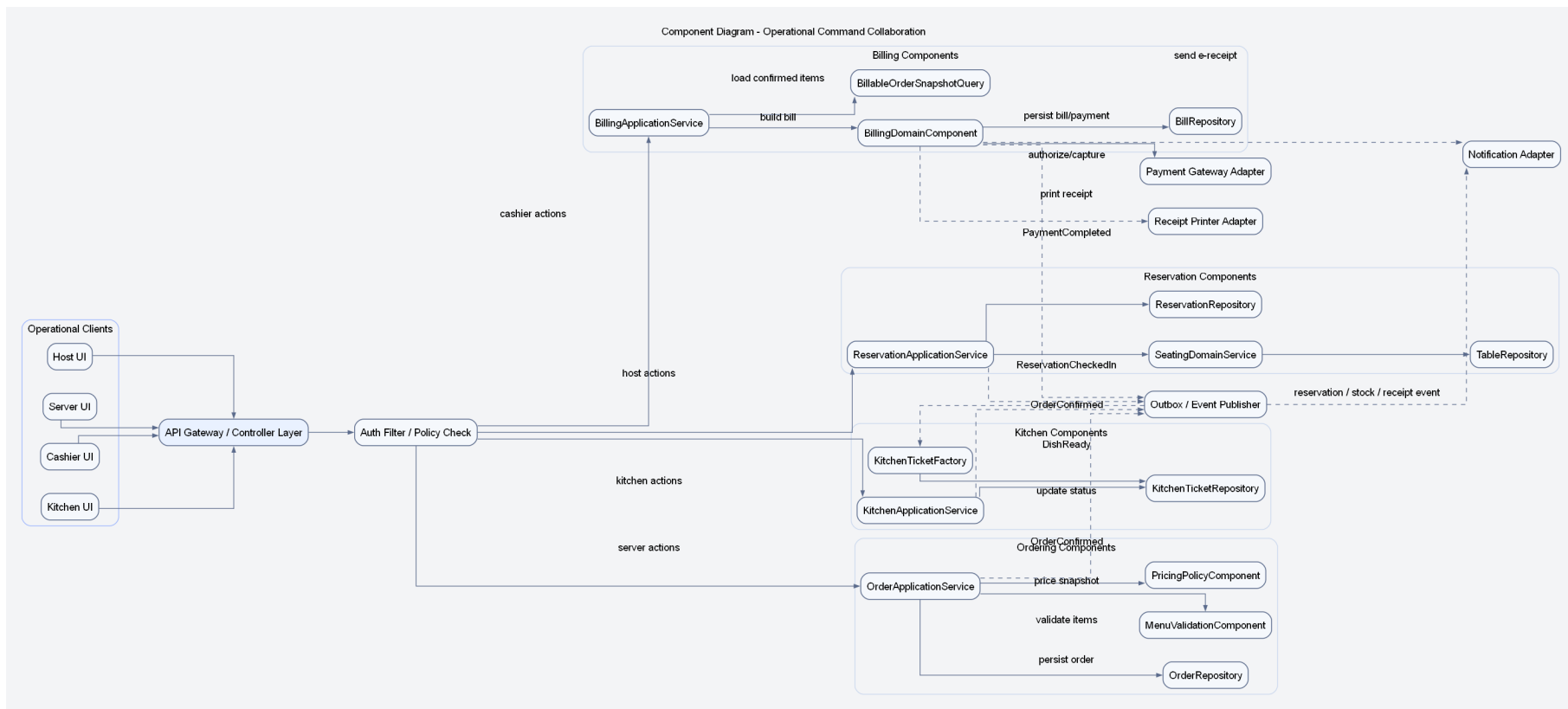
Lý do thiết kế của sơ đồ này.

- Sơ đồ chứng minh khối xử lý phía sau không phải một khối mờ, mà có cấu trúc nội bộ chặt chẽ và có chiều phụ thuộc rõ ràng.
- Việc tách lớp tiếp nhận, lớp điều phối ứng dụng, lớp miền nghiệp vụ và lớp hạ tầng giúp truy ra chính xác nơi nên đặt từng loại thay đổi.
- Đây là sơ đồ then chốt để nối góc nhìn thành phần với phần thiết kế chi tiết và phần cài đặt.

Điểm mạnh lớn nhất của cấu trúc này là khả năng giữ cho hệ thống phát triển lâu dài mà không bị rối. Đổi lại, nó đòi hỏi kỷ luật tổ chức mã nguồn ngay từ đầu: nhóm phát triển phải chấp nhận viết nhiều giao diện hơn, nhiều lớp điều phối hơn và nhiều lớp chuyển đổi dữ liệu hơn. Trong bối cảnh của IRMS, đó là chi phí hợp lý để đổi lấy một lõi nghiệp vụ không bị bào mòn bởi thay đổi công nghệ.



Sơ đồ thành phần cho tuyến lệnh vận hành



Hình 21: Sơ đồ thành phần cho tuyến lệnh vận hành của IRMS

Hình 21 được đưa vào để giải thích thật rõ đường đi của một lệnh vận hành điển hình, từ lúc người dùng thao tác trên giao diện đến lúc dữ liệu lỗi được ghi bền vững và các hệ quả phụ được chuẩn bị để xử lý tiếp. Đây là sơ đồ rất quan trọng vì IRMS không chỉ cần ?xử lý được?, mà còn cần xử lý theo một trình tự có thể giải thích và kiểm toán được.

Điểm đáng chú ý đầu tiên là yêu cầu đi qua lớp kiểm soát ở biên trước khi chạm vào lõi. Bộ lọc xác thực và kiểm tra chính sách không phải là chi tiết phụ. Chúng quyết định ai được phép gọi thao tác nào, trong bối cảnh nào, và với dữ liệu nào. Sau khi qua biên này, yêu cầu đi vào dịch vụ ứng dụng của từng vùng như **Reservation**, **Ordering**, **Kitchen** hoặc **Billing**. Dịch vụ ứng dụng ở đây chỉ nên điều phối: nó gọi đúng đối tượng miền nghiệp vụ, mở và đóng giao dịch, và quyết định thời điểm thích hợp để ghi hộp thư ra hoặc phát tín hiệu cho các phần hỗ trợ.

Điểm quan trọng thứ hai là sơ đồ tách rất rõ giữa **hoàn thành giao dịch lõi** và **xử lý hệ quả phụ**. Một thao tác như xác nhận đơn gọi món hay ghi nhận thanh toán phải đạt trạng thái bền vững trước. Chỉ sau khi phần đó an toàn, hệ thống mới nên phát thông tin sang bếp, làm mới báo cáo, gửi thông báo hoặc cập nhật phần đọc. Cách tổ chức này là nền tảng của luận điểm ?lõi đồng bộ, hỗ trợ bất đồng bộ?. Nó cho phép IRMS phản hồi nhanh ở tuyến đầu mà không đánh đổi tính đúng đắn của dữ liệu.

Điểm đáng đọc thứ ba là vị trí của các bộ kết nối như **Payment Gateway Adapter**, **Receipt Printer Adapter** và **Notification Adapter**. Chúng xuất hiện ở rìa thay vì ở giữa. Điều đó nói lên rằng cổng thanh toán, máy in hay kênh gửi thông báo không được phép định hình cấu trúc miền nghiệp vụ. Miền nghiệp vụ chỉ biết các năng lực mà nó cần; cách kết nối cụ thể được để cho lớp ngoài đảm nhiệm. Đây chính là điều làm cho thiết kế giữ được khả năng thay thế nhà cung cấp và cô lập lỗi khi tích hợp ngoài gặp trục trặc.

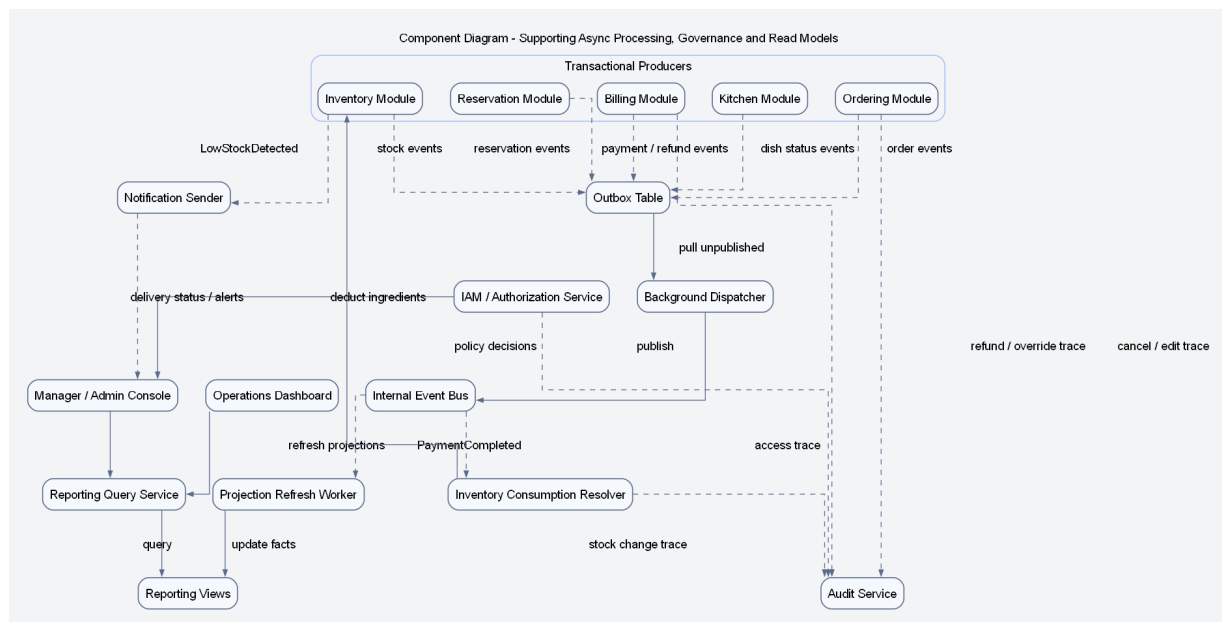
Các thành phần chính thể hiện trong hình.

- **Cổng vào giao diện lập trình ứng dụng và lớp bộ điều khiển** cùng **bộ lọc xác thực / kiểm tra chính sách**: tiếp nhận yêu cầu, chuẩn hóa dữ liệu vào, xác thực và chặn hành vi vượt quyền trước khi đi vào lõi.
- **Các dịch vụ ứng dụng** của từng vùng **Reservation**, **Ordering**, **Kitchen** và **Billing**: điều phối ca sử dụng, gọi đúng thành phần miền nghiệp vụ và kiểm soát biên giao dịch.
- **Các thành phần miền nghiệp vụ** và **kho lưu trữ**: hiện thực quy tắc miền và nơi lưu trạng thái bền vững.
- **Hộp thư ra / bộ phát sự kiện** cùng các bộ kết nối như **Payment Gateway Adapter**, **Receipt Printer Adapter** và **Notification Adapter**: tách bước chốt giao dịch khỏi bước tích hợp hoặc xử lý nền.

Lý do cần thêm sơ đồ này.

- Nó mô tả đường đi của một lệnh nghiệp vụ cụ thể, thay vì chỉ liệt kê danh mục thành phần.
- Nó bảo vệ luận điểm kiến trúc rằng những gì cần xác nhận ngay phải được chốt ở lõi trước, còn các hệ quả phụ đi sau qua cơ chế có kiểm soát.
- Nó làm rõ nơi đặt kiểm tra chính sách, giúp bảo mật và kiểm toán trở thành một phần hữu cơ của kiến trúc, không phải đoạn xử lý thêm về sau.

Sơ đồ thành phần cho xử lý nền, quản trị và mô hình đọc



Hình 22: Sơ đồ thành phần cho xử lý nền, quản trị và mô hình đọc

Hình 22 mô tả nửa còn lại của bức tranh thành phần: phần xử lý nền, phần quản trị và phần mô hình đọc. Nếu sơ đồ trước giải thích đường lệnh, sơ đồ này giải thích chuyện gì xảy ra sau khi giao dịch lỗi đã được chốt an toàn. Đây là nơi tư duy kiến trúc của IRMS bộc lộ rất rõ: hệ thống không đẩy mọi thứ vào đồng bộ, nhưng cũng không tùy tiện đưa mọi thao tác sang nền.

Điều cốt lõi của hình là nguyên tắc **sự kiện là hậu quả của giao dịch, không phải giao dịch tự nó**. Khi một thao tác lỗi hoàn tất, dữ liệu nghiệp vụ được ghi trước; sau đó Outbox Table, Background Dispatcher và Internal Event Bus mới tiếp nhận phần việc tiếp theo. Cách tổ chức này giúp giảm đáng kể nguy cơ mất đồng bộ kiểu ?thông báo đã gửi nhưng giao dịch gốc chưa được ghi? hoặc ?báo cáo đã cập nhật nhưng trạng thái gốc chưa chắc chắn?. Đối với một hệ thống có cả giao dịch tiền bạc lẫn điều phối vận hành như IRMS, đây là nguyên tắc sống còn.

Phần thứ hai cần chú ý là cách Projection Refresh Worker, Reporting Query Service và Reporting Views được đặt thành một chuỗi đọc riêng. Điều đó thể hiện một quyết định rõ ràng: báo cáo và bảng điều khiển không được tranh chấp trực tiếp với mô hình ghi nóng. Nhà quản lý có thể cần nhiều bộ lọc, nhiều thống kê, nhiều lát cắt dữ liệu theo giờ, theo ca, theo nhân viên hoặc theo trạm bếp; nhưng những nhu cầu đó không nên làm chậm giao diện phục vụ tại bàn, màn hình bếp hay giao diện thu ngân. Khi mô hình đọc được tách riêng, hệ thống có thể tối ưu phần đọc cho phân tích mà không làm ô nhiễm phần ghi giao dịch.

Phần thứ ba là vai trò của các thành phần quản trị như Audit Service, IAM / Authorization Service và Notification Sender. Hình này nhấn mạnh rằng kiểm toán, phân quyền và gửi thông báo không phải là những đoạn phụ dán thêm vào cuối chương trình. Chúng là các thành phần có vị trí kiến trúc rõ ràng, nhận dữ liệu từ tuyến giao dịch qua cơ chế có kiểm soát và thực hiện trách nhiệm của mình theo nhịp phù hợp. Chính nhờ cách tổ chức này mà IRMS vừa có thể vận hành mượt ở tuyến đầu, vừa giữ được năng lực kiểm soát, truy vết và quản trị ở tuyến sau.

Các thành phần chính thể hiện trong hình.

- **Các thành phần sinh giao dịch:** là nơi phát sinh sự kiện nghiệp vụ từ các mô-đun ghi như đặt bàn, gọi món, bếp, thanh toán và tồn kho.
- **Outbox Table, Background Dispatcher và Internal Event Bus:** tạo đường chuyển tiếp có kiểm soát từ giao dịch sang xử lý nền.

- **Projection Refresh Worker, Reporting Query Service và Reporting Views:** xây dựng phía đọc cho bảng điều khiển và báo cáo quản trị.
- **Audit Service, IAM / Authorization Service và Notification Sender:** hiện thực các trách nhiệm quản trị và tích hợp ngoài mà không ép chúng phải đồng giao dịch với gọi món hoặc thanh toán.

Điểm kiến trúc được nhấn mạnh bởi sơ đồ.

- Giao dịch lỗi phải được hoàn tất trước; phần còn lại đi sau theo một đường có thể thử lại, giám sát và truy vết.
- Báo cáo đọc từ mô hình đọc riêng, không tranh chấp trực tiếp với mô hình ghi trong giờ cao điểm.
- Kiểm toán và phân quyền được đặt đúng chỗ trong kiến trúc, cho thấy chúng là thành phần hạng nhất chứ không phải tiện ích phụ.

3.7 Góc nhìn triển khai

Ở góc nhìn triển khai, hệ thống được ánh xạ lên hạ tầng vật lý và hạ tầng chạy chương trình như sau:

1. Tầng thiết bị đầu cuối

- Máy tính bảng dành cho nhân viên phục vụ.
- Thiết bị lễ tân hoặc ki-ốt tiếp nhận khách.
- Màn hình bếp tại từng trạm chế biến.
- Máy thu ngân.
- Trình duyệt của quản lý.

2. Tầng cổng vào và phân phối

- Bộ đảo ngược lưu lượng hoặc **API Gateway**.
- Máy chủ phân phối giao diện tĩnh hoặc giao diện phía máy chủ.

3. Tầng ứng dụng

- Khối xử lý phía sau chứa các mô-đun đặt bàn, gọi món, điều phối bếp, thanh toán, tồn kho, báo cáo và quản lý truy cập.
- Bộ xử lý nền thực hiện tác vụ sự kiện, hàng đợi thử lại và làm mới ảnh chụp dữ liệu báo cáo.

4. Tầng dữ liệu

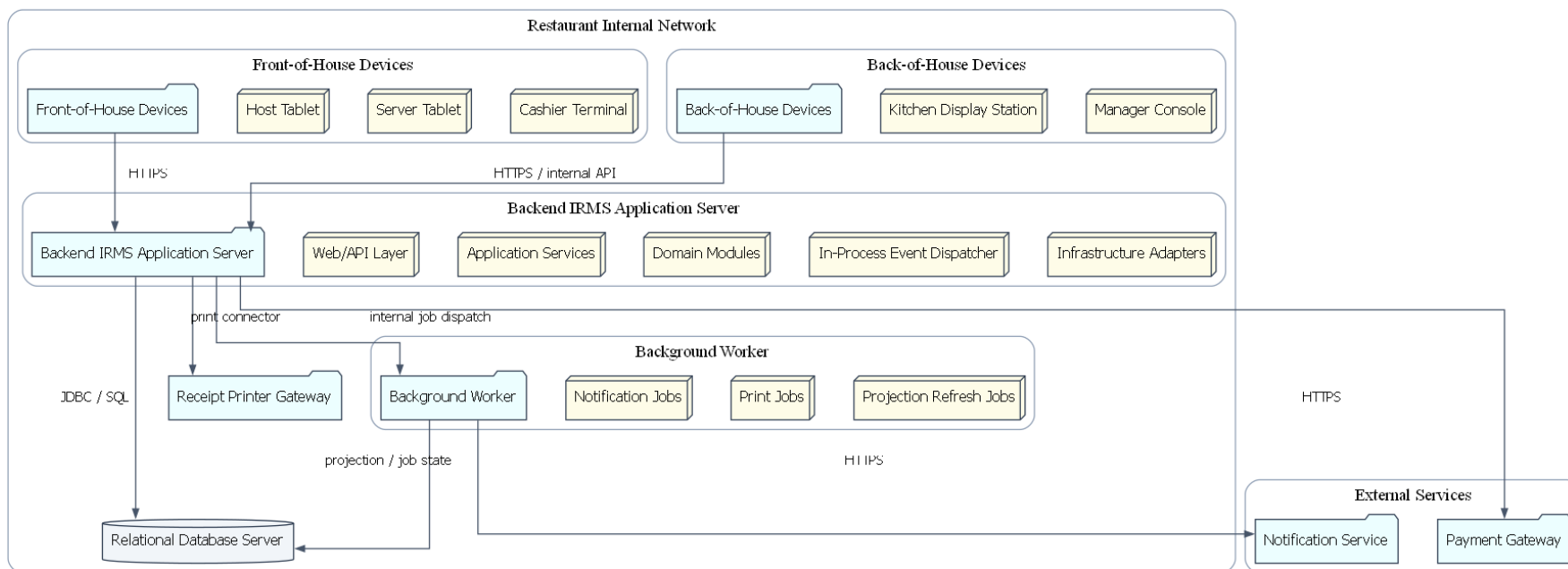
- Cơ sở dữ liệu giao dịch hoặc lược đồ giao dịch theo từng miền.
- Mô hình đọc và khung nhìn báo cáo.
- Kho lưu trữ đối tượng cho biên lai xuất ra hoặc tài nguyên của thực đơn.
- Hạ tầng nhật ký, số đo và vết thực thi.

Lý do thiết kế

- Thiết bị gọi món và thiết bị bếp phải đặt gần môi trường mạng nội bộ của nhà hàng và có cơ chế kết nối lại tốt vì đây là hai điểm bị ảnh hưởng trực tiếp nhất khi mạng dao động.
- Khối thanh toán và khối gọi món có yêu cầu sẵn sàng cao hơn các khối còn lại, nên cần được ưu tiên về giám sát, sao lưu và khả năng mở rộng.

- Khối báo cáo cần tách khỏi tuyến giao dịch để truy vấn phân tích không cản trở đường xử lý gọi món và thanh toán.
- Các nút dữ liệu phải có sao lưu và chính sách phân quyền riêng; dữ liệu thanh toán, hoàn tiền và nhật ký kiểm toán là nhóm cần ưu tiên bảo vệ cao.
- Việc tách bộ xử lý nền khối khỏi xử lý yêu cầu trực tuyến giúp những tác vụ như gửi thông báo, in biên lai điện tử hoặc cập nhật báo cáo không tranh chấp tài nguyên với thao tác tại quầy.

Giải thích chi tiết cho góc nhìn triển khai. Góc nhìn triển khai giúp chứng minh rằng kiến trúc được chọn không chỉ hợp lý trên sơ đồ logic mà còn có thể triển khai thực tế. Với IRMS, bản chất *nhiều điểm chạm vật lý* — quầy lễ tân, bàn phục vụ, trạm bếp, quầy thu ngân — làm cho góc nhìn triển khai quan trọng hơn nhiều so với các hệ thống chỉ có một giao diện web thông thường.



Hình 23: Sơ đồ triển khai tổng quan về bố trí vật lý của IRMS

Hình 23 là sơ đồ triển khai ở mức vật lý, tức mức trả lời câu hỏi hệ thống sẽ hiện diện như thế nào trong nhà hàng thật. Giá trị lớn nhất của hình này là nó kéo kiến trúc ra khỏi trạng thái trừu tượng thuần túy. Khi nhìn vào sơ đồ, người đọc không còn thấy các mô-đun dưới dạng khái niệm, mà thấy chúng gắn vào máy tính bảng gọi món, thiết bị lễ tân, màn hình bếp, quầy thu ngân, khối xử lý trung tâm, bộ xử lý nền, cơ sở dữ liệu và các hệ thống ngoài.

Điều quan trọng nhất cần rút ra là IRMS có **nhiều điểm chạm vật lý nhưng một lõi điều phối tập trung**. Đây là lựa chọn rất phù hợp với phạm vi của bài toán. Nhà hàng cần nhiều thiết bị tại hiện trường để thao tác gần nơi công việc diễn ra nhất, nhưng lại cần một lõi chung để giữ trạng thái thống nhất của bàn, đơn gọi món, bếp và hóa đơn. Nếu đẩy quá nhiều quyết định xuống từng thiết bị cục bộ, hệ thống sẽ rất dễ mất nhất quán khi mạng dao động hoặc khi hai vai trò thao tác chồng thời điểm. Vì vậy, hình này làm rõ rằng thiết bị đầu cuối nên gần người dùng, còn quyết định nghiệp vụ bền vững phải được neo ở khối xử lý trung tâm và hạ tầng dữ liệu.

Sơ đồ cũng cho thấy vì sao bộ xử lý nền cần đứng riêng. Nhiều tác vụ như gửi thông báo, phát hành biên lai điện tử, làm mới báo cáo hoặc xử lý một số hệ quả phụ không cần tranh tài nguyên trực tiếp với thao tác ở bàn, ở bếp hay ở quầy thu ngân. Khi chúng được tách ra, hệ thống giữ được nhịp phản hồi ổn định hơn ở tuyến trước khách hàng. Đây là lựa chọn triển khai có ý nghĩa thực tế rất lớn, vì giờ cao điểm của nhà hàng thường không cho phép những tác vụ phụ chen vào tuyến chính.

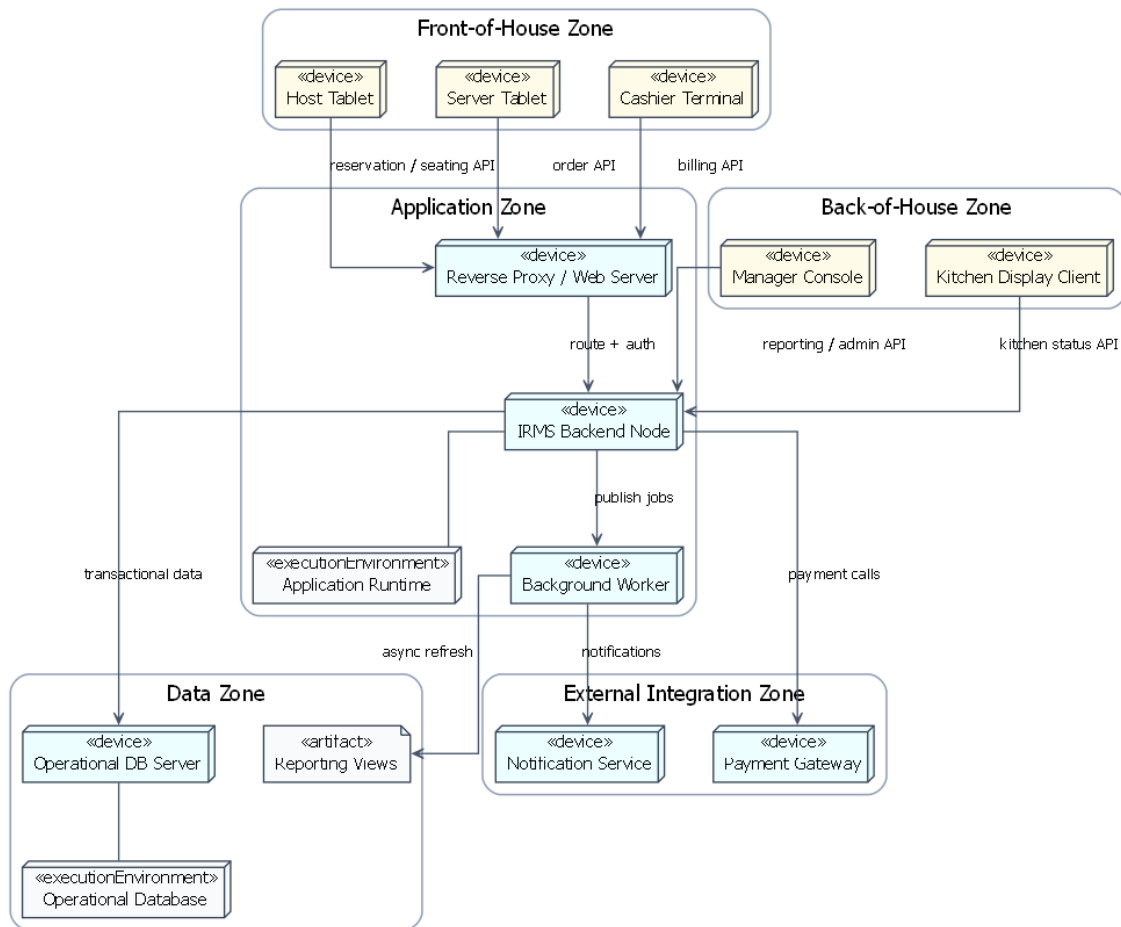
Các thành phần chính thể hiện trong hình.

- **Thiết bị đầu cuối:** gồm máy tính bảng gọi món, thiết bị lễ tân, màn hình bếp, máy thu ngân và trình duyệt quản lý.
- **Khối cổng vào:** tiếp nhận yêu cầu từ các thiết bị và chuyển tiếp vào khối xử lý.
- **Khối xử lý phía sau:** thực hiện các chức năng nghiệp vụ chính của hệ thống.
- **Bộ xử lý nền:** nhận tác vụ không cần phản hồi ngay như gửi thông báo hoặc cập nhật báo cáo.
- **Hạ tầng dữ liệu:** lưu trữ dữ liệu giao dịch, dữ liệu báo cáo và tệp đính kèm.
- **Các hệ thống ngoài:** gồm cổng thanh toán, kênh gửi thông báo và thiết bị hoặc dịch vụ in biên lai.

Lý do thiết kế của sơ đồ này.

- Bố trí một khối xử lý trung tâm giúp đơn giản hóa triển khai, dễ kiểm thử đầu-cuối và dễ giữ ranh giới giao dịch gọn.
- Tách bộ xử lý nền khỏi khối tiếp nhận yêu cầu trực tuyến giúp tác vụ phụ không tranh chấp quá mạnh với yêu cầu tại quầy.
- Đặt các hệ thống ngoài ở rìa giúp phân biệt rõ phần lõi của IRMS với phần tích hợp phụ thuộc đối tác.

Phương án này mạnh ở chỗ đơn giản và dễ vận hành, rất phù hợp với bài toán một nhà hàng hoặc một nhóm nhỏ chi nhánh. Đổi lại, khối xử lý trung tâm trở thành vùng cần được giám sát và bảo vệ cẩn thận hơn, vì nó là nơi hội tụ của nhiều quyết định nghiệp vụ.



Hình 24: Sơ đồ triển khai theo vùng mạng và ranh giới tin cậy

Hình 24 làm rõ góc nhìn mà sơ đồ vật lý tổng quan mới chỉ chạm tới ở mức khái quát: **ranh giới tin cậy**. Với một hệ thống như IRMS, bảo mật không chỉ là câu chuyện tài khoản và mật khẩu. Nó còn là câu chuyện thành phần nào được đặt ở vùng mạng nào, ai được phép nói chuyện trực tiếp với ai, và dữ liệu nhạy cảm nằm ở đâu trong toàn bộ mặt bằng triển khai.

Việc tách thành vùng phục vụ phía trước, vùng bếp, vùng ứng dụng, vùng dữ liệu và vùng tích hợp ngoài giúp kiến trúc thể hiện rõ những lớp bảo vệ khác nhau. Thiết bị ở hiện trường phải đủ gần nghiệp vụ để thao tác nhanh, nhưng không nên có quyền đi thẳng tới cơ sở dữ liệu. Cơ sở dữ liệu lại phải ở một vùng được bảo vệ hơn, tránh truy cập trực tiếp từ thiết bị đầu cuối hoặc từ các kết nối ngoài. Các hệ thống đối tác như cổng thanh toán hay dịch vụ gửi thông báo nằm ngoài biên tin cậy nội bộ, nên mọi lưu lượng đi qua chúng đều phải được kiểm soát, ghi nhật ký và hạn chế quyền truy cập. Nhờ vậy, sơ đồ này làm nổi bật một nguyên tắc quan trọng: không có ?một vùng an toàn duy nhất?, mà có nhiều lớp biên, mỗi lớp có mức tin cậy khác nhau.

Sơ đồ còn có giá trị vận hành thực tế. Khi xảy ra sự cố, cách chia vùng giúp khoanh nhanh câu hỏi: lỗi nằm ở lớp thiết bị hiện trường, ở vùng ứng dụng hay ở kết nối với hệ ngoài. Tương tự, khi thiết kế tường lửa, mã hóa đường truyền, nhật ký truy cập hay chính sách cấp quyền mạng, chính sơ đồ vùng mạng là cơ sở để quyết định cái gì phải bị chặn tuyệt đối, cái gì được phép đi qua một cổng duy nhất và cái gì chỉ được mở từ một vùng đã kiểm soát.

Các vùng chính thể hiện trong hình.

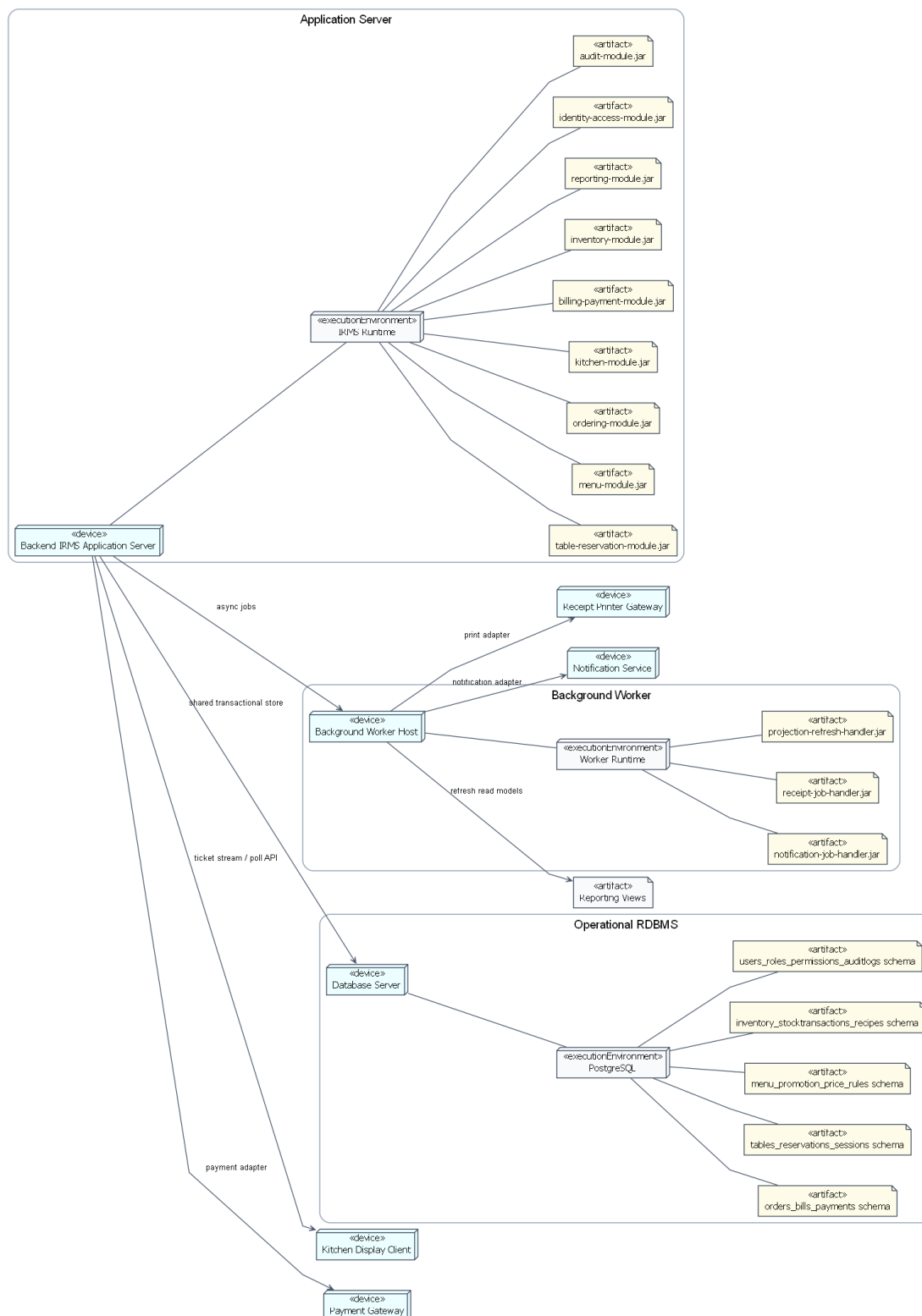
- **Vùng phục vụ phía trước:** chứa thiết bị của lễ tân, phục vụ và thu ngân.
- **Vùng bếp:** chứa màn hình bếp và các thiết bị phục vụ chế biến.

- **Vùng ứng dụng:** chứa cổng vào, khối xử lý phía sau và bộ xử lý nền.
- **Vùng dữ liệu:** chứa cơ sở dữ liệu giao dịch, mô hình đọc và kho lưu trữ đối tượng.
- **Vùng tích hợp ngoài:** chứa cổng thanh toán, dịch vụ gửi thông báo và các hệ thống ngoài khác.

Lý do thiết kế của sơ đồ này.

- Sơ đồ làm rõ nơi nào là mạng nội bộ hiện trường, nơi nào là vùng ứng dụng, nơi nào là vùng dữ liệu nhạy cảm và nơi nào là thế giới bên ngoài.
- Việc tách vùng dữ liệu thành một lớp riêng giúp giảm nguy cơ truy cập trực tiếp từ thiết bị đầu cuối hoặc từ Internet.
- Mọi tích hợp ngoài đi qua khối ứng dụng giúp tăng khả năng kiểm soát, ghi nhật ký và áp dụng chính sách bảo vệ dữ liệu.

Ưu điểm của cách biểu diễn theo vùng là giúp người đọc hiểu được nơi nào là nội bộ nhà hàng, nơi nào là biên kết nối ra ngoài và nơi nào là dữ liệu nhạy cảm. Nó cũng hỗ trợ suy nghĩ về tường lửa, cắt lớp mạng, mã hóa đường truyền, ghi nhật ký và ứng phó sự cố. Nhược điểm là sơ đồ này không diễn tả chi tiết luồng dữ liệu nghiệp vụ như sơ đồ trình tự, nên cần được đọc cùng góc nhìn sử dụng và góc nhìn thành phần.



Hình 25: Sơ đồ triển khai về ánh xạ dữ liệu và thực thi

Hình 25 là sơ đồ có ý nghĩa quyết định khi cần biện minh kiểu kiến trúc được chọn ở mức thực thi. Nó cho thấy các miền nghiệp vụ được đóng gói như các mô-đun trong cùng một khối xử lý trực tuyến, trong khi bộ xử lý nền đảm nhiệm các công việc chấp nhận độ trễ nhỏ hơn như gửi thông báo, phát hành biên lai điện tử và làm mới mô hình đọc. Nói cách khác, đây là sơ đồ chứng minh IRMS đi theo **đơn khối theo mô-đun** ở mức thực thi, chứ không phải một hệ phân tán nặng

ngay từ đầu.

Điểm mạnh nhất của cách bố trí này là giữ được cả hai lợi ích tưởng như khó đi cùng nhau. Một mặt, ranh giới mô-đun vẫn rõ nên hệ thống không trượt thành một khối mã hỗn loạn. Mặt khác, các giao dịch lỗi như tạo đơn gọi món, cập nhật trạng thái bếp, lập hóa đơn và thanh toán vẫn diễn ra trong một không gian thực thi tập trung hơn, từ đó tránh được rất nhiều phức tạp của giao dịch phân tán, đồng bộ lược đồ và phối hợp lỗi qua nhiều tiến trình mạng. Đây là lựa chọn rất hợp lý cho bối cảnh bài toán và quy mô dự án.

Sơ đồ cũng làm rõ vai trò của mô hình đọc và kho lưu trữ đối tượng. Mô hình đọc không phải là bản sao tùy hứng của dữ liệu giao dịch, mà là một mặt cắt phục vụ truy vấn quản trị. Kho lưu trữ đối tượng lại là nơi phù hợp cho biên lai, tệp đính kèm hoặc tài nguyên thực đơn khi cần. Nhờ tách như vậy, khối xử lý trực tuyến không phải ôm mọi nhu cầu lưu trữ vào cùng một hình thức.

Các thành phần chính thể hiện trong hình.

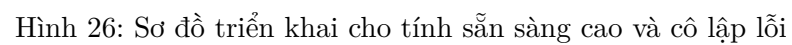
- **Khối xử lý trực tuyến:** chứa các mô-đun nghiệp vụ phục vụ yêu cầu thời gian thực.
- **Bộ xử lý nền:** xử lý các tác vụ chấp nhận độ trễ nhỏ.
- **Lược đồ giao dịch:** lưu dữ liệu giao dịch cốt lõi của hệ thống.
- **Mô hình đọc và dữ liệu báo cáo:** tối ưu cho truy vấn quản trị và thống kê.
- **Kho lưu trữ đối tượng:** giữ biên lai, tệp đính kèm hoặc tài nguyên thực đơn nếu có.

Lý do thiết kế của sơ đồ này.

- Đóng gói nhiều miền nghiệp vụ trong cùng một khối xử lý giúp giữ giao dịch lỗi đơn giản hơn so với việc tách thành nhiều dịch vụ mạng độc lập ngay từ đầu.
- Tách mô hình đọc khỏi dữ liệu giao dịch giúp báo cáo phong phú hơn mà không làm nặng tuyến xử lý trực tuyến.
- Giữ bộ xử lý nền ở ngoài khối trực tuyến giúp hệ thống vừa rõ ràng về thực thi, vừa linh hoạt khi sau này cần tách riêng một số năng lực lớn.

Ưu điểm của cách tổ chức này là thực dụng, nhất quán và phù hợp với phạm vi đề tài. Đổi lại, kiến trúc phải được bảo vệ bằng kỷ luật mô-đun bên trong; nếu không, việc cùng chạy trong một khối xử lý có thể dẫn tới tình trạng phụ thuộc chéo quá mức.

Sơ đồ triển khai cho tính sẵn sàng cao và cô lập lỗi



Hình 26 đi sâu vào một câu hỏi thường được đặt ra khi đánh giá kiến trúc vận hành: nếu một nút ứng dụng, một bộ xử lý nền hoặc một thành phần tích hợp gặp sự cố, IRMS còn giữ được điều gì và suy giảm dịch vụ ra sao. Đây là sơ đồ rất đáng giá vì nó buộc thiết kế phải trả lời về hành vi khi lỗi xảy ra, thay vì chỉ mô tả trạng thái lý tưởng khi mọi thứ đều hoạt động.

Điểm đầu tiên cần rút ra là **tuyến yêu cầu trực tuyến** và **tuyến xử lý nền** đều có phương án dự phòng. Bộ chuyển tiếp ngược hoặc bộ cân bằng tải đặt trước các nút ứng dụng giúp ứng dụng khách không gắn cứng vào một nút cụ thể. Nếu một nút ứng dụng lỗi, lưu lượng có thể được đưa sang nút còn lại mà không làm người dùng phải thay đổi điểm truy cập. Tương tự, việc có nhiều nút bộ xử lý nền giúp các tác vụ không bị tắc nghẽn toàn bộ khi một bộ tiêu thụ gặp lỗi.

Điểm thứ hai là sơ đồ làm rõ cách dữ liệu được bảo vệ. PostgreSQL Primary kết hợp với Read Replica / Warm Standby cho thấy dữ liệu giao dịch lỗi không nên chỉ có một nơi duy nhất. Đồng thời, phần đọc báo cáo và phần xử lý nền được đi qua hàng đợi hoặc hộp thư ra, nhờ đó sự cố ở bảng điều khiển hoặc một nhiệm vụ nền không làm sập ngay tuyến giao dịch. Đây là một dạng cô lập lỗi rất quan trọng trong nhà hàng: nếu kênh thông báo bị chậm, khách vẫn phải thanh toán được; nếu báo cáo chưa kịp làm mới, bàn vẫn phải mở và món vẫn phải ra đúng.

Điểm thứ ba là sơ đồ thể hiện rõ **đường suy giảm dịch vụ chấp nhận được**. Không phải mọi lỗi đều phải dẫn tới dừng toàn hệ thống. Một số lỗi chỉ nên làm chậm phần đọc, làm chậm thông báo hoặc chuyển một số tác vụ sang hàng chờ. Khả năng chấp nhận suy giảm có kiểm soát như vậy mới là dấu hiệu của một hệ thống vận hành trưởng thành.

Các ý chính cần rút ra từ sơ đồ.

- **Bộ chuyển tiếp ngược / bộ cân bằng tải** tách lớp ứng dụng khách khỏi từng nút ứng dụng riêng lẻ, nên lỗi một nút không đồng nghĩa lỗi toàn bộ điểm vào.
- **IRMS App Node A/B** cùng **Worker Node A/B** cho phép dự phòng ở cả tuyến xử lý yêu cầu và tuyến xử lý nền.
- **PostgreSQL Primary** cùng **bản sao đọc / nút dự phòng ấm** hỗ trợ tách truy vấn đọc hoặc phục hồi nhanh hơn sau lỗi nút chính.
- **Outbox / Job Queue** ngăn các tác vụ nền và việc làm mới mô hình chiếu làm nghẽn trực tiếp tuyến giao dịch.

Vì sao sơ đồ này cần có trong góc nhìn triển khai.

- Góc nhìn triển khai không chỉ nên cho thấy hệ thống đặt ở đâu, mà còn phải cho thấy nó phản ứng ra sao khi có lỗi.
- Với IRMS, nếu cổng thanh toán hoặc dịch vụ gửi thông báo chậm, hệ thống vẫn phải giữ đúng giao dịch đã hoàn tất trong lỗi; sơ đồ này minh họa chính xác cơ chế tách biệt đó.
- Nó cũng làm rõ hướng mở rộng sau này: có thể tăng số nút ứng dụng, nút bộ xử lý nền hoặc năng lực đọc mà không phải thiết kế lại mô hình miền nghiệp vụ.

3.8 Nguyên tắc thiết kế

Kiến trúc của IRMS bám theo các nguyên tắc thiết kế sau:

- **Kết tụ cao, phụ thuộc thấp**: mỗi mô-đun chỉ công khai phần giao diện thật sự cần thiết và tránh làm lộ chi tiết cài đặt.
- **Phân tách mối quan tâm**: bộ điều khiển, dịch vụ ứng dụng, miền nghiệp vụ và hạ tầng kỹ thuật không bị trộn vai trò.
- **Phụ thuộc thông qua lớp trừu tượng**: các tích hợp ngoài đi qua giao diện trừu tượng thay vì để lỗi nghiệp vụ phụ thuộc trực tiếp vào nhà cung cấp hoặc bộ thư viện cụ thể.

- **Lỗi đồng bộ, hỗ trợ bất đồng bộ:** các giao dịch lỗi như tạo đơn gọi món, cập nhật bếp và thanh toán ưu tiên xử lý đồng bộ; các hệ quả phụ như cảnh báo tồn kho, làm mới báo cáo và gửi thông báo đi qua sự kiện hoặc bộ xử lý nền.
- **Mô hình hóa bắt đầu từ miền nghiệp vụ:** sơ đồ lớp phải phản ánh hành vi thật của nhà hàng, không bị giản lược thành sơ đồ quan hệ dữ liệu thuần túy.

Các nguyên tắc trên không chỉ là danh sách khái niệm, mà là bộ tiêu chí nhất quán xuyên suốt toàn bộ tài liệu. Trong góc nhìn phân rã, chúng dẫn tới ranh giới năng lực nghiệp vụ rõ ràng; trong góc nhìn thành phần, chúng dẫn tới sự tách bạch giữa bộ điều khiển, dịch vụ ứng dụng, miền nghiệp vụ và bộ kết nối; trong góc nhìn triển khai, chúng dẫn tới quyết định tách bộ xử lý nền và đường báo cáo khỏi đường xử lý yêu cầu; còn trong sơ đồ lớp, chúng dẫn tới việc ưu tiên thực thể gốc, chính sách và giao diện trừu tượng thay vì chỉ vẽ các bảng dữ liệu rời rạc.

3.9 Áp dụng nguyên lý SOLID vào thiết kế

Bảng 6: Ánh xạ nguyên lý SOLID vào thiết kế của IRMS

Nguyên lý	Quyết định thiết kế chính	Biểu hiện trong IRMS
Trách nhiệm đơn nhất	tách trách nhiệm theo ca sử dụng, thực thể gốc và mối quan tâm tích hợp	bộ điều khiển chỉ tiếp nhận yêu cầu; dịch vụ ứng dụng chỉ điều phối; thực thể gốc giữ bất biến; bộ xử lý nền xử lý tác vụ phụ
Đóng để ổn định, mở để mở rộng	mở các điểm mở rộng qua chiến lược, chính sách và bộ kết nối	thêm phương thức thanh toán, kênh thông báo hoặc quy tắc khuyến mãi mà không phải sửa lỗi giao dịch
Thay thế tương thích	thiết kế hợp đồng rõ cho các lớp trừu tượng tích hợp	mọi cách hiện thực của <code>PaymentGateway</code> , <code>NotificationSender</code> , <code>ReceiptPrinter</code> phải thay thế được nhau mà không làm đổi hành vi mong đợi của lớp dùng chúng
Phân tách giao diện	tránh giao diện lớn kiểu ?làm mọi thứ?	<code>Billing</code> chỉ phụ thuộc cổng thanh toán và biên lai; <code>Inventory</code> chỉ phụ thuộc cổng cảnh báo; <code>Reporting</code> chỉ phụ thuộc giao diện truy vấn phù hợp
Đảo ngược phụ thuộc	giữ chiều phụ thuộc hướng vào lớp trừu tượng	dịch vụ nghiệp vụ phụ thuộc kho lưu trữ, chính sách và cổng kết nối trừu tượng thay vì phụ thuộc trực tiếp vào cách hiện thực cụ thể

Nguyên lý trách nhiệm đơn nhất

Nguyên lý này được áp dụng theo nhiều lớp khác nhau. Ở biên hệ thống, bộ điều khiển chỉ làm nhiệm vụ nhận yêu cầu, kiểm tra sơ bộ và chuyển cho dịch vụ ứng dụng; nó không chứa quy tắc nghiệp vụ cốt lõi. Ở lớp điều phối, mỗi dịch vụ ứng dụng chỉ chịu trách nhiệm cho một nhóm hành vi gần nhau: `OrderingApplicationService` không xử lý hoàn tiền, còn `BillingApplicationService` không làm nhiệm vụ điều phối bếp. Ở miền nghiệp vụ, các thực thể gốc như `Order`, `Bill` hay `InventoryItem` giữ bất biến của riêng mình thay vì đẩy toàn bộ quy tắc vào một lớp trung tâm khổng lồ. Nhờ vậy, khi một chính sách thay đổi, phạm vi sửa đổi được khoanh gọn đúng nơi có trách nhiệm.

Nguyên lý đóng để ổn định, mở để mở rộng

IRMS có nhiều điểm biến động cao: đối tác thanh toán, kênh gửi thông báo, cách phát hành biên lai, quy tắc khuyến mãi, quy tắc cảnh báo mức tồn thấp. Nếu lỗi nghiệp vụ gọi thẳng từng nhà

cung cấp hoặc mã hóa cứng từng biến thể, mỗi thay đổi sẽ buộc phải sửa phần trung tâm. Thiết kế vì thế ưu tiên các lớp trừu tượng như `PaymentGateway`, `NotificationSender`, `ReceiptPrinter`, cùng các chính sách riêng cho định giá, khả dụng và phân quyền. Nhờ đó, hệ thống mở cho mở rộng nhưng vẫn giữ được phần lõi ổn định.

Nguyên lý thay thế tương thích

Nguyên lý này đặc biệt quan trọng ở các bộ kết nối tích hợp. Một cách hiện thực mới của `PaymentGateway` phải tuân thủ cùng một hợp đồng hành vi về giao dịch thành công, thất bại, hoàn tiền, tính chống xử lý lặp và mã tham chiếu. Tương tự, một `NotificationSender` mới có thể gửi thư điện tử, tin nhắn hay thông báo đẩy, nhưng không được khiến dịch vụ ứng dụng phải viết nhánh xử lý đặc biệt chỉ dành riêng cho nó. Khi một lớp trừu tượng được thiết kế đúng, lớp dùng nó không cần biết bộ kết nối phía sau là nhà cung cấp nào.

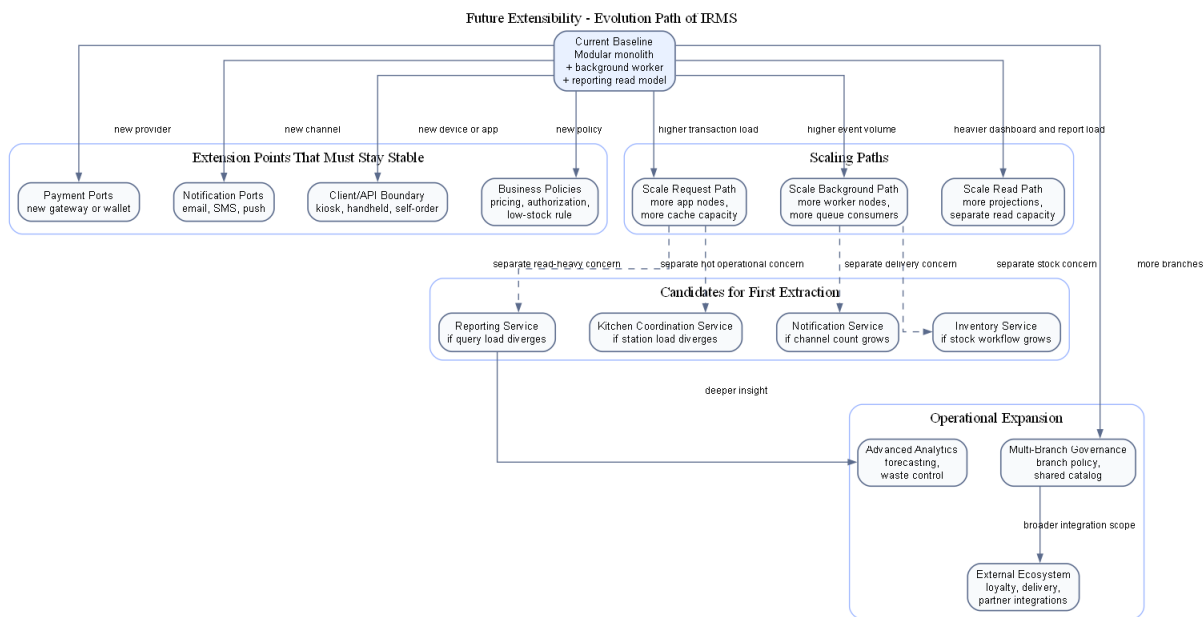
Nguyên lý phân tách giao diện

IRMS tránh các giao diện khổng lồ kiểu “dịch vụ tiện ích tổng hợp” hoặc “quản lý mọi hệ ngoại?”. Thay vào đó, mỗi mô-đun chỉ phụ thuộc vào đúng giao diện mà nó cần. `Billing` chỉ cần cổng thanh toán và cổng biên lai. `Inventory` chỉ cần cổng cảnh báo. `Reporting` chỉ cần giao diện truy vấn hoặc giao diện làm mới mô hình đọc phù hợp. Cách chia này làm cho việc kiểm thử đơn vị gọn hơn, giảm phụ thuộc chéo và tránh việc mô-đun tuyến đầu phải nhìn thấy những giao diện quản trị không liên quan.

Nguyên lý đảo ngược phụ thuộc

Đây là nguyên lý bảo vệ lõi nghiệp vụ khỏi công nghệ cụ thể. Dịch vụ ứng dụng và mô hình miền nghiệp vụ phụ thuộc vào giao diện kho lưu trữ, giao diện chính sách và giao diện bộ kết nối; còn lớp hạ tầng mới hiện thực các giao diện đó bằng `PostgreSQL`, giao diện thanh toán hay nhà cung cấp thông báo cụ thể. Nhờ vậy, chiều phụ thuộc được giữ nhất quán: chi tiết công nghệ phụ thuộc vào nhu cầu nghiệp vụ, chứ không phải nghiệp vụ bị ép đi theo công nghệ. Chính điều này làm cho kiến trúc có thể tiến hóa dần mà không làm nứt phần lõi.

3.10 Khả năng mở rộng trong tương lai



Hình 27: Lộ trình mở rộng kiến trúc của IRMS trong tương lai

Hình 27 tập trung riêng vào câu hỏi mà hầu như mọi đề án kiến trúc đều phải trả lời tường minh: nếu phạm vi vận hành lớn hơn, số thiết bị tăng lên, số chi nhánh tăng lên hoặc yêu cầu tích hợp đa dạng hơn, kiến trúc hiện tại sẽ tiến hóa theo đường nào mà không phải viết lại phần lõi. Hình này không mô tả một trạng thái giả định mơ hồ trong tương lai. Nó mô tả **đường tiến hóa hợp lý** từ kiến trúc hiện tại sang những khả năng mở rộng có xác suất xảy ra cao nhất đối với IRMS.

Ở trạng thái hiện tại, IRMS vận hành hiệu quả nhất như một **modular monolith theo năng lực nghiệp vụ** với bộ xử lý nền và mô hình đọc tách riêng. Đây là nền tảng tốt vì giao dịch lõi vẫn gọn, ranh giới mô-đun vẫn rõ và chi phí vận hành chưa bị đẩy lên mức của hệ phân tán nặng. Từ nền này, hướng tiến hóa hợp lý nhất là tách dần Reporting, Notification và một phần Kitchen Coordination hoặc Inventory ra trước khi nghĩ đến việc tách tuyến giao dịch lõi. Lý do là các vùng đó đã có nhịp xử lý, kiểu tải và mức nhất quán khác với giao dịch đồng bộ ở tuyến đầu.

Điểm quan trọng của hình là nó nêu rõ **điều kiện để mở rộng mà không phá kiến trúc**. Nếu muốn thêm phương thức thanh toán mới, hệ thống chỉ nên chạm vào lớp cổng thanh toán và chính sách liên quan. Nếu muốn thêm kênh thông báo, thay đổi phải chủ yếu nằm ở Notification Integration. Nếu muốn mở thêm chi nhánh, hệ thống cần tăng sức chứa ở lớp cấu hình chi nhánh, vùng dữ liệu và tầng triển khai, nhưng không được buộc viết lại mô hình đơn gọi món hay quyết toán. Nếu muốn có kiosk tự phục vụ hoặc thiết bị cầm tay mới, thay đổi chủ yếu nằm ở lớp client và giao diện lập trình ứng dụng, còn chuỗi miền nghiệp vụ chính vẫn giữ nguyên. Những điều kiện đó chính là thước đo thật sự của extensibility, chứ không chỉ là khả năng thêm lớp mã mới.

Sơ đồ cũng chỉ ra các **điểm mở rộng chiến lược** cần được giữ ổn định từ sớm: cổng thanh toán, cổng gửi thông báo, chính sách phân quyền, quy tắc định giá, quy tắc cảnh báo tồn kho, mô hình đọc báo cáo và các giao diện ứng dụng khách. Khi những điểm này được giữ ở đúng ranh giới, hệ thống có thể tiến hóa dần từng năng lực mà không cần đánh đổi sự ổn định của tuyến lõi. Đây là lý do mục 'Future Extensibility' không nên chỉ được nói bằng một đoạn văn chung chung; nó cần một hình kiến trúc riêng để cho thấy đường phát triển của hệ thống là hữu hình và có kiểm soát.

Các hướng tiến hóa chính thể hiện trong hình.

- **Mở rộng tích hợp:** thêm phương thức thanh toán, kênh thông báo, thiết bị in hoặc thiết bị đầu cuối mới thông qua các cổng và bộ kết nối hiện có.
- **Mở rộng theo tải:** tăng số nút ứng dụng, nút bộ xử lý nền, năng lực đọc báo cáo hoặc tách riêng một số mô-đun có tải đặc thù.
- **Mở rộng theo phạm vi vận hành:** thêm nhiều chi nhánh, thêm chính sách theo chi nhánh và tăng năng lực quản trị tập trung mà không phải làm lại mô hình miền lõi.
- **Mở rộng theo dịch vụ:** chỉ tách thành dịch vụ mạng độc lập với những vùng có nhịp thay đổi hoặc mức tải thực sự khác biệt, thay vì tách đồng loạt vì lý do hình thức.

4 Thiết kế chi tiết

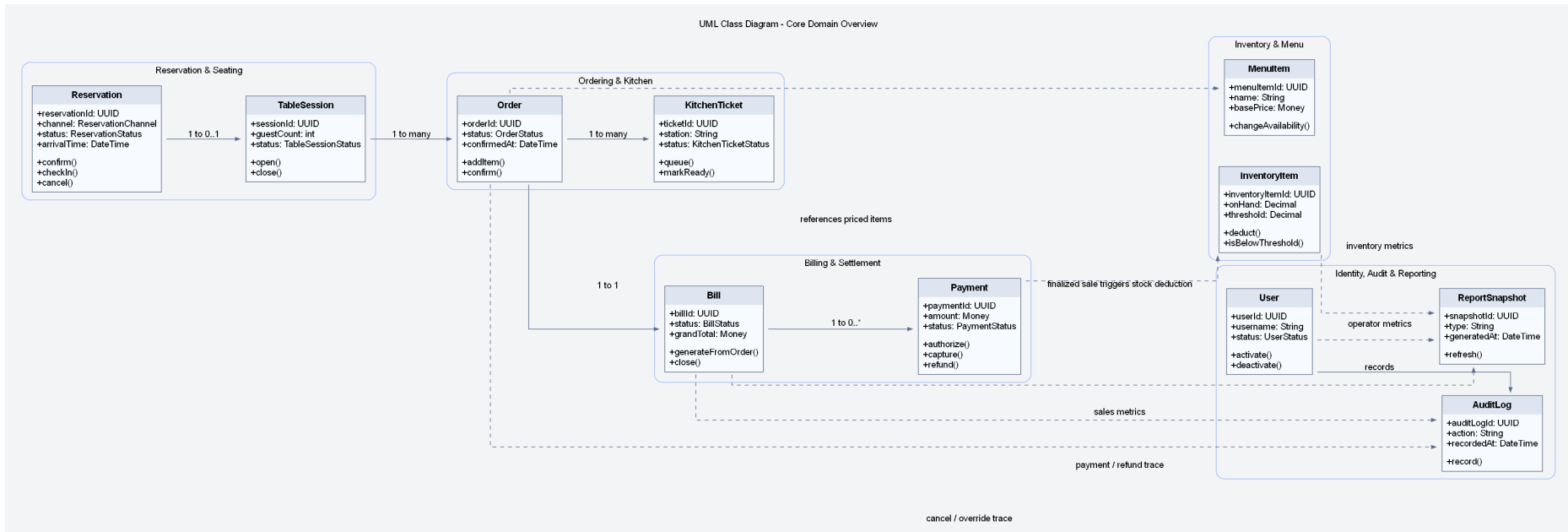
4.1 Góc nhìn lớp

Phạm vi bao phủ của sơ đồ lớp

Sơ đồ lớp được tổ chức theo năm hướng chính để bao phủ đầy đủ miền nghiệp vụ:

1. **Ngữ cảnh khách hàng và chi nhánh:** thêm `CustomerProfile` và `Branch` để đặt bàn, hóa đơn, tồn kho và phân công nhân sự có phạm vi rõ ràng theo địa điểm và khách hàng.
2. **Bao phủ tốt hơn tùy biến món:** thêm `ModifierOption` và `OrderItemModifier` để ghi nhận lựa chọn chi tiết, không chỉ dừng ở mức nhóm modifier.
3. **Bao phủ khuyến mãi và giảm giá:** thêm `PromotionCampaign` và `AppliedDiscount` để hóa đơn phản ánh đúng các chương trình ưu đãi.
4. **Bao phủ nhân sự ca làm:** thêm `ShiftAssignment` vì đề bài IRMS có nêu công cụ quản trị và sắp lịch nhân viên.
5. **Giữ rõ ràng các giao diện mở rộng:** các giao diện cho thanh toán, thông báo và phân quyền thể hiện rõ những điểm mở rộng và những chính sách nhạy cảm.

Sơ đồ lớp trong báo cáo này không được hiểu như một ERD thuần dữ liệu. Nó được dùng để biểu diễn **hành vi nghiệp vụ, ranh giới aggregate và các điểm mở rộng**. Điều đó giải thích vì sao nhiều lớp có operation như `confirm()`, `split()`, `markReady()`, `authorize()`, `issue()` thay vì chỉ liệt kê thuộc tính. Cách mô hình hóa này phù hợp hơn với mục tiêu của báo cáo, vì nó làm rõ hệ thống *vận hành như thế nào*, không chỉ *lưu gì trong database*.



Hình 28: Sơ đồ lớp UML tổng thể của miền lõi IRMS

Hình 28 là sơ đồ lớp ở mức nhìn toàn hệ thống, có nhiệm vụ cho thấy tất cả các cụm nghiệp vụ quan trọng và những liên kết chính giữa chúng. Thay vì chỉ tập trung vào vài lớp giao dịch quen thuộc như `Order` hay `Bill`, sơ đồ này giữ lại đầy đủ ngữ cảnh khách hàng, chi nhánh, đặt bàn, gọi món, bếp, thanh toán, tồn kho, quản trị, phân quyền và báo cáo. Nhờ đó, người đọc có thể nhìn được **bức tranh miền nghiệp vụ đầy đủ** trước khi đi sâu vào từng cụm lớp chi tiết hơn.

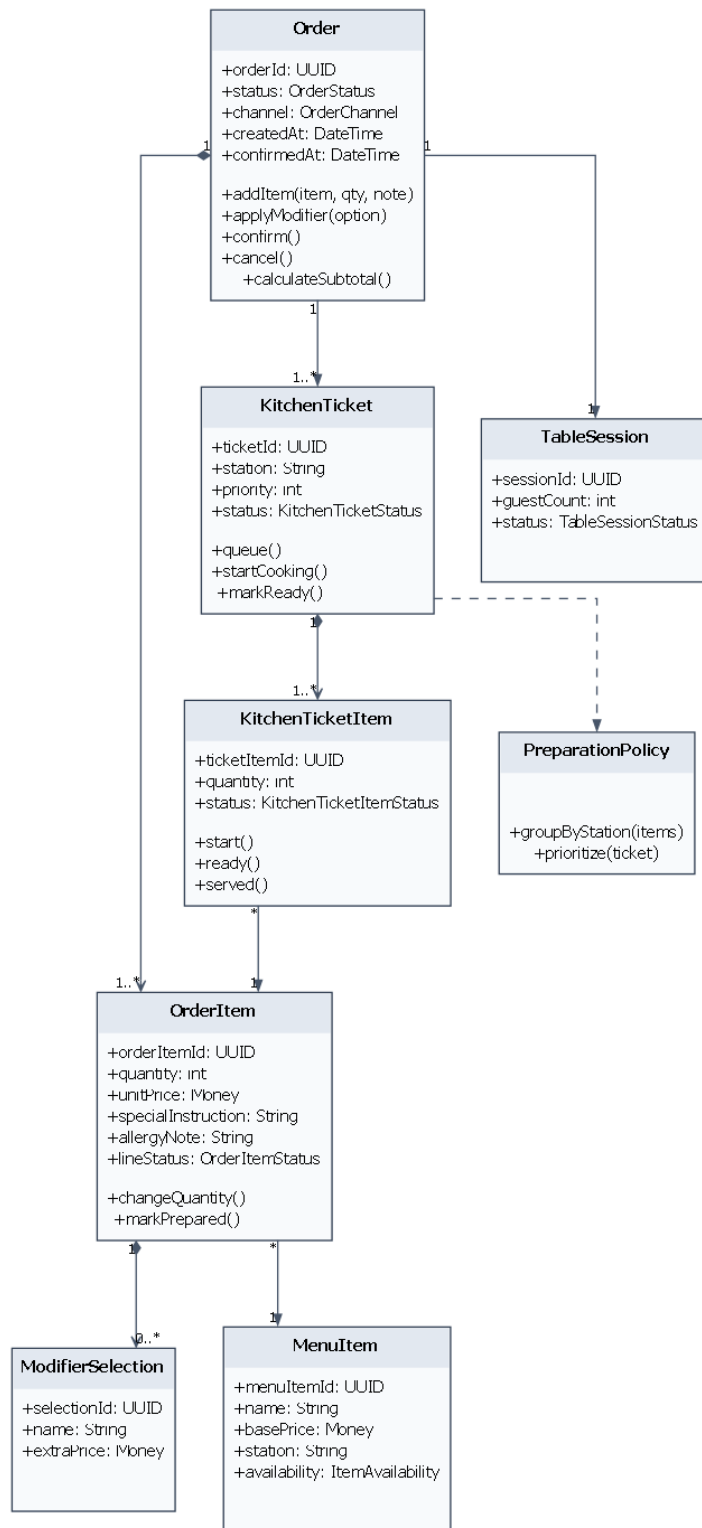
Các nhóm lớp chính thể hiện trong hình.

- **Ngữ cảnh khách hàng và chi nhánh:** `CustomerProfile` và `Branch` tạo nền cho việc gắn dữ liệu nghiệp vụ vào đúng khách hàng và đúng địa điểm vận hành.
- **Đặt bàn và phục vụ tại bàn:** `Reservation`, `WaitlistEntry`, `DiningTable` và `TableSession` biểu diễn vòng đời trước khi khách vào bàn và trong suốt thời gian bàn hoạt động.
- **Thực đơn và gọi món:** `MenuCategory`, `MenuItem`, `ModifierGroup`, `ModifierOption`, `Order`, `OrderItem` và `OrderItemModifier` mô tả cả cấu hình thực đơn lẫn dữ liệu giao dịch tại thời điểm khách gọi món.
- **Điều phối bếp:** `KitchenStation`, `KitchenTicket` và `KitchenTicketItem` cho thấy đường đi từ đơn gọi món sang đơn vị chế biến thực tế.
- **Quyết toán và thanh toán:** `Bill`, `BillSplit`, `Payment`, `Refund`, `Receipt`, `PromotionCampaign` và `AppliedDiscount` bao phủ đầy đủ luồng tính tiền của nhà hàng.
- **Tồn kho, cảnh báo và quản trị:** `InventoryItem`, `RecipeLine`, `StockTransaction`, `LowStockRule`, `User`, `Role`, `Permission`, `AuditLog`, `AuthorizationPolicy`, `ShiftAssignment` và `ReportSnapshot` đảm nhiệm phần kiểm soát, vận hành và phân tích.

Lý do thiết kế của sơ đồ này.

- Sơ đồ tổng thể được đặt ở mức đủ rộng để người đọc thấy ranh giới giữa các miền nghiệp vụ, nhưng chưa đi quá sâu vào chi tiết cài đặt của từng mô-đun.
- Các quan hệ được chọn theo tiêu chí nghiệp vụ trước, nghĩa là chỉ thể hiện các liên kết thực sự có ý nghĩa vận hành như đặt bàn gắn với bàn ăn, phiên bàn sinh đơn gọi món, đơn gọi món sinh phiếu bếp, và hóa đơn quyết toán phiên bàn.
- Các điểm mở rộng như `PaymentGateway`, `NotificationSender` và `AuthorizationPolicy` xuất hiện ngay trong sơ đồ tổng thể để cho thấy mô hình lớp không chỉ nói về dữ liệu, mà còn nói về **ranh giới mở rộng và kiểm soát**.

Điểm mạnh của sơ đồ tổng thể là giúp kiểm tra nhanh xem tài liệu có thực sự bao phủ đủ phạm vi IRMS hay chưa. Nếu thiếu một cụm lớp ở hình này, khả năng rất cao là cụm nghiệp vụ tương ứng cũng sẽ bị trình bày thiếu ở các phần sau. Theo nghĩa đó, đây là sơ đồ dùng để khóa phạm vi thiết kế trước khi đi vào chi tiết.



Hình 29: Sơ đồ lớp UML của cụm gọi món, thực đơn và bếp

Hình 29 đi vào cụm lớp có mật độ tương tác cao nhất trong vận hành hằng ngày của nhà hàng. Đây là nơi luồng nghiệp vụ bắt đầu từ dữ liệu thực đơn, đi qua quá trình chọn món và tùy biến, sau đó chuyển thành các đơn vị điều phối ở bếp. Vì vậy sơ đồ này có vai trò đặc biệt quan trọng trong việc chứng minh rằng mô hình gọi món của IRMS không bị giản lược thành một bảng dữ liệu phẳng.

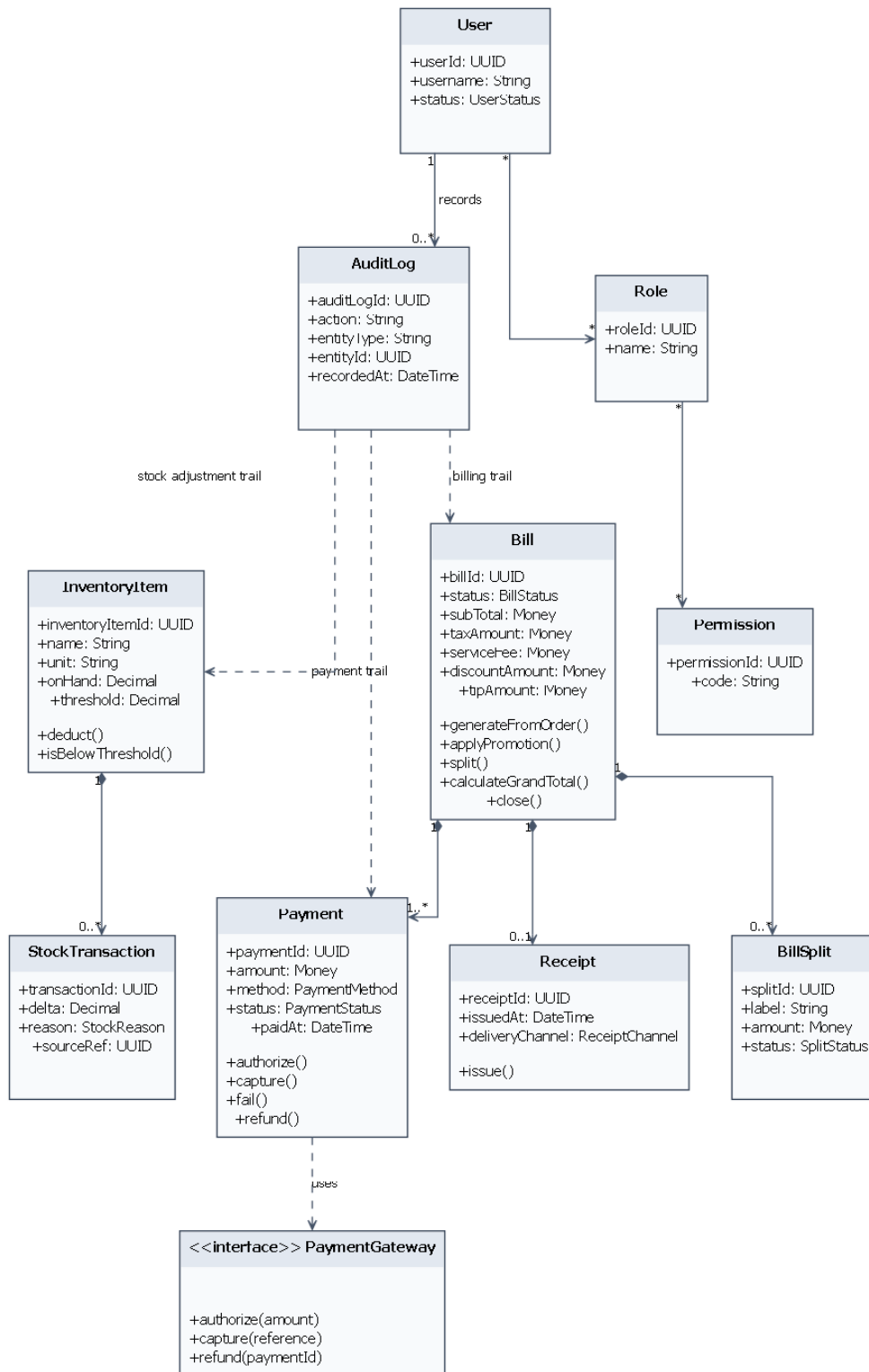
Các lớp trọng tâm thể hiện trong hình.

- **MenuCategory** và **MenuItem**: biểu diễn cấu trúc thực đơn và là nguồn gốc của giá cơ bản, trạng thái còn bán và thông tin điều phối.
- **ModifierGroup** và **ModifierOption**: chịu trách nhiệm ràng buộc việc tùy biến món, ví dụ giới hạn số lựa chọn tối thiểu và tối đa hoặc thay đổi giá do tùy chọn thêm.
- **Order** và **OrderItem**: là lõi của giao dịch gọi món; **Order** giữ trạng thái đơn tổng thể, còn **OrderItem** là đơn vị bán hàng cụ thể.
- **OrderItemModifier**: lưu ảnh chụp của tùy biến đã chọn tại thời điểm gọi món, giúp hệ thống không bị lệ thuộc ngược vào cấu hình thực đơn hiện tại.
- **KitchenStation**, **KitchenTicket** và **KitchenTicketItem**: chuyển dữ liệu từ miền bán hàng sang miền chế biến, giữ được trạng thái của từng trạm bếp thay vì ép toàn bộ tiến độ vào **Order**.

Lý do thiết kế của sơ đồ này.

- Tách **OrderItem** khỏi **KitchenTicketItem** để phân biệt rõ **đơn vị bán hàng** với **đơn vị điều phối chế biến**; đây là quyết định bắt buộc nếu một món cần được định tuyến sang trạm bếp cụ thể.
- Dùng cơ chế ảnh chụp ở **OrderItem** và **OrderItemModifier** để bảo vệ tính đúng đắn của hóa đơn và phiếu bếp ngay cả khi thực đơn thay đổi sau thời điểm gọi món.
- Giữ **KitchenStation** như một lớp riêng để quy tắc định tuyến, mức ưu tiên và trạng thái bếp không làm phình to aggregate **Order**.

Ưu điểm của cụm lớp này là phản ánh được đồng thời **cấu hình thực đơn**, **dữ liệu giao dịch** và **trạng thái chế biến**. Nếu thiếu một trong ba lớp nghĩa đó, hệ thống rất dễ rơi vào tình trạng hoặc là khó mở rộng nghiệp vụ, hoặc là không theo dõi được đường đi thật sự của món từ màn hình gọi món xuống bếp.



Hình 30: Sơ đồ lớp UML của cụm thanh toán, tồn kho và quản trị

Hình 30 tập trung vào vùng nhạy cảm nhất của kiến trúc, nơi các quyết định của hệ thống tác động trực tiếp đến tiền, tồn kho và quyền thao tác của người dùng. Vì đây là nơi dễ phát sinh sai sót có hậu quả vận hành lớn, sơ đồ được trình bày chi tiết hơn để người đọc thấy rõ các lớp nào chịu trách nhiệm giao dịch, lớp nào chịu trách nhiệm truy vết, và lớp nào chỉ đóng vai trò cổng mở rộng.

Các lớp trọng tâm thể hiện trong hình.

- **Cụm quyết toán:** Bill, BillSplit, AppliedDiscount và PromotionCampaign cùng nhau biểu diễn logic tổng hợp tiền, tách hóa đơn và áp dụng khuyến mãi.
- **Cụm thanh toán:** Payment, Refund, Receipt và PaymentGateway mô tả việc thu tiền, hoàn tiền, phát hành biên lai và giao tiếp với hệ thống ngoài qua giao diện trừu tượng.
- **Cụm tồn kho:** InventoryItem, RecipeLine, StockTransaction, LowStockRule và NotificationSender mô tả đường suy diễn từ món bán ra tới biến động tồn kho và cảnh báo mức tồn thấp.
- **Cụm quản trị và kiểm soát:** User, Role, Permission, AuditLog, AuthorizationPolicy, ShiftAssignment và ReportSnapshot tạo nền cho phân quyền, kiểm toán, lịch ca và báo cáo.

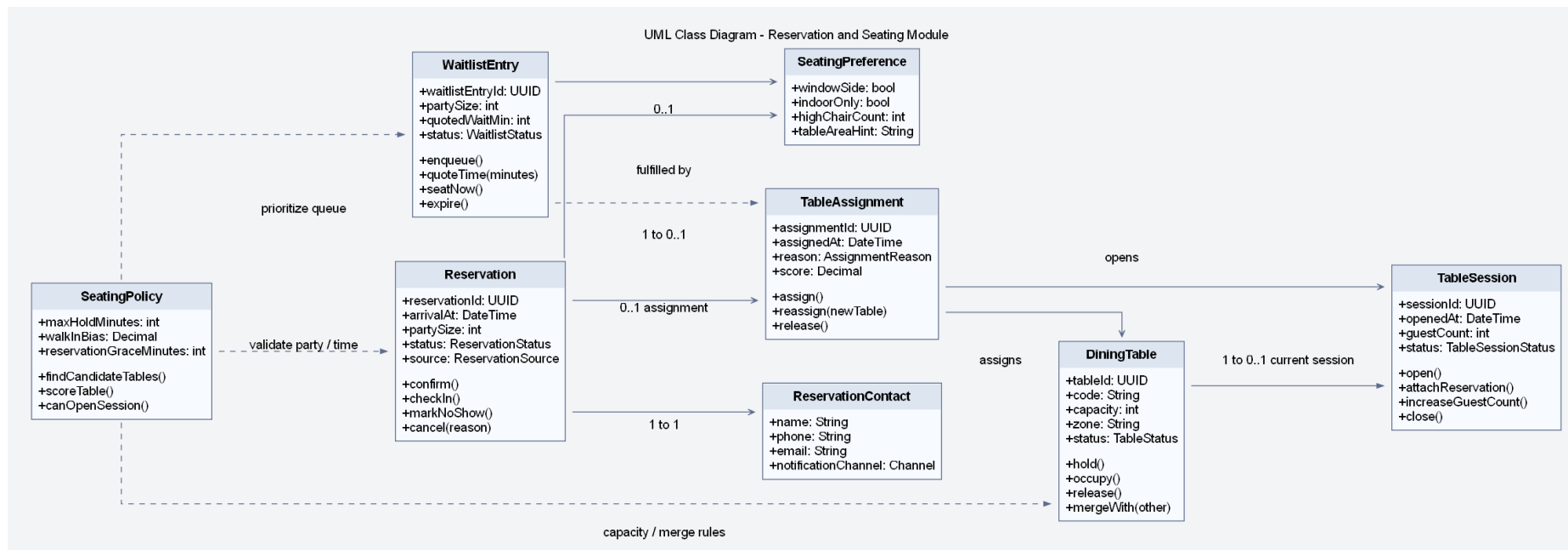
Lý do thiết kế của sơ đồ này.

- Đặt PaymentGateway và NotificationSender dưới dạng giao diện để miền nghiệp vụ không phụ thuộc trực tiếp vào một nhà cung cấp cụ thể.
- Tách Refund ra khỏi Payment để phản ánh đúng việc hoàn tiền là một hành động nghiệp vụ riêng, có lý do, ngưỡng phê duyệt và trạng thái theo dõi riêng.
- Đưa AuditLog và AuthorizationPolicy vào cùng sơ đồ với giao dịch để nhấn mạnh rằng kiểm soát quyền và truy vết không phải phần phụ, mà là một phần của thiết kế lõi.
- Giữ ReportSnapshot ở cụm này để thể hiện rõ dữ liệu báo cáo được suy diễn từ dữ liệu vận hành, thay vì can thiệp trực tiếp vào tuyến giao dịch nóng.

Ưu điểm lớn nhất của sơ đồ này là nó cho thấy kiến trúc lớp của IRMS không chỉ giải quyết việc ?chạy đúng chức năng?, mà còn giải quyết việc ?kiểm soát được hậu quả? khi xảy ra hoàn tiền, điều chỉnh kho, ghi đề giá hay tranh chấp thanh toán. Đây là điểm làm cho phần thiết kế chặt hơn dưới góc nhìn kiến trúc phần mềm ở môi trường vận hành thật.

4.2 Giải thích theo từng cụm lớp

Phần này đi theo nhịp *nhìn sơ đồ* → *đọc các cụm trách nhiệm* → *đọc lý do mô hình hóa*. Cách trình bày đó giúp phần sơ đồ lớp không dừng ở mức liệt kê lớp, mà làm rõ vai trò trong miền, ranh giới trách nhiệm và quyết định thiết kế cho từng mô-đun.



Hình 31: Sơ đồ lớp chi tiết cho module Reservation & Seating

Hình 31 đi vào module Reservation & Seating ở mức chi tiết và đúng tinh thần mô hình miền. Trọng tâm của module này không phải chỉ là ?giữ danh sách bàn?, mà là quản lý **năng lực phục vụ theo thời gian**: ai đến trước, ai có cam kết trước, bàn nào phù hợp, khi nào được mở phiên, và vì sao một quyết định gán bàn lại được đưa ra. Bởi vậy, sơ đồ được tổ chức quanh ba trục: **nguồn nhu cầu** (Reservation, WaitlistEntry), **tài nguyên phục vụ** (DiningTable, TableSession) và **quy tắc điều phối** (TableAssignment, SeatingPolicy, SeatingPreference).

Các cụm lớp và trách nhiệm thể hiện trong hình.

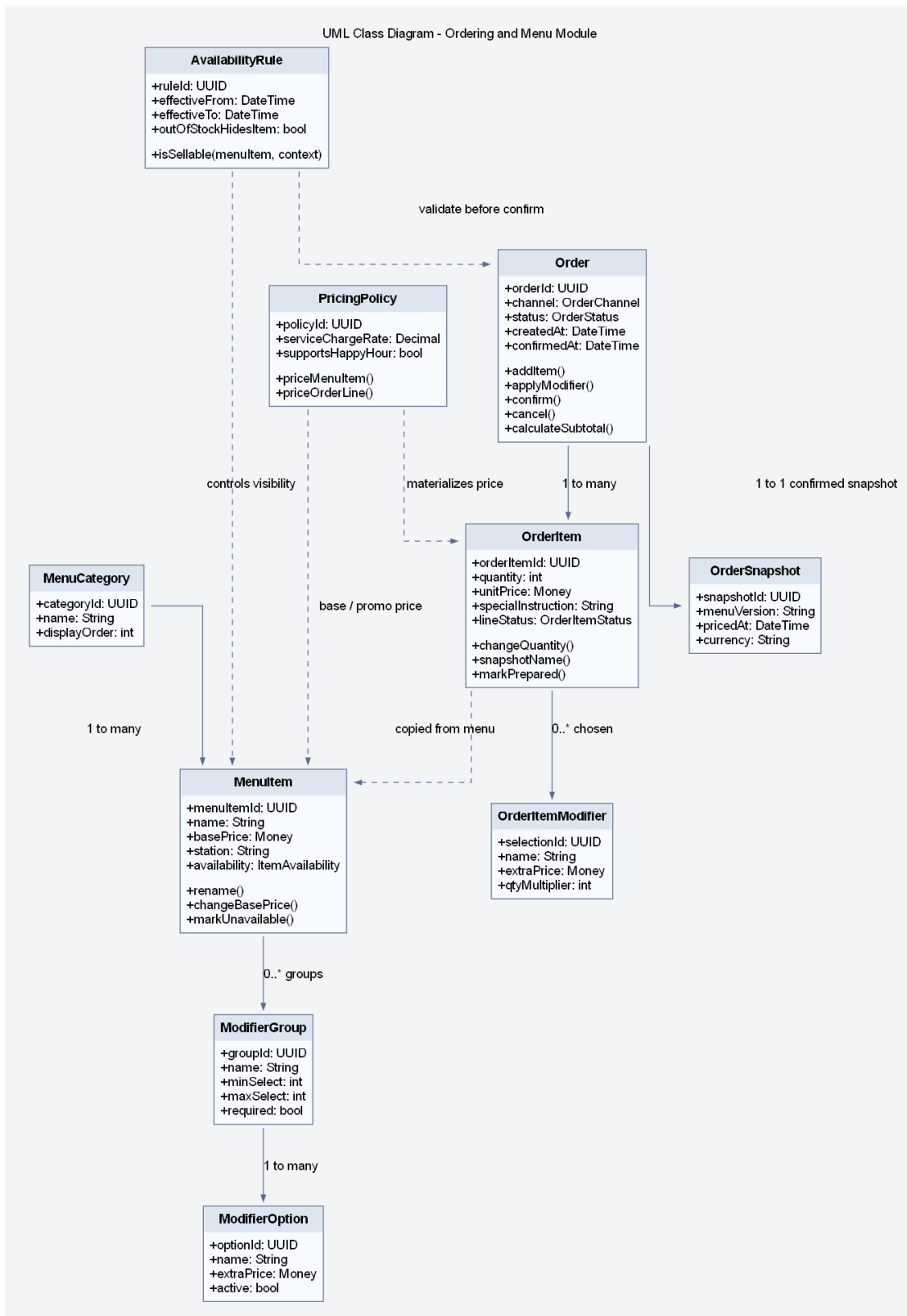
- **Nguồn nhu cầu trước khi phục vụ:** Reservation giữ thông tin đặt trước, trạng thái và hành vi như xác nhận, check-in, no-show và hủy; ReservationContact tách riêng thông tin liên hệ để tránh nhồi dữ liệu truyền thông vào thực thể đặt bàn.
- **Nguồn nhu cầu chưa được đáp ứng ngay:** WaitlistEntry mô hình hóa khách vãng lai hoặc trường hợp chưa có bàn phù hợp tại thời điểm hiện tại; điều này rất quan trọng vì waitlist không nên bị ép vào cùng vòng đời với reservation.
- **Tài nguyên vật lý và phiên vận hành:** DiningTable biểu diễn bàn vật lý với sức chứa và vùng; TableSession biểu diễn phiên phục vụ thực tế, độc lập với việc bàn đó có được đặt trước hay không.
- **Lớp điều phối:** TableAssignment lưu quyết định gán bàn cụ thể, còn SeatingPolicy và SeatingPreference làm rõ phần ?vì sao bàn này được chọn? thay vì để quy tắc nằm rải rác trong service.

Giải thích chi tiết theo từng cụm lớp.

- **Reservation không đồng nhất với TableSession.** Một reservation có thể bị hủy, no-show hoặc dời giờ mà không sinh phiên phục vụ. Ngược lại, một table session có thể được mở cho khách walk-in mà không có reservation. Tách hai lớp này là quyết định mô hình hóa quan trọng nhất của module.
- **WaitlistEntry là đối tượng hạng nhất của miền.** Nếu chỉ xem waitlist là một trạng thái phụ của reservation, hệ thống sẽ khó diễn tả đúng khách vãng lai, khách chưa chắc ngồi, khách đổi ý hoặc khách chấp nhận vùng bàn khác sau một khoảng chờ.
- **TableAssignment giữ lịch sử quyết định gán bàn.** Điều này hữu ích cho giải thích vận hành, tái gán bàn, và về sau có thể nối với audit hoặc analytics để đo hiệu quả xếp bàn theo giờ cao điểm.
- **SeatingPolicy là nơi đặt rule thay vì hard-code trong controller.** Các rule như thời gian giữ chỗ, ưu tiên walk-in hay reservation, giới hạn mở session và năng lực gộp bàn nên được gom vào chính sách có tên gọi rõ ràng.

Lý do thiết kế của sơ đồ này.

- Tách ReservationContact và SeatingPreference giúp thực thể Reservation gọn hơn, chỉ giữ phần nhận diện và trạng thái lõi.
- Dùng TableAssignment như đối tượng trung gian để lưu lý do và điểm chấm bàn, nhờ đó quyết định xếp bàn có thể giải thích được.
- Giữ TableSession như bản ghi riêng bảo vệ toàn bộ tuyến gọi món, hóa đơn và thanh toán khỏi việc phụ thuộc trực tiếp vào đặt bàn.



Hình 32: Sơ đồ lớp chi tiết cho module Ordering & Menu

Hình 32 mô tả mô-đun nóng nhất của toàn hệ thống. Về mặt miền, mô-đun này phải giải được mâu thuẫn giữa **thực đơn sống** và **giao dịch bất biến**. Thực đơn có thể thay đổi liên tục: ẩn món, đổi giá, thêm tùy chọn món, đổi giờ phục vụ. Trong khi đó, đơn gọi món đã xác nhận phải giữ nguyên đúng tên món, giá, tùy chọn và điều kiện áp dụng tại thời điểm xác nhận. Vì thế, sơ đồ tách rõ hai lớp đối tượng: **Menu*** là phần cấu hình có thể thay đổi, còn **Order*** là phần giao dịch đã

chốt.

Các cụm lớp và trách nhiệm thể hiện trong hình.

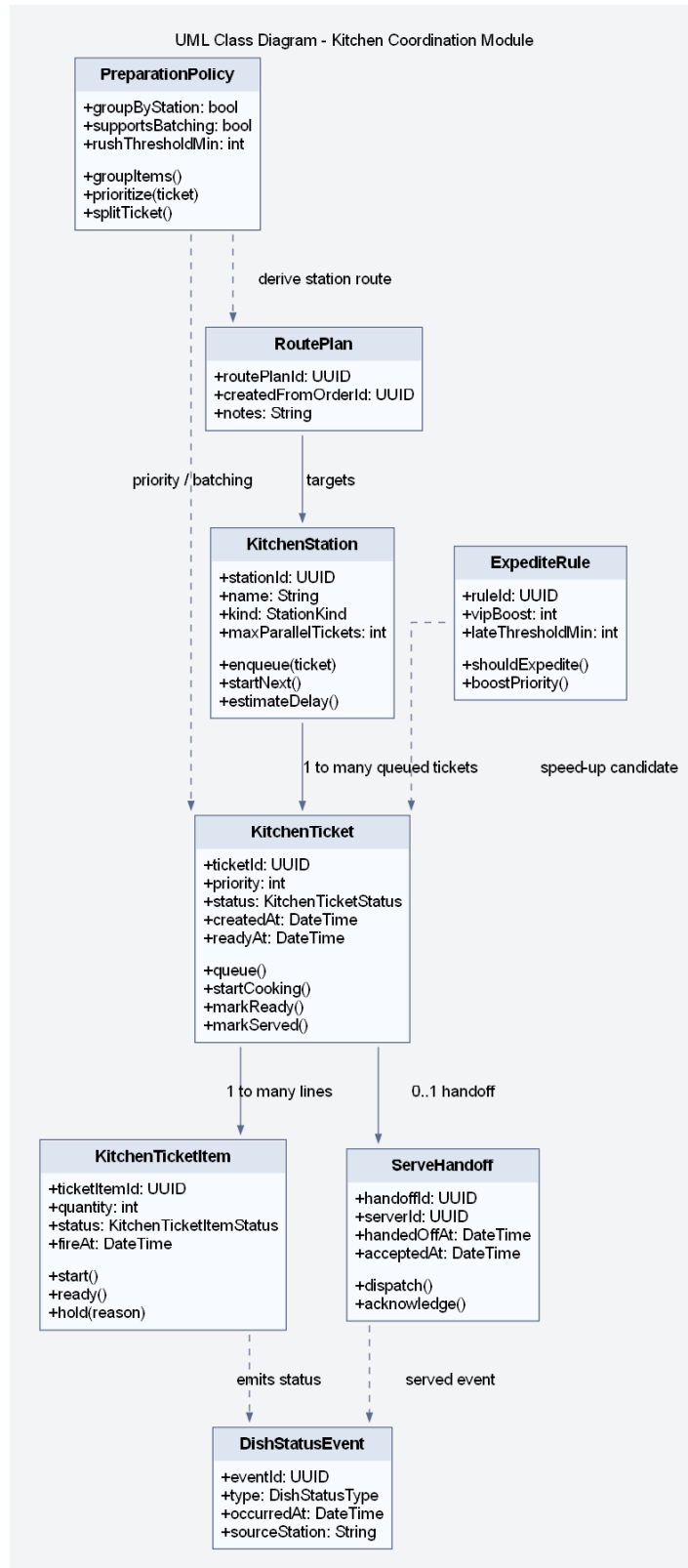
- **Cấu hình thực đơn:** `MenuCategory`, `MenuItem`, `ModifierGroup` và `ModifierOption` tạo nên cấu trúc menu mà quản lý có thể thay đổi.
- **Quy tắc khả dụng và định giá:** `AvailabilityRule` và `PricingPolicy` gom logic giờ bán, trạng thái hết hàng, giá cơ sở và các quy tắc điều chỉnh giá.
- **Giao dịch đơn gọi món:** `Order`, `OrderItem` và `OrderItemModifier` lưu lại món đã chọn, tùy chọn đã chọn, số lượng, ghi chú và trạng thái dòng món.
- **Ảnh chụp giao dịch:** `OrderSnapshot` cho thấy tại thời điểm xác nhận, hệ thống có điểm neo để lần vết lại phiên bản menu và cách tính giá đã được áp dụng.

Giải thích chi tiết theo từng cụm lớp.

- **`MenuItem` không nên bị dùng trực tiếp như `OrderItem`.** Nếu dùng trực tiếp, chỉ cần đổi tên món hoặc giá món sau đó là lịch sử giao dịch và hóa đơn sẽ bị biến dạng.
- **`ModifierGroup` và `ModifierOption` là rule-bearing structure.** Chúng không chỉ là dữ liệu hiển thị; chúng còn quyết định số lựa chọn bắt buộc, số lượng tối đa và phần cộng giá thêm.
- **`PricingPolicy` nên đứng riêng khỏi `MenuItem`.** Giá một món không chỉ là `basePrice`; còn có khung giờ, phí phục vụ, chương trình áp dụng hoặc logic định giá theo bối cảnh. Tách chính sách này giúp mô-đun tuân thủ trách nhiệm đơn nhất và dễ mở rộng hơn.
- **`OrderSnapshot` khóa ngữ nghĩa tại thời điểm xác nhận.** Đây là điểm rất quan trọng khi về sau cần kiểm toán, đối soát hóa đơn, hoặc tái hiện đơn hàng trong tranh chấp.

Lý do thiết kế của sơ đồ này.

- Tách rule và data giúp module vừa linh hoạt cho quản trị menu, vừa chắc chắn cho tuyến giao dịch.
- Đặt `AvailabilityRule` và `PricingPolicy` ở cùng sơ đồ để nhấn mạnh rằng validate và price phải là một phần của domain model chứ không chỉ là service logic.
- Duy trì `OrderItemModifier` như thực thể riêng để ghi nhận modifier thực tế đã chọn thay vì chỉ giữ quan hệ tham chiếu tới modifier cấu hình.



Hình 33: Sơ đồ lớp chi tiết cho module Kitchen Coordination

Hình 33 làm rõ việc điều phối bếp là một miền riêng chứ không chỉ là một trạng thái phụ của đơn gọi món. Khi đơn gọi món đã được xác nhận, hệ thống bắt đầu giải quyết một bài toán khác: chia việc theo trạm, xếp hàng đợi, đổi mức ưu tiên, theo dõi tiến độ, và bàn giao món đã hoàn tất cho tuyến phục vụ. Nếu nhét toàn bộ logic này vào `Order` hoặc `OrderItem`, mô hình sẽ nhanh chóng lẫn lộn giữa góc nhìn bán hàng và góc nhìn chế biến.

Các cụm lớp và trách nhiệm thể hiện trong hình.

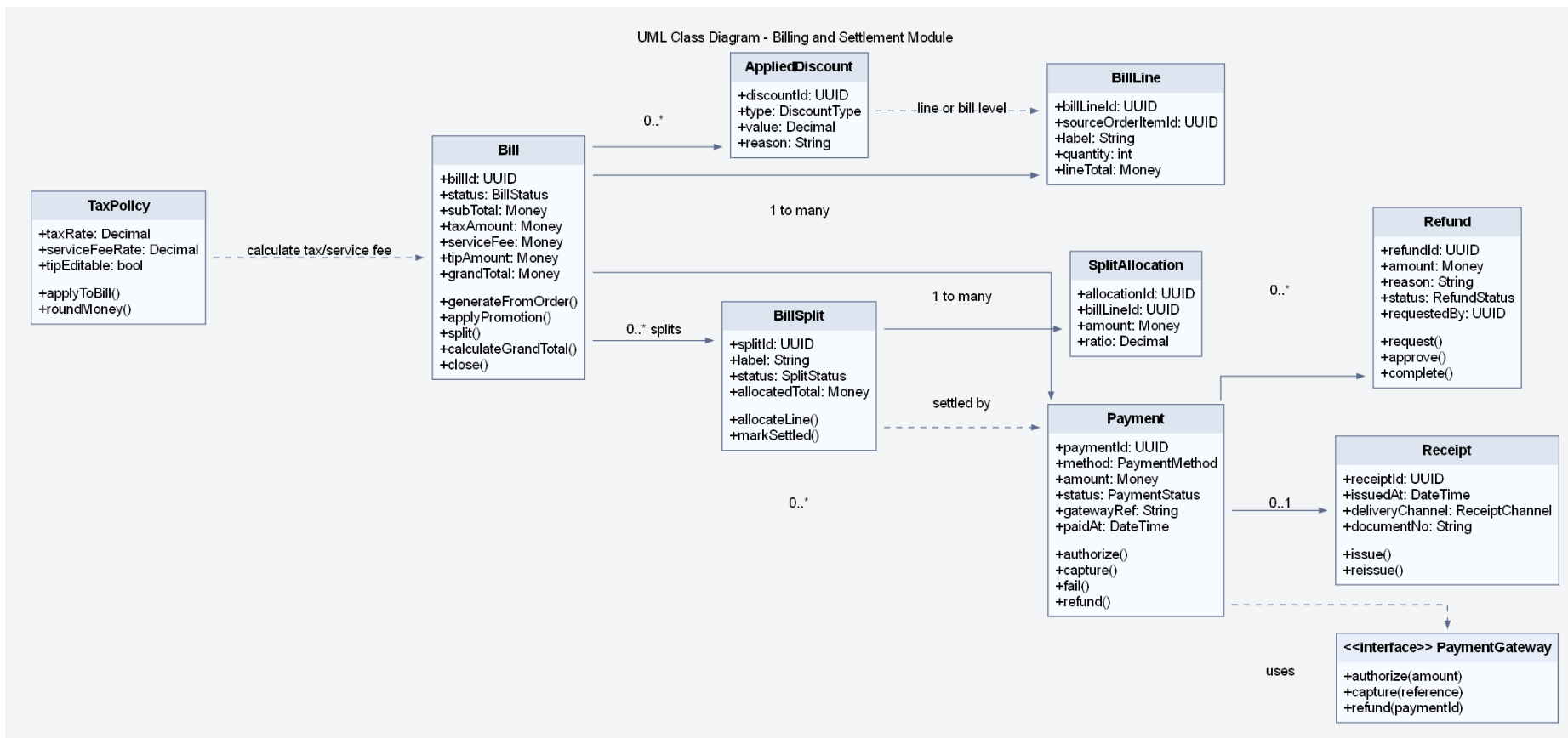
- **Tài nguyên bếp:** KitchenStation biểu diễn từng trạm với năng lực xử lý, loại trạm và hàng chờ tương ứng.
- **Đơn vị điều phối:** KitchenTicket là phiếu công việc ở mức phiếu bếp; KitchenTicketItem là đơn vị thực thi cụ thể theo món hoặc nhóm món.
- **Chính sách điều phối:** PreparationPolicy, RoutePlan và ExpediteRule gom logic tách phiếu, nhóm món, định tuyến và tăng ưu tiên.
- **Bàn giao trạng thái:** ServeHandoff và DishStatusEvent neo đường truyền trạng thái món ra khỏi bếp.

Giải thích chi tiết theo từng cụm lớp.

- **KitchenTicket là aggregate riêng.** Nó có vòng đời chờ, đang làm, sẵn sàng, đã bàn giao; vòng đời này không nên bị gộp vào trạng thái đơn gọi món.
- **KitchenTicketItem cho phép xử lý hạt mịn.** Trong thực tế, không phải toàn bộ phiếu bếp luôn đi cùng tốc độ; từng dòng có thể bị giữ, sẵn sàng hay bị chậm riêng.
- **PreparationPolicy và ExpediteRule là nơi đặt thuật toán điều phối.** Nhờ đó, logic ưu tiên VIP, món bị trễ hoặc gom nhóm theo trạm bếp có chỗ đứng rõ trong mô hình.
- **ServeHandoff tạo điểm chốt giữa bếp và phục vụ.** Không phải cứ món ready là đã served; thêm lớp bàn giao giúp phân biệt rõ hai giai đoạn này.

Lý do thiết kế của sơ đồ này.

- Tách miền bếp thành một module có lớp riêng giúp hệ thống mở rộng dễ hơn khi cần nhiều station hoặc màn hình bếp khác nhau.
- Dùng lớp sự kiện DishStatusEvent để cho phép mô hình đọc, thông báo hay phân tích theo dõi tiến độ mà không chạm vào aggregate bếp.
- Mô hình này làm rõ vì sao góc nhìn thành phần cần KitchenTicketFactory và KitchenApplicationService riêng.



Hình 34: Sơ đồ lớp chi tiết cho module Billing & Settlement

Hình 34 trình bày vùng quyết toán với đúng chiều sâu cần có trong báo cáo. Ở IRMS, bài toán quyết toán không dừng ở ?tính tổng tiền?: hệ thống còn phải biểu diễn dòng hóa đơn, tách hóa đơn, phân bổ dòng vào từng phần thanh toán, áp khuyến mãi, tính thuế và phí, thu tiền theo một hay nhiều giao dịch thanh toán, phát hành biên lai và theo dõi hoàn tiền. Mỗi quyết định trong vùng này đều có hậu quả tài chính, vì vậy mô hình lớp phải phân tách đủ rõ để tránh một lớp Bill khổng lồ ôm hết mọi trách nhiệm.

Các cụm lớp và trách nhiệm thể hiện trong hình.

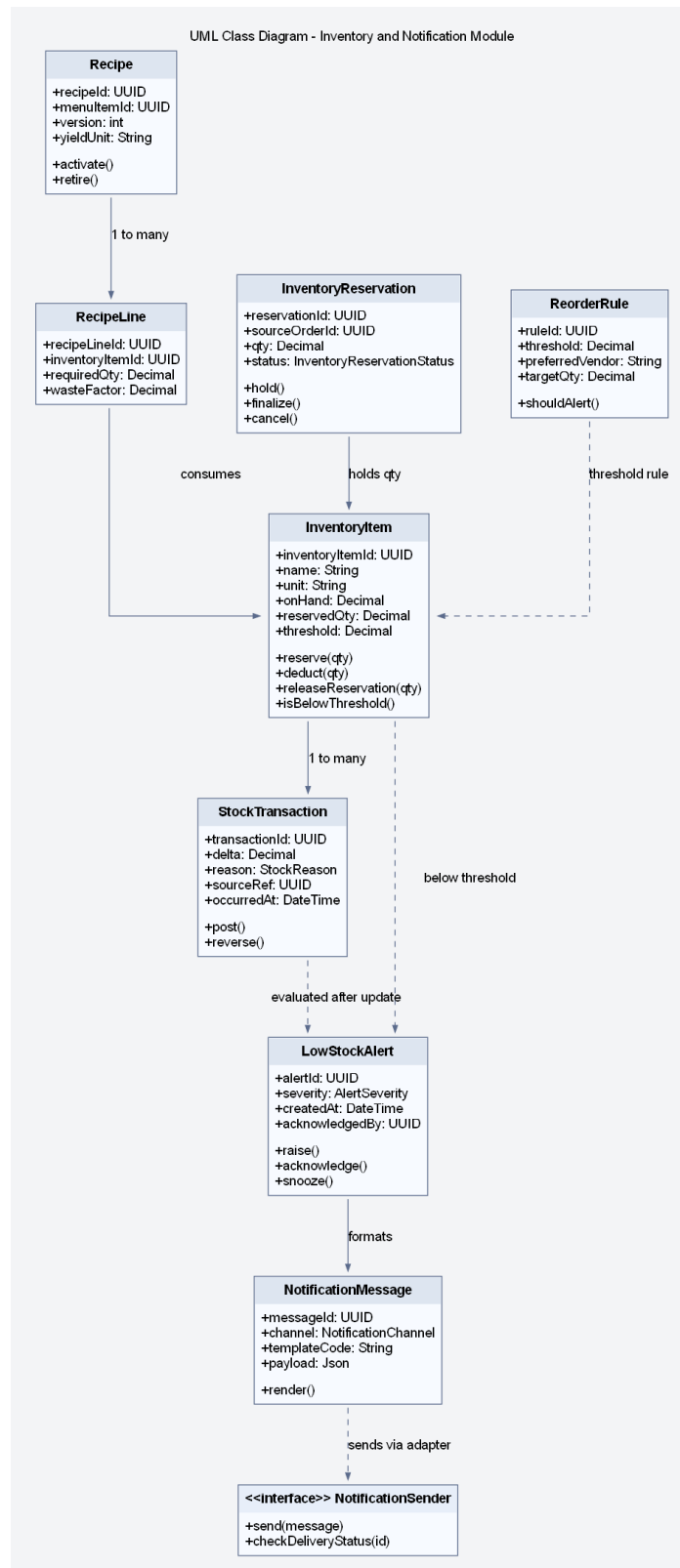
- **Hóa đơn và dòng hóa đơn:** Bill và BillLine là trung tâm quyết toán, nơi dữ liệu đơn gọi món đã xác nhận được gom lại thành đối tượng tài chính.
- **Tách hóa đơn:** BillSplit và SplitAllocation biểu diễn đúng nghiệp vụ chia hóa đơn, thay vì coi split chỉ là vài cờ hoặc danh sách tạm.
- **Thanh toán và hoàn tiền:** Payment, Refund và Receipt tạo thành vòng đời thu tiền, hoàn tiền và chứng từ.
- **Quy tắc quyết toán:** AppliedDiscount, TaxPolicy và giao diện PaymentGateway gom logic giảm giá, làm tròn, thuế và tích hợp ngoài.

Giải thích chi tiết theo từng cụm lớp.

- **BillSplit là đối tượng hạng nhất.** Với nhà hàng, việc tách hóa đơn không phải thao tác hiếm; nó có nhãn, phạm vi, tổng phân bổ và trạng thái riêng. Mô hình hóa tường minh giúp hỗ trợ chia theo người, theo dòng món hoặc theo tỷ lệ.
- **Payment tách khỏi Bill.** Một hóa đơn có thể có nhiều giao dịch thanh toán; một giao dịch có thể thất bại, cần thử lại, hoặc bị hoàn tiền một phần. Dồn logic này vào hóa đơn sẽ làm Bill mang quá nhiều lý do thay đổi.
- **Refund là thực thể riêng, không phải chỉ là số âm của payment.** Nó cần lý do, người yêu cầu, trạng thái và về sau có thể cần phê duyệt.
- **PaymentGateway phải là lớp trừu tượng.** Lỗi quyết toán cần thao tác nghiệp vụ ủy quyền, ghi nhận giao dịch và hoàn tiền, nhưng không nên phụ thuộc vào SDK hoặc giao thức đặc thù của nhà cung cấp.

Lý do thiết kế của sơ đồ này.

- Mô hình này bám sát tuyến giao dịch nhưng vẫn chứa đúng điểm mở cho bộ kết nối.
- Việc tách TaxPolicy và AppliedDiscount làm rõ đâu là dữ liệu nghiệp vụ, đâu là quy tắc tính toán.
- Sơ đồ cũng chuẩn bị tốt cho kiểm toán: hoàn tiền, tách hóa đơn và giao dịch thanh toán đều đã có chỗ neo riêng để nối với nhật ký kiểm toán hoặc luồng phê duyệt.



Hình 35: Sơ đồ lớp chi tiết cho module Inventory & Notification

Hình 35 mở rộng phần tồn kho theo hướng **truy vết được nguồn gốc biến động** và **phân biệt rõ dữ liệu kho với kênh cảnh báo**. Một hệ thống nhà hàng thực tế không thể chỉ có trường `onHand`; nó phải biết món nào dùng nguyên liệu nào, thời điểm nào thì giữ tạm nguyên liệu, khi nào thì trừ thật, và khi nào cần phát ra cảnh báo mức tồn thấp. Bởi vậy, sơ đồ thêm các lớp **Recipe**, **RecipeLine**, **InventoryReservation**, **ReorderRule**, **LowStockAlert** và **NotificationMessage**.

Các cụm lớp và trách nhiệm thể hiện trong hình.

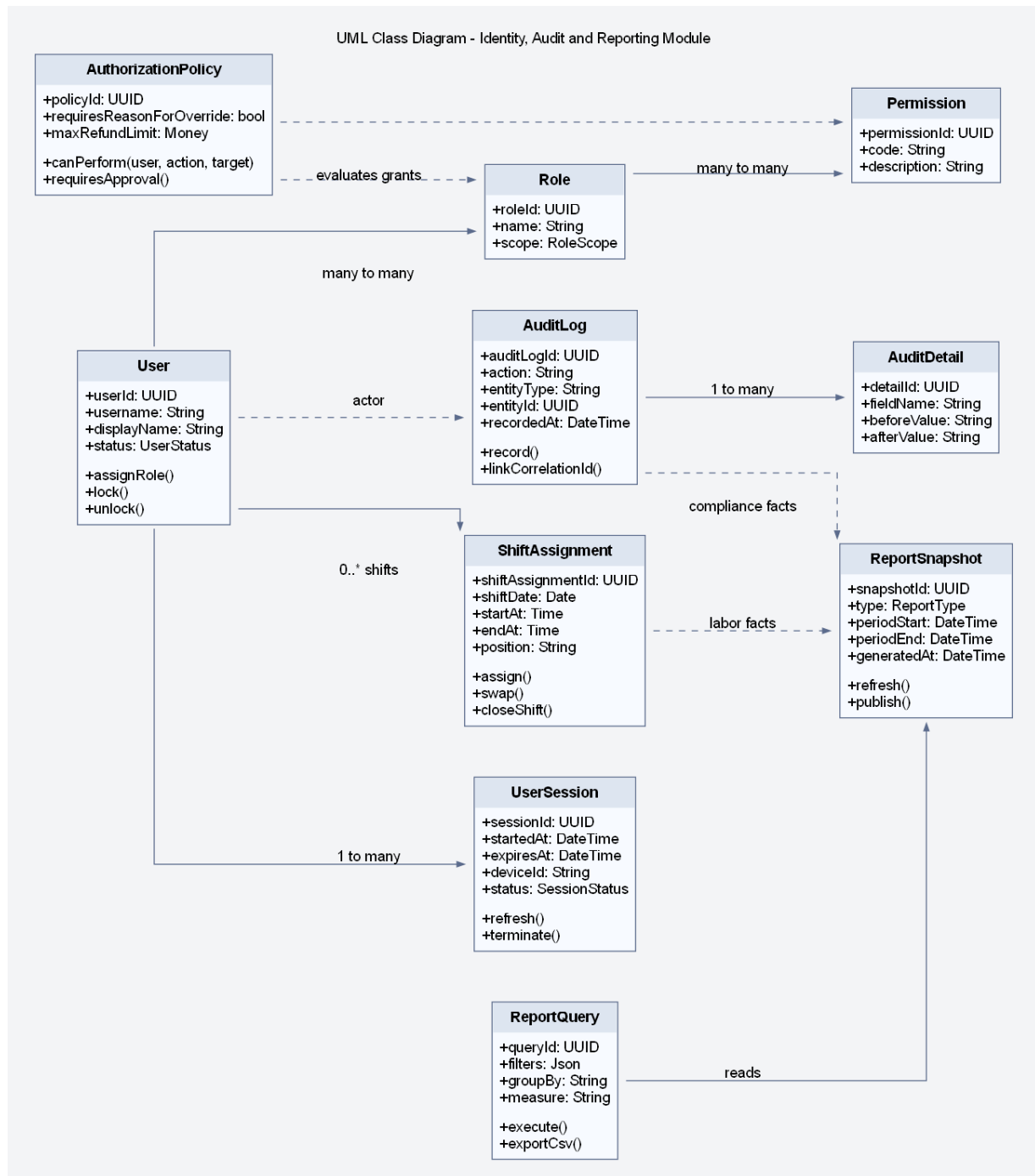
- **Số dư và giao dịch kho:** `InventoryItem` và `StockTransaction` giữ trạng thái kho hiện tại và lịch sử biến động.
- **Suy diễn mức tiêu hao:** `Recipe` và `RecipeLine` nối món bán ra với nguyên liệu cần dùng.
- **Giữ chỗ và chốt trừ:** `InventoryReservation` cho phép mô tả giai đoạn giữ tạm nguyên liệu trước khi thanh toán hoàn tất hoặc đơn gọi món được chốt.
- **Cảnh báo và kênh gửi:** `ReorderRule`, `LowStockAlert`, `NotificationMessage` và giao diện `NotificationSender` tách rõ điều kiện phát cảnh báo khỏi hạ tầng chuyển thông báo.

Giải thích chi tiết theo từng cụm lớp.

- **Recipe là cầu nối giữa thực đơn và kho.** Nếu thiếu nó, phần tiêu hao nguyên liệu sẽ bị mã hóa cứng vào logic gọi món hoặc quyết toán.
- **InventoryReservation giúp mô hình linh hoạt hơn.** Một số đội triển khai muốn trừ kho ngay khi xác nhận đơn gọi món, số khác chỉ chốt hẳn khi thanh toán hoàn tất; có lớp giữ chỗ giúp mở cả hai chiến lược.
- **StockTransaction là bằng chứng, không chỉ là lịch sử phụ.** Nó cho phép giải thích vì sao tồn kho thay đổi, giúp reconciliation và audit sau ca.
- **Notification phải đi qua lớp trừu tượng.** Cảnh báo tồn thấp là nghiệp vụ; gửi email, SMS hay thông báo đẩy lại là bài toán của bộ kết nối.

Lý do thiết kế của sơ đồ này.

- Mô hình tách được tồn kho khỏi kênh thông báo nhưng vẫn giữ liên hệ chặt qua `LowStockAlert`.
- Nó hỗ trợ tốt cho sơ đồ thành phần bất đồng bộ, nơi bộ xử lý nền có thể gửi cảnh báo sau khi mô-đun tồn kho phát hiện ngưỡng.
- Nó cũng giúp phần báo cáo doanh thu-tồn kho sau này có điểm nối dữ liệu sạch hơn.



Hình 36: Sơ đồ lớp chi tiết cho module Identity, Audit & Reporting

Hình 36 trình bày phần governance ở mức đúng với kiến trúc của một hệ thống vận hành thật. Trong IRMS, user, role và permission mới chỉ là phần đầu. Hệ thống còn cần mô tả phiên đăng nhập (UserSession), chính sách kiểm tra quyền theo bối cảnh (AuthorizationPolicy), nhật ký kiểm toán đa thuộc tính (AuditLog, AuditDetail), phân công ca (ShiftAssignment) và ảnh chụp báo cáo cùng truy vấn báo cáo (ReportSnapshot, ReportQuery). Chỉ khi các lớp này được biểu diễn tường minh, phần quản trị, kiểm toán, báo cáo mới thoát khỏi mức khẩu hiệu.

Các cụm lớp và trách nhiệm thể hiện trong hình.

- **Danh tính và quyền:** User, Role, Permission và UserSession giữ nền tảng xác thực–phân quyền.
- **Chính sách truy cập:** AuthorizationPolicy gom logic bối cảnh như giới hạn hoàn tiền,

yêu cầu nêu lý do ghi đề hoặc yêu cầu phê duyệt.

- **Dấu vết thay đổi:** AuditLog và AuditDetail lưu hành động, thực thể bị tác động và giá trị trước-sau.
- **Nhân sự và phân tích:** ShiftAssignment, ReportSnapshot và ReportQuery nối dữ liệu vận hành với góc nhìn quản trị.

Giải thích chi tiết theo từng cụm lớp.

- **AuthorizationPolicy phải là lớp riêng.** Nếu mọi quy tắc quyền đều nằm ở bộ điều khiển hoặc chú giải, hệ thống sẽ khó diễn tả các quyết định phụ thuộc ngữ cảnh nghiệp vụ như số tiền hoàn tiền hay lý do ghi đề.
- **AuditDetail giúp nhật ký kiểm toán có giá trị thực tế.** Chỉ lưu ?ai làm gì? là chưa đủ; với thao tác nhạy cảm, cần biết trường nào đã đổi từ gì sang gì.
- **UserSession là đối tượng đáng mô hình hóa.** Nó hỗ trợ khóa phiên, truy vết thiết bị và quản lý timeout, vốn là nhu cầu thực trong nhà hàng khi đổi ca hoặc bàn giao máy.
- **ReportSnapshot tách khỏi ReportQuery.** Snapshot là dữ liệu tổng hợp bền vững cho một kỳ; query là cách người dùng truy cập và lọc dữ liệu đó.

Lý do thiết kế của sơ đồ này.

- Module governance được biểu diễn chi tiết giúp phần bảo mật, kiểm toán và báo cáo đứng đúng vai trò kiến trúc, không bị xem như lớp chức năng phụ.
- Nó tạo điểm nối rõ ràng giữa sơ đồ lớp với góc nhìn thành phần hỗ trợ và quản trị đã trình bày ở phần trước.
- Nó cũng hỗ trợ rất tốt cho phân biện minh nguyên lý SOLID, đặc biệt là SRP, OCP và DIP ở vùng policy, audit và reporting.

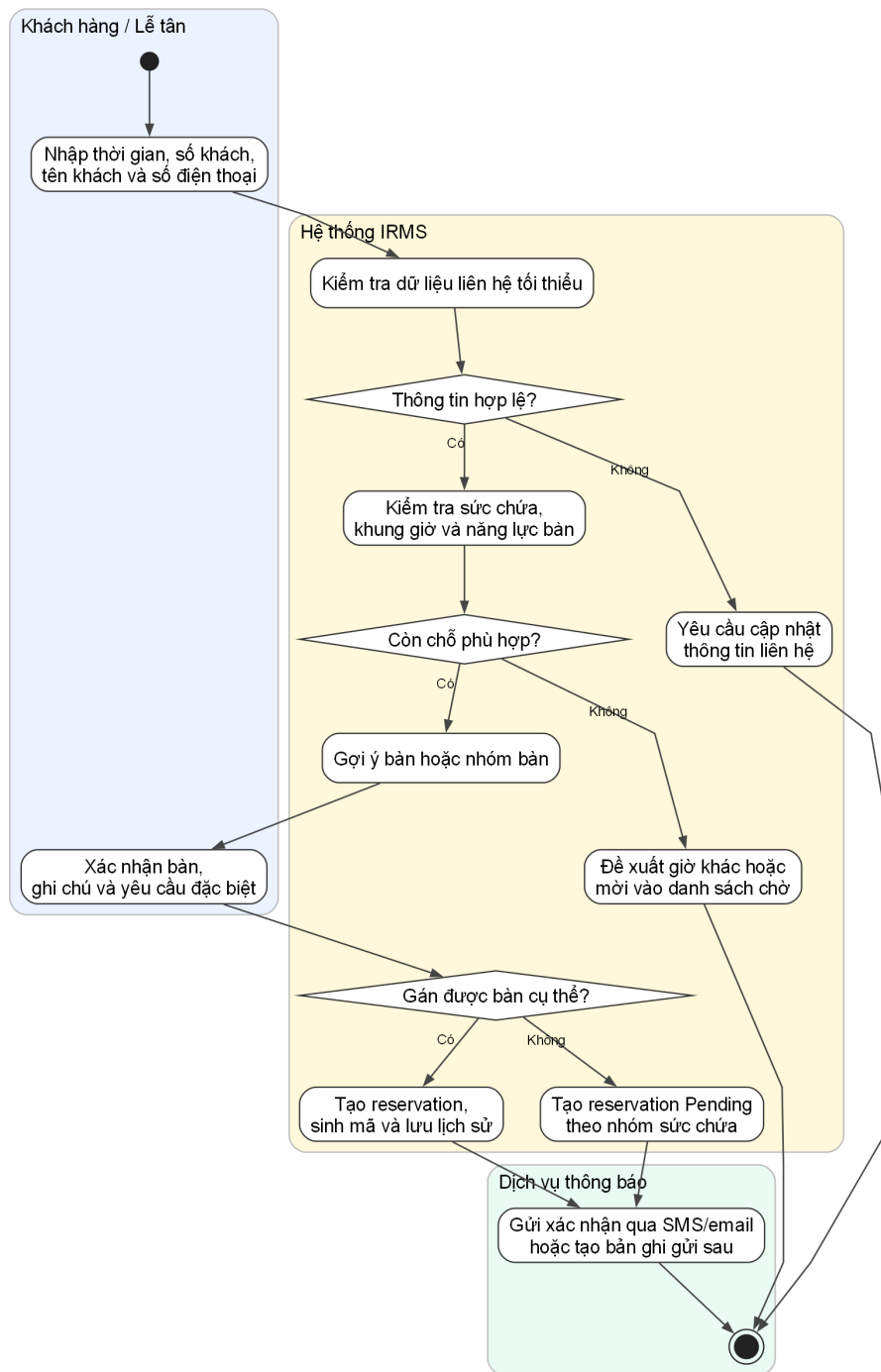
4.3 Thiết kế hành vi dựa trên ca sử dụng

Phần này mở rộng trực tiếp từ các ca sử dụng đã đặc tả ở trên. Phần đặc tả ca sử dụng xác định tác nhân, mục tiêu, điều kiện và kết quả nghiệp vụ; còn sơ đồ hoạt động diễn đạt thứ tự xử lý, điểm rẽ nhánh và nơi phát sinh ngoại lệ trong từng luồng vận hành. Vì vậy, các sơ đồ dưới đây được vẽ theo đúng ranh giới vận hành hiện tại của IRMS: lễ tân, phục vụ, bếp, thu ngân, quản lý và hệ thống IRMS.

Một điểm quan trọng của nhóm sơ đồ này là mức độ chi tiết không hoàn toàn giống nhau. Những ca sử dụng nằm trên tuyến nghiệp vụ nóng như nhận khách, gọi món, gửi bếp, cập nhật chế biến, tạo hóa đơn, thanh toán, hoàn tiền và cập nhật tồn kho được mở rộng sâu hơn vì đây là những nơi quyết định trực tiếp đến độ trễ phản hồi, độ tin cậy giao dịch và khả năng truy vết của toàn hệ thống.

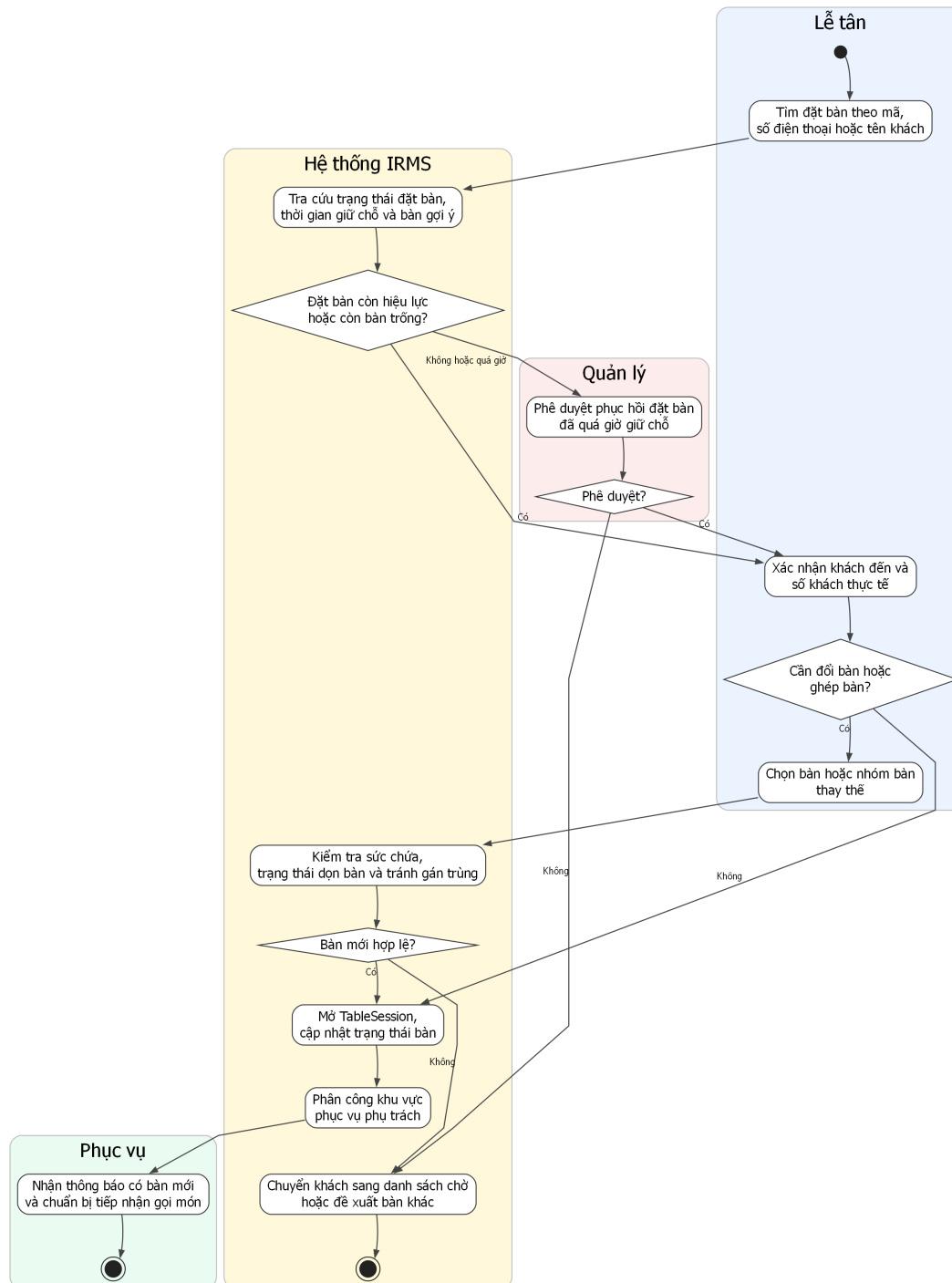
Nhóm đặt bàn và bàn ăn

UC-RES-01 -- Tạo đặt bàn



Hình 37: Sơ đồ hoạt động cho UC-RES-01 – Tạo đặt bàn

UC-RES-02 — Nhận khách và sắp bàn

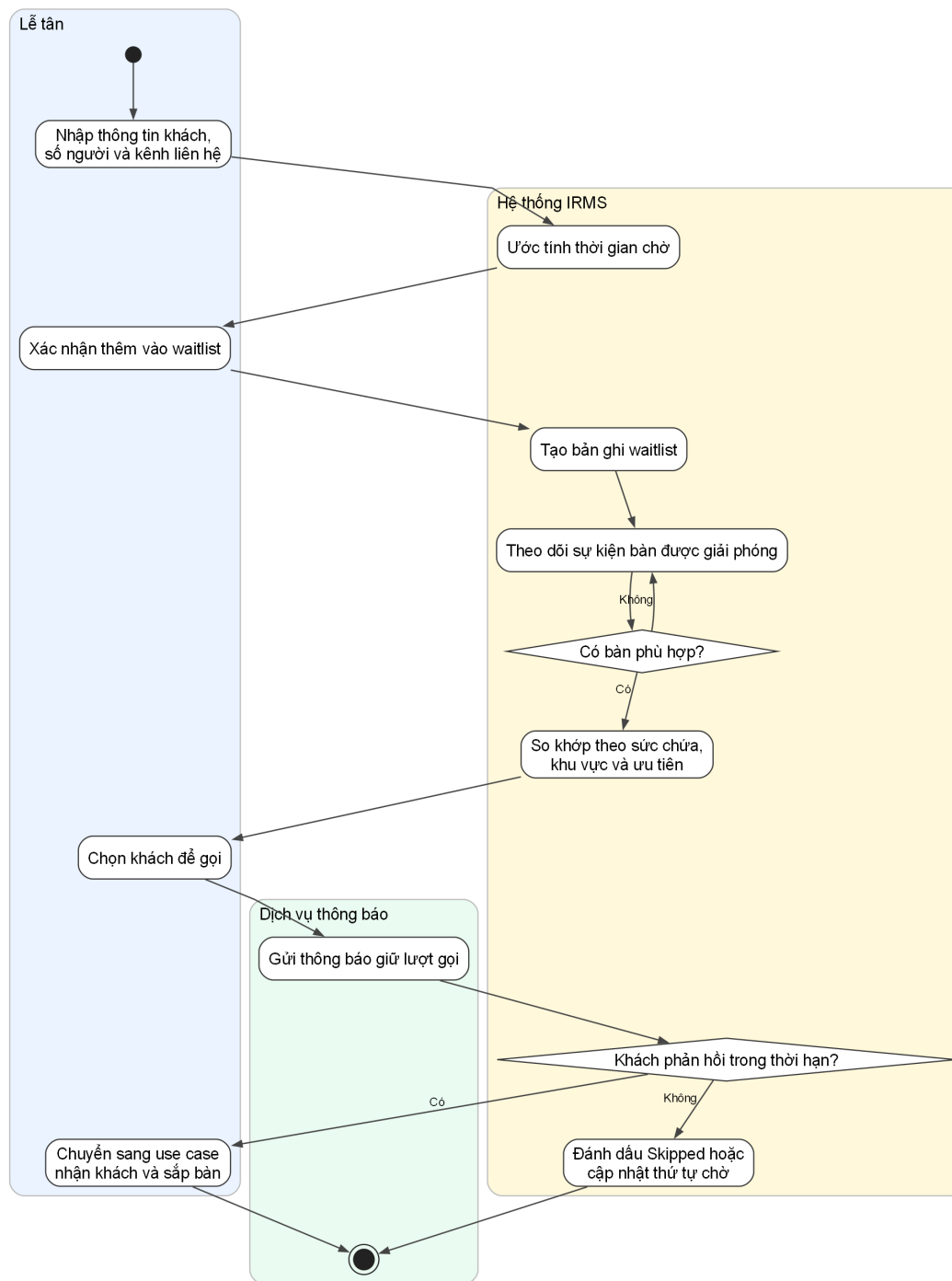


Hình 38: Sơ đồ hoạt động cho UC-RES-02 – Nhận khách và sắp bàn

Phân tích sơ đồ UC-RES-02. Luồng hoạt động ở đây bám đúng thực tế tại quầy lễ tân: tìm đặt bàn, kiểm tra hiệu lực, đối chiếu số khách thực tế, quyết định có cần đổi bàn hoặc ghép bàn hay không, sau đó mới mở `TableSession`. Điểm mấu chốt là việc **mở phiên phục vụ bàn** luôn xảy ra sau bước kiểm tra bàn khả dụng, nhờ đó báo cáo giữ được bất biến quan trọng là một bàn không có hai phiên phục vụ hoạt động cùng lúc.

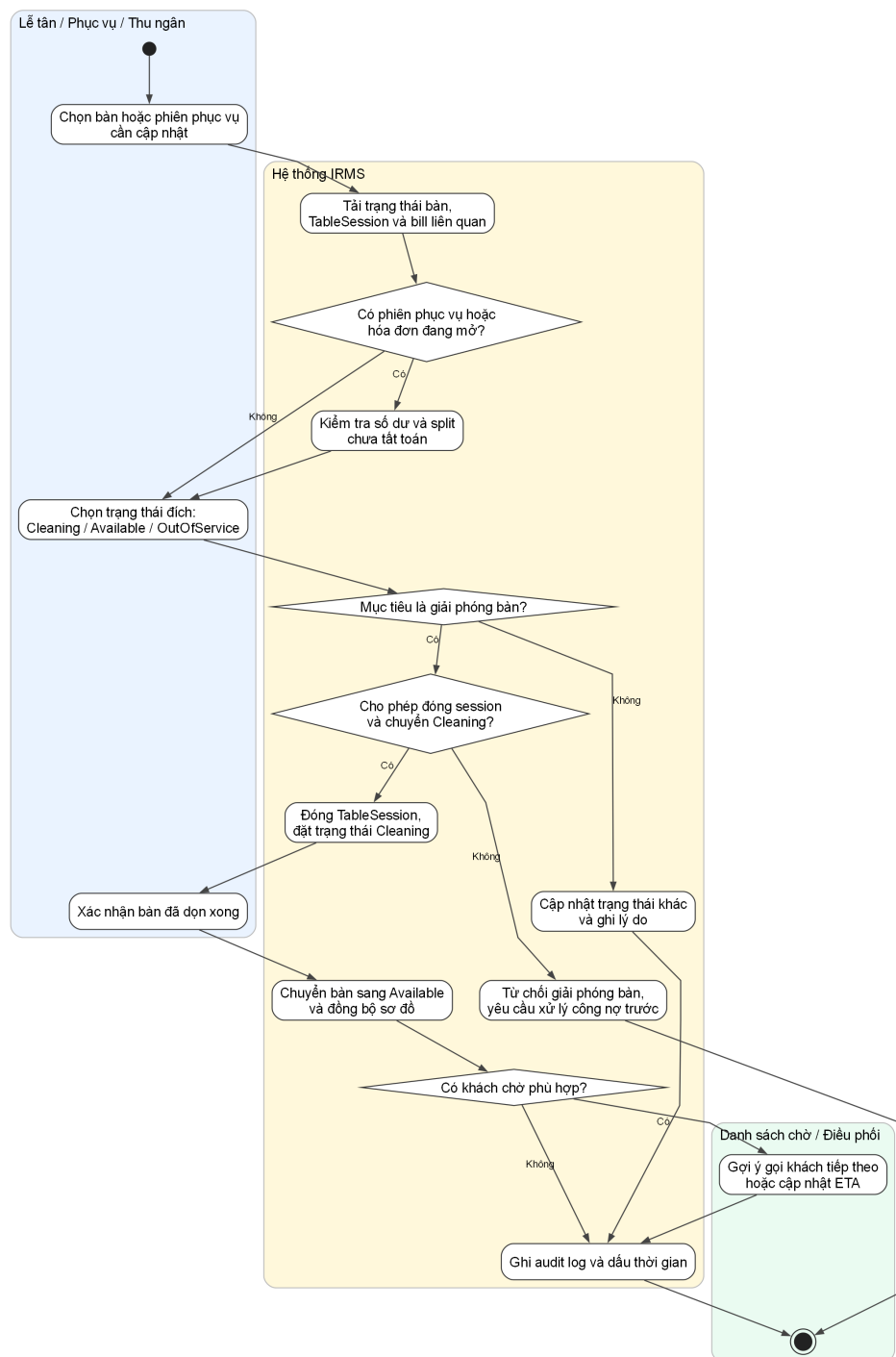
Ngoài ra, sơ đồ cũng làm rõ nhánh ngoại lệ khi đặt bàn đã quá giờ giữ chỗ. Khi đó hệ thống không tự động bỏ qua, mà chuyển sang bước phê duyệt của quản lý. Cách mô tả này bám sát phần đặc tả ca sử dụng phía trên và phản ánh đúng yêu cầu kiểm soát nghiệp vụ của IRMS.

UC-RES-03 -- Quản lý danh sách chờ



Hình 39: Sơ đồ hoạt động cho UC-RES-03 – Quản lý danh sách chờ

UC-RES-04 -- Cập nhật trạng thái bàn và giải phóng bàn



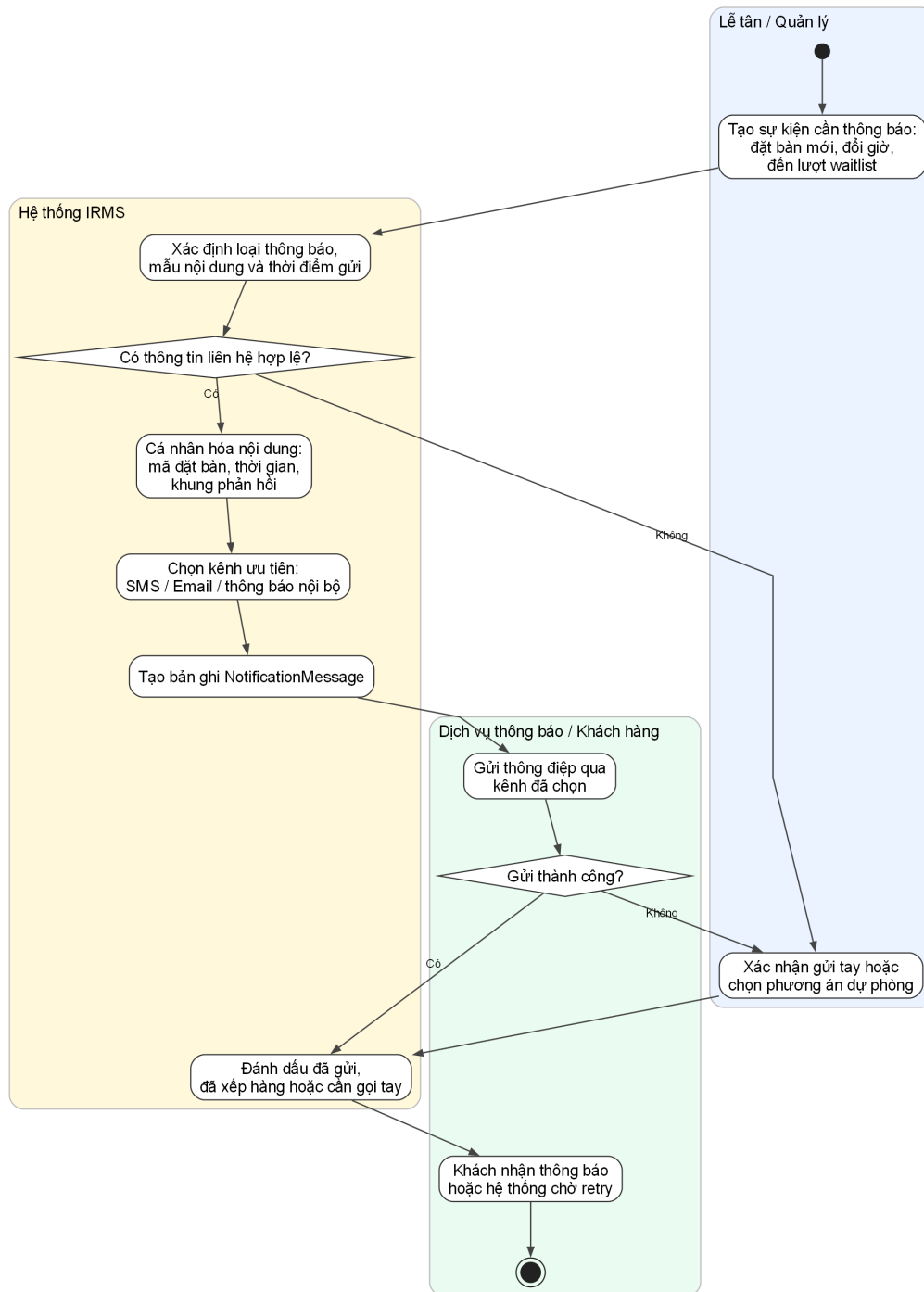
Hình 40: Sơ đồ hoạt động cho UC-RES-04 – Cập nhật trạng thái bàn và giải phóng bàn

Phân tích sơ đồ UC-RES-04. Luồng này làm rõ phần nằm giữa **kết thúc phục vụ** và **bắt đầu vòng quay bàn mới**. Trong vận hành thực tế, việc một bàn trở thành **Available** không thể chỉ là hệ quả ngầm sau thanh toán. Nó phải đi qua chuỗi kiểm tra rõ ràng: còn phiên phục vụ nào đang mở không, còn split hoặc công nợ nào chưa tất toán không, bàn đã chuyển sang **Cleaning** chưa, và nhân viên đã xác nhận dọn xong chưa. Nhờ vậy, trạng thái bàn trên sơ đồ vận hành phản ánh đúng thực tế thay vì chỉ là ước đoán theo cảm tính của từng bộ phận.

Điểm giá trị nhất của sơ đồ là nó nối được ba miền vốn thường bị mô tả rời nhau: **TableSession**, **Billing** và **Waitlist**. Khi bàn được giải phóng thành công, hệ thống không chỉ đổi một nhãn trạng

thái; nó còn cập nhật nền cho việc gọi khách tiếp theo, tính vòng quay bàn và tránh lỗi gán trùng trong giờ cao điểm. Đây là lý do use case này cần đứng riêng thay vì bị ẩn trong một câu mô tả chung về ?đóng bàn?.

UC-RES-05 -- Gửi thông báo cho khách hàng



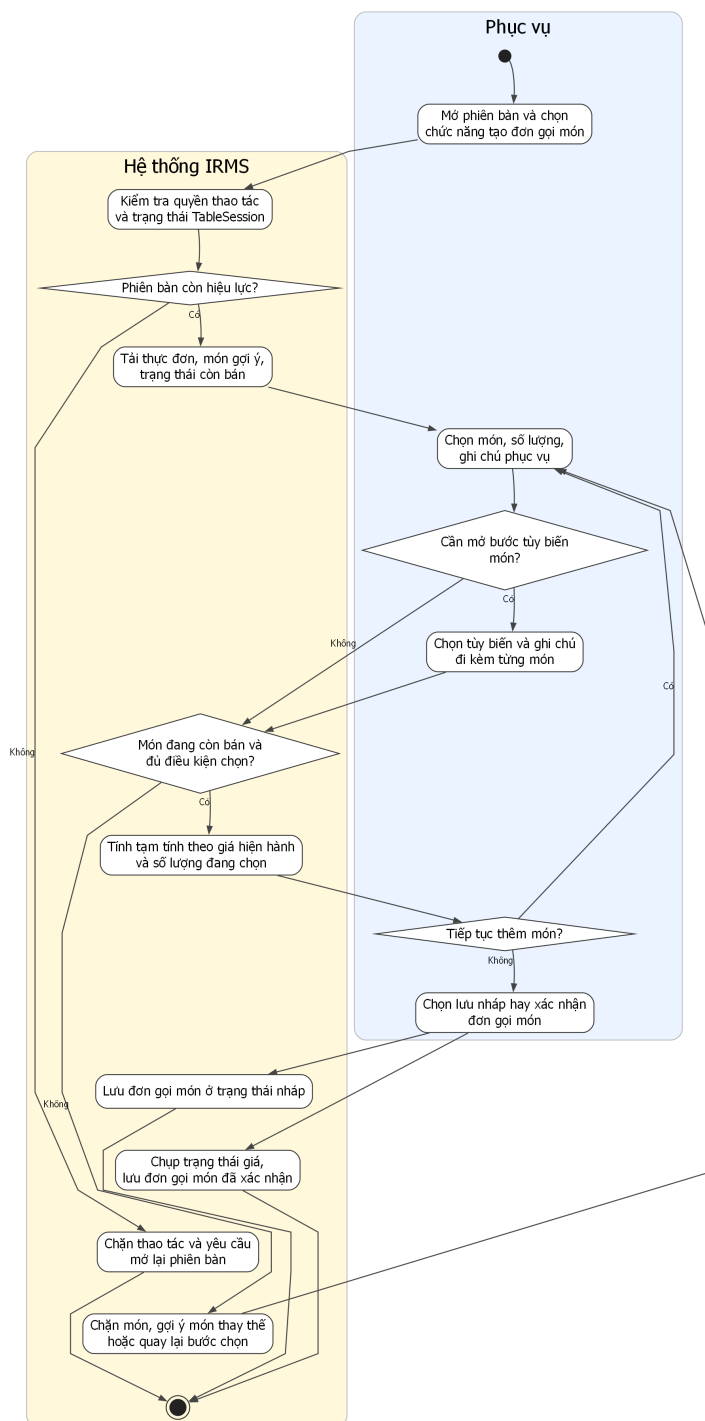
Hình 41: Sơ đồ hoạt động cho UC-RES-05 – Gửi thông báo cho khách hàng

Phân tích sơ đồ UC-RES-05. Sơ đồ thông báo khách hàng cho thấy đây không phải một thao tác phụ có thể bỏ qua, mà là một luồng nghiệp vụ có đầu vào, điều kiện và trạng thái riêng. Trong IRMS, việc thông báo đặt bàn thành công, nhắc giờ đến, gọi khách từ danh sách chờ hay xác nhận thay đổi không nên được mô tả như vài dòng tích hợp kỹ thuật ở rìa hệ thống. Trái lại, nó phải được hiểu là **hệ quả được kiểm soát của sự kiện nghiệp vụ**. Cách mô tả này giúp tách rõ

phần nào thuộc trách nhiệm của lỗi nghiệp vụ, phần nào thuộc bộ kết nối gửi thông báo và phần nào cần cơ chế retry hoặc fallback.

Nhóm gọi món và điều phối bếp

UC-ORD-01 — Tạo đơn gọi món tại bàn



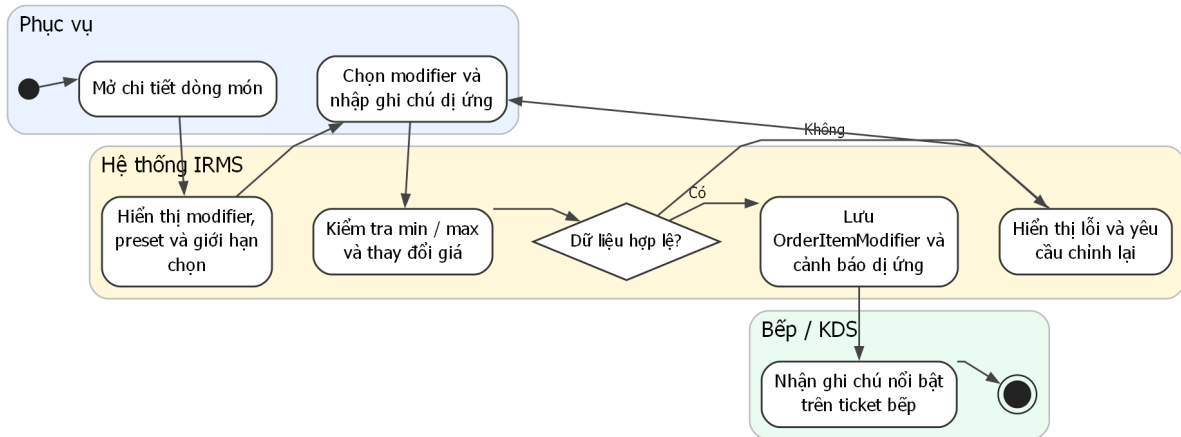
Hình 42: Sơ đồ hoạt động cho UC-ORD-01 – Tạo đơn gọi món tại bàn

Phân tích sơ đồ UC-ORD-01. Phần hoạt động này làm rõ các bước tải thực đơn, kiểm tra quyền thao tác trên phiên bản, chọn món, kiểm tra trạng thái còn bán, quay lại vòng chọn món nếu món vừa hết hàng, tính tạm tính và tách rõ hai nhánh ?lưu nhập? và ?xác nhận đơn?. Điều này quan trọng vì ca sử dụng gọi món không chỉ là thao tác nhập dữ liệu; nó còn là điểm chụp trạng thái giá và tính hợp lệ của món tại thời điểm phục vụ thao tác.

Sơ đồ cũng giữ đúng ranh giới với ca sử dụng gửi bếp. Ở đây, đơn gọi món chỉ được tạo và xác

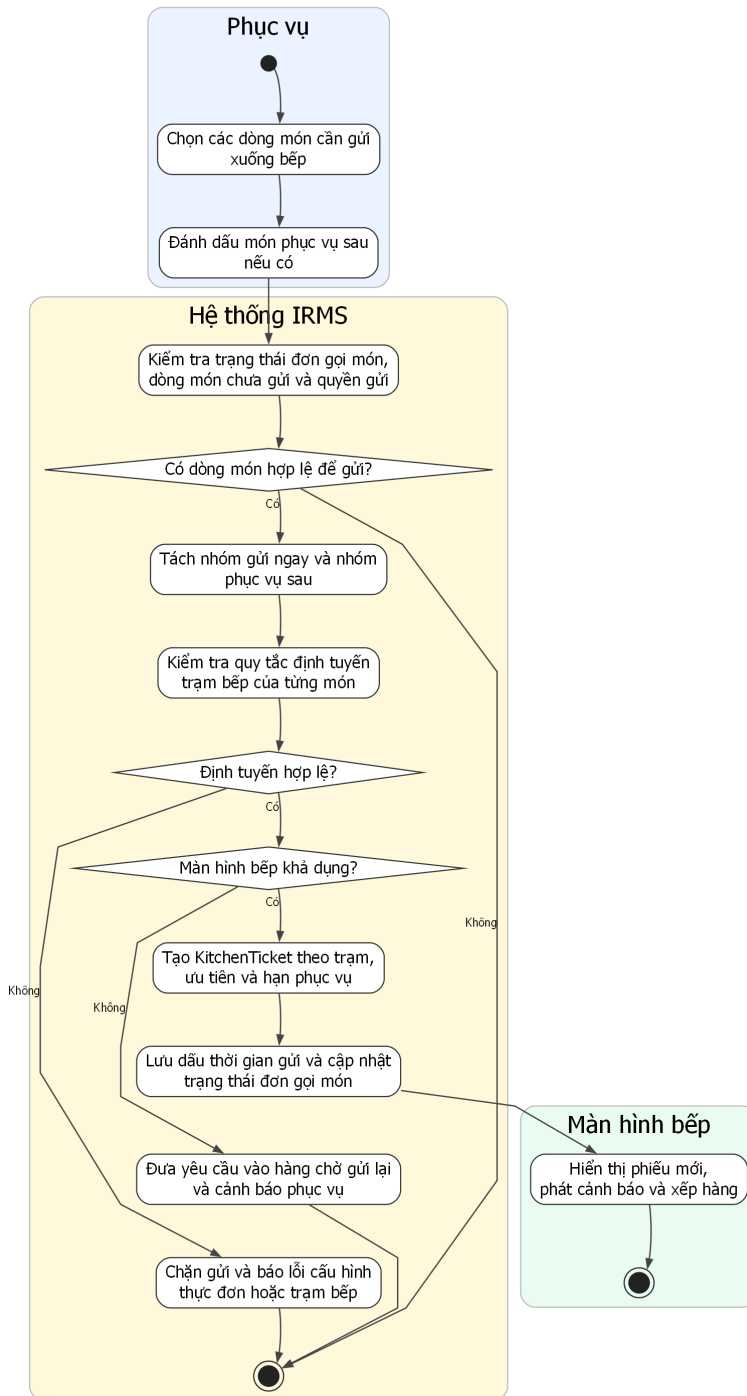
nhận ở mức nghiệp vụ gọi món; việc tạo phiếu bếp và điều phối sang trạm bếp được tách sang sơ đồ riêng, nhờ đó ranh giới trách nhiệm giữa Ordering và Kitchen Coordination rõ ràng hơn.

UC-ORD-02 — Tùy biến món và ghi chú dị ứng



Hình 43: Sơ đồ hoạt động cho UC-ORD-02 – Tùy biến món và ghi chú dị ứng

UC-ORD-03 — Gửi đơn gọi món sang bếp



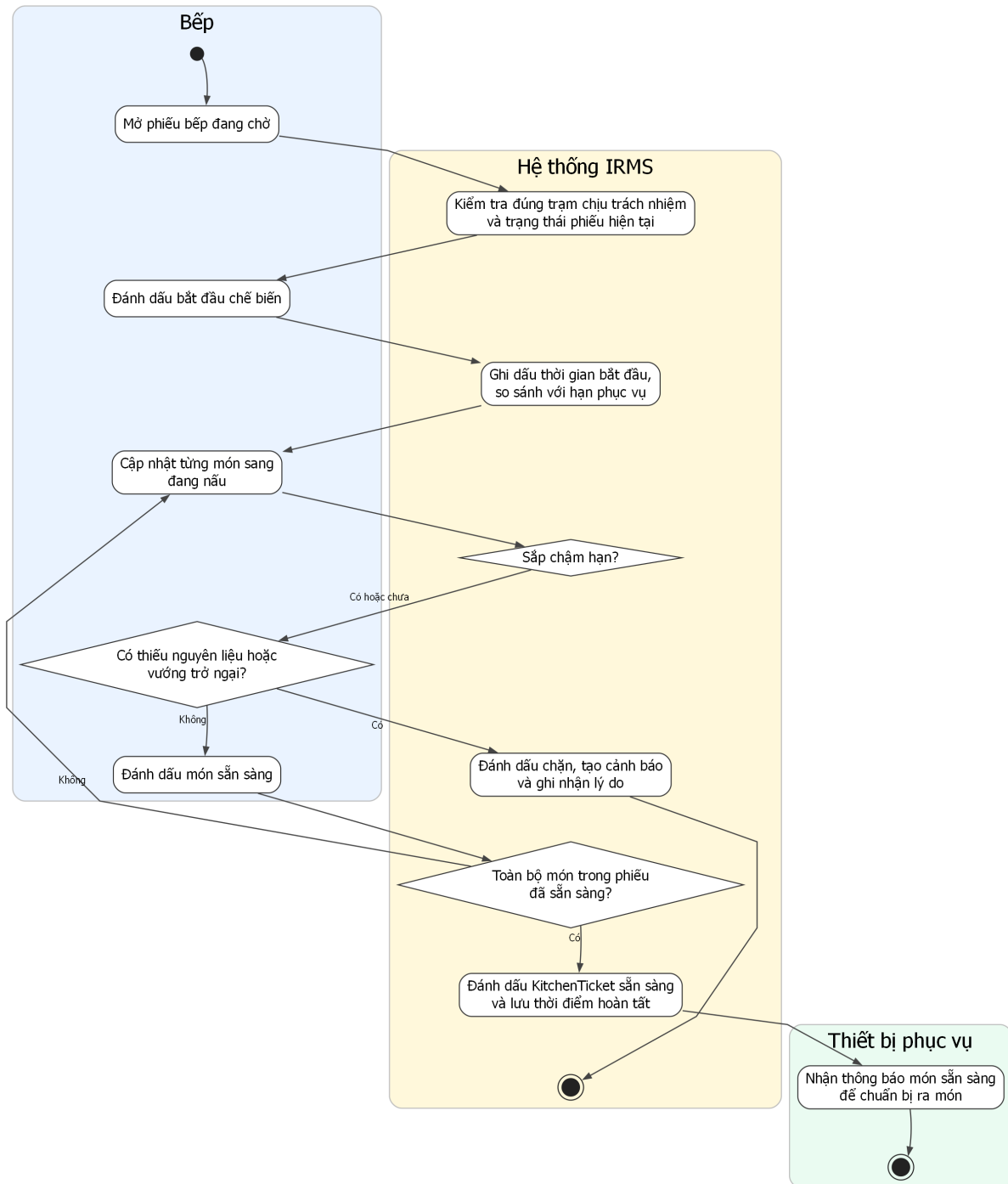
Hình 44: Sơ đồ hoạt động cho UC-ORD-03 – Gửi đơn gọi món sang bếp

Phân tích sơ đồ UC-ORD-03. Đây là một trong những sơ đồ hoạt động quan trọng nhất của báo cáo vì nó nằm đúng trên ranh giới giữa tuyến gọi món của phục vụ và tuyến điều phối của bếp. Luồng hoạt động này làm rõ các bước kiểm tra điều kiện gửi, tách riêng các dòng món gửi ngay và các dòng món phục vụ sau, kiểm tra quy tắc định tuyến theo trạm, tạo KitchenTicket, xếp hàng theo độ ưu tiên và cập nhật trạng thái đơn gọi món sau khi tạo phiếu.

Nhánh ngoại lệ cũng được mô tả rõ hơn: nếu không xác định được trạm bếp hoặc màn hình bếp không khả dụng, hệ thống không được phép làm mất dữ liệu đã xác nhận mà phải chuyển sang hàng chờ gửi lại và phát cảnh báo cho phục vụ. Đây là đúng tinh thần của phần yêu cầu phi chức

năng về độ tin cậy và không làm mất đơn đã xác nhận.

UC-KIT-01 — Cập nhật tiến độ chế biến

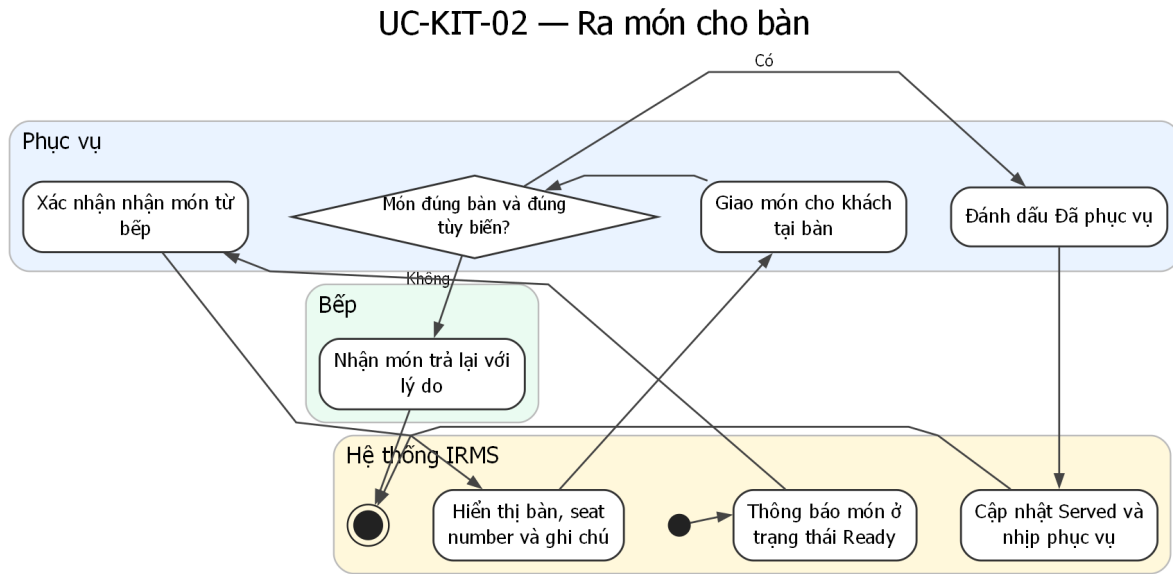


Hình 45: Sơ đồ hoạt động cho UC-KIT-01 – Cập nhật tiến độ chế biến

Phân tích sơ đồ UC-KIT-01. Luồng hoạt động ở đây bám đúng nhịp vận hành của bếp: nhận phiếu, kiểm tra trạm chịu trách nhiệm, đánh dấu bắt đầu, chuyển sang chế biến, ghi dấu thời gian cho từng lần đổi trạng thái, đánh giá nguy cơ chậm hạn và cuối cùng xác nhận món sẵn sàng. Mức chi tiết này giúp phần báo cáo không chỉ dừng ở mô tả ?bếp cập nhật trạng thái?, mà còn cho thấy trạng thái nào cần được ghi nhận để phục vụ phân tích vận hành về sau.

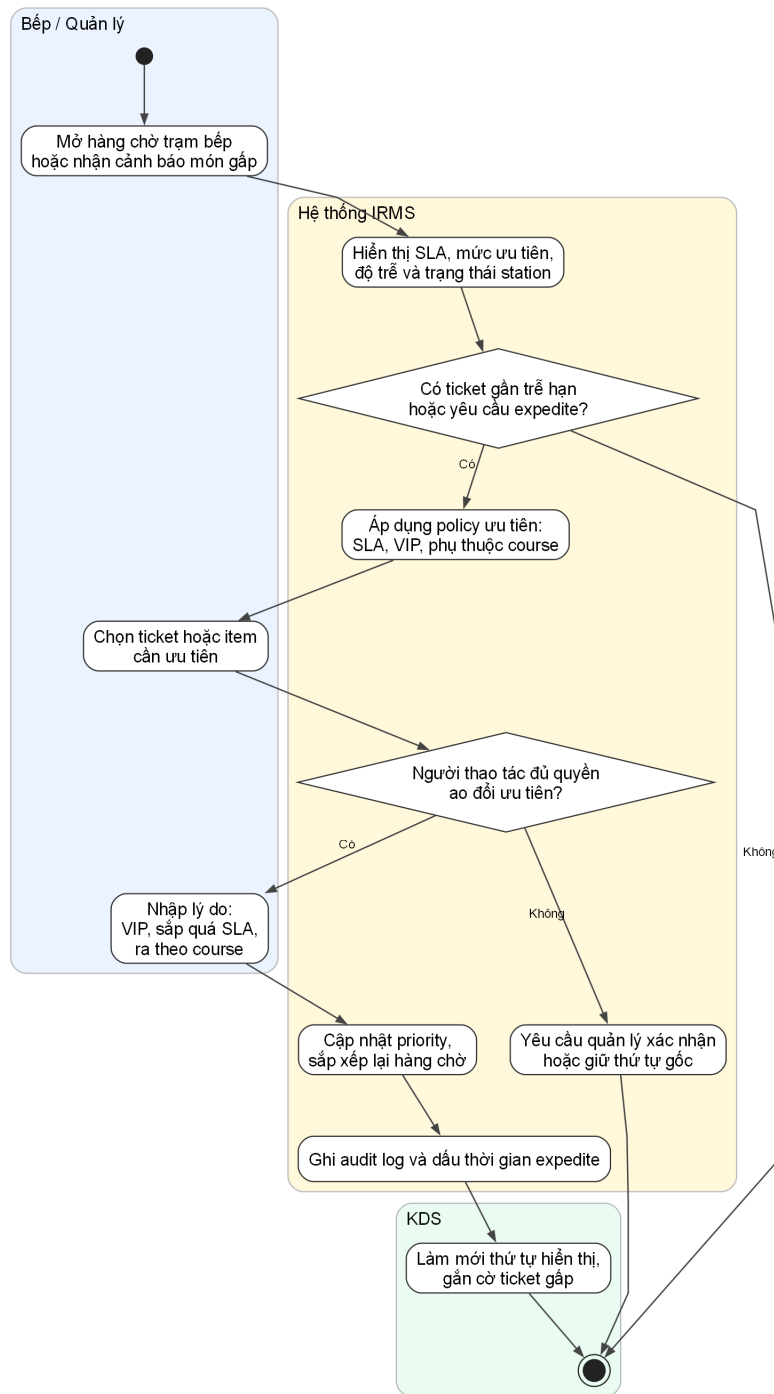
Phần nhánh ?thiếu nguyên liệu hoặc bị chặn? cũng rất quan trọng. Khi đó hệ thống không kết thúc im lặng mà phải đánh dấu chặn, phát cảnh báo và trả thông tin ngược về phía phục vụ. Điều

này giữ cho luồng giao tiếp giữa bếp và khu vực phục vụ thống nhất với phần kiến trúc đã mô tả ở các góc nhìn trước.



Hình 46: Sơ đồ hoạt động cho UC-KIT-02 – Ra món cho bàn

UC-KIT-03 -- Ưu tiên hàng chờ bếp và xử lý món gấp

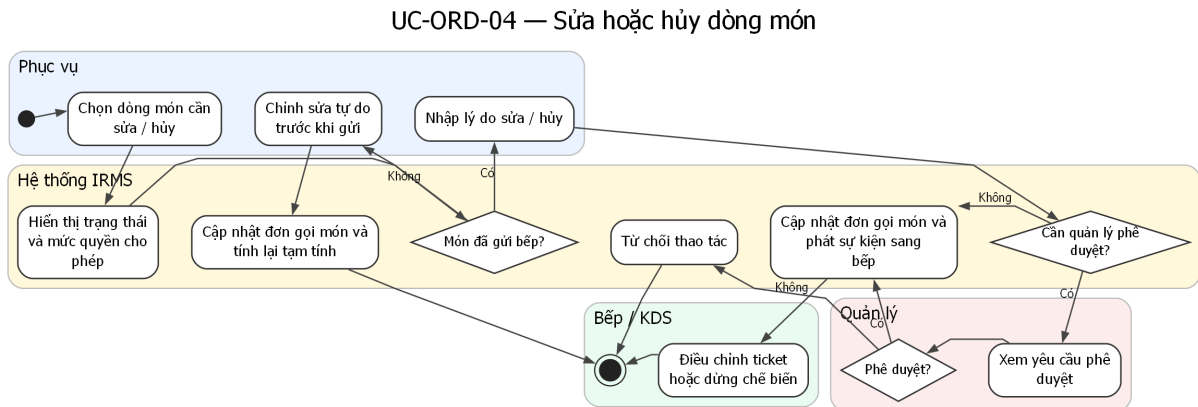


Hình 47: Sơ đồ hoạt động cho UC-KIT-03 – Ưu tiên hàng chờ bếp và xử lý món gấp

Phân tích sơ đồ UC-KIT-03. Luồng này làm rõ một điểm mà các tài liệu nhà hàng rất hay nói tới nhưng lại ít mô hình hóa đến nơi đến chốn: **bếp không chỉ nhận phiếu bếp, mà còn phải điều phối thứ tự thực thi**. Trong giờ cao điểm, nếu mọi ticket đều bị xử lý theo đúng thứ tự đến mà không xét SLA, cấu trúc course, khách VIP hay món sắp trễ hạn, hệ thống KDS sẽ chỉ là bảng hiển thị thụ động. Vì vậy, ca sử dụng ưu tiên hàng chờ bếp được trình bày như một luồng độc lập để phản ánh đúng yêu cầu *“prioritize queues”* của đề bài.

Về mặt kiến trúc, sơ đồ cũng cho thấy quyết định ưu tiên phải đi qua chính sách có kiểm soát, có ghi lý do và có giới hạn quyền, chứ không phải bất kỳ nhân viên nào cũng có thể tùy ý đổi thứ

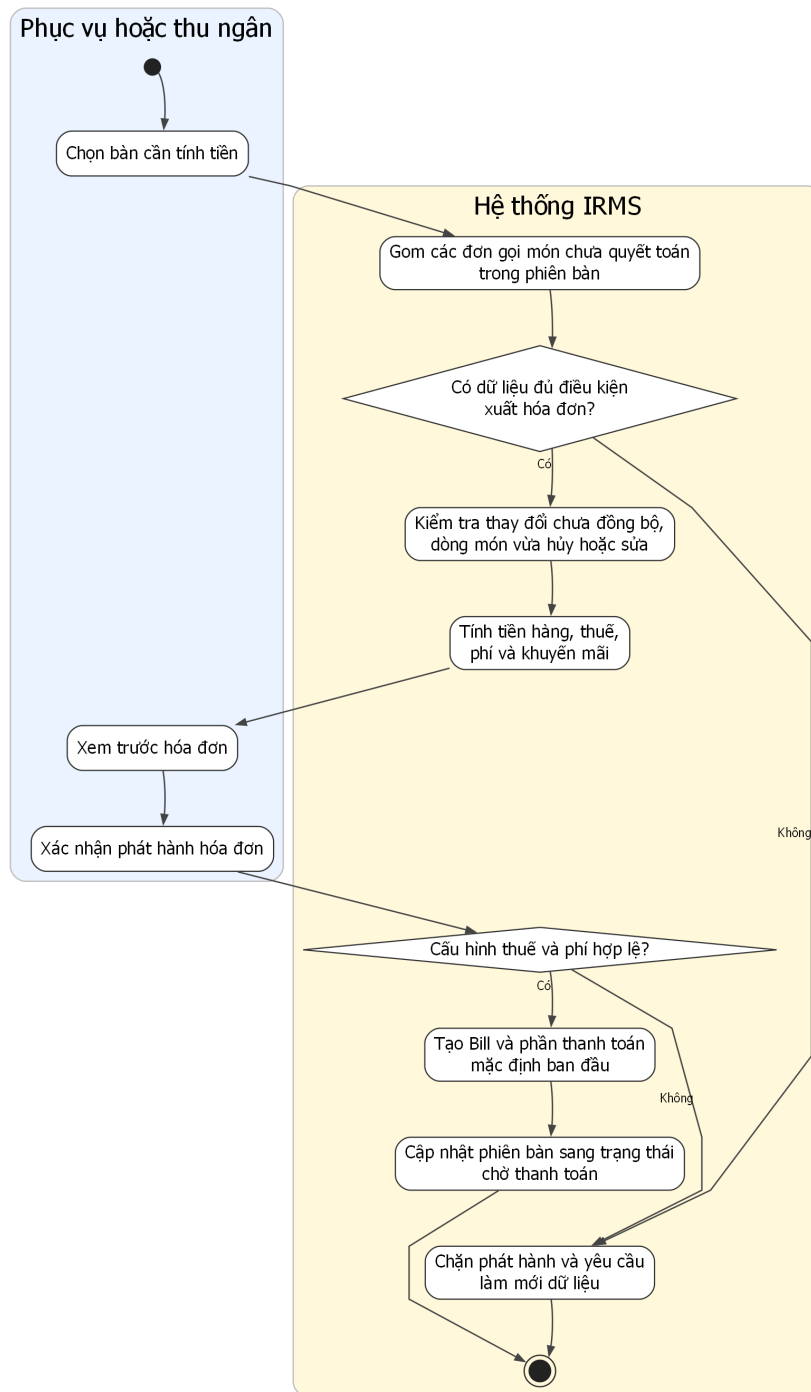
tự chế biến. Điều này giúp IRMS vừa hỗ trợ vận hành linh hoạt, vừa giữ được khả năng truy vết khi có khiếu nại về món ra chậm hoặc món bị đẩy lùi không hợp lý.



Hình 48: Sơ đồ hoạt động cho UC-ORD-04 – Sửa hoặc hủy dòng món

Nhóm hóa đơn, thanh toán và biên lai

UC-BIL-01 — Tạo hóa đơn cho bàn

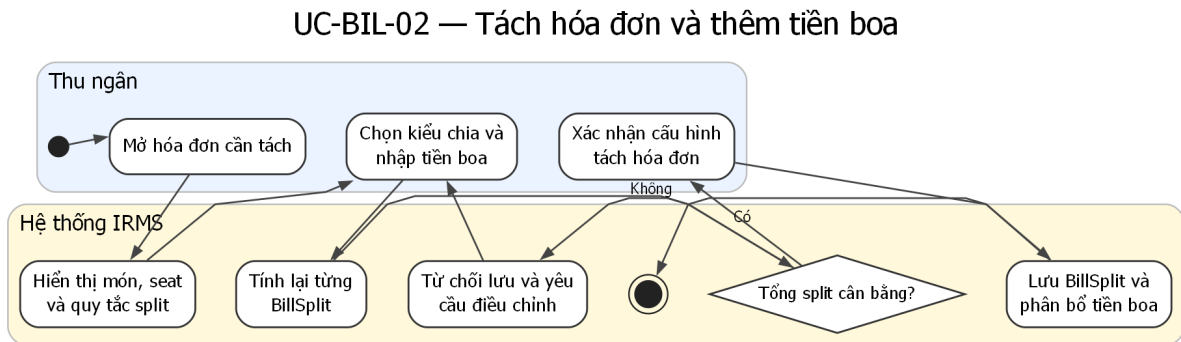


Hình 49: Sơ đồ hoạt động cho UC-BIL-01 – Tạo hóa đơn cho bàn

Phân tích sơ đồ UC-BIL-01. Sơ đồ tạo hóa đơn phản ánh đúng thứ tự xử lý của phần quyết toán: gom các đơn gọi món chưa quyết toán, kiểm tra điều kiện xuất hóa đơn, tính tiền hàng, thuế, phí và khuyến mãi, sau đó mới cho phép phục vụ hoặc thu ngân xem trước và xác nhận phát hành. Việc đặt bước ?xem trước hóa đơn? trước ?phát hành? là cần thiết vì đây là điểm cuối cùng để phát hiện sai lệch nếu một dòng món vừa bị hủy hoặc vừa có thay đổi cấu hình giá.

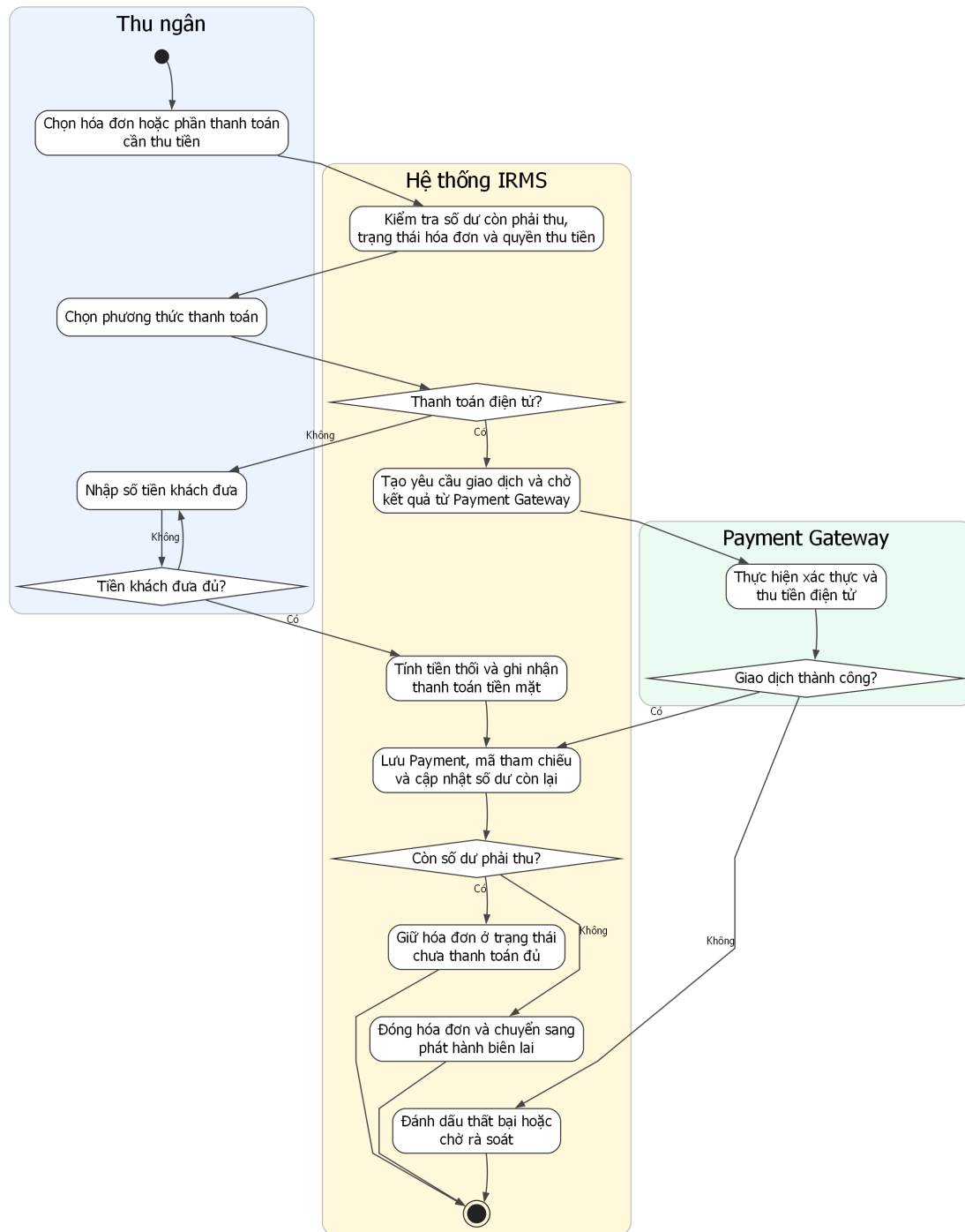
Sơ đồ cũng thể hiện rõ nhánh chặn phát hành khi dữ liệu chưa hợp lệ. Điều này phù hợp với

đặc tả ca sử dụng ở trên và làm nổi bật rằng Bill không phải chỉ là phép cộng cuối cùng, mà là một thực thể nghiệp vụ có điều kiện phát hành chặt chẽ.



Hình 50: Sơ đồ hoạt động cho UC-BIL-02 – Tách hóa đơn và thêm tiền boa

UC-PAY-01 — Xử lý thanh toán



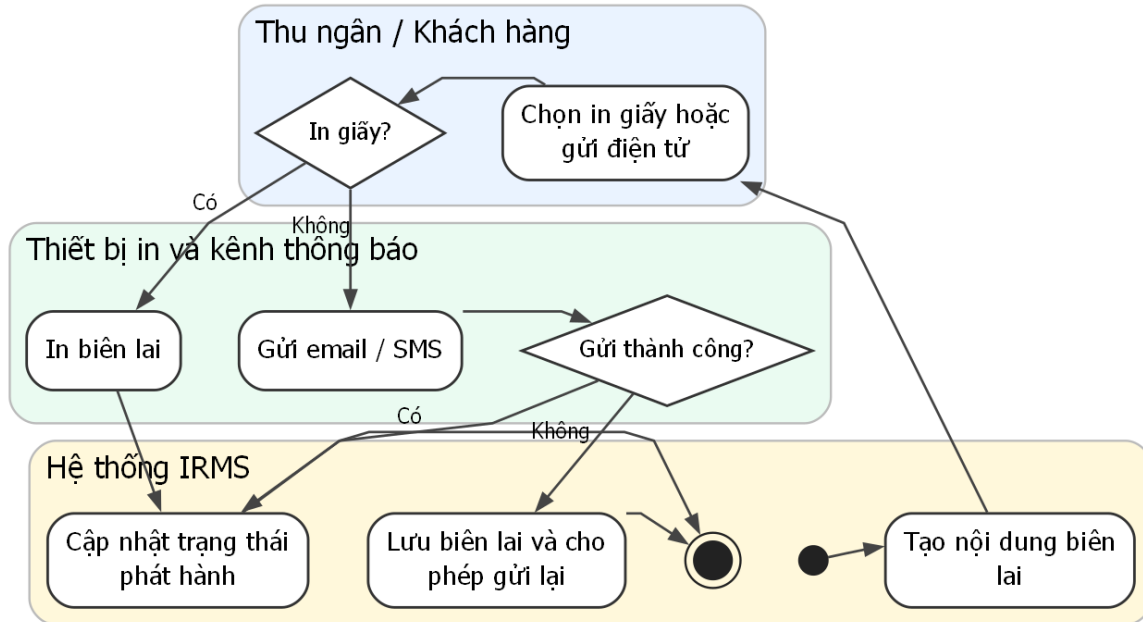
Hình 51: Sơ đồ hoạt động cho UC-PAY-01 – Xử lý thanh toán

Phân tích sơ đồ UC-PAY-01. Đây là sơ đồ hoạt động có ý nghĩa kiểm soát cao nhất trong toàn bộ phần hành vi. Luồng này làm rõ hai tuyến thanh toán khác nhau: thanh toán điện tử qua **Payment Gateway** và thanh toán tiền mặt tại quầy. Với thanh toán điện tử, hệ thống phải tạo yêu cầu giao dịch, gửi sang cổng thanh toán, nhận kết quả, lưu mã tham chiếu và xử lý riêng các trạng thái thành công, thất bại hoặc chờ rà soát. Với thanh toán tiền mặt, hệ thống tính số tiền thối rồi mới cập nhật số dư còn lại.

Sau bước ghi nhận thanh toán, hệ thống còn phải đánh giá hóa đơn đã được thanh toán đủ hay chưa. Nếu chưa đủ, hóa đơn vẫn mở; nếu đủ, hóa đơn được đóng và luồng phát hành biên lai mới

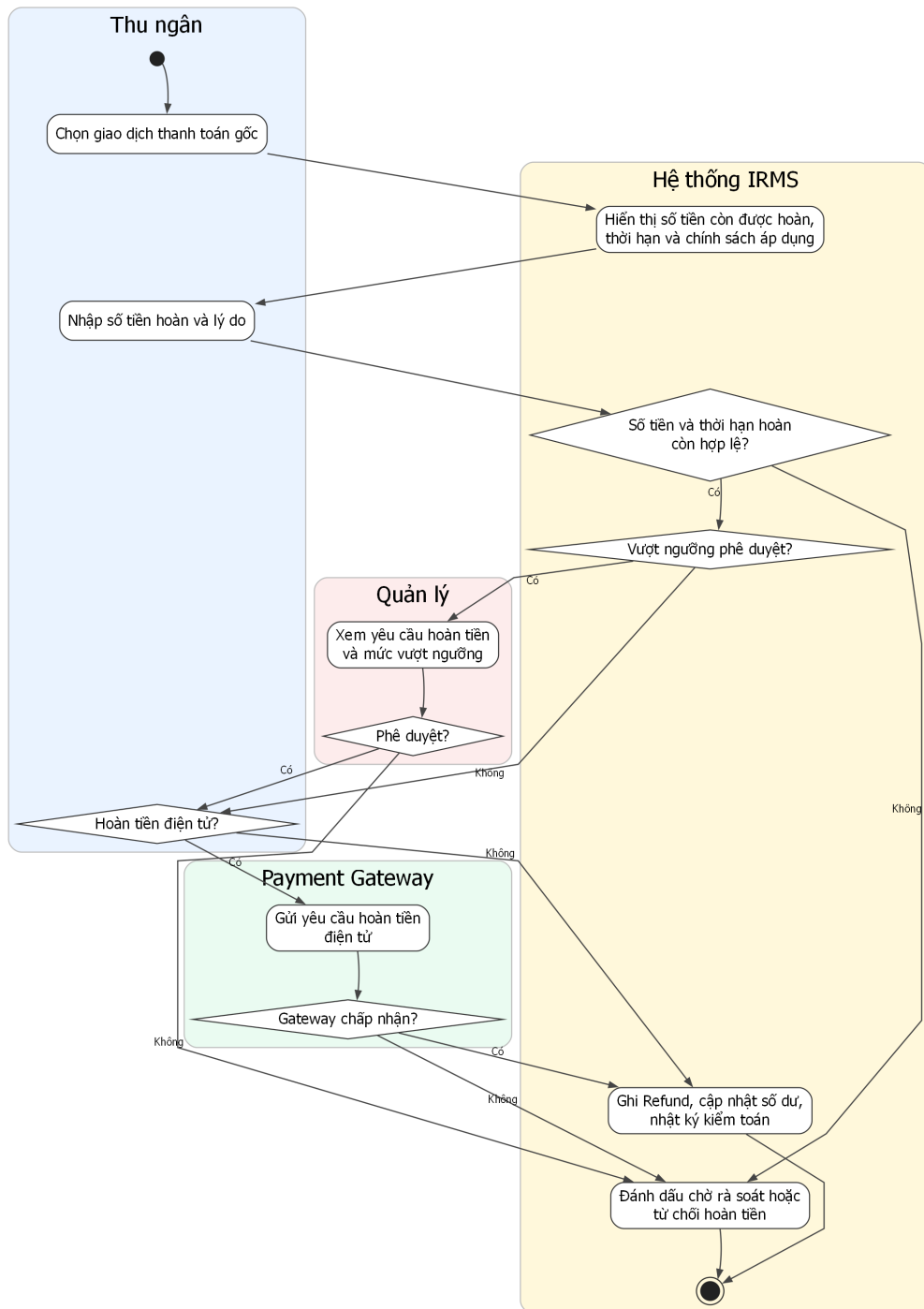
được mở ra. Cách chi tiết này bám đúng kiến trúc hiện tại của IRMS và tránh nhập nhằng giữa ?đã có một giao dịch thanh toán? và ?đã tắt toán hóa đơn?.

UC-PAY-02 — Phát hành biên lai



Hình 52: Sơ đồ hoạt động cho UC-PAY-02 – Phát hành biên lai

UC-PAY-03 — Hoàn tiền



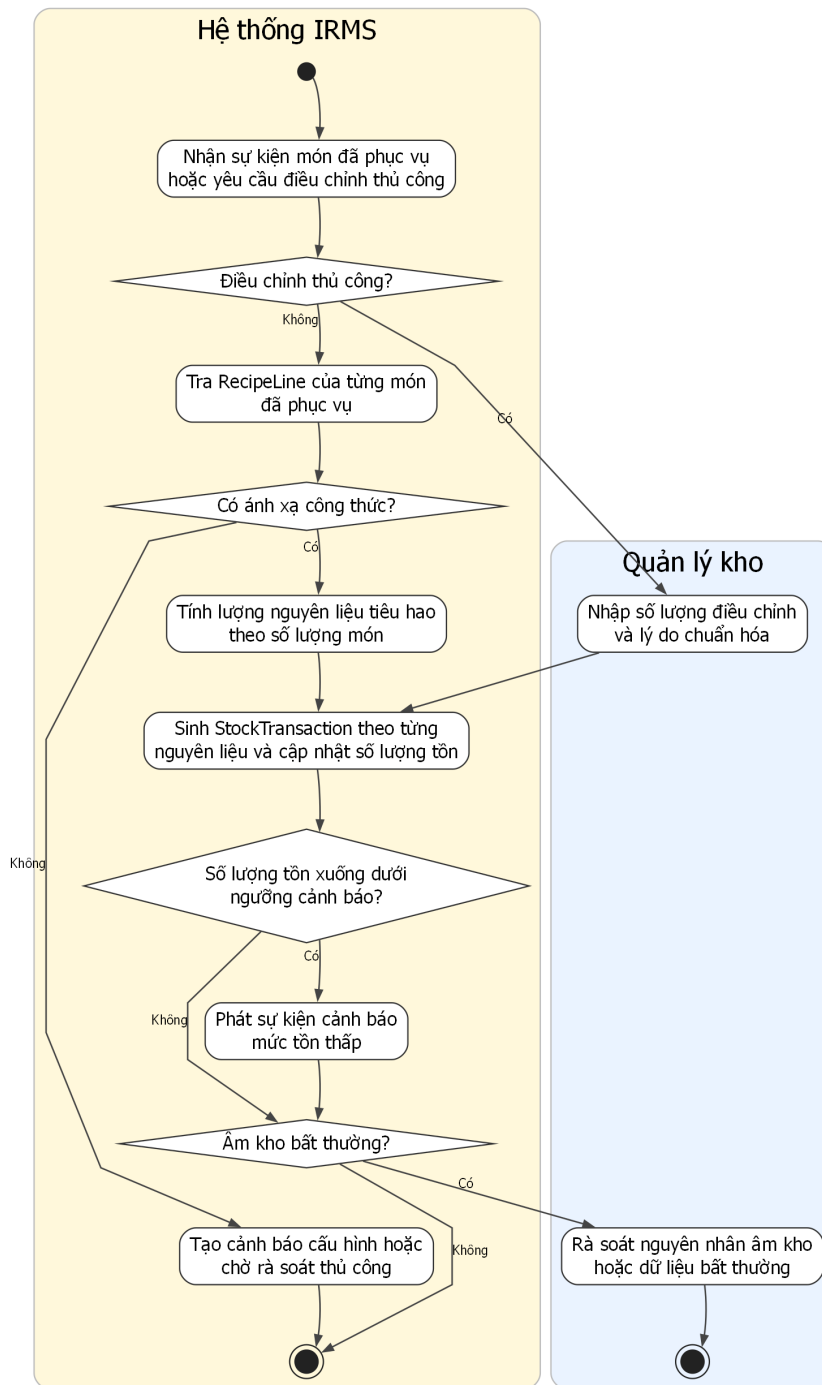
Hình 53: Sơ đồ hoạt động cho UC-PAY-03 – Hoàn tiền

Phân tích sơ đồ UC-PAY-03. Sơ đồ hoàn tiền hiện đã thể hiện đầy đủ hơn các điểm kiểm soát bắt buộc: chọn giao dịch gốc, kiểm tra số tiền còn được hoàn, nhập lý do, so ngưỡng phê duyệt, xin xác nhận của quản lý nếu cần, gửi yêu cầu hoàn tiền tới **Payment Gateway** hoặc ghi nhận hoàn tiền thủ công, rồi mới cập nhật sổ kiểm toán. Điều này rất quan trọng vì hoàn tiền là thao tác nhạy cảm nhất của vùng thanh toán.

Một chi tiết đáng chú ý là sơ đồ đã tách rõ nhánh ?chờ rà soát? khi cổng thanh toán không chấp nhận hoặc phản hồi không rõ ràng. Nhờ đó báo cáo có thể giải thích trạng thái nghiệp vụ chặt chẽ hơn, thay vì gom mọi trường hợp không thành công vào một nhãn lỗi duy nhất.

Nhóm tồn kho, cảnh báo và phân tích

UC-INV-01 — Cập nhật tiêu hao tồn kho sau phục vụ

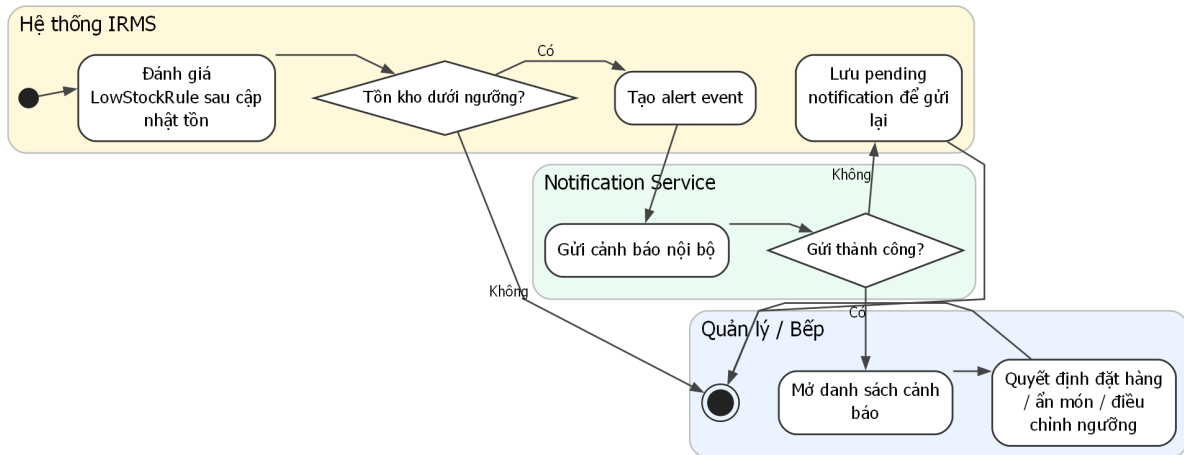


Hình 54: Sơ đồ hoạt động cho UC-INV-01 – Cập nhật tiêu hao tồn kho sau phục vụ

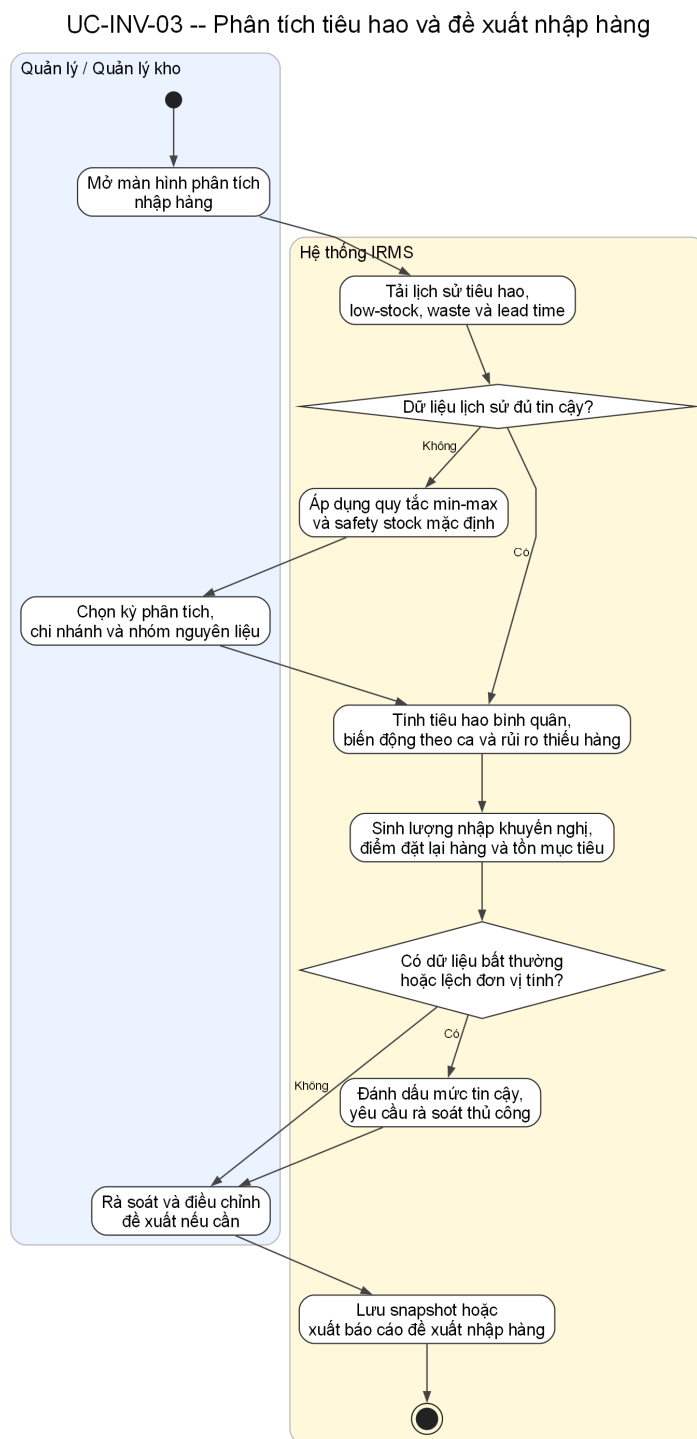
Phân tích sơ đồ UC-INV-01. Sơ đồ cập nhật tiêu hao tồn kho phản ánh đúng bản chất suy diễn của vùng tồn kho: nhận sự kiện từ món đã phục vụ hoặc điều chỉnh thủ công, tra công thức nguyên liệu, tính lượng tiêu hao theo số lượng món, sinh **StockTransaction** theo từng nguyên liệu, cập nhật số lượng tồn hiện có, rồi đánh giá tiếp các điều kiện bất thường. Đây là mô tả phù hợp với kiến trúc của IRMS, nơi vùng tồn kho không tự suy đoán món bán ra từ giao diện mà dựa vào tín hiệu nghiệp vụ từ tuyến phục vụ.

Sơ đồ cũng làm rõ hai nhánh ngoại lệ quan trọng: thiếu ánh xạ công thức và âm kho bất thường. Nhờ đó phần thiết kế hành vi bám sát hơn với ca sử dụng đã đặc tả, đồng thời nổi được với sơ đồ cảnh báo sắp hết hàng ngay bên dưới.

UC-INV-02 — Nhận cảnh báo sắp hết hàng



Hình 55: Sơ đồ hoạt động cho UC-INV-02 – Nhận cảnh báo sắp hết hàng



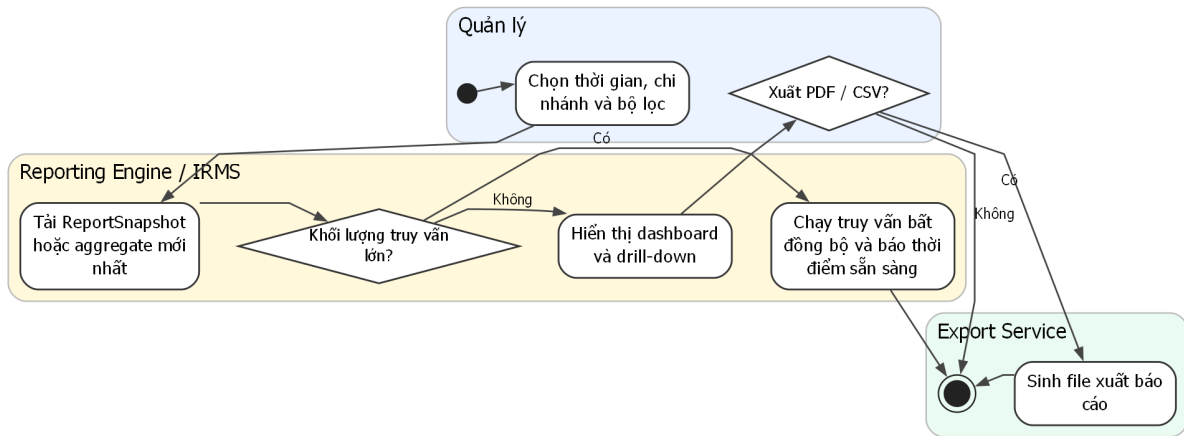
Hình 56: Sơ đồ hoạt động cho UC-INV-03 – Phân tích tiêu hao và đề xuất nhập hàng

Phân tích sơ đồ UC-INV-03. Ca sử dụng này làm cho phần tồn kho bám sát hơn với yêu cầu?historical usage helps predict reorder quantities and manage waste?. Nếu chỉ dừng ở cập nhật tiêu hao và cảnh báo sắp hết hàng, IRMS mới hỗ trợ phản ứng vận hành ngắn hạn; nó chưa hỗ trợ tốt cho quyết định chuẩn bị hàng hóa ở chu kỳ ngày, tuần hoặc kỳ cao điểm. Vì vậy, sơ đồ chuyển trọng tâm của vùng tồn kho từ **theo dõi hiện trạng** sang **hỗ trợ quyết định**.

Điểm quan trọng của sơ đồ là nó không mô tả việc dự báo như một thuật toán mơ hồ, mà làm rõ chuỗi dữ liệu đầu vào: tiêu hao lịch sử, low-stock alert, hao hụt, lead time và mức tồn an toàn. Điều này rất có ích trong báo cáo kiến trúc, vì nó cho thấy đề xuất nhập hàng là một sản phẩm

của mô hình dữ liệu và luồng xử lý đã có, không phải một chức năng đặt thêm ở phần quản trị mà không có nền dữ liệu đủ tin cậy.

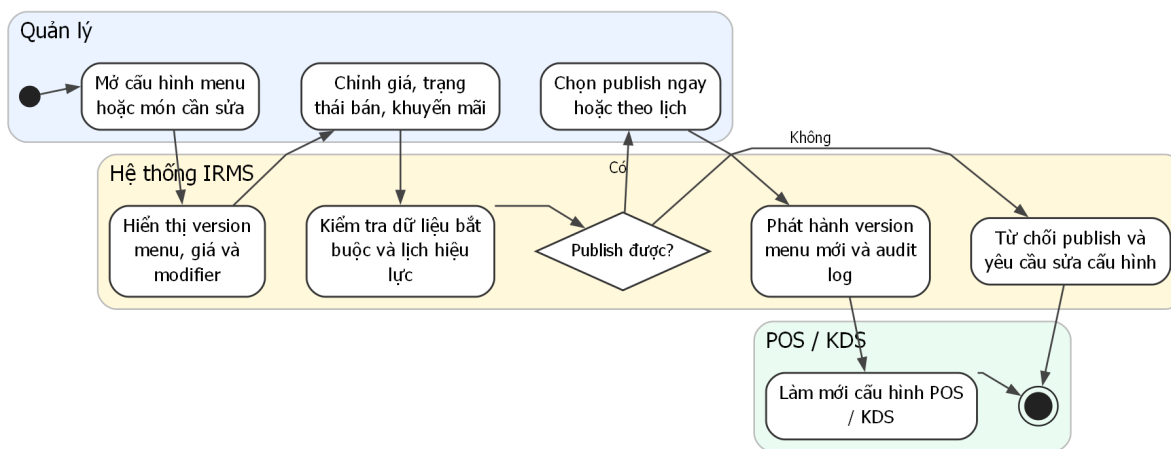
UC-REP-01 — Xem báo cáo doanh thu và vận hành



Hình 57: Sơ đồ hoạt động cho UC-REP-01 – Xem báo cáo doanh thu và vận hành

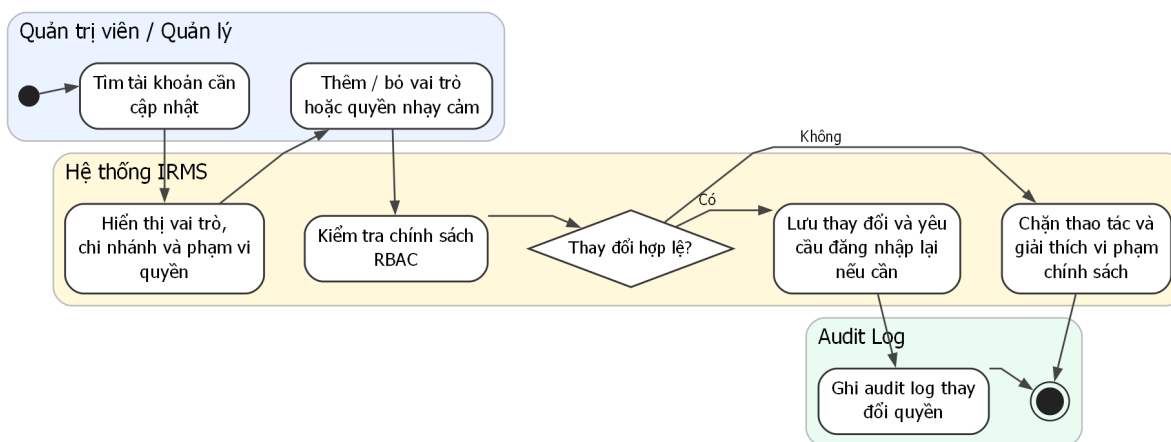
Nhóm quản trị cấu hình và vận hành

UC-ADM-01 — Quản lý menu và giá bán



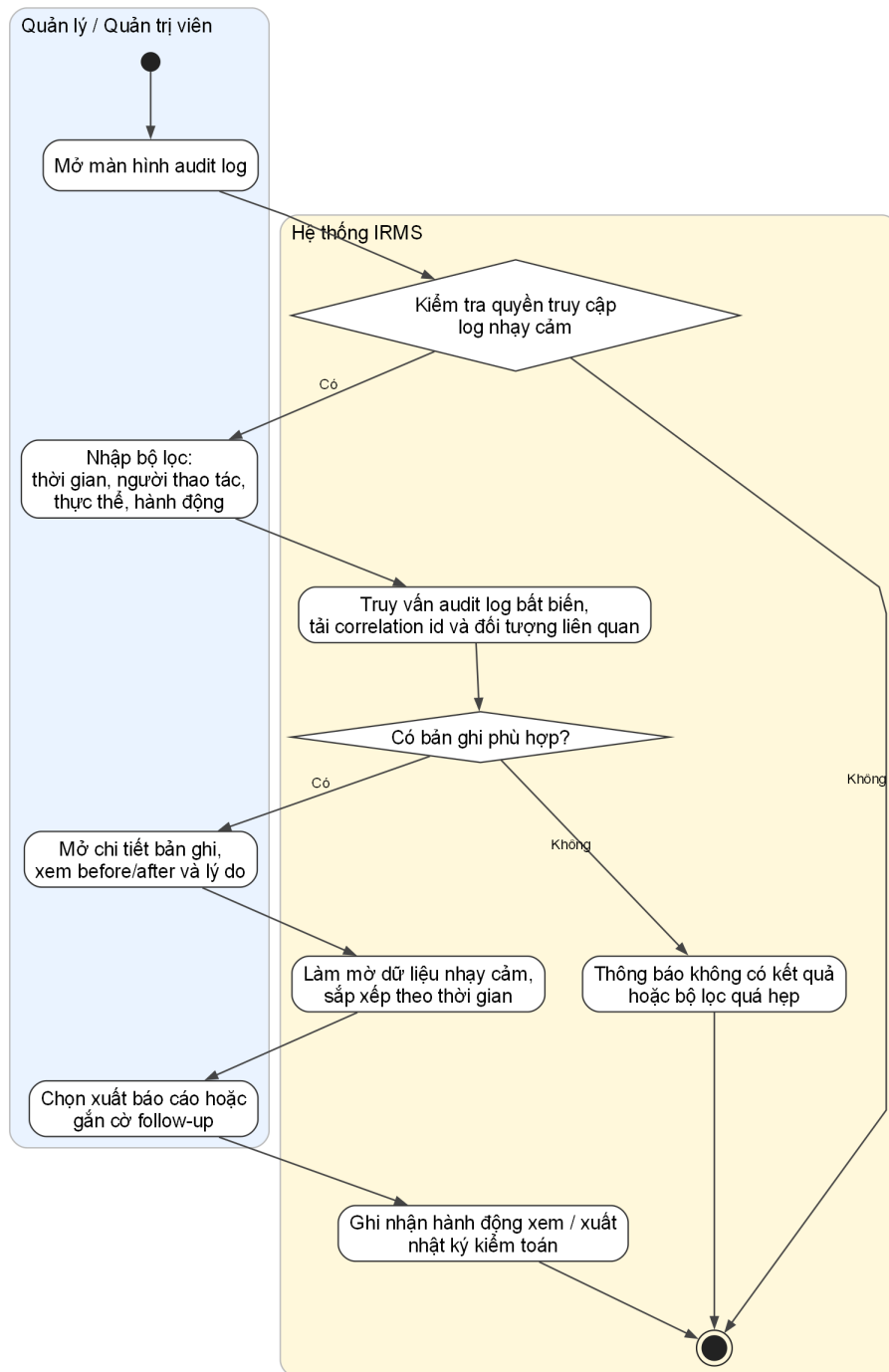
Hình 58: Sơ đồ hoạt động cho UC-ADM-01 – Quản lý thực đơn và giá bán

UC-ADM-02 — Quản lý vai trò và phân quyền



Hình 59: Sơ đồ hoạt động cho UC-ADM-02 – Quản lý vai trò và phân quyền

UC-ADM-03 -- Xem nhật ký kiểm toán và truy vết thao tác nhạy cảm



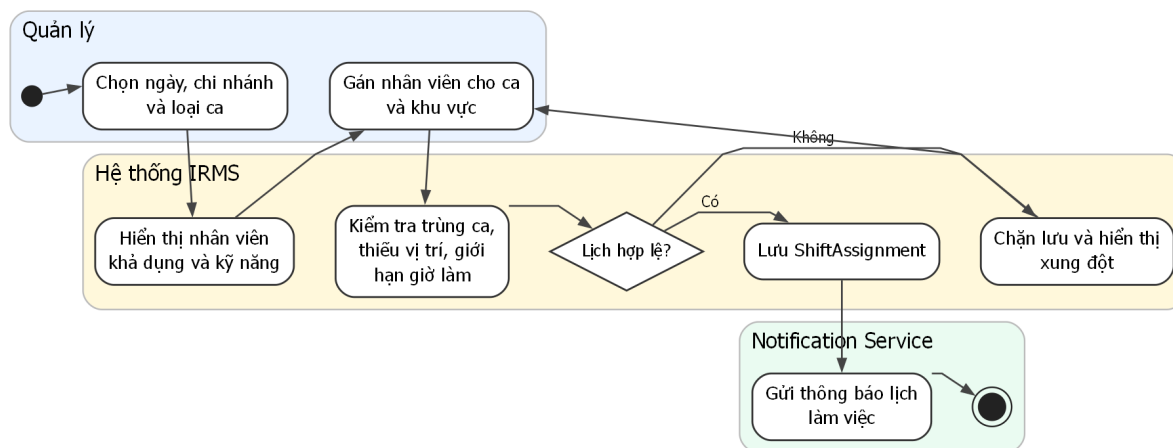
Hình 60: Sơ đồ hoạt động cho UC-ADM-03 – Xem nhật ký kiểm toán và truy vết thao tác nhạy cảm

Phân tích sơ đồ UC-ADM-03. Phần hoạt động này làm rõ rằng nhật ký kiểm toán không chỉ là nơi hệ thống ?ghi lại cho có?, mà là một luồng sử dụng thật sự của khối quản trị. Khi có hoàn tiền bất thường, hủy món sau khi gửi bếp, ghi đè giá, mở lại hóa đơn hoặc thay đổi quyền, quản lý cần một ca sử dụng riêng để truy ngược chuỗi hành động, xem trước-sau, lý do và mã liên kết tới giao dịch gốc. Nếu thiếu ca sử dụng đó, phần audit trong báo cáo sẽ dễ bị hiểu như một thuộc tính kỹ thuật thay vì một khả năng kiểm soát vận hành.

Sơ đồ cũng nhấn mạnh rằng việc xem audit log nhạy cảm tự nó là một hành động cần kiểm soát. Đây là chi tiết quan trọng vì dữ liệu kiểm toán thường chạm vào thông tin nhân sự, quyết

định tài chính và dấu vết thay đổi cấu hình. Do đó, mô hình hành vi không dừng ở bước ?truy vấn log?, mà còn thêm bước kiểm tra quyền, làm mờ dữ liệu nhạy cảm và ghi nhận việc xem hoặc xuất log khi cần.

UC-OPS-01 — Quản lý ca làm nhân viên



Hình 61: Sơ đồ hoạt động cho UC-OPS-01 – Quản lý ca làm nhân viên

4.4 Phản tư về toàn bộ báo cáo và định hướng hiện thực

Điểm mạnh lớn nhất của báo cáo là đã giữ được một đường chỉ dẫn tương đối nhất quán từ bối cảnh nghiệp vụ, yêu cầu phi chức năng, ca sử dụng, kiến trúc, sơ đồ lớp cho tới phần hành vi. Tài liệu không đặt các sơ đồ như những hình minh họa rời rạc, mà dùng mỗi góc nhìn để làm rõ một lớp quyết định riêng của cùng một hệ thống. Nhờ vậy, người đọc có thể lần theo một logic liên tục: từ áp lực vận hành của nhà hàng, sang nhu cầu tách miền nghiệp vụ, sang cách tổ chức mô-đun, thành phần, dữ liệu và các cơ chế xử lý nền.

Điểm mạnh thứ hai nằm ở việc báo cáo đưa **ranh giới năng lực nghiệp vụ** vào vai trò trung tâm. Các phần đặt bàn, gọi món, bếp, quyết toán, tồn kho, báo cáo, phân quyền và kiểm toán đều được mô tả như những miền có trách nhiệm riêng thay vì bị hòa thành một khối xử lý chung. Nhờ vậy, góc nhìn mô-đun, góc nhìn thành phần và góc nhìn lớp hỗ trợ lẫn nhau khá chặt. Khi xuất hiện mô-đun **Billing & Settlement** trong phần mô-đun, người đọc cũng thấy cụm lớp tương ứng trong phần lớp và thấy tuyến xử lý tương ứng trong phần thành phần. Sự nhất quán này làm cho lập luận kiến trúc có sức nặng hơn nhiều so với cách trình bày chỉ dừng ở danh sách chức năng.

Một kết quả tích cực khác là các nguyên lý **SOLID** không bị trình bày như phần lý thuyết tách rời. **SRP** thể hiện ở việc mỗi mô-đun và mỗi lớp điều phối giữ một phạm vi trách nhiệm rõ. **OCP** xuất hiện ở các điểm mở rộng như **PaymentGateway**, **NotificationSender**, **ReceiptPrinter** và các chính sách định giá, phân quyền. **LSP** được phản ánh qua yêu cầu mọi bộ kết nối cùng một vai trò phải giữ hợp đồng hành vi nhất quán. **ISP** thể hiện ở việc tránh tạo các giao diện lớn ôm mọi tích hợp ngoài. **DIP** là nguyên lý giữ ổn định phần lõi khi các chi tiết công nghệ như cổng thanh toán, kênh thông báo hay cách lưu trữ thay đổi. Nhờ được gắn vào cấu trúc mô-đun, thành phần và lớp, **SOLID** ở đây mang ý nghĩa kiến trúc thực sự thay vì chỉ là phần diễn giải bổ sung.

Tuy nhiên, báo cáo cũng có những giới hạn cần được nhìn nhận thẳng thắn. Hệ thống được mô tả khá đầy đủ ở quy mô một nhà hàng hoặc một cụm nhỏ chi nhánh, nên các quyết định như modular monolith, mô hình đọc riêng cho báo cáo và bộ xử lý nền tách biệt là phù hợp. Dù vậy, khi quy mô tăng lên theo hướng nhiều chi nhánh, nhiều thiết bị, nhiều tích hợp thanh toán và nhiều chính sách giá khác nhau, kiến trúc sẽ cần tiến thêm một bước ở vùng triển khai, khả năng quan sát và chiến lược phân tách dữ liệu. Ngoài ra, một số phần như dự báo nhập hàng, điều phối bếp

nâng cao hay xử lý sự cố tích hợp mới dừng ở mức kiến trúc và miền nghiệp vụ, chưa đi sâu tới mức thuật toán hay chính sách vận hành chi tiết.

Phần phản tư quan trọng nhất nằm ở chính kinh nghiệm tổ chức lập luận kiến trúc. Khi bám theo danh sách chức năng, tài liệu rất dễ dừng lại ở việc hệ thống có những gì. Khi chuyển trọng tâm sang giao dịch lõi, hệ quả phụ, biên mô-đun, điểm mở rộng và ranh giới dữ liệu, báo cáo mới bắt đầu thể hiện đúng bản chất của một tài liệu kiến trúc. Từ đó có thể rút ra một nguyên tắc rõ ràng: chất lượng báo cáo không nằm ở số lượng sơ đồ, mà nằm ở việc mỗi sơ đồ có bảo vệ được một quyết định thiết kế cụ thể hay không.

Giá trị lâu dài của tài liệu này là nó tạo ra một khung quyết định có thể tiếp tục dùng trong giai đoạn hiện thực. Những ranh giới như **Reservation & Seating, Ordering, Kitchen Coordination, Billing & Settlement, Inventory, Reporting** và **IAM / Audit** đã được neo tương đối rõ; vì vậy, phần mã nguồn về sau có thể dùng chính các ranh giới đó làm chuẩn để tự kiểm tra chất lượng tổ chức mã. Nếu giữ được vai trò này, báo cáo không chỉ là tài liệu trình bày, mà còn là bản thiết kế có khả năng dẫn hướng cho việc phát triển hệ thống.

A Phụ lục

A.1 Lý do chọn kiến trúc và hồ sơ quyết định kiến trúc

So sánh các kiểu kiến trúc ứng viên

Bảng 7: So sánh các kiểu kiến trúc ứng viên cho IRMS

Kiểu kiến trúc	Ưu điểm	Hạn chế trong bối cảnh IRMS
Layered Monolith thuần	dễ dựng ban đầu, ít thành phần triển khai, giao dịch đơn giản	rất dễ trượt thành khối lớn theo tầng kỹ thuật; gọi món, thanh toán, bếp và báo cáo dễ chồng lấn trách nhiệm; khó giữ ranh giới mô-đun rõ theo năng lực nghiệp vụ
Service-Based phân tán	mỗi năng lực có thể triển khai và mở rộng độc lập hơn; tách cấu trúc thực thi rõ hơn	làm tăng số lượng bước nhảy mạng, nhu cầu truy vết, thử lại, tiến hóa lược đồ và chi phí phối hợp; vượt quá nhu cầu của đề tài nếu hầu hết năng lực vẫn còn cần giao dịch gần nhau
Microservices	cô lập mạnh về triển khai, phù hợp khi mỗi năng lực có đội riêng và nhịp phát hành riêng	chi phí vận hành cao nhất; phát sinh nhiều vấn đề giao dịch phân tán, khả năng quan sát và tiến hóa hợp đồng; không tương xứng với phạm vi học phần và một IRMS quy mô vừa
Event-Driven thuần	hỗ trợ phát tán sự kiện, thử lại, mô hình chiều và các hệ quả phụ tốt; phù hợp tồn kho, báo cáo và thông báo	không phù hợp để dùng cho toàn bộ hệ thống vì gọi món và thanh toán cần phản hồi tức thời và ngữ nghĩa lỗi rõ ràng; dùng thuần sự kiện dễ làm tuyến đầu khó giải thích trạng thái
Kiến trúc lai: Modular Monolith theo năng lực + phân lớp nội bộ + luồng hỗ trợ hướng sự kiện	giữ được ranh giới năng lực rõ; giao dịch lõi đơn giản hơn; dễ triển khai trong học phần; vẫn hỗ trợ hệ quả phụ qua sự kiện và bộ xử lý nền; phù hợp với các sơ đồ triển khai hiện có	khối xử lý phía sau vẫn còn tập trung; một số năng lực lớn lên sau này có thể cần tách dần; đòi hỏi kỷ luật mã nguồn để không phá ranh giới mô-đun bên trong

Từ các sơ đồ mô-đun, thành phần và triển khai, phương án phù hợp nhất cho IRMS không phải là **kiến trúc dịch vụ thuần** theo nghĩa mỗi năng lực là một dịch vụ mạng độc lập, mà là một **kiến trúc lai**. Ở mức logic, hệ thống được phân rã rất rõ theo năng lực nghiệp vụ như đặt bàn và bàn ăn, thực đơn, gọi món, bếp, quyết toán, tồn kho, báo cáo, IAM và kiểm toán. Nhưng ở mức thực thi, các năng lực này vẫn được đặt bên trong một khối xử lý phía sau tập trung, có bộ xử lý nền cho các hệ quả phụ và có mô hình đọc hoặc mô hình chiều cho báo cáo. Vì vậy, cách gọi chính xác hơn là **modular monolith theo năng lực**, bên trong áp dụng **kiến trúc phân lớp**, đồng thời dùng **luồng hỗ trợ hướng sự kiện** cho thông báo, làm mới mô hình đọc và một phần tồn kho, báo cáo.

ADR-01: Lựa chọn kiến trúc lai cho IRMS

- Vấn đề:** IRMS phải đồng thời hỗ trợ đặt bàn và xếp bàn, gọi món, điều phối bếp, quyết toán và thanh toán, theo dõi tồn kho, báo cáo, RBAC và kiểm toán. Kiến trúc được chọn cần giữ ranh giới nghiệp vụ đủ rõ để dễ bảo trì, nhưng không được đẩy chi phí triển khai và vận hành lên mức của một hệ phân tán nặng khi phạm vi học phần vẫn cần ưu tiên tính

khả thi.

2. **Quyết định:** Chọn kiến trúc lai gồm modular monolith theo năng lực ở mức triển khai lõi, phân lớp ở bên trong khối xử lý phía sau, luồng hỗ trợ hướng sự kiện cho thông báo và mô hình đọc, cùng luồng lõi đồng bộ cho đơn gọi món, hóa đơn và thanh toán.
3. **Trạng thái:** ĐÃ PHÊ DUYỆT.
4. **Lập luận:** Phương án này giữ được giao dịch lõi gọn hơn so với hệ phân tán, nhưng vẫn tách đủ rõ các ranh giới mô-đun để phục vụ bảo trì, kiểm thử và tiến hóa về sau.
5. **Hệ quả kiến trúc:** Ở mức logic, các năng lực vẫn được mô tả như dịch vụ hoặc mô-đun riêng. Ở mức thực thi hiện tại, chúng có thể cùng chạy trong một backend runtime. Báo cáo, thông báo và làm mới mô hình chiếu là các ứng viên tách ra trước nếu tải tăng.

ADR-02: Sử dụng API Gateway làm điểm vào thống nhất

1. **Vấn đề:** IRMS có nhiều loại ứng dụng khách như máy tính bảng POS, màn hình KDS, thiết bị thu ngân và ứng dụng web cho quản lý. Nếu từng ứng dụng gọi trực tiếp từng dịch vụ, việc quản lý xác thực, phiên bản giao diện lập trình ứng dụng, giới hạn tốc độ và quan sát hệ thống sẽ bị phân tán.
2. **Quyết định:** Đặt **API Gateway** làm điểm vào duy nhất cho mọi ứng dụng khách. Gateway chịu trách nhiệm xác thực mã truy cập, định tuyến yêu cầu, giới hạn tốc độ, ghi nhật ký ở biên và áp dụng các chính sách chung trước khi chuyển tiếp tới các dịch vụ phía sau.
3. **Trạng thái:** ĐÃ PHÊ DUYỆT.
4. **Lập luận:** Gateway giảm phụ thuộc chặt giữa ứng dụng khách và cấu trúc nội bộ, giúp chuẩn hóa bảo mật, kiểm toán và phát hành giao diện lập trình ứng dụng.

ADR-03: Kết hợp REST đồng bộ với sự kiện miền bất đồng bộ

1. **Vấn đề:** Một số tương tác trong IRMS cần phản hồi ngay, ví dụ tra cứu thực đơn, tạo đơn gọi món hoặc ghi nhận thanh toán; trong khi một số luồng khác phù hợp hơn với xử lý bất đồng bộ như gửi phiếu sang bếp, cập nhật tồn kho, làm mới báo cáo hay gửi cảnh báo.
2. **Quyết định:** Dùng **REST đồng bộ** cho lệnh và truy vấn nghiệp vụ ở ranh giới người dùng, đồng thời dùng **sự kiện miền** cho các tác vụ phát tán hoặc cần nhất quán sau cùng như định tuyến sang bếp, khấu trừ tồn kho, cảnh báo sắp hết hàng và làm mới báo cáo.
3. **Trạng thái:** ĐÃ PHÊ DUYỆT.
4. **Lập luận:** Cách kết hợp này giữ được trải nghiệm thao tác nhanh ở tuyến đầu, đồng thời tách rời các luồng nền có thể xử lý lại hoặc mở rộng độc lập.

ADR-04: Tách dữ liệu theo miền nghiệp vụ và dùng read model cho báo cáo

1. **Vấn đề:** Nếu toàn bộ đặt bàn, gọi món, bếp, quyết toán, tồn kho và báo cáo cùng đọc ghi trên một lược đồ dùng chung, hệ thống sẽ khó kiểm soát khóa, khó mở rộng, và báo cáo phân tích dễ tác động ngược lại đường giao dịch trực tuyến.
2. **Quyết định:** Mỗi dịch vụ hoặc mô-đun sở hữu dữ liệu giao dịch theo miền của mình. Phân báo cáo không đọc trực tiếp đường ghi nghiệp vụ mà xây dựng mô hình đọc hoặc ảnh chụp từ sự kiện miền hoặc một đường ống đồng bộ nhẹ.
3. **Trạng thái:** ĐÃ PHÊ DUYỆT.

4. **Lập luận:** Tách dữ liệu theo miền giảm phụ thuộc chặt, tăng khả năng kiểm soát lược đồ và cho phép mở rộng theo điểm nóng. Mô hình đọc báo cáo giúp bảng điều khiển quản lý phong phú nhưng không cản trở POS và KDS ở giờ cao điểm.

ADR-05: Áp dụng RBAC và nhật ký kiểm toán cho tác vụ nhạy cảm

1. **Vấn đề:** IRMS chứa các thao tác nhạy cảm như ghi đề giá, hủy món sau khi gửi bếp, hoàn tiền, điều chỉnh tồn kho, công bố thực đơn và thay đổi phân quyền. Nếu không kiểm soát và ghi vết đầy đủ, hệ thống khó đáp ứng nhu cầu vận hành và kiểm tra sau sự cố.
2. **Quyết định:** Triển khai **RBAC** ở tầng IAM và Gateway, đồng thời thực thi thêm **AuthorizationPolicy** trong dịch vụ cho các thao tác nghiệp vụ nhạy cảm. Song song với đó, ghi **AuditLog** cho mọi hành động quan trọng với thông tin ai thực hiện, khi nào, thao tác nào và trên thực thể nào.
3. **Trạng thái:** ĐÃ PHÊ DUYỆT.
4. **Lập luận:** Kết hợp RBAC với chính sách trong dịch vụ vừa chặn truy cập sai vai trò, vừa xử lý được các điều kiện nghiệp vụ tinh hơn. Nhật ký kiểm toán tạo nền cho đối soát giao dịch, điều tra hoàn tiền và đánh giá hiệu suất vận hành.

A.2 Phân công công việc

Bảng 8: Phân công công việc của nhóm 8

MSSV	Thành viên	Phạm vi đóng góp chính
2311896	Đỗ Quang Long	Tổng hợp bối cảnh nghiệp vụ, yêu cầu chức năng và phi chức năng; hoàn thiện phần ca sử dụng và rà soát tính nhất quán giữa use case với kiến trúc.
2311982	Lê Hoàng Luân	Tổ chức cấu trúc dự án LaTeX, chuẩn hóa sơ đồ, tích hợp tài liệu, hoàn thiện các góc nhìn kiến trúc, bố cục báo cáo và bản dựng cuối cùng.
2310990	Lê Thúy Hiền	Xây dựng phần đặc tả use case, hoạt động nghiệp vụ và diễn giải các luồng đặt bàn, gọi món, thanh toán theo ngữ cảnh vận hành nhà hàng.
2213698	Nguyễn Hải Trung	Hoàn thiện các góc nhìn mô-đun, thành phần và kết nối; tham gia lập luận lựa chọn phong cách kiến trúc và hồ sơ quyết định kiến trúc.
2211384	Tô Thế Hưng	Phát triển các sơ đồ lớp, rà soát mô hình miền nghiệp vụ, đối chiếu nguyên lý SOLID với cấu trúc lớp và các điểm mở rộng.
2310737	Trần Nhật Đăng	Hoàn thiện phần triển khai, phân bổ, phụ trách phần phản tư và rà soát chất lượng trình bày toàn báo cáo.

Phân công trên phản ánh vai trò chính của từng thành viên trong quá trình xây dựng báo cáo. Trên thực tế, toàn bộ tài liệu đã được rà soát chéo nhiều vòng để thống nhất thuật ngữ, sơ đồ và lập luận kiến trúc trước khi chốt bản cuối.